

Corso di Onboarding DevOps

1. Introduzione

Benvenuto/a a bordo!

Come è organizzato questo corso

Il contesto di lavoro: cosa fa il tuo team, in parole semplici

Il tuo percorso: da Junior Project Manager a Scrum Master

Come affrontare lo studio: qualche consiglio pratico

La mappa del corso

Pronto/a per iniziare?

- Esercizi pratici

- Collegamenti

- Risorse

2. Fondamenti di informatica

Obiettivi della sezione

2.1 Cos'è un computer: hardware e software

2.2 La CPU: il cervello che calcola

2.3 La RAM: la memoria di lavoro temporanea

2.4 Il disco: la memoria permanente

2.5 Il Sistema Operativo: chi coordina tutto

2.6 Processi e Thread

2.7 Il File System: come sono organizzati i file

2.8 La rete: cos'è una rete di computer

2.9 TCP/IP: come i computer si "spediscono lettere"

2.10 HTTP e HTTPS: il protocollo del web

2.11 DNS: la rubrica telefonica di internet

2.12 API: come i software si parlano tra loro

2.13 REST: uno stile comune per costruire API

2.14 JSON e XML: i formati per scambiarsi dati

2.15 Database relazionali e SQL

2.16 Database NoSQL

2.17 Virtual Machine: un computer dentro un computer

2.18 Container: più leggeri di una VM

2.19 Docker: lo strumento più diffuso per i container

2.20 Kubernetes: l'orchestratore di container

2.21 Riepilogo: come si incastrano tutti questi pezzi

- Checklist di autoverifica

- Esercizi pratici

- Collegamenti

- Risorse

3. Come nasce un software

Obiettivi della sezione

Il ciclo di vita del software

Requisiti: funzionali e non funzionali

- Bug: quando qualcosa va storto
- Feature: una nuova funzionalità
- Versioning: perché numeriamo il software
- Branching: lavorare in parallelo
 - Dal branch al merge: merge request e code review
- Come si collegano tutti questi concetti
 - Esercizi pratici
 - Collegamenti
 - Risorse

4. Git e GitLab

Obiettivi della sezione

- 4.1 Cos'è Git
- 4.2 Cos'è GitLab (e non è la stessa cosa di Git!)
- 4.3 Repository: il “progetto” versionato
- 4.4 Commit: uno scatto fotografico delle modifiche
- 4.5 Branch: un ramo di lavoro parallelo
- 4.6 Merge: unire due rami
- 4.7 Merge Request: il cuore della collaborazione su GitLab
- 4.8 Issue: tracciare bug e richieste
- 4.9 Release: una versione pubblicata ufficialmente
- 4.10 Tag: etichettare un punto preciso della storia
- 4.11 Git Flow: un workflow strutturato a più branch
- 4.12 Trunk Based Development: il workflow semplificato
- 4.13 Riepilogo dei comandi visti in questa sezione
 - Esercizi pratici
 - Collegamenti
 - Risorse

5. Agile

Obiettivi della sezione

- 5.1 Perché nasce l'Agile: il problema del modello a “cascata”
- 5.2 Il Manifesto Agile: i 4 valori fondamentali
- 5.3 I 12 principi Agile: raggruppati per temi
- 5.4 Il “mindset” Agile: più di un metodo, un modo di pensare
- 5.5 Waterfall vs Agile: il confronto visivo
- 5.6 Dall'Agile ai framework concreti: Scrum e Kanban
 - Esercizi pratici
 - Collegamenti
 - Risorse

6. Scrum

Obiettivi della sezione

- 6.1 Cos'è Scrum e perché è ovunque
- 6.2 I tre ruoli di Scrum
- 6.3 Gli eventi di Scrum
- 6.4 Gli artefatti di Scrum
- 6.5 User Story: come si descrive un requisito in Scrum
- 6.6 Story Point: stimare senza usare ore o giorni

- 6.7 Velocity: quanto il team riesce a fare in uno sprint
- 6.8 Definition of Ready: quando una storia può entrare in sprint
- 6.9 Definition of Done: quando una storia è davvero completata
- 6.10 Il flusso completo, in sintesi
- 6.11 Il ruolo dello Scrum Master nella pratica quotidiana

Esercizi pratici

Collegamenti

Risorse

7. Kanban

Obiettivi della sezione

- 7.1 Da dove viene Kanban
- 7.2 La board Kanban: colonne e card
- 7.3 WIP: il lavoro “in corso” e i suoi limiti
- 7.4 Lead Time e Cycle Time
- 7.5 Kanban vs Scrum: quando usare cosa
- 7.6 Riepilogo

Esercizi pratici

Collegamenti

Risorse

8. Project Management

Obiettivi della sezione

- 8.1 Stakeholder: chi ha interesse nel progetto
- 8.2 RAID Log: la mappa dei problemi (prima che diventino problemi)
- 8.3 RACI: chi fa cosa (e chi deve solo saperlo)
- 8.4 Milestone: un traguardo, non una scadenza qualsiasi
- 8.5 Rischio: identificare, valutare, mitigare
- 8.6 Pianificazione in un contesto Agile: livelli, non un unico piano
- 8.7 Monitoraggio: come si tiene traccia dell'andamento
- 8.8 KPI: gli indicatori chiave per un team Agile/DevOps
- 8.9 Reportistica: cosa riportare, a chi, e con quale livello di dettaglio
- 8.10 Riepilogo

Esercizi pratici

Collegamenti

Risorse

9. DevOps

Obiettivi della sezione

- 9.1 Cos'è DevOps: non è un tool, è una cultura
- 9.2 Perché nasce DevOps: il muro della confusione
- 9.3 Il ciclo DevOps: un percorso senza fine
- 9.4 La cultura DevOps: collaborazione, responsabilità, fiducia
- 9.5 Automazione: il motore pratico di DevOps
- 9.6 Continuous Integration (CI): integrare spesso, non alla fine
- 9.7 Continuous Delivery e Continuous Deployment: la differenza che confonde tutti
- 9.8 Infrastructure as Code (IaC): l'infrastruttura scritta come codice
- 9.9 Monitoring: sapere se le cose stanno andando bene
- 9.10 Observability: capire non solo il “cosa”, ma il “perché”

9.11 Logging: la memoria degli eventi del sistema

9.12 Riepilogo

Esercizi pratici

Collegamenti

Risorse

10. Azure DevOps

Obiettivi della sezione

10.1 Cos'è Azure DevOps

10.2 Boards: la gestione del lavoro

10.3 Repos: il codice sorgente

10.4 Pipelines: build e rilascio automatizzati

10.5 Test Plans: qualità e collaudo

10.6 Artifacts: i pacchetti software

10.7 Dashboard: lo stato del progetto a colpo d'occhio

10.8 Il flusso end-to-end: dalla User Story alla produzione

10.9 Mappa dei concetti: cosa hai già visto, con un altro nome

10.10 Riepilogo

Esercizi pratici

Collegamenti

Risorse

11. CI/CD

Obiettivi della sezione

11.1 Ripasso: cosa sono CI e CD

11.2 Anatomia di una pipeline: le fasi tipiche

11.3 Trigger: cosa fa scattare una pipeline

11.4 Ambienti: dove "vive" il codice, fase per fase

11.5 Artifact: il "pacco" che viaggia lungo la pipeline

11.6 Pipeline as Code: la pipeline è codice, non un click di troppo

11.7 Un esempio di pipeline in YAML

11.8 Quality Gate: i controlli che possono bloccare tutto

11.9 Rollback: cosa succede se un deploy va male comunque

11.10 Strumenti concreti della pipeline

11.11 Riepilogo

Esercizi pratici

Collegamenti

Risorse

12. Architetture software

Obiettivi della sezione

12.1 Cos'è un'architettura software

12.2 Architettura Monolitica

12.3 Architettura a Microservizi

12.4 Come comunicano i componenti

12.5 Frontend vs Backend

12.6 Client-Server: il concetto base

12.7 Architettura a 3 livelli

12.8 Un accenno al Serverless

12.9 Riepilogo: cosa ti serve ricordare

- Checklist di autoverifica
- Esercizi pratici
- Collegamenti
- Risorse

13. Cloud

Obiettivi della sezione

- 13.1 Cos'è il cloud computing
- 13.2 I tre modelli principali: IaaS, PaaS, SaaS
- 13.3 Confronto: chi gestisce cosa
- 13.4 Azure: panoramica generale
- 13.5 AWS: la principale alternativa
- 13.6 Scalabilità: verticale vs orizzontale
- 13.7 Pay-as-you-go: paghi solo quello che usi
- 13.8 Regioni, data center e alta disponibilità
- 13.9 Riepilogo: cosa ti serve ricordare

- Checklist di autoverifica
- Esercizi pratici
- Collegamenti
- Risorse

14. Sicurezza

Obiettivi della sezione

- 14.1 Perché la sicurezza riguarda anche un PM, non solo i tecnici
- 14.2 Autenticazione vs Autorizzazione
- 14.3 Password, credenziali e MFA: le buone pratiche di base
- 14.4 HTTPS e crittografia: un richiamo veloce
- 14.5 Vulnerabilità comuni: perché il codice va "testato" anche per la sicurezza
- 14.6 OWASP Top 10: il riferimento standard del settore
- 14.7 DevSecOps: la sicurezza fin dall'inizio ("shift left")
- 14.8 Scan di sicurezza nella pipeline CI/CD
- 14.9 GDPR e privacy dei dati: un accenno essenziale
- 14.10 Backup e disaster recovery: il piano B quando qualcosa va storto
- 14.11 Riepilogo

- Checklist di autoverifica
- Esercizi pratici
- Collegamenti
- Risorse

15. Ambienti di sviluppo

Obiettivi della sezione

- 15.1 Perché servire diversi ambienti: la "prova costume"
- 15.2 Ambiente di Sviluppo (Dev)
- 15.3 Ambiente di Test / QA
- 15.4 Ambiente di Staging (Pre-produzione)
- 15.5 Ambiente di Produzione
- 15.6 I quattro ambienti, fase per fase
- 15.7 "Funziona sul mio computer": perché gli ambienti devono somigliarsi

15.8 Dati di test vs dati reali

15.9 Configurazioni per ambiente

15.10 Chi ha accesso a quali ambienti

Confronto tra i quattro ambienti

15.11 Riepilogo

- Esercizi pratici

- Collegamenti

Risorse

16. Glossario

Mappa dei concetti: come si collegano

Collegamenti

Risorse

17. Piano di studio

Come usare questo piano

Vista d'insieme del percorso

Settimana 1 — Le fondamenta: come funziona un computer, come nasce un software

Settimana 2 — Come nasce un software e il controllo di versione con Git

Settimana 3 — Agile e Scrum: il mindset e il framework che il team usa ogni giorno

Settimana 4 — Kanban e Project Management: un altro modo di visualizzare il lavoro, e come si tiene sotto controllo un progetto

Settimana 5 — DevOps: la cultura e i concetti che uniscono sviluppo e operations

Settimana 6 — Azure DevOps e CI/CD in dettaglio: dalla teoria alla pipeline reale

Settimana 7 — Il contesto tecnico e trasversale: architetture, cloud, sicurezza, ambienti

Settimana 8 — Consolidamento: ripasso generale, autovalutazione e prossimi passi

Tabella riepilogativa delle 8 settimane

Collegamenti

Risorse

18. Libri consigliati

La logica dei 5 livelli

Livello 0 – Fondamenti di informatica

Livello 1 – Sviluppo software

Livello 2 – Agile

Livello 3 – Project Management

Livello 4 – DevOps

Tabella riepilogativa

Collegamenti

Risorse

19. Risorse online

Come usare questa sezione

1. Fondamenti di informatica

2. Sviluppo software e Git/GitLab

3. Agile, Scrum, Kanban

4. Project Management

5. DevOps, Azure DevOps, CI/CD

6. Architetture, Cloud, Sicurezza

Come restare aggiornato nel tempo

Collegamenti

CORSO DI ONBOARDING

Da neolaureato a Junior Project Manager / Scrum Master

1. Introduzione

Benvenuto/a a bordo!

Ciao, e benvenuto/a in azienda!

Se stai leggendo questa pagina, probabilmente ti sei appena seduto/a alla tua scrivania (fisica o virtuale) con un mucchio di acronimi nuovi in testa — Scrum, Sprint, DevOps, CI/CD, board, backlog — e la sensazione di dover imparare un'intera lingua straniera in poche settimane.

Respira: è normale, ed è esattamente per questo che esiste questo corso.

Sei stato/a assunto/a come **Junior Project Manager** e, nell'arco dei prossimi 2-3 mesi, affiancherai (e poi gradualmente sostituirai) una collega che oggi lavora come **Scrum Master / Project Manager** su un team che si occupa di una **piattaforma DevOps a supporto dello sviluppo software**. Non sai cosa significhi tutto questo? Perfetto, è il punto di partenza giusto: questo corso è pensato apposta per chi, come te, arriva da un percorso di studi gestionale (Ingegneria Gestionale) e parte da zero dal punto di vista tecnico e informatico.

Non ti verrà chiesto di scrivere codice o di diventare uno sviluppatore. Ti verrà chiesto di **capire come funziona il mondo in cui il tuo team lavora**, per poterlo organizzare, facilitare e rappresentare al meglio — verso il team stesso, verso i responsabili e verso il cliente.

Come è organizzato questo corso

Il corso è diviso in **19 sezioni numerate**, pensate per essere seguite **in ordine progressivo**: ogni sezione si appoggia sui concetti spiegati in quella precedente, un po' come i mattoncini di un Lego — non puoi costruire il tetto se non hai ancora messo le fondamenta.

Le sezioni si possono raggruppare in quattro grandi blocchi:

1. **Fondamenta tecniche** (sezioni 2-4): cosa è un computer, come “nasce” un software, cos'è Git/GitLab. Ti servono per capire il linguaggio di base che sviluppatori e tecnici usano ogni giorno.
2. **Metodologie di lavoro** (sezioni 5-8): Agile, Scrum, Kanban e Project Management. Sono i “metodi di organizzazione del lavoro” — qui il tuo background gestionale ti aiuterà moltissimo, perché sono concetti di organizzazione applicati al software.
3. **Mondo DevOps** (sezioni 9-14): DevOps, Azure DevOps, CI/CD, architetture software, cloud e sicurezza. È il cuore tecnico del corso, quello più legato al ruolo che andrai a occupare.
4. **Strumenti di supporto e consultazione** (sezioni 15-19): ambienti di sviluppo, glossario, piano di studio, libri e risorse online. Non sono da “studiare” una volta e basta: sono pensate per essere consultate spesso, come un dizionario o un'agenda.

Esempio pratico: se in una riunione senti dire “la pipeline è rossa, il rilascio è bloccato”, non serve che tu capisca subito cosa significhi nel dettaglio. Ti basta pensare “questo è un argomento del Blocco C, probabilmente della sezione 11 sul CI/CD” e prendere nota mentale per approfondirlo quando arrivi lì, invece di sentirti perso/a o di dover fingere di aver capito.

Oltre alle 19 sezioni, trovi due strumenti trasversali che ti accompagneranno per tutto il percorso:

- **Il Glossario** (sezione 16): ogni volta che incontri un termine che non ricordi — sia in questo corso, sia durante una riunione di lavoro — puoi cercarlo lì. Non c'è nulla di male nel non ricordare a memoria cosa significa “backlog” la terza volta che lo senti: è normale, ci vuole ripetizione.
- **Il Piano di studio** (sezione 17): un programma settimanale su **8 settimane** che ti dice, settimana per settimana, cosa leggere, quanto tempo dedicarci, e con quali attività pratiche verificare di aver capito (ad esempio: “questa settimana osserva una Daily Scrum del team e prendi nota di cosa succede”).

Consiglio pratico: la prima volta che apri il corso, dai una scorsa veloce a tutti i titoli delle 19 sezioni (li trovi anche nel menu laterale). Non devi capire tutto subito: ti basta avere una mappa mentale di “cosa troverò più avanti”, così quando in una sezione si accenna a un concetto che verrà spiegato dopo, saprai che è normale e che arriverà il suo momento.

Il contesto di lavoro: cosa fa il tuo team, in parole semplici

Prima di addentrarci nei dettagli tecnici (che vedremo dalla sezione 2 in poi), proviamo a capire **a grandi linee** cosa significa lavorare su “una piattaforma DevOps a supporto dello sviluppo software”. Niente gergo tecnico per ora: solo un'analogia.

Immagina una grande cucina di un ristorante che deve servire centinaia di piatti ogni giorno, in modo veloce, corretto e senza errori. In quella cucina:

- ci sono **cuochi** che preparano i piatti (gli **sviluppatori**, che scrivono il codice del software);
- ci sono **linee di produzione** che passano il piatto da una stazione all'altra — dalla preparazione, alla cottura, all'impattamento, al controllo qualità, fino al tavolo del cliente (questo, nel mondo software, si chiama **pipeline**, e lo vedremo nella sezione 11 sul CI/CD);
- ci sono **strumenti e attrezzature** condivise da tutta la cucina — i forni, i frigoriferi, i banchi di lavoro (nel mondo software, sono i **server, gli ambienti di test, gli strumenti di collaborazione**: lo vedremo nelle sezioni su DevOps, Azure DevOps e Cloud);
- e c'è un **responsabile di sala/cucina** che si assicura che gli ordini arrivino in tempo, che i cuochi non siano sovraccarichi, che eventuali problemi (un piatto tornato indietro, un ingrediente finito) vengano gestiti e comunicati al cliente. Questo è, in sostanza, il ruolo di **Project Manager / Scrum Master**: non cucini tu, ma fai in modo che la cucina funzioni, che il team lavori bene insieme, e che il cliente sia informato e soddisfatto.

Una **piattaforma DevOps** è l'insieme di strumenti e processi che permettono a questa “cucina” di funzionare in modo automatico, veloce e sicuro: dal momento in cui un cuoco scrive una nuova ricetta (scrive codice), al momento in cui quel piatto arriva davanti al cliente (il software è “in produzione”, cioè realmente usato dagli utenti finali), passando per tutti i controlli di qualità intermedi.

Non preoccuparti se questa descrizione ti sembra ancora vaga: è voluto. Da qui in avanti, sezione dopo sezione, ogni singolo “ingrediente” di questa cucina (Git, Scrum, Kanban, CI/CD, Cloud, Sicurezza...) verrà spiegato nel dettaglio, con esempi concreti tratti dal lavoro quotidiano del team.

Per rendere questi esempi davvero concreti (e non un'accozzaglia di casi slegati tra loro), in tutto il corso useremo sempre lo stesso scenario: **ShopFacile**, una piattaforma e-commerce (catalogo prodotti, carrello, ordini, pagamenti, sconti) sviluppata da un piccolo team interno. Il team è composto da **Marco** e **Giulia**, developer con esperienza medio-alta (Giulia è particolarmente attenta a test e qualità), **Ahmed**, developer più junior e in crescita, **Sara**, Product Owner che rappresenta le esigenze del business, e **Luca**, Scrum Master/Team Lead — il ruolo che tu stesso/a stai imparando a coprire. Ogni volta che nelle prossime sezioni troverai un esempio pratico, sarà quasi sempre un episodio della vita di ShopFacile e del suo team: così, invece di dover ricordare scenari diversi ogni volta, potrai concentrarti sul concetto nuovo riconoscendo persone e progetto già familiari.

Esempio pratico: Ahmed, il developer più junior del team, completa una nuova funzionalità di ShopFacile (un nuovo filtro di ricerca nel catalogo prodotti) e la carica su Git (il “quaderno delle ricette”, lo vedremo nella sezione 4). Da quel momento, in automatico, una serie di controlli (i “controlli qualità” della pipeline) verificano che il codice non rompa nulla di esistente. Se tutto va bene, la funzionalità arriva prima in un ambiente di test, poi — dopo l'ok di Giulia in fase di review — in produzione, cioè visibile ai clienti reali del negozio online. Il tuo ruolo, in questo flusso, non è scrivere codice: è assicurarti che tutti (incluse Sara e Luca) sappiano cosa sta succedendo, che eventuali blocchi (un test che fallisce, un ambiente non disponibile) vengano comunicati subito, e che il cliente sia aggiornato su quando la funzionalità sarà disponibile.

Il tuo percorso: da Junior Project Manager a Scrum Master

Il tuo percorso di crescita in questi 2-3 mesi seguirà tre fasi, che si sovrappongono gradualmente:

Diagramma 1

Diagramma 1

- **Fase 1 - Osservazione:** parteciperai alle riunioni del team (le vedremo nella sezione 6 su Scrum: si chiamano “cerimonie” o “eventi”, come la Daily Scrum o lo Sprint Planning) semplicemente osservando. In questa fase il tuo compito principale è studiare le sezioni di questo corso e collegare quello che leggi a quello che vedi succedere dal vivo.
- **Fase 2 - Affiancamento attivo:** comincerai a condurre tu stesso/a alcune attività — ad esempio facilitare una Daily Scrum, aggiornare la board del progetto, preparare un report per il cliente — mentre la tua collega osserva e ti dà feedback.

- **Fase 3 - Autonomia guidata:** gestirai in autonomia le attività di routine del ruolo, con la tua collega disponibile per domande su casi più complessi, finché il passaggio di consegne non sarà completo.

Questo non è un percorso “accademico” fine a se stesso: è un percorso pensato per portarti, passo dopo passo, a **sostituire operativamente** una persona con un ruolo di responsabilità. Per questo è fondamentale che, oltre a leggere le sezioni, tu **osservi il lavoro reale del team** e faccia domande — tante domande — alla tua collega e al resto del team.

Esempio pratico: nella Fase 1, durante una Daily Scrum, ti limiti ad ascoltare e prendere appunti su chi dice cosa. Nella Fase 2, è la tua collega a chiederti: “oggi la conduci tu, io resto in ascolto e ti do feedback alla fine” — magari dovrai gestire un membro del team che si dilunga troppo su un problema tecnico, riportando la discussione sui binari giusti. Nella Fase 3, sei tu a organizzare la Daily Scrum in autonomia, e la tua collega interviene solo se le chiedi aiuto su un caso particolare (ad esempio, come gestire un conflitto tra due membri del team sulle priorità).

Come affrontare lo studio: qualche consiglio pratico

Prima di iniziare, qualche indicazione su come vivere questo percorso senza scoraggiarti:

1. **Non devi capire tutto la prima volta.** Concetti come “pipeline CI/CD” o “architettura a microservizi” richiedono più di una lettura per sedimentare. È previsto che tu debba tornare indietro e rileggere: non è un tuo limite, è la normalità quando si impara qualcosa di completamente nuovo.
2. **Fai domande, sempre.** Non esiste una domanda “troppo banale” in questo percorso. Chi lavora ogni giorno con questi strumenti a volte dà per scontati concetti che per te sono nuovi: chiedere è il modo più veloce per colmare quel divario, molto più veloce che aspettare di capire tutto da solo/a leggendo.
3. **Collega la teoria a quello che vedi sul campo.** Ogni volta che in una riunione o in una chat di lavoro senti un termine nuovo, provalo a ricercare nel [Glossario](#) o pensa a quale sezione del corso lo tratta. Questo doppio binario (teoria + osservazione pratica) è il modo più efficace per imparare in questo contesto.
4. **Prova gli esempi, non limitarti a leggerli.** Dove il corso propone un esempio pratico (una board Kanban, un comando Git, un diagramma di pipeline), se possibile provalo tu stesso/a in un ambiente di prova. Capire leggendo è utile, ma capire “con le mani” resta molto più solido.
5. **Usa il piano di studio come bussola, non come gabbia.** Il [Piano di studio](#) a 8 settimane è una guida, non un obbligo rigido: se una settimana hai bisogno di più tempo su un argomento (capita spessissimo con i fondamenti di informatica, sezione 2), prenditelo. È molto meglio capire bene una sezione in più tempo che correre e arrivare in fondo con basi fragili.
6. **A fine settimana, fai un check-in con la tua collega o il tuo mentor.** Un breve confronto settimanale ti permette di validare cosa hai capito, chiarire dubbi rimasti in sospeso e ricevere indicazioni su cosa osservare la settimana successiva nel lavoro reale del team.

Esempio pratico: leggi nella sezione 7 cos'è un limite di WIP (Work In Progress) su una board Kanban. Il giorno dopo, durante una riunione, senti la tua collega dire “non possiamo prendere in carico un'altra card, siamo già al limite in ‘In Corso’”. Invece di lasciarlo passare, collegalo subito a quanto letto e magari chiedi: “è questo il limite di WIP di cui si parlava nella sezione 7?”. Questo piccolo gesto — collegare una lettura a una frase sentita dal vivo — è esattamente il “doppio binario” di cui si parla al punto 3.

La mappa del corso

Il diagramma seguente mostra come le 19 sezioni si collegano tra loro a grandi linee. Non è necessario memorizzarlo: serve solo per farti un'idea di dove ti troverai, sezione dopo sezione.

Diagramma 2

Diagramma 2

In sintesi: prima impari **il linguaggio di base** del software (Blocco A), poi **il modo in cui i team si organizzano** per costruirlo (Blocco B), poi **gli strumenti e i processi specifici** della piattaforma DevOps che gestirai (Blocco C), e infine hai a disposizione un **kit di strumenti di consultazione** (Blocco D) da usare per tutta la durata del percorso e anche dopo.

Pronto/a per iniziare?

Nella prossima sezione, [2. Fondamenti di informatica](#), partiremo dalle basi più elementari: cosa è un computer, come funziona una rete, cosa sono un server e un database. Sono i mattoncini su cui si costruirà tutto il resto del corso, quindi prenditi il tempo che serve.

Buon percorso!

Esercizi pratici

Prima di passare alla sezione 2, prova questi esercizi: non richiedono ancora conoscenze tecniche, servono solo a farti “digerire” la mappa del corso e il contesto in cui lavorerai.

1. **Disegna la mappa del corso a modo tuo.** Su un foglio o in un documento, riscrivi con parole tue (senza copiare) i quattro blocchi (Fondamenta tecniche, Metodologie di lavoro, Mondo DevOps, Strumenti di supporto) e per ciascuno annota una sola parola chiave che ti aiuti a ricordarlo. **Come verificare:** se riesci a spiegare a voce alta, in meno di 90 secondi e senza guardare il foglio, “di cosa parla ciascuno dei quattro blocchi”, l'esercizio è riuscito.

2. **Racconta l'analogia della cucina a qualcuno.** Prova a spiegare a un amico o familiare (anche non del settore) l'analogia della cucina del ristorante usata in questa sezione, sostituendo i termini tecnici con quelli che hai appena imparato (cuochi = sviluppatori, pipeline = linea di produzione, ecc.). **Come verificare:** se la persona a cui lo racconti riesce a ripeterti, con parole sue, cosa fa un Project Manager/Scrum Master in questa analogia, hai comunicato bene il concetto.
3. **Individua le tue tre fasi nel calendario.** Segna sul tuo calendario (anche approssimativamente) le date di inizio previste delle tre fasi del tuo percorso (Osservazione, Affiancamento attivo, Autonomia guidata) in base a quando sei stato/a assunto/a. **Come verificare:** mostra il calendario alla tua collega Scrum Master/PM e chiedile se le date che hai stimato sono realistiche rispetto al carico di lavoro reale del team in quel periodo.
4. **Fai una prima esplorazione del Glossario.** Apri il [Glossario](#) e scorri per 5 minuti senza l'ansia di memorizzare nulla: individua 3 termini che hai già sentito nominare in azienda (anche di striscio, in corridoio o in chat) ma che non sapevi spiegare con precisione. **Come verificare:** scrivi i 3 termini scelti e una definizione con parole tue in una nota; a fine settimana 1 del piano di studio, riguardala e controlla se la tua definizione è ancora corretta o va corretta.
5. **Identifica chi è chi nel tuo contesto di lavoro.** Chiedi alla tua collega o a un membro del team di aiutarti a fare un elenco veloce (basta un elenco a punti, non serve un documento formale) di chi sono, nel progetto reale su cui lavorerai, gli "sviluppatori/cuochi", chi si occupa di "operations" e chi è il referente lato cliente. **Come verificare:** se dopo questo esercizio sai dire, senza esitare, a chi rivolgerti per un problema tecnico e a chi per una domanda del cliente, l'esercizio ha funzionato.

Collegamenti

- [2. Fondamenti di informatica](#) — le basi tecniche da cui parte tutto il resto del corso
- [16. Glossario](#) — da consultare ogni volta che incontri un termine nuovo
- [17. Piano di studio](#) — il programma dettagliato delle 8 settimane
- [19. Risorse online](#) — materiali extra se vuoi approfondire fin da subito

Risorse

- [Atlassian – Agile Coach: cos'è un team Agile](#) — introduzione generale al mindset Agile, utile come primo assaggio prima delle sezioni 5-7
- [GitLab Docs – Get started with Git](#) — guida ufficiale per chi non ha mai usato Git, utile in anticipo sulla sezione 4
- [Microsoft Learn – Cos'è DevOps](#) — panoramica ufficiale Microsoft sul concetto di DevOps, come anteprima della sezione 9
- [Project Management Institute – What is Project Management](#) — visione generale del ruolo di Project Manager, come richiamo al tuo percorso di studi

2. Fondamenti di informatica

Questa è una delle sezioni più importanti di tutto il corso. Non preoccuparti se non hai mai scritto una riga di codice o non sai cosa sia un server: qui costruiamo insieme, da zero, tutto il vocabolario tecnico che ti servirà per capire il resto del percorso (Git, Agile, DevOps, Cloud...).

Non serve leggere tutto in una sola sessione. Prendi questa sezione a piccoli pezzi, prova a rileggere gli esempi con i tuoi tempi e, se un concetto non è chiaro, chiedi al tuo mentor. È normale che alcuni termini richiedano più di una lettura per “entrare” davvero.

Obiettivi della sezione

Al termine di questa sezione saprai:

- distinguere hardware e software, e conoscere i componenti principali di un computer (CPU, RAM, disco);
- spiegare cosa fa un sistema operativo e cosa sono processi e thread;
- capire come sono organizzati i file su un computer;
- spiegare in modo semplice come funziona una rete, internet, e i protocolli TCP/IP, HTTP/HTTPS, DNS;
- capire cos'è un'API, cosa significa REST, e leggere un semplice JSON;
- capire la differenza tra database relazionali e NoSQL;
- capire cosa sono Virtual Machine, Container, Docker e Kubernetes — i mattoncini su cui si basa gran parte del mondo DevOps che incontrerai nel team.

2.1 Cos'è un computer: hardware e software

Partiamo dalla domanda più semplice possibile: **cos'è un computer?**

Un computer è una macchina che esegue istruzioni. Punto. Tutto quello che fa — mostrarti una pagina web, farti scrivere un documento, farti guardare un video — è il risultato di miliardi di piccole istruzioni eseguite a altissima velocità.

Per capire come funziona, dobbiamo separare due concetti fondamentali:

- **Hardware:** sono i componenti fisici, quelli che puoi toccare. Lo schermo, la tastiera, il “cervello” interno (CPU), la memoria, il disco.
- **Software:** sono le istruzioni, i programmi. Non li puoi toccare, ma “dicono” all'hardware cosa fare. Windows, Excel, Chrome, WhatsApp sono tutti software.

Analogia: il corpo e la mente

Pensa al computer come a una persona:

- l'**hardware** è il **corpo**: braccia, gambe, occhi, cervello (come organo fisico). Serve per agire nel mondo fisico.
- il **software** è la **mente**: i pensieri, le conoscenze, le decisioni. È ciò che dice al corpo cosa fare e come farlo.

Un corpo senza una mente che lo guidi non farebbe nulla di utile: sta lì, fermo. Una mente senza un corpo non potrebbe agire nel mondo. Hardware e software funzionano allo stesso modo: uno senza l'altro non serve a niente.

Diagramma 1

Diagramma 1

Nelle prossime sezioni vediamo nel dettaglio i pezzi principali dell'hardware (CPU, RAM, disco) e poi il software che li coordina (il sistema operativo).

Esempio pratico

Il laptop di lavoro di Marco è hardware (schermo, tastiera, CPU, RAM). Su di esso girano software come il sistema operativo, il browser e il client per collegarsi in VPN al progetto. Allo stesso modo, un server che ospita **ShopFacile**, la piattaforma e-commerce su cui lavora il team, è hardware (fisico o virtuale, in cloud) su cui girano software come il sistema operativo Linux, il motore container e l'applicazione stessa.

2.2 La CPU: il cervello che calcola

Abbiamo detto che l'hardware è il "corpo" del computer: cominciamo a sezionarlo, partendo dal componente che fa più "pensare" tra tutti, la CPU.

La **CPU** (Central Processing Unit, "unità centrale di elaborazione") è il componente che esegue le istruzioni. È lei che fa i calcoli, prende le decisioni logiche ("se questo è vero, fai questo, altrimenti fai quello") e coordina il lavoro di tutti gli altri componenti.

Analogia: la persona che fa i calcoli

Immagina un ufficio dove arrivano richieste continue: "calcola questo", "confronta questi due numeri", "copia questo dato qui". La CPU è la persona seduta alla scrivania che esegue materialmente queste operazioni, una dopo l'altra, a una velocità incredibile: miliardi di operazioni al secondo.

Alcuni concetti utili da conoscere (non servono i dettagli tecnici, solo il significato generale, perché li sentirai nominare):

- **Core (nucleo)**: una CPU moderna ha più “core”, cioè più unità di calcolo indipendenti nello stesso chip. È come avere più persone alla scrivania invece di una sola: possono lavorare su cose diverse in parallelo.
- **Frequenza (GHz)**: quante operazioni al secondo può fare, in miliardi di cicli. Più alta è la frequenza, più velocemente lavora (semplificando molto).

Perché ti interessa? Perché quando in futuro sentirai parlare di “server con 8 core” o “la CPU è sotto stress”, saprai che si parla della capacità di calcolo della macchina.

Esempio pratico

Durante un picco di traffico sulla piattaforma, il team nota su una dashboard di monitoring che la CPU di un server è al 95% di utilizzo per diversi minuti. Questo è un segnale che la macchina fa fatica a gestire tutte le richieste in arrivo, e il team decide di aggiungere un altro container (ne parleremo più avanti) per distribuire il carico su più “persone alla scrivania”.

2.3 La RAM: la memoria di lavoro temporanea

La CPU, però, non calcola nel vuoto: ha bisogno di un posto dove tenere a portata di mano i dati su cui sta lavorando in questo preciso istante. Quel posto è la RAM.

La **RAM** (Random Access Memory) è la memoria che il computer usa mentre sta lavorando. È velocissima, ma ha due caratteristiche importanti:

1. è **temporanea**: quando spegni il computer, tutto quello che c'era in RAM viene perso;
2. è **limitata**: ha uno spazio finito (es. 8 GB, 16 GB, 32 GB).

Analogia: la scrivania

Pensa alla RAM come alla tua **scrivania** mentre lavori. Ci metti sopra i fogli, i libri, gli oggetti che ti servono in questo momento, perché prenderli dallo scaffale ogni volta sarebbe troppo lento. La scrivania è comoda e veloce da usare, ma ha spazio limitato: se hai troppi documenti, devi rimetterne alcuni nell'armadio per farne spazio ad altri. E quando vai via alla fine della giornata (spegni il PC), la scrivania viene “sgomberata”.

Quando apri un programma (es. il browser), il computer copia le istruzioni e i dati necessari dal disco (l'armadio, vedi sotto) alla RAM (la scrivania), perché lavorarci da lì è molto più rapido.

Se hai “troppe cose aperte” e il computer diventa lento, spesso è perché la RAM è piena: non c'è più spazio sulla scrivania.

Esempio pratico

Un'applicazione del progetto va in errore con un messaggio del tipo "out of memory" (memoria esaurita) perché il container in cui gira ha un limite di RAM troppo basso per il carico di lavoro reale. Il team operativo interviene aumentando il limite di memoria assegnato al container nella configurazione, e il problema si risolve.

2.4 Il disco: la memoria permanente

Abbiamo visto che la RAM si svuota allo spegnimento: ma allora dove restano salvati i dati di ShopFacile (ordini, prodotti, clienti) anche quando i server vengono riavviati? Serve una memoria che non "dimentichi" nulla.

Il **disco** (hard disk o, oggi più spesso, SSD - Solid State Drive) è la memoria dove i dati restano salvati **anche quando spegni il computer**. È più lento della RAM, ma ha molto più spazio ed è permanente.

Analogia: l'armadio

Il disco è come l'**armadio** di casa tua: ci conservi tutto quello che ti serve nel lungo periodo (documenti, vestiti, ricordi). Non è comodo come la scrivania per lavorare (aprire l'armadio, cercare la cosa giusta, richiuderlo richiede più tempo), ma è lì che le cose restano anche quando esci di casa o vai a dormire.

	RAM (scrivania)	Disco (armadio)
Velocità	Molto veloce	Più lenta (SSD comunque abbastanza rapido)
Persistenza	Si perde tutto allo spegnimento	Resta salvato
Spazio tipico	Pochi GB (es. 16 GB)	Centinaia di GB / TB
Uso	Lavoro "in corso"	Archiviazione permanente

Questa distinzione RAM/disco tornerà utile più avanti nel corso, ad esempio quando parleremo di database (dati salvati su disco) o di cache (dati tenuti in RAM per velocità).

Esempio pratico

Il team riceve un alert automatico perché il disco del server che ospita il database è occupato all'85%. Se non si interviene (ad esempio archiviando o cancellando log vecchi, oppure aumentando lo spazio disponibile), il database rischia di non poter più scrivere nuovi dati quando il disco si riempie completamente.

CPU, RAM e Disco insieme

Diagramma 2

Diagramma 2

La CPU lavora sui dati che trova in RAM (velocissima ma volatile); la RAM, a sua volta, recupera dal disco (lento ma permanente) ciò che serve quando serve. Il Sistema Operativo, che vediamo ora, è il “direttore” che organizza tutto questo traffico.

2.5 Il Sistema Operativo: chi coordina tutto

Il **Sistema Operativo** (Operating System, OS) è il software che gestisce tutte le risorse hardware del computer (CPU, RAM, disco, schermo...) e permette agli altri programmi (Excel, Chrome, ecc.) di funzionare senza dover “parlare” direttamente con l’hardware.

I sistemi operativi più diffusi sono:

- **Windows** (Microsoft) — il più usato sui PC aziendali e privati;
- **macOS** (Apple) — sui computer Apple;
- **Linux** — molto usato sui **server** (i computer che erogano servizi via internet) e nel mondo DevOps che vedrai più avanti nel corso.

Analogia: il direttore d'albergo

Immagina un albergo con tanti ospiti (i programmi) che hanno bisogno di risorse condivise: le camere (memoria), il personale (CPU), la cucina (disco). Il direttore dell'albergo (il sistema operativo) decide chi occupa cosa, per quanto tempo, evita che due ospiti litighino per la stessa risorsa, e fa in modo che tutto funzioni in armonia, senza che gli ospiti debbano parlarsi direttamente tra loro per organizzarsi.

Il sistema operativo si occupa, tra le altre cose, di:

- decidere quale programma può usare la CPU in un dato istante;
- gestire quale programma può usare quanta RAM;
- organizzare i file sul disco (vedi il “File System” più sotto);
- gestire la comunicazione con periferiche (stampanti, rete, mouse...);
- fornire un'interfaccia con cui l'utente interagisce (le finestre, le icone, o — su Linux — anche solo una riga di comando testuale, il “terminale”).

Nel mondo di cui ti occuperai come Project Manager su una piattaforma DevOps, sentirai parlare spesso di server **Linux**, perché è il sistema operativo più usato per far funzionare applicazioni web e infrastrutture.

Esempio pratico

Il team pianifica una finestra di manutenzione notturna per applicare un aggiornamento di sicurezza al sistema operativo Linux dei server di produzione, perché l'aggiornamento richiede il riavvio delle macchine: si sceglie un orario a basso traffico per minimizzare l'impatto sugli utenti.

2.6 Processi e Thread

Abbiamo detto che il sistema operativo decide chi usa la CPU e quanta RAM assegnare: ma a “chi”, concretamente? La risposta è: ai processi e ai thread, le unità di lavoro che il sistema operativo gestisce ogni istante.

Quando apri un programma (ad esempio il browser), il sistema operativo crea un **processo**: un'istanza del programma in esecuzione, con la sua porzione di memoria RAM dedicata, isolata dagli altri programmi.

Un processo può, a sua volta, dividersi in più **thread**: filoni di esecuzione più piccoli che lavorano sullo stesso processo, condividendone la memoria, ma potendo procedere in parti diverse del lavoro (anche in parallelo, se la CPU ha più core).

Analogia: un'azienda (processo) e i suoi dipendenti (thread)

Pensa a un **processo** come a un **reparto aziendale**: ha un proprio budget, un proprio spazio (l'ufficio), delle proprie risorse, separate da quelle degli altri reparti. Se un reparto va in crisi, in genere non manda in crisi automaticamente gli altri reparti (isolamento).

I **thread** sono i **dipendenti di quel reparto**: lavorano nello stesso spazio, condividono le stesse risorse (scrivanie, documenti, strumenti), e possono lavorare contemporaneamente su compiti diversi dello stesso progetto per essere più veloci. Se un dipendente combina un disastro con un documento condiviso, però, può creare problemi anche ai colleghi dello stesso reparto, perché condividono lo stesso spazio.

Esempio concreto: quando apri il browser (processo) e navighi su più schede, ogni scheda può essere gestita con thread (o addirittura processi) diversi, in modo che se una pagina web “va in crash” non blocchi necessariamente tutte le altre schede aperte.

Concetto	Cos'è	Analogia
Processo	Programma in esecuzione, con memoria propria isolata	Un reparto aziendale con il proprio ufficio
Thread	Filone di lavoro dentro un processo, condivide memoria con gli altri thread dello stesso processo	I dipendenti dello stesso reparto

Questo concetto tornerà utile quando, più avanti, parlerai di applicazioni che devono gestire **tante richieste contemporaneamente** (es. un sito web con migliaia di visitatori): il modo in cui un software gestisce processi e thread incide molto sulle sue performance.

2.7 Il File System: come sono organizzati i file

Processi e thread lavorano sui dati in RAM, ma quei dati — codice, immagini, configurazioni — devono comunque “vivere” da qualche parte sul disco quando non sono in uso: è qui che entra in gioco il file system.

Il **file system** è il modo in cui il sistema operativo organizza e memorizza i file sul disco. Praticamente ogni informazione che il computer salva in modo permanente (documenti, foto, programmi, configurazioni) è un **file**, e i file sono organizzati in **cartelle** (chiamate anche “directory”), che possono contenere altre cartelle, in una struttura ad albero.

Analogia: l'archivio con cartelline

Pensa a un grande archivio fisico con tanti raccoglitori (cartelle), che possono contenere altri raccoglitori più piccoli, che a loro volta contengono i documenti (file). Per trovare un documento specifico, segui un percorso: “vai nell’armadio Contabilità, poi nel raccoglitore 2024, poi nella cartellina Gennaio, prendi il foglio Fattura_012”.

In informatica, questo percorso si chiama **path** (percorso), e si scrive così:

```
/contabilità/2024/gennaio/fattura_012.pdf
```

oppure, su Windows:

```
C:\Contabilità\2024\Gennaio\fattura_012.pdf
```

Ogni file ha tipicamente un **nome** e un’**estensione** che indica il suo tipo: **.pdf**, **.docx**, **.jpg**, **.txt**, **.py** (codice Python), **.json** (vedi più avanti), ecc.

Diagramma 3

Diagramma 3

Capire i path ti servirà moltissimo più avanti, ad esempio quando vedremo **Git** (sezione 4): i progetti software sono organizzati esattamente come cartelle e file su un file system.

Esempio pratico

Marco ti scrive in chat: “il file di configurazione di ShopFacile è in **/app/config/produzione.json**”. Grazie a quello che hai appena imparato, sai leggere quel percorso: parti dalla radice (**/**), entri nella cartella **app**, poi in **config**, e lì trovi il file **produzione.json**. Non serve saperlo modificare, ma saperlo “leggere” ti permette di seguire la conversazione senza sentirti perso.

2.8 La rete: cos'è una rete di computer

Finora abbiamo parlato di un singolo computer — CPU, RAM, disco, sistema operativo, file. Ma un server con ShopFacile installato sopra, da solo e isolato, non servirebbe a nessuno: deve poter essere raggiunto dai clienti che navigano da casa loro. Il valore vero della tecnologia moderna nasce quando i computer **si parlano tra loro**: questo è possibile grazie alle **reti**.

Una **rete di computer** è semplicemente un insieme di dispositivi collegati tra loro, capaci di scambiarsi informazioni. Può essere piccola (la rete Wi-Fi di casa tua, con il tuo PC, telefono, smart TV) o enorme: **internet** è la rete di reti più grande del mondo, che collega miliardi di dispositivi.

Analogia: il sistema postale

Pensa a una rete come al **sistema postale**: ogni casa ha un indirizzo univoco, le lettere vengono instradate attraverso uffici postali intermedi fino a raggiungere il destinatario giusto, indipendentemente da dove si trovi. Anche i computer hanno un “indirizzo” (si chiama **indirizzo IP**) e le informazioni viaggiano in “pacchetti” (come lettere) attraverso una serie di dispositivi intermedi (router) finché non arrivano a destinazione.

Esempio pratico

I clienti segnalano che ShopFacile “non si carica più”. Marco, che si occupa spesso di infrastruttura, scopre che il problema non è nell'applicazione stessa, ma in un guasto di rete tra due data center che impedisce ai server di raggiungersi: un classico problema “di rete”, da distinguere da un bug nel codice.

2.9 TCP/IP: come i computer si “spediscono lettere”

Abbiamo detto che una rete permette a dispositivi diversi di scambiarsi informazioni: ma con quali regole precise avviene questo scambio? Le definisce un protocollo di base che sta sotto (quasi) tutto il resto, TCP/IP.

TCP/IP è l'insieme di regole (un “protocollo”, cioè un linguaggio comune condiviso) che permette a due computer di scambiarsi dati su una rete, compreso internet. È, in un certo senso, l'alfabeto di base su cui si basa tutta la comunicazione online.

Semplificando molto:

- **IP** (Internet Protocol) si occupa dell'**indirizzamento**: assegna a ogni dispositivo un indirizzo univoco (l'**indirizzo IP**, es. **192.168.1.10**) e si occupa di far viaggiare i dati verso l'indirizzo giusto, passando per i router intermedi — proprio come il sistema postale smista le lettere in base all'indirizzo.

- **TCP** (Transmission Control Protocol) si occupa di **spezzare i dati in pacchetti**, spedirli, e assicurarsi che arrivino tutti, nell'ordine giusto, e — se qualcosa si perde per strada — di rispeditirlo. È come spedire un libro intero per posta spezzandolo in tante lettere numerate: il destinatario le riceve, le rimette in ordine con i numeri, e se manca la lettera 5 chiede di rispeditirla.

Analogia riassuntiva

Immagina di dover spedire un grosso pacco a un amico in un'altra città:

1. lo dividi in scatole più piccole numerate (TCP: divisione in pacchetti e controllo dell'ordine/integrità);
2. scrivi l'indirizzo del destinatario su ogni scatola (IP: indirizzamento);
3. le scatole passano per vari centri di smistamento (i router) prima di arrivare a destinazione;
4. il destinatario le riceve, controlla che siano tutte arrivate e le ricompone nell'ordine giusto.

Non hai bisogno di sapere i dettagli tecnici di TCP/IP per il tuo lavoro da PM, ma è utile sapere che **è il livello base** su cui si costruiscono protocolli più “di alto livello” che userai spesso a parole, come HTTP.

Esempio pratico

In un ticket di supporto si legge: “il server non risponde sulla porta 443”. Il collega di infrastruttura controlla che quella porta TCP (usata per HTTPS) sia effettivamente aperta sul firewall del server: se è chiusa o bloccata, nessuna richiesta può arrivare a destinazione, anche se il server è acceso e funzionante.

2.10 HTTP e HTTPS: il protocollo del web

TCP/IP garantisce che i dati arrivino a destinazione, ma non dice nulla su **come** un browser deve chiedere una pagina web a un server: per quello serve un protocollo di livello più alto, costruito sopra TCP/IP, che è HTTP.

HTTP (HyperText Transfer Protocol) è il protocollo usato per trasferire pagine web e dati tra un **client** (es. il tuo browser) e un **server** (il computer che “ospita” il sito). È costruito sopra TCP/IP: se TCP/IP è “come spedire un pacco”, HTTP è “il modulo che compili per chiedere qualcosa e come è strutturata la risposta”.

Il funzionamento di base è semplice: **richiesta e risposta**.

1. Il client manda una **richiesta** (“dammi la pagina della home”);
2. il server elabora la richiesta e manda una **risposta** (il contenuto della pagina, oppure un errore come il famoso “404 - pagina non trovata”).

Diagramma 4

Diagramma 4

La “S” di sicurezza: HTTPS

HTTPS è la versione **sicura** di HTTP: la “S” sta per “Secure”. La differenza è che i dati scambiati vengono **cifrati** (criptati), cioè trasformati in un codice illeggibile per chiunque li intercetti lungo il tragitto, e solo il client e il server legittimi possono “decifrarli” e leggerli.

Analogia: cartolina vs busta sigillata

- **HTTP** è come spedire una **cartolina postale**: chiunque la maneggi lungo il percorso (postini, magazzinieri) può leggerne il contenuto.
- **HTTPS** è come spedire una **lettera in una busta sigillata e cifrata**, che solo il destinatario ha la “chiave” per aprire e leggere.

Per questo, oggi, praticamente tutti i siti seri usano HTTPS: quando navighi e vedi il lucchetto nella barra degli indirizzi del browser, significa che la connessione è protetta con HTTPS.

Esempio pratico

Durante un test di sicurezza su ShopFacile, viene segnalato che una vecchia pagina interna è ancora raggiungibile in HTTP (senza cifratura). Giulia, sempre attenta alla qualità, la corregge configurando un redirect automatico verso la versione HTTPS, così chi provasse ad accedere alla versione non sicura viene reindirizzato automaticamente a quella protetta.

2.11 DNS: la rubrica telefonica di internet

Nella sezione precedente abbiamo visto che HTTP fa viaggiare richiesta e risposta tra client e server, ma un server, come ogni computer su internet, è identificato da un **indirizzo IP** (es. **142.250.180.4**). Ma un cliente, per visitare il sito, digita nel browser un nome facile da ricordare come **www.shopfacile.it**, non una sequenza di numeri. Chi fa la traduzione da nome a indirizzo IP?

Il **DNS** (Domain Name System)!

Analogia: la rubrica telefonica

Il DNS funziona come una **rubrica telefonica**: tu vuoi chiamare “Luca”, non ricordi il suo numero di telefono a memoria, quindi consulti la rubrica, trovi il nome “Luca” e la rubrica ti restituisce il numero corretto da chiamare. Il DNS fa esattamente questo per internet: un cliente chiede **www.shopfacile.it**, il DNS gli risponde con l'indirizzo IP del server giusto, e solo dopo il suo browser può effettivamente contattarlo.

Diagramma 5

Diagramma 5

Senza il DNS dovremmo ricordare a memoria stringhe di numeri per ogni sito che vogliamo visitare: praticamente impossibile.

Esempio pratico

Dopo aver spostato ShopFacile su un nuovo server, il team aggiorna il record DNS del progetto perché punti al nuovo indirizzo IP. Per alcune ore, però, alcuni clienti continuano a raggiungere il vecchio server: è un fenomeno normale, chiamato “propagazione DNS”, perché la modifica impiega un po' di tempo a diffondersi su tutti i server DNS del mondo.

2.12 API: come i software si parlano tra loro

Finora abbiamo visto come un client raggiunge un server (DNS, HTTP). Ma una volta raggiunto, come chiede esattamente “dammi la lista prodotti” o “registra questo ordine” in un linguaggio che il server capisca? Con le API.

API sta per **Application Programming Interface** (“interfaccia di programmazione delle applicazioni”). È il modo in cui un software espone delle “funzionalità” che altri software possono usare, senza dover conoscere i dettagli interni di come funziona.

Analogia: il cameriere al ristorante

Pensa a un ristorante: tu (il **cliente/client**) non entri in cucina a prepararti da mangiare da solo. Ordini un piatto al **cameriere**, che porta la tua richiesta in **cucina** (il **server**), aspetta che il piatto sia pronto, e te lo riporta al tavolo.

Il cameriere è l'**API**: è l'intermediario con **regole precise** (il menù: puoi ordinare solo quello che è scritto lì, in un formato preciso) che permette a te (client) di ottenere qualcosa dalla cucina (server) senza sapere come funzionano i fornelli o le pentole.

In termini informatici:

- un'app di meteo sul tuo telefono non “misura” la temperatura da sola: fa una **richiesta API** a un servizio esterno che gestisce i dati meteorologici, e riceve una **risposta** con i dati richiesti;
- ShopFacile, al momento del pagamento, usa le API di un servizio di pagamento esterno per gestire il pagamento con carta, senza dover costruire da zero un intero sistema di pagamenti.

Diagramma 6

Diagramma 6

Le API sono uno dei concetti più importanti che incontrerai lavorando su una piattaforma software: praticamente tutte le applicazioni moderne sono fatte di tanti “pezzi” (servizi) che comunicano tra loro tramite API.

Esempio pratico

Sara chiede al team di aggiungere l'invio di notifiche via SMS ai clienti di ShopFacile quando un ordine viene spedito. Invece di costruire da zero un intero sistema per inviare SMS (server, connessioni con gli operatori telefonici, ecc.), Marco integra le API REST di un servizio esterno specializzato: basta inviare una richiesta con numero e testo del messaggio, e il servizio esterno si occupa di tutto il resto.

2.13 REST: uno stile comune per costruire API

Le API, come abbiamo appena visto, permettono a due software di parlarsi. Ma perché tutti riescano a capirsi senza reinventare ogni volta le regole del “dialogo”, serve un modo condiviso di costruirle: lo stile REST.

REST (REpresentational State Transfer) non è una tecnologia specifica, ma uno **stile**, un insieme di convenzioni condivise su come costruire le API in modo semplice, standard e prevedibile, sfruttando proprio HTTP che abbiamo visto sopra.

Le API costruite seguendo lo stile REST si chiamano **API REST** (o “RESTful”), e sono lo standard più diffuso oggi per far comunicare applicazioni web.

L'idea di base: si usano gli indirizzi web (URL) per identificare le **risorse** (es. “un cliente”, “un ordine”), e si usano dei **verbi HTTP** standard per indicare l'azione da compiere su quella risorsa:

Verbo HTTP	Significato	Analogia
GET	Leggi/recupera dati	“Fammi vedere l'ordine numero 42”
POST	Crea un nuovo elemento	“Crea un nuovo ordine”
PUT / PATCH	Modifica un elemento esistente	“Aggiorna l'indirizzo di spedizione dell'ordine 42”
DELETE	Elimina un elemento	“Cancella l'ordine 42”

Esempio pratico: l'API REST che ShopFacile usa per gestire gli ordini potrebbe avere un indirizzo come:

```
GET https://api.shopfacile.it/ordini/42
```

per “recuperare i dettagli dell'ordine con id 42”.

Non avrai bisogno di scrivere codice per costruire API REST, ma sentirai questo termine costantemente parlando con gli sviluppatori del team, quindi è importante che il concetto ti sia chiaro.

2.14 JSON e XML: i formati per scambiarsi dati

Abbiamo visto come un'API REST identifica una risorsa come "l'ordine 42" e come intervenireci (GET, POST...). Ma in che formato viaggiano concretamente i dati di quell'ordine dentro la richiesta e la risposta? Qui entrano in gioco JSON e XML.

Quando due software si scambiano dati (ad esempio tramite un'API), devono usare un **formato comune**, condiviso, che entrambi sappiano "leggere". I due formati più diffusi sono **JSON** e **XML**.

JSON

JSON (JavaScript Object Notation) è oggi il formato più usato per scambiare dati tra applicazioni web, perché è semplice da leggere sia per un umano che per un computer. Organizza i dati in coppie **chiave: valore**, proprio come una piccola scheda informativa.

Esempio: i dati di un ordine in formato JSON potrebbero essere così:

```
{
  "id_ordine": 42,
  "cliente": "Mario Rossi",
  "totale": 129.90,
  "spedito": false,
  "prodotti": [
    "Tastiera",
    "Mouse"
  ]
}
```

Si legge in modo abbastanza naturale: "l'ordine numero 42, del cliente Mario Rossi, con un totale di 129,90, non ancora spedito, contiene una tastiera e un mouse". Le parentesi graffe `{ }` delimitano un "oggetto" (una scheda di dati), le parentesi quadre `[ ]` delimitano una lista di valori.

XML

XML (eXtensible Markup Language) è un formato più "vecchio" ma ancora molto usato in certi contesti aziendali (soprattutto in sistemi legacy o in alcuni settori come banking e pubblica amministrazione). Usa dei **tag** che racchiudono i dati, un po' come l'HTML delle pagine web:

```
<ordine>
  <id>42</id>
  <cliente>Mario Rossi</cliente>
  <totale>129.90</totale>
  <spedito>>false</spedito>
</ordine>
```

Quale scegliere?

Nella maggior parte dei progetti moderni con cui avrai a che fare, il formato predefinito sarà **JSON**: è più compatto, più leggibile, e più facile da usare nella programmazione moderna. XML lo incontrerai più raramente, magari in integrazioni con sistemi più datati.

2.15 Database relazionali e SQL

Il JSON dell'ordine visto poco fa deve comunque essere salvato da qualche parte in modo permanente e interrogabile — non basta scambiarlo in una risposta API, va anche conservato. È qui che entrano in gioco i database.

Un **database** (base di dati) è un sistema organizzato per **salvare, organizzare e recuperare grandi quantità di dati** in modo efficiente e strutturato — molto più potente e affidabile di un semplice file Excel per gestire dati che crescono in volume e complessità.

I **database relazionali** organizzano i dati in **tabelle**, esattamente come un foglio Excel: righe e colonne.

Analogia: il foglio Excel evoluto

Pensa a un database relazionale come a un insieme di fogli Excel collegati tra loro:

- ogni **tabella** è un foglio (es. “Clienti”, “Ordini”, “Prodotti”);
- ogni **riga** è un singolo elemento (es. un cliente specifico);
- ogni **colonna** è un attributo di quell'elemento (es. nome, cognome, email).

Le tabelle possono essere **collegate** tra loro: ad esempio, la tabella “Ordini” può avere una colonna che indica “a quale cliente appartiene questo ordine” — questa è la “relazione” da cui prende il nome “database relazionale”.

Esempio, tabella **Clienti** del database di ShopFacile:

id	nome	email
1	Mario Rossi	mario.rossi@email.com
2	Anna Bianchi	anna.bianchi@email.com

Esempio, tabella **Ordini**:

id	id_cliente	totale
101	1	129.90
102	2	59.00

SQL: il linguaggio per “interrogare” il database

SQL (Structured Query Language) è il linguaggio standard usato per “interrogare” (query) un database relazionale: chiedere dati, inserirne di nuovi, modificarli o eliminarli.

Esempio semplicissimo: per chiedere “dammi il nome e l'email di tutti i clienti”, in SQL scriveresti:

```
SELECT nome, email FROM Clienti;
```

Per chiedere “dammi solo gli ordini con un totale superiore a 100 euro”:

```
SELECT * FROM Ordini WHERE totale > 100;
```

Non ti verrà chiesto di scrivere SQL nel tuo ruolo di PM, ma capire questa logica di base ti aiuterà a seguire discussioni tecniche sul team riguardo “performance del database”, “query lente”, “migrazione dati”, ecc.

Esempi di database relazionali diffusi: **Microsoft SQL Server, PostgreSQL, MySQL, Oracle Database**.

2.16 Database NoSQL

Le tabelle rigide di SQL funzionano benissimo per dati regolari come clienti e ordini. Ma cosa succede quando i dati da salvare hanno una struttura molto variabile, come il catalogo prodotti di ShopFacile? Qui SQL comincia a scricchiolare, ed entrano in gioco i database NoSQL.

I database **NoSQL** (“Not Only SQL”) sono un’alternativa ai database relazionali, pensati per casi in cui la rigida struttura a tabelle non è la scelta migliore.

Perché esistono

Immagina di dover salvare dati molto **variabili** nella struttura (ad esempio: i prodotti nel catalogo di ShopFacile, dove ogni categoria ha attributi diversi — un libro ha “autore” e “numero pagine”, uno smartphone ha “memoria” e “colore”). Costringere questi dati in tabelle rigide e uniformi sarebbe scomodo. I database NoSQL permettono maggiore **flessibilità**.

Analogia: cartelle e documenti vs archivio rigido

Se il database relazionale è come un archivio con moduli standard identici per tutti (ogni modulo ha esattamente gli stessi campi da riempire), un database NoSQL è più come una serie di **cartelline con documenti liberi**: ogni documento può avere una struttura leggermente diversa, senza dover rispettare uno schema fisso e identico per tutti.

Molti database NoSQL salvano i dati proprio in formato **JSON** (quello che abbiamo visto sopra):

```
{
  "prodotto": "Smartphone X",
  "colore": "nero",
  "memoria_gb": 128
}
```

```
{
  "prodotto": "Libro Y",
  "autore": "M. Verdi",
  "pagine": 320
}
```

Due prodotti, campi diversi, nello stesso database: con un database relazionale classico sarebbe più complicato da gestire in modo pulito.

Quando si usano

- **Relazionali (SQL)**: dati molto strutturati, con relazioni chiare tra entità (es. sistemi gestionali, contabilità, ordini). Garantiscono forte coerenza dei dati.
- **NoSQL**: dati con struttura variabile o che cambia spesso, grandi volumi con necessità di scalare velocemente, casi come cataloghi prodotto, log, dati di sessione, messaggistica.

Esempi di database NoSQL diffusi: **MongoDB**, **Cosmos DB** (Azure), **Redis**, **Cassandra**.

Non esiste “il migliore in assoluto”: la scelta dipende dal tipo di dati e dal problema da risolvere. Sentirai il team discutere di questa scelta nelle fasi di progettazione di un nuovo servizio.

2.17 Virtual Machine: un computer dentro un computer

Fin qui abbiamo parlato di dati (SQL, NoSQL): ora torniamo all’infrastruttura che fa girare ShopFacile. Un modo classico per ricavare più “computer” indipendenti da un solo server fisico è la macchina virtuale.

Una **Virtual Machine** (VM, macchina virtuale) è un software che **simula un computer completo** dentro un altro computer fisico. Dal punto di vista di chi la usa, si comporta esattamente come un computer vero e proprio: ha il suo sistema operativo, le sue applicazioni, il suo spazio disco — ma “vive” dentro un computer fisico più grande (l'**host**), condividendo con altre eventuali VM le risorse hardware reali.

Il software che crea e gestisce le macchine virtuali si chiama **hypervisor**.

Analogia: un condominio con appartamenti indipendenti

Pensa a un edificio (il computer fisico, “host”) diviso in **appartamenti** indipendenti (le VM). Ogni appartamento ha la sua porta blindata, i suoi mobili, la sua cucina — è completamente **isolato** dagli altri, anche se condividono le stesse fondamenta, gli stessi impianti elettrici e idraulici dell’edificio. Se un appartamento va a fuoco, in teoria gli altri restano al sicuro (isolamento).

Perché sono utili? Perché su un solo computer fisico potente puoi far “vivere” tante macchine virtuali diverse, ciascuna con il proprio sistema operativo e le proprie applicazioni, risparmiando sui costi hardware e semplificando la gestione (puoi creare, cancellare, spostare una VM molto più facilmente di un computer fisico).

Diagramma 7

Diagramma 7

Esempio pratico

Marco deve testare un aggiornamento importante prima di applicarlo ai server di produzione di ShopFacile. Crea una nuova VM di test partendo da un’immagine standard identica a quella di produzione, prova l’aggiornamento lì sopra senza alcun rischio per i clienti reali, e solo dopo aver verificato che tutto funziona procede con l’aggiornamento vero e proprio.

2.18 Container: più leggeri di una VM

Le VM funzionano, ma portare con sé un intero sistema operativo per ogni appartamento del “condominio” ha un costo in termini di peso e velocità. Esiste un’alternativa più leggera, molto usata oggi anche per ShopFacile: i container.

Un **container** è un altro modo per “isolare” un’applicazione e le sue dipendenze, ma in modo molto più **leggero** rispetto a una VM: invece di simulare un computer intero con il suo sistema operativo, un container condivide il sistema operativo della macchina che lo ospita, isolando solo l’applicazione e ciò che le serve per funzionare (librerie, configurazioni, file necessari).

Analogia: la valigia pronta all’uso

Se la VM è un intero appartamento indipendente, il **container** è più come una **valigia già pronta con tutto il necessario**: vestiti, spazzolino, caricabatterie — tutto quello che serve per il viaggio, organizzato e pronto. Puoi portare questa valigia in qualsiasi “casa” (computer/server) e sarai subito operativo, perché hai già con te tutto ciò che ti serve, senza dover allestire un’intera casa da zero (come faresti con una VM).

Questo risolve un problema molto concreto e comune nello sviluppo software: *“sul mio computer funzionava, perché sul server non funziona?”*. Il container “porta con sé” esattamente le stesse versioni di librerie, configurazioni e dipendenze usate in fase di sviluppo, garantendo che l'applicazione si comporti allo stesso modo ovunque venga eseguita.

VM vs Container: il confronto visivo

Diagramma 8

Diagramma 8

Diagramma 9

Diagramma 9

La differenza chiave: ogni VM porta con sé un **intero sistema operativo** (pesante, lento da avviare), mentre i container **condividono** il sistema operativo della macchina host e sono quindi molto più leggeri, veloci da avviare (secondi, invece di minuti) e più efficienti in termini di risorse.

	Virtual Machine	Container
Cosa isola	Un intero computer virtuale, con proprio OS	Solo l'applicazione e le sue dipendenze
Peso	Pesante (GB), OS completo	Leggero (MB), condivide l'OS host
Tempo di avvio	Minuti	Secondi
Isolamento	Molto forte (OS separato)	Buono, ma leggermente meno forte della VM

Esempio pratico

Un cliente segnala un bug su ShopFacile in produzione. Ahmed, invece di provare a indovinare cosa non funzioni, lancia in locale sul proprio computer lo stesso identico container usato in produzione (stesse librerie, stessa versione del linguaggio, stessa configurazione) e riesce a riprodurre il problema in pochi minuti, con la certezza di lavorare in un ambiente identico a quello reale.

2.19 Docker: lo strumento più diffuso per i container

Abbiamo appena descritto cosa sono i container, ma “chi” li crea e li fa girare concretamente? Nella grande maggioranza dei casi, incluso il progetto ShopFacile, lo strumento usato è Docker.

Docker è oggi lo strumento più popolare e diffuso per creare, distribuire ed eseguire container. È diventato talmente centrale nel mondo dello sviluppo software moderno che spesso “container” e “Docker” vengono usati quasi come sinonimi (anche se Docker è solo uno degli strumenti possibili).

Concetti chiave da conoscere (senza bisogno di saperli usare tecnicamente):

- **Immagine (image)**: è il “modello” o la “ricetta” del container: contiene l'applicazione e tutto ciò che le serve per funzionare (un po' come la lista precisa del contenuto della valigia, prima ancora di farla). È come una fotografia di uno stato pronto all'uso.
- **Container**: è un'immagine “in esecuzione”, cioè la valigia effettivamente in uso, attiva, con l'applicazione che sta girando.
- **Dockerfile**: è un file di testo con le istruzioni su come costruire un'immagine (es. “parti da questo sistema base, installa queste librerie, copia questi file, avvia questo comando”).
- **Registry** (es. Docker Hub): è un “magazzino” online dove si salvano e si condividono le immagini Docker, pronte per essere scaricate e usate.

Analogia riassuntiva

- il **Dockerfile** è la ricetta scritta;
- l'**immagine** è la valigia già fatta secondo quella ricetta, pronta ma non ancora “in viaggio”;
- il **container** è la valigia che stai effettivamente usando durante il viaggio (in esecuzione).

Nel team con cui lavorerai, sentirai spesso frasi come “buildiamo l'immagine”, “il container è andato in crash”, “pushiamo l'immagine sul registry”: ora sai cosa significano a grandi linee.

Esempio pratico

Marco scrive un Dockerfile per ShopFacile che parte da un'immagine base con il linguaggio di programmazione già installato, copia il codice dell'applicazione e installa le librerie necessarie. La pipeline di CI/CD (ne parleremo più avanti nel corso) usa questo Dockerfile per costruire automaticamente una nuova immagine ogni volta che il codice cambia, e la pubblica su un registry privato del progetto, pronta per essere distribuita.

2.20 Kubernetes: l'orchestratore di container

Docker ti permette di creare e avviare un singolo container, ma ShopFacile in produzione non ne ha uno solo: ne ha decine, distribuiti su più macchine. Gestirli a mano diventa presto impossibile, e qui entra in gioco Kubernetes.

Quando un'applicazione è composta da tanti container (magari decine o centinaia, distribuiti su tante macchine diverse per gestire tanti utenti contemporaneamente), diventa molto complicato gestirli a mano: avviarli, fermarli, sostituirli se si “rompono”, distribuirli sulle macchine giuste, farli comunicare tra loro...

Kubernetes (spesso abbreviato **K8s**) è lo strumento più diffuso per **orchestrare** (coordinare automaticamente) grandi quantità di container.

Analogia: il direttore d'orchestra

Pensa a un'orchestra con decine di musicisti (i container). Ogni musicista suona il suo strumento (esegue la sua applicazione), ma senza un **direttore** che coordini tempi, entrate e uscite di ciascuno, il risultato sarebbe caos. Il **direttore d'orchestra** (Kubernetes) decide:

- quando far “entrare” un nuovo musicista se ne serve uno in più (es. più utenti stanno usando l'app: servono più container per gestire il traffico);
- cosa fare se un musicista si sente male e deve uscire (un container si blocca: Kubernetes lo rileva e ne avvia automaticamente uno nuovo al suo posto, senza intervento umano);
- come distribuire i musicisti sul palco (su quali macchine far girare quali container, in base alle risorse disponibili).

Kubernetes gestisce automaticamente compiti come:

- **Scalabilità**: aumentare o diminuire il numero di container attivi in base al traffico/carico di lavoro;
- **Self-healing** (auto-guarigione): se un container si blocca o si rompe, Kubernetes lo rileva e lo riavvia o lo sostituisce automaticamente;
- **Distribuzione del carico**: smistare le richieste in arrivo tra i vari container disponibili, in modo che nessuno sia sovraccaricato mentre altri sono inattivi;
- **Deployment controllato**: aggiornare l'applicazione a una nuova versione gradualmente, riducendo il rischio di interruzioni per gli utenti.

Diagramma 10

Diagramma 10

Non dovrai mai configurare Kubernetes con le tue mani come Project Manager, ma sentirai questo nome citato molto spesso parlando dell'infrastruttura del progetto e dei tempi di rilascio: sapere cosa fa a grandi linee ti aiuterà a capire meglio le conversazioni tecniche del team e a stimare meglio la complessità di certe attività.

Esempio pratico

Durante il Black Friday, con un picco di clienti su ShopFacile per una promozione con molto traffico, Kubernetes rileva l'aumento del carico e scala automaticamente da 3 a 10 container della stessa applicazione, per distribuire meglio le richieste. Passato il picco, riduce di nuovo il numero di container a 3, risparmiando risorse — tutto senza che nessuno del team debba intervenire manualmente nel cuore della notte.

2.21 Riepilogo: come si incastrano tutti questi pezzi

Facciamo un ultimo passaggio per vedere come i concetti di questa sezione si incastrano in uno scenario reale — ad esempio, quando un utente usa un sito web che il tuo team ha costruito:

Diagramma 11

Diagramma 11

In questo singolo scenario compaiono quasi tutti i concetti visti in questa sezione: DNS, HTTPS, container orchestrati (Kubernetes), database e SQL, formato dati JSON scambiato via API. Non è un caso: questi sono davvero i “mattoncini” fondamentali di praticamente ogni sistema software moderno, e li ritroverai continuamente nel resto del corso.

Checklist di autoverifica

Prima di passare alla sezione successiva, prova a rispondere (anche solo a voce, o scrivendo due righe) a queste domande. Se qualcuna ti mette in difficoltà, torna a rileggere il paragrafo corrispondente:

- Sapresti spiegare a un amico la differenza tra hardware e software?
- Sai spiegare la differenza tra RAM e disco con un'analogia tua?
- Sai dire a cosa serve un sistema operativo?
- Sai spiegare la differenza tra processo e thread?
- Sai spiegare, con parole tue, cosa fa il DNS?
- Sai spiegare perché HTTPS è più sicuro di HTTP?
- Sai spiegare cos'è un'API con l'analogia del ristorante?
- Sapresti leggere un piccolo blocco di dati in formato JSON?
- Sai spiegare la differenza tra database relazionale e NoSQL?
- Sai spiegare la differenza tra una VM e un container?
- Sai spiegare a cosa serve Kubernetes, a grandi linee?

Esercizi pratici

Gli esercizi che seguono ti aiutano a trasformare il vocabolario di questa sezione in comprensione reale. Non serve saper scrivere codice per farli: servono soprattutto occhi curiosi e la disponibilità a fare qualche domanda ai colleghi.

1. **Guarda “sotto il cofano” del tuo computer.** Apri il Task Manager (Windows) o il Monitoraggio Attività (macOS) e osserva per un paio di minuti quanta CPU e quanta RAM stanno usando le applicazioni aperte. Prova ad aprire molte schede del browser insieme e osserva come cambiano i numeri. **Come verificare:** sai indicare quale processo, in quel momento, sta consumando più CPU e quale più RAM, e sai spiegare la differenza tra le due cose usando le analogie di questa sezione (scrivania vs persona che calcola)?
2. **Traccia il percorso di una richiesta web.** Apri gli strumenti di sviluppo del browser (F12 o tasto destro → “Ispeziona”), vai sulla scheda “Network”/“Rete”, visita un sito qualsiasi e osserva la prima richiesta HTTP che parte: nota il metodo (**GET**), il codice di risposta (es. **200**) e se il protocollo usato è HTTP o HTTPS. **Come verificare:** sai indicare, guardando la schermata, se la connessione era protetta (HTTPS) e sai spiegare cosa significa il codice di risposta che hai visto?

3. **Interroga un DNS a mano.** Da riga di comando (terminale su Mac/Linux, Prompt dei comandi o PowerShell su Windows), esegui il comando `nslookup www.google.com` (o `ping www.google.com`) e osserva l'indirizzo IP che viene restituito. **Come verificare:** sai spiegare a parole tue cosa ha fatto il comando, collegandolo all'analogia della "rubrica telefonica" vista in questa sezione?
4. **Leggi e "traduci" un JSON reale.** Chiedi a un developer del team di mostrarti un piccolo esempio di risposta JSON restituita da un'API del progetto (o, in alternativa, apri in un browser un endpoint pubblico come `https://gitlab.com/api/v4/users?username=gitlab-org`). Prova a identificare le coppie chiave-valore principali. **Come verificare:** sapresti riscrivere a voce, in una frase in italiano, cosa dice quel JSON, come abbiamo fatto con l'esempio dell'ordine in questa sezione?
5. **Confronta VM e container con parole tue.** Senza guardare il testo, scrivi in 3-4 righe la differenza tra una macchina virtuale e un container, e almeno un motivo per cui un team potrebbe preferire il secondo per distribuire un'applicazione. **Come verificare:** fai leggere quello che hai scritto a un collega tecnico (o confrontalo con la tabella di questa sezione): la tua spiegazione coglie sia la differenza di "peso" che quella di isolamento?
6. **Disegna lo schema di una richiesta completa.** Su un foglio (anche a mano), disegna il percorso di una richiesta utente che visita una pagina della piattaforma: browser → DNS → server/container → database → e ritorno, etichettando ogni passaggio con il termine giusto (IP, HTTP/S, API, JSON, SQL...), ispirandoti al diagramma della sezione 2.21. **Come verificare:** riesci a spiegare il tuo disegno a un collega non tecnico in meno di 3 minuti, senza dover consultare gli appunti?

Collegamenti

- Prossima sezione: [3. Come nasce un software](#)
- [4. Git e GitLab](#) — vedrai come i concetti di file e cartelle si applicano al codice
- [9. DevOps](#) — dove Docker e Kubernetes tornano centrali
- [12. Architetture software](#) — dove API e REST vengono approfonditi
- [13. Cloud](#) — dove VM e container vengono usati concretamente su Azure/AWS
- [16. Glossario](#) — per ripassare rapidamente ogni termine visto qui

Risorse

- [MDN Web Docs – Come funziona internet](#) — spiegazione approfondita di rete, DNS, HTTP
- [MDN Web Docs – Panoramica su HTTP](#) — approfondimento sul protocollo HTTP/HTTPS
- [Microsoft Learn – Cos'è un container?](#) — confronto ufficiale container vs VM
- [Documentazione ufficiale Docker – Introduzione](#) — guida introduttiva ufficiale a Docker
- [Documentazione ufficiale Kubernetes – Concetti base](#) — introduzione ufficiale a Kubernetes (disponibile anche in italiano)

3. Come nasce un software

Nella sezione precedente hai imparato cosa sono hardware, software, reti e altri concetti di base dell'informatica. Ora facciamo un passo avanti: come nasce, concretamente, un software? Chi lo progetta? Chi lo scrive? Chi decide quando è pronto per essere usato dalle persone?

Capire questo processo è fondamentale per te, futuro Junior Project Manager, perché il tuo lavoro consisterà proprio nel coordinare le persone e le attività che compongono questo processo. Non dovrai scrivere codice, ma dovrai capire di cosa parlano gli sviluppatori, cosa significa “abbiamo trovato un bug in produzione” o “la merge request è ancora in review”, e come si inserisce ogni attività nel percorso complessivo di un progetto software.

Pensa a questa sezione come alla mappa di un territorio che esplorerai per tutta la tua carriera: oggi impari i nomi delle vie principali, poi con l'esperienza scoprirai i dettagli di ogni singolo quartiere.

Per non restare sul piano astratto, seguiremo come filo conduttore **ShopFacile**, la piattaforma e-commerce che hai già incontrato nella sezione precedente, con il suo piccolo team interno: Marco e Giulia (developer, lei molto attenta a test e qualità), Ahmed (developer junior, in crescita), Sara (Product Owner) e Luca (Scrum Master).

Obiettivi della sezione

Alla fine di questa sezione saprai:

- Descrivere le fasi del ciclo di vita di un software e cosa succede in ciascuna
- Distinguere un requisito funzionale da un requisito non funzionale
- Spiegare la differenza tra bug e feature
- Capire perché il software viene “numerato” con le versioni
- Comprendere, a livello concettuale, cosa sono branch, merge request, code review e merge

Il ciclo di vita del software

Immagina di dover costruire una casa. Non inizi comprando mattoni a caso: prima parli con la famiglia che ci vivrà per capire quante stanze servono (requisiti), poi un architetto disegna la piantina (analisi), poi gli operai costruiscono davvero la casa (sviluppo), poi un ingegnere controlla che tutto sia sicuro e a norma (testing), poi la famiglia ci si trasferisce (rilascio), e infine, anno dopo anno, la casa avrà bisogno di manutenzione: una caldaia da sostituire, un tetto da riparare (manutenzione).

Il software funziona esattamente allo stesso modo. Questo percorso si chiama **ciclo di vita del software** (in inglese *Software Development Life Cycle*, o **SDLC**), ed è composto da fasi che si ripetono in ogni progetto, grande o piccolo.

Diagramma 1

Diagramma 1

Nota la freccia tratteggiata che torna da “Manutenzione” a “Requisiti”: il ciclo di vita non è quasi mai un percorso lineare che finisce una volta per tutte. È più simile a una spirale: ogni volta che il software viene usato nel mondo reale, emergono nuove esigenze, si scoprono difetti, arrivano richieste di nuove funzionalità, e il ciclo riparte. Nei prossimi capitoli (in particolare nella sezione sull'Agile) vedrai che i team moderni ripetono questo ciclo molte volte, anche ogni due settimane, invece di farlo una sola volta in anni.

Vediamo ora ogni fase nel dettaglio.

1. Requisiti: cosa deve fare il software

È la fase in cui si stabilisce **cosa** deve fare il software, senza ancora preoccuparsi di **come** lo farà tecnicamente. È come quando parli con un architetto e gli dici “vogliamo tre camere, una cucina abitabile e un balcone”: stai descrivendo un bisogno, non stai disegnando la pianta.

In questa fase il project manager, il cliente e gli analisti si incontrano per capire cosa serve davvero. Il risultato è un documento (o una lista di elementi in uno strumento come Jira o Azure DevOps, che vedremo più avanti) che elenca i **requisiti**.

Esempio pratico: durante una riunione su ShopFacile, il cliente dice “vogliamo che i nostri utenti possano cercare i prodotti nel catalogo per nome o categoria”. Sara, la Product Owner, trascrive questa richiesta in una card del backlog, magari intitolata “Ricerca prodotti nel catalogo”, senza ancora sapere se si useranno filtri, una barra di ricerca testuale o entrambe le cose: quella scelta arriverà nella fase successiva.

Approfondiamo questo concetto nella prossima sezione perché è cruciale.

2. Analisi: dai requisiti alla soluzione

Una volta chiaro **cosa** serve, bisogna capire **come** realizzarlo. È il lavoro dell'architetto che trasforma “vogliamo tre camere” in una piantina con misure precise, posizione dei muri portanti, degli impianti elettrici e idraulici.

Nel software, questa fase si chiama **analisi** (o analisi tecnica/progettazione). Chi si occupa di analisi — spesso un architetto software o un tech lead — decide:

- Come sarà strutturato il software (quali “pezzi” lo compongono e come comunicano tra loro — ne parleremo nella sezione sulle architetture software)
- Quali tecnologie usare (linguaggi di programmazione, database, strumenti)
- Come i dati saranno organizzati e salvati
- Quali rischi tecnici ci sono e come affrontarli

Esempio pratico: il requisito è “l'utente deve poter recuperare la password se la dimentica”. Marco, che spesso si occupa di infrastruttura e di questo tipo di scelte tecniche, stabilisce come: si invierà un'email con un link temporaneo, quel link scadrà dopo 30 minuti, i dati della password saranno salvati in un certo modo per motivi di sicurezza. È il passaggio dall'idea al progetto tecnico.

Una volta che si sa “come” farlo, resta solo un passaggio: farlo davvero. Passiamo quindi alla fase di sviluppo.

3. Sviluppo: si scrive il codice

Questa è la fase che tutti immaginano quando pensano all'informatica: gli sviluppatori “scrivono codice”. Ma cosa significa davvero?

Scrivere codice significa dare istruzioni precise e dettagliate al computer, usando un linguaggio di programmazione (Python, Java, C#, JavaScript, e tanti altri). Il computer non capisce l'italiano o l'inglese naturale: capisce solo istruzioni scritte in un modo molto rigido e specifico, un po' come una ricetta di cucina scritta per una persona che esegue esattamente e solo quello che è scritto, senza improvvisare nulla.

Un'analogia utile: se dici a un amico “fammi un caffè”, lui capisce dal contesto cosa fare. Se dovessi scrivere le istruzioni per un robot che non ha mai visto un caffè, dovresti specificare ogni singolo passaggio: “apri lo sportello della macchina, prendi la capsula dal contenitore in alto a destra, inseriscila nell'apposito slot, chiudi lo sportello, premi il tasto con la scritta ESPRESSO, attendi che il led diventi verde...”. Scrivere codice è simile: bisogna essere estremamente precisi, perché il computer esegue letteralmente quello che gli viene scritto, senza intuire le intenzioni.

Lo sviluppatore prende i documenti di analisi e li traduce in queste istruzioni dettagliate, organizzate in file di testo (i “file di codice”) che insieme compongono il software. Il codice viene poi salvato e condiviso con il resto del team usando strumenti come Git (che vedremo nella prossima sezione).

Esempio pratico: l'analisi ha stabilito che serve una funzione che calcoli lo sconto finale su un carrello di ShopFacile. Marco scrive, in un linguaggio come Python o Java, una serie di istruzioni tipo “prendi il totale del carrello, verifica se esiste un codice sconto valido, se sì calcola la percentuale da sottrarre, altrimenti restituisci il totale invariato”. Ogni singolo caso (carrello vuoto, codice sconto scaduto, sconto superiore al totale) deve essere previsto esplicitamente: il computer non “capisce” cosa intendevi, esegue solo quello che hai scritto.

Il codice è scritto, ma “sembra funzionare” non basta: prima di fidarsene bisogna verificarlo sistematicamente. È qui che entra in gioco il testing.

4. Testing: si verifica che funzioni

Immaginiamo che gli operai abbiano finito di costruire la casa. Prima di far entrare la famiglia, un ingegnere collaudatore verifica che gli impianti funzionino, che le porte si aprano bene, che non ci siano crepe nei muri. Non basta che la casa “sembri” pronta: bisogna testarla.

Nel software questa fase si chiama **testing** (o test, o collaudo). Consiste nel verificare, in modo sistematico, che il software faccia esattamente quello che deve fare, senza comportamenti indesiderati.

Esistono diversi tipi di test, con diversi livelli di dettaglio:

- **Test unitari:** verificano un singolo “pezzettino” di codice isolato, ad esempio una funzione che calcola uno sconto. È come testare una singola presa elettrica prima di collegare l'intero impianto.
- **Test di integrazione:** verificano che più pezzi di codice funzionino bene insieme, ad esempio che il sistema di pagamento comunichi correttamente con il catalogo prodotti. È come verificare che l'impianto elettrico e quello idraulico non entrino in conflitto.

- **Test manuali:** una persona (spesso chiamata tester o QA, *Quality Assurance*) usa il software come farebbe un utente reale, cliccando in giro, provando casi limite, cercando di “romperlo”. È come far vivere qualcuno nella casa per un weekend di prova prima del trasferimento definitivo.

Perché si testa? Perché è molto più economico e veloce trovare un errore prima che il software arrivi agli utenti, piuttosto che scoprirlo quando migliaia di persone lo stanno già usando (e magari perdendo dati o soldi a causa di quell'errore).

Esempio pratico: per la nuova funzionalità di ricerca prodotti di ShopFacile, Ahmed scrive un test unitario che verifica che la funzione di ricerca restituisca risultati corretti con un solo prodotto nel catalogo; Giulia, sempre molto attenta alla qualità, scrive un test di integrazione che verifica che la ricerca funzioni correttamente insieme al filtro dei prezzi, e poi prova lei stessa a digitare caratteri strani (accenti, emoji, campo vuoto) nella barra di ricerca per vedere se qualcosa si rompe. Solo dopo che tutti e tre i livelli di test danno esito positivo, la funzionalità viene considerata pronta per il passo successivo.

I test sono tutti verdi: la funzionalità può finalmente lasciare l'ambiente del team ed essere messa nelle mani degli utenti veri.

5. Rilascio (Deploy): si va “in produzione”

Quando il software ha superato i test, è pronto per essere usato davvero dagli utenti finali. Questo passaggio si chiama **rilascio** o, con il termine tecnico che sentirai usare moltissimo, **deploy**.

“Mettere il software in produzione” significa renderlo disponibile e funzionante nell'ambiente reale, quello usato dai clienti veri — a differenza degli ambienti di test o sviluppo, che sono come delle “copie di prova” usate solo internamente dal team (ne parleremo nella sezione sugli ambienti di sviluppo).

Un'analogia: è il giorno del trasferimento nella casa nuova. Tutto quello che è stato progettato, costruito e verificato diventa finalmente reale e utilizzabile dalla famiglia.

Il deploy può essere un'operazione delicata: si tratta di installare la nuova versione del software sui sistemi che gli utenti usano davvero, a volte senza nemmeno interrompere il servizio (pensa a un aggiornamento di un'app che non ti fa nemmeno notare che è cambiato qualcosa). Nella sezione dedicata al DevOps e alla CI/CD scoprirai come oggi questo processo venga spesso automatizzato per essere più rapido e sicuro.

Esempio pratico: il team decide di rilasciare la nuova funzionalità di ricerca prodotti di ShopFacile di notte, in una fascia orario con pochissimi utenti collegati, per limitare l'impatto nel caso qualcosa vada storto. Il deploy viene fatto prima solo su una piccola parte dei server (un cosiddetto “rilascio graduale”): se dopo un'ora tutto funziona bene, la nuova versione viene estesa a tutti gli utenti; se emergono problemi, si può tornare rapidamente alla versione precedente (un'operazione chiamata *rollback*).

Il software è finalmente in produzione, ma la storia non finisce qui: da questo momento inizia una fase che dura molto più a lungo di tutte le altre messe insieme, la manutenzione.

6. Manutenzione: la cura continua

Una casa, una volta costruita, non resta perfetta per sempre: la caldaia si guasta, il tetto va rifatto, magari dopo qualche anno si decide di ristrutturare la cucina. Lo stesso vale per il software.

La **manutenzione** comprende tutte le attività che avvengono dopo il rilascio:

- Correggere errori che emergono durante l'uso reale (i famosi **bug**, che vedremo tra poco)
- Aggiornare il software per farlo funzionare con nuove versioni di sistemi operativi o browser
- Migliorare le prestazioni
- Aggiungere piccole modifiche richieste dagli utenti

Un progetto software di successo può restare “in manutenzione” per anni, anche decenni. È una fase spesso sottovalutata da chi è alle prime armi, ma in realtà occupa una parte enorme del tempo e del budget di un progetto informatico nel lungo periodo.

Esempio pratico: sei mesi dopo il rilascio, la funzionalità di ricerca prodotti di ShopFacile inizia a rispondere sempre più lentamente perché il catalogo è cresciuto da 500 a 50.000 prodotti. Nessun requisito iniziale parlava di questo scenario: è un'attività di manutenzione, in cui Marco introduce un meccanismo di **caching** (una sorta di “memoria veloce” che conserva i risultati delle ricerche più frequenti) per riportare i tempi di risposta a livelli accettabili.

Abbiamo visto le sei fasi in azione. Ma per parlarne con precisione nel lavoro quotidiano, serve anche il vocabolario giusto: cominciamo dai requisiti, distinguendo due categorie che confonderai spesso all'inizio.

Requisiti: funzionali e non funzionali

Torniamo sui requisiti, perché è un concetto che userai costantemente nel tuo ruolo di Project Manager.

Un **requisito** è una descrizione di qualcosa che il software deve fare o di come deve comportarsi. Si dividono in due grandi categorie.

Requisiti funzionali

Descrivono **cosa** il software deve fare, cioè le sue funzionalità concrete. Rispondono alla domanda: “quali azioni deve poter compiere l'utente (o il sistema)?”

Esempi:

- “L'app deve permettere all'utente di effettuare il login con email e password”
- “Il sistema deve permettere di aggiungere un prodotto al carrello”
- “L'utente deve poter scaricare la fattura in formato PDF”

Sono requisiti facili da immaginare, perché descrivono azioni concrete e visibili, un po' come l'elenco delle stanze di una casa: “vogliamo una cucina, due camere, un bagno”.

Requisiti non funzionali

Descrivono **come** il software deve comportarsi, cioè la qualità con cui svolge le sue funzioni. Non riguardano una singola azione specifica, ma una caratteristica generale del sistema. Rispondono alla domanda: “quanto bene, quanto velocemente, quanto in sicurezza deve funzionare?”

Esempi:

- “L'app deve rispondere in meno di 2 secondi a ogni richiesta”
- “Il sistema deve poter gestire fino a 10.000 utenti collegati contemporaneamente”
- “I dati personali degli utenti devono essere cifrati”
- “L'applicazione deve essere disponibile (funzionante) almeno il 99,9% del tempo in un anno”

Tornando all'analogia della casa: se il requisito funzionale è “vogliamo una cucina”, il requisito non funzionale è “la cucina deve essere isolata acusticamente e avere abbastanza corrente per usare più elettrodomestici insieme senza far scattare il salvavita”. Non è una stanza in più, è una qualità che tutte le stanze devono avere.

Aspetto	Requisito funzionale	Requisito non funzionale
Domanda a cui risponde	Cosa fa?	Quanto bene lo fa?
Esempio	“Login con email e password”	“Login in meno di 2 secondi”
Facile da vedere?	Sì, si nota subito se manca	No, spesso si nota solo quando manca (es. sistema lento)
Chi lo richiede spesso	Cliente, utenti	Team tecnico, esigenze di business (sicurezza, scalabilità)

Un errore comune di chi inizia in questo settore è concentrarsi solo sui requisiti funzionali, perché sono i più “visibili” e i clienti li richiedono esplicitamente. Ma un buon Project Manager sa che i requisiti non funzionali (velocità, sicurezza, affidabilità) sono altrettanto critici: un'app che fa tutto quello che deve, ma è lentissima o insicura, è comunque un fallimento.

Anche il requisito meglio scritto, però, non garantisce che il codice finale si comporti come previsto: a volte qualcosa va storto, ed è lì che entra in scena il bug.

Bug: quando qualcosa va storto

Un **bug** è un difetto nel software: un comportamento non corretto, non previsto o non voluto. Il termine in inglese significa letteralmente “insetto”, e c'è una storia (in parte leggendaria, ma bellissima) dietro questo nome.

Curiosità storica: si racconta che nel 1947, mentre lavorava su uno dei primi grandi calcolatori (il Harvard Mark II), la programmatrice e informatica Grace Hopper e il suo team trovarono una falena vera e propria incastrata tra i contatti di un relè elettromeccanico, che causava un malfunzionamento. Il team incollò l'insetto su un foglio del registro di laboratorio con la scritta

“First actual case of bug being found” (primo caso reale di un bug trovato). Da allora, “bug” è diventato il termine universale per indicare un errore nel software (anche se il termine era già usato in ambito ingegneristico prima di quell’episodio).

Esempio concreto di bug: su ShopFacile, Giulia scopre durante un test che se metti nel carrello più di 99 pezzi dello stesso prodotto, il prezzo totale diventa negativo per un errore nel calcolo. L’utente potrebbe finire per “guadagnare” acquistando, invece di pagare! Questo è esattamente il tipo di comportamento inatteso che un bug produce: qualcosa che nessuno ha previsto o voluto, che si scopre spesso solo quando il software viene usato in modi che gli sviluppatori non avevano immaginato.

I bug possono essere piccoli e quasi invisibili (un testo scritto con il colore sbagliato) o gravissimi (un sistema bancario che addebita importi errati). Parte del lavoro di un Project Manager è proprio aiutare a **classificare** i bug per gravità e priorità, per decidere quali vanno risolti immediatamente e quali possono aspettare: il bug del prezzo negativo di ShopFacile, per esempio, è abbastanza grave da bloccare tutto il resto.

Feature: una nuova funzionalità

Un bug corregge qualcosa che non va, ma non tutto il lavoro del team nasce da un errore da correggere: molto lavoro nasce dal desiderio di aggiungere qualcosa di nuovo e utile. Questo si chiama feature.

Una **feature** è, al contrario del bug, qualcosa di voluto: una nuova funzionalità che si aggiunge al software per offrire più valore agli utenti.

Esempio: aggiungere la possibilità di pagare con Apple Pay in un’app che finora accettava solo carta di credito è una feature. È una scelta, una decisione di business, non la correzione di un errore.

La differenza tra bug fix (correzione di un bug) e feature è quindi:

	Bug fix	Feature
Perché si fa	Qualcosa non funziona come dovrebbe	Si vuole aggiungere qualcosa di nuovo
È previsto dai requisiti originali?	Sì, il comportamento corretto era già previsto ma non funzionava	No, è un’aggiunta rispetto a quanto già esisteva
Urgenza tipica	Spesso alta, soprattutto se blocca l’uso del software	Variabile, spesso pianificata per una versione futura
Esempio	“Il bottone ‘Acquista’ non risponde al click su alcuni telefoni”	“Aggiungere la possibilità di salvare i prodotti preferiti”

Questa distinzione è fondamentale perché, come Project Manager, dovrai spesso gestire un elenco (detto **backlog**, termine che approfondiremo nella sezione sull’Agile) contenente sia bug da correggere che feature da sviluppare, e aiutare il team a decidere le priorità.

■ Ogni volta che un bug viene corretto o una feature aggiunta, il software cambia rispetto a prima: serve un modo per identificare con precisione “quale versione” stiamo usando o discutendo in un dato momento.

Versioning: perché numeriamo il software

Hai mai notato che le app sul telefono si aggiornano e mostrano numeri come “2.4.1” o “10.0.3”? Questo si chiama **versioning** (o numerazione delle versioni).

Perché è importante? Immagina di dover parlare con un collega di un problema e dire semplicemente “l'app fa una cosa strana”. Quale app? In che momento? Con quali funzionalità presenti? Senza un numero di versione preciso, sarebbe come descrivere un'auto senza dire il modello o l'anno: informazioni troppo vaghe per essere utili.

Il numero di versione permette di:

- Sapere esattamente quale “fotografia” del software si sta usando o discutendo
- Capire se un problema è già stato risolto in una versione più recente
- Comunicare chiaramente ai clienti cosa cambia da una versione all'altra

Un sistema molto diffuso si chiama **Semantic Versioning** (versionamento semantico), che usa tre numeri nel formato **MAJOR.MINOR.PATCH** (ad esempio **1.2.3**):

- **MAJOR** (il primo numero, “1”): cambia quando si introducono modifiche importanti, che potrebbero non essere compatibili con le versioni precedenti. Ad esempio, se cambia completamente il modo in cui l'app comunica con altri sistemi.
- **MINOR** (il secondo numero, “2”): cambia quando si aggiungono nuove feature, ma tutto quello che c'era prima continua a funzionare come prima.
- **PATCH** (il terzo numero, “3”): cambia quando si correggono bug, senza aggiungere nuove funzionalità.

Esempio pratico: se il software è alla versione 1.2.3 e il team:

- corregge un bug → diventa 1.2.4
- aggiunge una nuova feature (es. il pagamento con Apple Pay) → diventa 1.3.0
- cambia qualcosa in modo così profondo da rompere la compatibilità con l'esterno → diventa 2.0.0

Questo sistema è uno standard molto diffuso nel mondo dello sviluppo software, adottato da moltissime librerie e applicazioni.

Numerare le versioni presuppone però che il codice arrivi in modo ordinato a uno stato “pubblicabile” — e questo, con più persone che scrivono codice insieme, non è affatto scontato. Vediamo come si organizza concretamente il lavoro di squadra sul codice.

Branching: lavorare in parallelo

Immagina un team di 5 sviluppatori — pensa a Marco, Giulia, Ahmed e ai loro colleghi di ShopFacile — che lavorano tutti insieme, contemporaneamente, sullo stesso identico file di codice. Cosa potrebbe succedere? Molto probabilmente, un disastro: le modifiche di uno cancellerebbero quelle di un altro, e sarebbe impossibile capire chi ha cambiato cosa.

Per questo esiste il concetto di **branch** (in italiano “ramo”). Un branch è come una copia parallela e temporanea del progetto, su cui una persona (o un piccolo gruppo) può lavorare in isolamento, senza disturbare il lavoro degli altri, per poi ricongiungere le modifiche al progetto principale quando sono pronte.

L'analogia migliore è quella dell'albero: il tronco principale rappresenta la versione “ufficiale” e stabile del software (spesso chiamata branch `main` o `master`). Ogni volta che uno sviluppatore deve lavorare su qualcosa — una nuova feature, la correzione di un bug — crea un ramo che si distacca dal tronco, ci lavora in tranquillità, e alla fine quel ramo viene “ricongiunto” al tronco principale.

Diagramma 2

Diagramma 2

Questo concetto sarà spiegato con molti più dettagli pratici (comandi, strumenti, esempi passo-passo) nella prossima sezione dedicata a Git e GitLab. Per ora ti basta capire l'idea: il branching permette a più persone di lavorare in parallelo sullo stesso progetto senza pestarsi i piedi.

Esempio pratico: mentre Ahmed lavora sul branch `feature-ricerca-prodotti` per aggiungere la barra di ricerca, Giulia lavora in parallelo sul branch `fix-carrello` per correggere il bug del prezzo negativo visto prima. Nessuno dei due deve aspettare che l'altro finisca: lavorano entrambi sul proprio ramo, isolati, e uniranno le modifiche al tronco principale quando saranno pronti, ciascuno con i suoi tempi.

Dal branch al merge: merge request e code review

Un branch isolato, però, non può restare isolato per sempre: prima o poi il lavoro va ricongiunto al tronco principale. Ma una volta che uno sviluppatore ha finito di lavorare sul proprio ramo (ad esempio ha completato la feature “login con email”), non può semplicemente “ricongiungere” le sue modifiche al tronco principale senza controlli. Sarebbe rischioso: e se il suo codice avesse errori? E se rompesse qualcosa che già funzionava?

Per questo, prima di unire le modifiche, si passa attraverso due passaggi fondamentali: la **merge request** e la **code review**.

Merge Request: “propongo questa modifica”

Una **merge request** (spesso abbreviata **MR** — su altre piattaforme lo stesso concetto può avere un nome diverso, ma la logica è identica) è, in parole semplici, una richiesta formale: “ho finito di lavorare su questo ramo, per favore controllate il mio lavoro e, se va bene, unitelo al progetto principale”.

È come quando, dopo aver scritto una relazione di lavoro, non la invii direttamente al cliente, ma la mandi prima al tuo responsabile per un controllo. La merge request è esattamente quella richiesta di controllo, applicata al codice.

Esempio pratico: Ahmed, che ha finito la barra di ricerca prodotti, apre una merge request con un titolo come “Aggiunta ricerca prodotti per nome e categoria” e una descrizione che spiega cosa cambia, come è stato testato e magari uno screenshot del risultato. A quel punto la merge request compare in una lista visibile a tutto il team (su GitLab o Azure DevOps), pronta per essere revisionata.

Aprire la merge request, però, non basta: qualcuno deve davvero leggere quel codice prima che diventi parte ufficiale del progetto. È il momento della code review.

Code Review: il controllo dei colleghi

La **code review** è il momento in cui uno o più colleghi (spesso sviluppatori più esperti, o semplicemente altri membri del team) leggono il codice scritto e verificano che:

- Faccia davvero quello che deve fare
- Sia scritto in modo chiaro e mantenibile (cioè che altre persone, in futuro, possano capirlo e modificarlo facilmente)
- Non introduca bug o problemi di sicurezza
- Segua le convenzioni e gli standard adottati dal team

Perché è importante che sia qualcun altro a controllare, e non solo chi ha scritto il codice? Per lo stesso motivo per cui è utile far leggere una mail importante a un collega prima di inviarla: chi scrive tende a non vedere i propri errori, mentre un occhio esterno nota cose che altrimenti sfuggirebbero. La code review, inoltre, aiuta a diffondere la conoscenza del progetto tra i membri del team, così che non ci sia una sola persona che “sa tutto” e nessun altro capisce quel pezzo di codice.

Se durante la code review emergono problemi, chi ha scritto il codice apporta le correzioni richieste, e il processo si ripete finché tutti sono soddisfatti.

Esempio pratico: Giulia, revisionando la merge request della ricerca prodotti, lascia un commento del tipo “qui la ricerca fa distinzione tra maiuscole e minuscole, un utente che scrive ‘scarpe’ con la S minuscola non troverebbe ‘Scarpe’ scritto con la maiuscola: puoi correggerlo?”. Ahmed corregge il codice, aggiorna la merge request, e Giulia la approva.

Merge: l'unione finale

La code review è stata superata: non resta che rendere ufficiale il lavoro. Quando la merge request è stata approvata dai colleghi durante la code review, si procede al **merge**: l'unione definitiva delle modifiche del ramo (branch) al progetto principale. A questo punto, quella nuova funzionalità o correzione diventa parte ufficiale del software, pronta per le fasi successive (ulteriori test, e infine il rilascio).

Esempio pratico: dopo l'approvazione, Ahmed (o un collega con i permessi adeguati) clicca sul bottone “Merge” nello strumento usato dal team. Da quel momento, il codice della barra di ricerca fa parte del branch principale `main` di ShopFacile, insieme al lavoro di tutti gli altri membri del team, e sarà incluso nel prossimo rilascio pianificato.

Diagramma 3

Diagramma 3

Questo flusso — branch, merge request, code review, merge — è oggi lo standard adottato dalla ^{*}stragrande maggioranza dei team di sviluppo professionali nel mondo, e lo ritroverai costantemente nel tuo lavoro quotidiano come Project Manager, anche solo per monitorare lo stato di avanzamento delle attività del team.

Come si collegano tutti questi concetti

Abbiamo visto tanti termini uno alla volta: requisiti, bug, feature, versioning, branch, merge request, code review, merge. Facciamo un piccolo riepilogo con un esempio end-to-end sul progetto ShopFacile, per vedere come tutti i concetti di questa sezione si incastrano insieme in un caso reale (semplificato):

1. Il cliente chiede una nuova funzionalità: “vogliamo che gli utenti possano recuperare la password” → è un **requisito funzionale**, che Sara fa finire nel backlog di ShopFacile come **feature**.
2. Marco fa l'**analisi**: decide di implementarla con un'email contenente un link temporaneo.
3. Ahmed crea un **branch** dedicato e inizia lo **sviluppo**, scrivendo il codice necessario.
4. Durante lo sviluppo scopre e corregge anche un piccolo **bug** preesistente in una funzione che invia le email.
5. Finito il lavoro, apre una **merge request**.
6. Giulia esegue la **code review**, chiede una piccola modifica per rispettare un **requisito non funzionale** (il link deve scadere dopo 30 minuti per motivi di sicurezza).
7. Ahmed corregge, Giulia approva, si fa il **merge**.
8. Il team esegue il **testing** (unitario, di integrazione e manuale) sulla nuova funzionalità.
9. Tutto ok: si prepara il **rilascio** e si assegna un nuovo numero di **versione** a ShopFacile, ad esempio da 2.3.1 a 2.4.0 (è una nuova feature, quindi cambia il numero MINOR).
10. Il software va in **produzione**. Da qui in avanti, entra nella fase di **manutenzione**: se emergono nuovi bug legati a questa funzionalità, il ciclo ripartirà da capo su un nuovo branch.

☑ Come vedi, tutti questi termini che oggi possono sembrare astratti sono in realtà i mattoni con cui è costruito il lavoro quotidiano di qualsiasi team software, ShopFacile incluso. Nelle prossime sezioni approfondirai molti di questi concetti con strumenti e pratiche concrete.

Esercizi pratici

Questi esercizi ti aiutano a consolidare i concetti di questa sezione prima di passare a Git e GitLab. Non servono strumenti particolari: bastano carta, penna e, dove indicato, una breve chiacchierata con un collega.

1. **Mappa il ciclo di vita su un caso reale.** Pensa a una funzionalità che sai essere stata rilasciata di recente nel progetto (chiedi a un collega se non ne conosci una) e scrivi, riga per riga, come pensi si siano svolte le sei fasi del ciclo di vita (requisiti, analisi, sviluppo, testing, rilascio,

manutenzione) per quel caso specifico. Non preoccuparti di essere preciso al 100%: l'obiettivo è esercitarti a “vedere” le fasi in un esempio concreto. **Come verificare:** mostra la tua mappa a un collega del team e chiedigli di correggere i punti in cui hai sbagliato o semplificato troppo.

2. **Classifica 5 requisiti.** Scrivi 5 frasi che potrebbero essere requisiti di un software (puoi inventarle o prenderle da una card reale del backlog del progetto) e per ciascuna indica se è un requisito funzionale o non funzionale, motivando la scelta in una riga. **Come verificare:** rileggi ogni frase chiedendoti “risponde a cosa fa?” (funzionale) o a “quanto bene lo fa?” (non funzionale); se la risposta non è immediata, probabilmente la frase è scritta in modo ambiguo — prova a riformulare.
3. **Distingui bug da feature su casi reali.** Chiedi a un developer o al tuo referente di mostrarti 3-4 elementi del backlog del progetto (o della board del team) e, prima che te lo dicano loro, provaci a indovinare quali sono bug e quali sono feature, spiegando il perché della tua scelta. **Come verificare:** confronta le tue risposte con la classificazione reale usata dal team; se hai sbagliato qualcosa, capisci perché (spesso la differenza sta nel fatto che un bug viola un comportamento già previsto, mentre la feature è un'aggiunta nuova).
4. **Simula un versioning.** Immagina che il software del progetto sia alla versione 3.4.2. Elenca 4 eventi ipotetici (2 bug fix, 1 nuova feature, 1 cambiamento che rompe la compatibilità) e scrivi, in ordine, come cambierebbe il numero di versione dopo ciascun evento. **Come verificare:** rileggi le regole MAJOR.MINOR.PATCH di questa sezione e verifica che ogni tuo passaggio le rispetti (ad esempio, un nuovo MAJOR deve riportare MINOR e PATCH a zero).
5. **Disegna il flusso branch → merge con un esempio del progetto.** Su un foglio (anche a mano), disegna il flowchart “branch, sviluppo, merge request, code review, merge” visto in questa sezione, ma sostituendo le etichette generiche con un esempio verosimile del progetto (es. “branch fix-notifiche-email” invece di “creazione del branch”). Poi racconta il disegno a voce alta, come se lo spiegassi a un collega nuovo arrivato. **Come verificare:** se riesci a raccontare il disegno senza guardare gli appunti e senza incepparti su nessun passaggio, hai interiorizzato bene il flusso.

Collegamenti

- [4. Git e GitLab](#) — per capire in pratica, con comandi ed esempi, come funzionano davvero branch, commit, merge request e merge
- [5. Agile](#) — per scoprire come i team moderni organizzano il ciclo di vita del software in iterazioni brevi e continue
- [9. DevOps](#) — per approfondire come le fasi di sviluppo, testing e rilascio vengono oggi collegate e automatizzate
- [11. CI/CD](#) — per capire come build, test e deploy possono essere automatizzati end-to-end

Risorse

- [Atlassian — What is SDLC? Software Development Life Cycle Explained](#) — panoramica chiara sul ciclo di vita del software

- [Semantic Versioning 2.0.0](#) — la specifica ufficiale del versionamento semantico (in inglese, ma con esempi semplici)
- [GitLab Docs — About merge requests](#) — documentazione ufficiale su cosa sono e come funzionano le merge request
- [Atlassian — Code Review Best Practices](#) — guida pratica sul perché e come si fa code review
- [Wikipedia — Software bug \(storia del termine, inclusa la storia della falena di Grace Hopper\)](#) — approfondimento storico e tecnico sull'origine del termine “bug”

4. Git e GitLab

Se hai mai lavorato su un documento Word con altre persone, probabilmente conosci già l'incubo dei file chiamati `Progetto_finale_v2_DEFINITIVO_uso_questo.docx`. Qualcuno ha modificato una frase, qualcun altro ha cancellato per errore un paragrafo importante, e nessuno ricorda più qual è la versione "buona".

Git e GitLab esistono per risolvere esattamente questo problema, ma applicato al codice sorgente di un software: migliaia di file di testo che decine di persone modificano ogni giorno, in parallelo, senza pestarsi i piedi a vicenda.

Questa è probabilmente la sezione più "pratica" del corso: gli strumenti che imparerai qui li vedrai citati in ogni *standup*, in ogni *merge request*, in ogni pipeline di CI/CD. Non serve che tu diventi uno sviluppatore, ma è fondamentale che tu capisca il vocabolario e la logica di fondo, perché è il linguaggio con cui il tuo team lavora ogni giorno.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- spiegare la differenza tra Git e GitLab;
- capire cosa sono repository, commit, branch e merge;
- capire come funziona una Merge Request e perché è il cuore della collaborazione su GitLab;
- distinguere Issue, Release e Tag;
- conoscere due modelli di lavoro (workflow) molto diffusi: **Git Flow** e **Trunk Based Development**, e capire quando si usa l'uno o l'altro.

4.1 Cos'è Git

Git è un **sistema di controllo di versione** (in inglese *Version Control System*, o VCS). In parole semplici: è un programma che tiene traccia di **ogni modifica** fatta ai file di un progetto, salvando nel tempo tutta la "cronologia" di quelle modifiche.

Analogia: pensa alla cronologia delle modifiche di Google Docs o alla funzione "Traccia modifiche" di Word, ma molto più potente. Con Git non solo puoi vedere chi ha cambiato cosa e quando, ma puoi anche: - tornare indietro a una versione precedente in qualsiasi momento; - far lavorare **più persone in parallelo** sullo stesso progetto senza che si sovrascrivano il lavoro a vicenda; - "unire" automaticamente le modifiche fatte da persone diverse.

Git è stato creato nel 2005 da Linus Torvalds (lo stesso creatore di Linux) per gestire lo sviluppo del kernel Linux, un progetto con migliaia di collaboratori. Oggi è lo standard de facto per versionare codice, usato praticamente da ogni azienda software al mondo.

Un punto fondamentale: **Git è distribuito**. Questo significa che ogni persona che lavora su un progetto ha, sul proprio computer, una **copia completa** di tutta la cronologia del progetto — non solo l'ultima versione, ma *tutta la storia* fin dall'inizio. Questo è molto diverso dai vecchi sistemi centralizzati, dove esisteva un solo “archivio” centrale e se quello andava giù, si perdeva tutto.

Diagramma 1

Diagramma 1

Git funziona da **riga di comando** (il terminale), ma esistono anche interfacce grafiche (come GitKraken, o l'integrazione diretta in Visual Studio Code) che rendono tutto più visuale. In questo corso vedrai comandi da terminale a scopo illustrativo: non devi memorizzarli a memoria, l'obiettivo è che tu capisca **la logica**.

4.2 Cos'è GitLab (e non è la stessa cosa di Git!)

Nel diagramma di prima abbiamo scritto “GitLab” sopra il server remoto un po' di sfuggita: è il momento di chiarire di cosa si tratta davvero. Uno degli equivoci più comuni per chi inizia, infatti, è pensare che Git e GitLab siano la stessa cosa. Non lo sono.

	Git	GitLab
Cos'è	Uno strumento (software)	Una piattaforma online (un servizio)
Dove vive	Sul tuo computer	Su internet (cloud), o su un server aziendale (self-hosted)
Cosa fa	Gestisce la cronologia delle modifiche	Ospita i repository online e aggiunge funzionalità di collaborazione
Necessario?	Sì, è il motore	No, è un servizio che <i>usa</i> Git

Analogia: Git è come il motore di un'auto: fa il lavoro tecnico di spostare la macchina. GitLab è come una concessionaria con parcheggio, assistenza e servizi extra: ospita l'auto, la rende visibile ad altri, offre strumenti di manutenzione condivisa. Potresti guidare l'auto (usare Git) senza mai passare dalla concessionaria (senza mai usare GitLab) — ma se lavori in team, avere un posto centrale dove parcheggiare e collaborare è estremamente comodo.

GitLab, in concreto, offre:

- uno spazio online dove “ospitare” i repository (i progetti);
- strumenti di collaborazione come **Merge Request**, **Issue**, revisioni del codice, commenti;
- automazioni (**GitLab CI/CD**, che vedremo nella sezione CI/CD);
- gestione di permessi, team, sicurezza.

GitLab **non è l'unica piattaforma** di questo tipo. L'alternativa più diffusa in contesti aziendali come il tuo è:

- **Azure DevOps Repos**: la componente di versionamento codice della suite Azure DevOps di Microsoft (che vedremo nella sezione 10) — in alcuni contesti aziendali è la piattaforma principale.

Il concetto di fondo è sempre lo stesso (Git sotto il cofano), cambia la piattaforma “concessionaria” sopra. In questo corso useremo GitLab come riferimento perché è molto diffusa sia in versione cloud sia **self-hosted** (installata sui server dell'azienda, ad esempio su un indirizzo come `gitlab.tuaazienda.it`), ed è una scelta molto comune nelle realtà aziendali che vogliono mantenere il controllo completo della propria infrastruttura.

4.3 Repository: il “progetto” versionato

GitLab ospita i progetti, dicevamo: ma cosa contiene esattamente uno di quei progetti da un punto di vista tecnico? La risposta è il repository. Un **repository** (spesso abbreviato “repo”) è semplicemente **la cartella di un progetto**, ma tracciata da Git. Contiene tutti i file del progetto (codice, documentazione, configurazioni) più una cartella speciale invisibile chiamata `.git`, dove Git conserva tutta la cronologia delle modifiche.

Analogia: se il progetto è un libro, il repository è sia il libro stesso sia l'archivio completo di tutte le bozze precedenti, capitolo per capitolo, con la data e l'autore di ogni modifica.

Un repository può essere: - **locale**: la copia che vive sul tuo computer; - **remoto**: la copia ospitata online (su GitLab, cloud o self-hosted), che funge da punto di incontro condiviso tra tutti i membri del team.

```
# Creare un nuovo repository da zero, dentro una cartella
git init

# Oppure "clonare" (scaricare) un repository che esiste già online
git clone https://gitlab.com/nome-organizzazione/nome-progetto.git
```

Dopo un `git clone`, avrai sul tuo computer una copia completa del progetto, compresa tutta la sua storia — pronta per essere modificata.

Esempio pratico: immagina che Marco ti mandi il link al repository GitLab del progetto, ad esempio `https://gitlab.com/shopfacile/backend-api` (o, se l'azienda usa un'istanza self-hosted, qualcosa come `https://gitlab.tuaazienda.it/shopfacile/backend-api`). Aprendolo nel browser vedrai: una lista di cartelle e file (il codice), un pulsante blu “Clone” per clonarlo, il numero di branch attivi, e una voce “Commits” con la cronologia di tutte le modifiche

fatte finora. Se lo clonassi sul tuo computer con

```
git clone https://gitlab.com/shopfacile/backend-api.git
```

, ti ritroveresti in una cartella `backend-api` con dentro tutti quei file, pronti per essere letti — anche se non li modifichi mai.

4.4 Commit: uno scatto fotografico delle modifiche

Il repository conserva “tutta la cronologia delle modifiche”, abbiamo detto: ma di cosa è fatta, concretamente, questa cronologia? È fatta di commit. Un **commit** è un salvataggio “ufficiale” di un insieme di modifiche, con un messaggio che spiega **cosa** e **perché** è stato cambiato.

Analogia: immagina di scattare una fotografia allo stato attuale del progetto. Ogni commit è uno scatto: cattura esattamente come erano i file in quel momento, con un’etichetta (il messaggio di commit) che descrive cosa è successo da uno scatto all’altro. Con tante fotografie in sequenza, hai un album completo della storia del progetto.

Il flusso tipico per creare un commit è:

```
# 1. Vedo quali file ho modificato
git status

# 2. "Metto in valigia" le modifiche che voglio salvare (staging)
git add nome-file.txt
# oppure, per aggiungere tutti i file modificati:
git add .

# 3. Creo il commit con un messaggio descrittivo
git commit -m "Aggiunge la validazione del campo email nel form di registrazione"
```

Alcune buone pratiche sui messaggi di commit (le vedrai spesso menzionate nelle Merge Request del team):

- il messaggio deve spiegare **il perché**, non solo il “cosa” (il “cosa” si vede già dal codice cambiato);
- meglio tanti commit piccoli e mirati che un unico commit enorme con scritto “fix vari”;
- molti team adottano convenzioni standard, ad esempio i [Conventional Commits](#) (`feat:`, `fix:`, `docs:`, `chore:` ...).

Ogni commit ha un **identificativo univoco** (un codice esadecimale, es. `a1b2c3d`), che permette di riferirsi esattamente a quello scatto in qualsiasi momento futuro — utile per capire, ad esempio, “in quale commit è stato introdotto questo bug?”.

Esempio pratico: dopo tre commit di Marco su ShopFacile, se lanci `git log --oneline` vedresti qualcosa così:

```
a1b2c3d Aggiunge la validazione del campo email nel form di registrazione
9f8e7d6 Aggiunge il form di registrazione utente
3c4d5e6 Setup iniziale del progetto
```

Ogni riga è un commit: il codice a sinistra (es. `a1b2c3d`) è l'identificativo univoco, il testo a destra è il messaggio. Se un giorno Giulia nota un bug nella validazione email, potrà usare `git log` per capire esattamente in quale commit — e quindi in che momento — è stata introdotta quella modifica.

4.5 Branch: un ramo di lavoro parallelo

Ogni commit di Marco si accumula in sequenza sulla stessa linea temporale: ma cosa succede quando Giulia e Ahmed devono lavorare *contemporaneamente* su cose diverse, senza pestarsi i piedi? Serve poter deviare dalla linea principale, ed è qui che entra in gioco il branch.

Un **branch** (letteralmente “ramo”) è una linea di sviluppo indipendente che parte da un punto della storia del progetto. Permette di lavorare su qualcosa di nuovo (una funzionalità, una correzione) **senza toccare** la versione “stabile” del progetto, che di solito vive nel branch principale chiamato `main` (in passato spesso chiamato `master`).

Analogia: pensa alla linea principale della storia (`main`) come alla strada maestra di un progetto: deve restare sempre percorribile e affidabile. Un branch è come un cantiere laterale in cui provi cose nuove: se qualcosa va storto, il cantiere non blocca la strada principale. Quando il lavoro nel cantiere è pronto e testato, lo “colleghi” alla strada maestra.

```
# Creo un nuovo branch chiamato "feature/login-utente"
git branch feature/login-utente

# Mi sposto su quel branch per iniziare a lavorarci
git checkout feature/login-utente

# In alternativa, questi due comandi si possono unire in uno solo:
git checkout -b feature/login-utente
```

Convenzioni di naming comuni per i branch (utili da riconoscere quando le vedi nelle Merge Request):

- `feature/...` → nuova funzionalità (es. `feature/login-utente`)
- `fix/...` o `bugfix/...` → correzione di un bug

- `hotfix/...` → correzione urgente su produzione
- `release/...` → preparazione di una nuova versione

Lavorare su branch separati è ciò che permette a più persone del team di lavorare **in parallelo sullo stesso progetto** senza pestarsi i piedi.

Esempio pratico: il team di ShopFacile deve sviluppare due cose in parallelo: una nuova pagina di login e la correzione di un bug nel checkout. Marco e Ahmed lanceranno:

```
git checkout -b feature/login-utente      # Marco
git checkout -b fix/checkout-prezzo-errato # Ahmed
```

Da questo momento lavorano su due “cantieri” separati: i commit di Marco non toccano i file su cui lavora Ahmed (a meno che non modifichino esattamente gli stessi file), e nessuno dei due tocca `main`, che resta stabile per tutti gli altri.

4.6 Merge: unire due rami

Marco e Ahmed, però, non possono lavorare per sempre su rami separati: prima o poi il loro lavoro deve tornare a far parte di un'unica versione condivisa. L'operazione che lo rende possibile è il merge. Il **merge** è l'operazione che **unisce le modifiche** fatte su un branch dentro un altro branch (tipicamente, si uniscono le modifiche di un branch di feature dentro `main`, una volta che il lavoro è completo e testato).

Analogia: riprendendo l'immagine di prima, il merge è il momento in cui il cantiere laterale viene collegato alla strada principale: da quel momento, chi percorre la strada maestra passa anche dal nuovo tratto.

```
# Mi sposto sul branch che deve "ricevere" le modifiche
git checkout main

# Unisco le modifiche del branch di feature dentro main
git merge feature/login-utente
```

Git è molto bravo a unire automaticamente modifiche fatte su parti diverse dei file. Ma se due persone hanno modificato **esattamente la stessa riga** in modo diverso, Git non sa quale versione tenere: si genera un **conflitto di merge**, che va risolto manualmente decidendo quale modifica (o quale combinazione) mantenere. Non preoccuparti: è un evento normalissimo nel lavoro quotidiano, non un errore grave.

Esempio pratico: supponiamo che il branch `feature/login-utente` sia pronto. Da `main` lanci:

```
git checkout main
git merge feature/login-utente
```

Se nessuno ha toccato le stesse righe su `main` nel frattempo, Git risponde con qualcosa come `Fast-forward` o `Merge made by the 'ort' strategy` e il lavoro è unito automaticamente. Se invece Giulia aveva modificato lo stesso file nello stesso punto, Git si ferma e restituisce un messaggio come `CONFLICT (content): Merge conflict in login.js` — a quel punto tocca aprire il file, decidere quale versione tenere (o combinarle), e completare il merge con `git add + git commit`.

Ecco un diagramma che riassume l'intero ciclo di vita di un branch di feature, dalla creazione al merge:

Diagramma 2

Diagramma 2

Cosa racconta questo diagramma: mentre il branch `feature/login-utente` procedeva con i suoi commit, anche `main` è avanzato con un piccolo fix. Il branch di feature ha “assorbito” quell'aggiornamento (merge di `main` dentro il branch), per essere sicuro di lavorare sulla versione più recente, e infine è stato unito (merge) dentro `main`, portando tutto il lavoro fatto nella linea principale.

4.7 Merge Request: il cuore della collaborazione su GitLab

Abbiamo visto come funziona un merge da riga di comando, ma nella pratica quotidiana su GitLab il merge non viene quasi mai lanciato “a freddo”: passa prima per uno strumento che formalizza la richiesta e apre lo spazio per la revisione, la Merge Request. Una **Merge Request** (spesso abbreviata **MR**) è una **richiesta formale** di unire le modifiche di un branch dentro un altro branch (di solito dentro `main`), passando prima per una **revisione** da parte di altri membri del team.

Questo è probabilmente il concetto più importante di tutta la sezione, perché è il meccanismo attraverso cui lavora quasi ogni team che usa GitLab. (Su Azure DevOps, che vedremo nella sezione 10, lo stesso identico concetto si chiama “Pull Request” — non farti confondere dal nome diverso: la logica è la stessa.)

Analogia: è come sottoporre una bozza di documento a un collega prima di renderla definitiva. Non modifichi direttamente il documento ufficiale: proponi le tue modifiche, il collega le legge, lascia commenti, magari chiede correzioni, e solo quando tutti sono d'accordo la bozza diventa la versione ufficiale.

Il flusso tipico di una Merge Request:

1. Uno sviluppatore (ad esempio Marco) crea un branch e ci lavora (con i suoi commit).
2. Quando il lavoro è pronto, apre una **Merge Request** su GitLab, proponendo di unire quel branch dentro `main` (o dentro un altro branch di destinazione).
3. Altri membri del team fanno la **code review**: leggono le modifiche, lasciano commenti, chiedono chiarimenti o correzioni.
4. Spesso, in parallelo, delle **pipeline automatiche** (CI/CD, che vedremo nella sezione 11) eseguono automaticamente test e controlli di qualità sul codice della MR.
5. Quando tutto è approvato (“approved”) e i controlli automatici sono passati (i cosiddetti “check verdi”), la Merge Request viene unita (merge) — spesso con un semplice click sul pulsante “Merge” su GitLab.

Diagramma 3

Diagramma 3

Perché le Merge Request sono così importanti per un Project Manager? Perché sono il punto in cui puoi **misurare visivamente lo stato di avanzamento** del lavoro: quante MR sono aperte, quante sono in attesa di revisione, quanto tempo restano aperte prima di essere unite (un indicatore utile di quanto è fluido il processo del team).

Esempio pratico: Marco ha finito la feature login su ShopFacile. I passaggi concreti su GitLab sarebbero: 1. Push del branch: `git push origin feature/login-utente`. 2. Su GitLab compare un banner “Create merge request”: clicca, scrivi un titolo (es. “Aggiunge login utente con validazione email”) e una descrizione di cosa cambia e perché. 3. Assegna Giulia come reviewer/approver. 4. Giulia lascia commenti tipo “Perché qui usiamo `==` invece di `===` ?” — Marco risponde o corregge con un nuovo commit sullo stesso branch, che aggiorna automaticamente la MR. 5. Quando i commenti sono risolti e i check automatici (CI/CD) sono verdi, Giulia clicca “Approve”, poi qualcuno clicca “Merge”.

Da Project Manager, vedresti questa MR nella sezione “Merge requests” del repository con un’etichetta verde “Open” che diventa viola “Merged” a fine processo.

4.8 Issue: tracciare bug e richieste

Finora abbiamo parlato di codice già scritto: branch, commit, merge request. Ma da dove nasce il lavoro stesso, cioè “cosa” bisogna scrivere o correggere? Spesso nasce da una Issue. Una **Issue** è una “voce” che descrive un problema da risolvere o una richiesta da realizzare: un bug da correggere, una nuova funzionalità da sviluppare, un miglioramento da valutare.

Analogia: è come un “biglietto” (ticket) di un help desk: qualcuno segnala un problema o una richiesta, il biglietto viene assegnato a qualcuno, discusso, e chiuso quando risolto.

Una Issue tipicamente contiene: - un titolo chiaro; - una descrizione del problema o della richiesta (spesso con passi per riprodurre un bug, screenshot, comportamento atteso vs. osservato); - **etichette** (label), es. `bug`, `enhancement`, `documentazione`; - un **assegnatario** (chi ci lavora); - commenti di discussione.

Le Issue si possono collegare direttamente alle Merge Request: è comune vedere in una MR la frase “Closes #42”, che indica che, una volta unita quella MR, la Issue numero 42 verrà chiusa automaticamente perché risolta.

Molti team di progetto (incluso probabilmente il tuo) usano le Issue di GitLab — o l'equivalente “Work Item” in Azure DevOps — come base per la gestione del backlog che vedremo nelle sezioni su Agile, Scrum e Kanban.

Esempio pratico: un cliente di ShopFacile segnala che il pulsante “Conferma ordine” non funziona su un certo browser. Sara apre una Issue così: - **Titolo:** “Il pulsante Conferma ordine non risponde su Safari” - **Descrizione:** passi per riprodurlo, screenshot, browser e versione usati - **Label:** `bug`, `priorità-alta` - **Assegnatario:** Ahmed, che se ne occuperà

Ahmed apre un branch `fix/conferma-ordine-safari`, risolve il problema, e nella descrizione della Merge Request scrive “Closes #57” (dove 57 è il numero della Issue): una volta unita la MR, GitLab chiude automaticamente la Issue numero 57.

4.9 Release: una versione pubblicata ufficialmente

La Issue #57 è stata chiusa, il fix è unito in `main`: ma quando quel fix arriva davvero nelle mani degli utenti, con quale nome viene comunicato? Con quello di una Release. Una **Release** è un punto preciso della storia del progetto che viene “pubblicato” ufficialmente come versione consegnabile agli utenti finali (o al cliente).

Analogia: se i commit sono gli scatti fotografici quotidiani di un album, la release è la **foto scelta per la copertina** di un capitolo: un momento speciale, marcato e comunicato a tutti come “questa è la versione 2.3.0, disponibile da oggi”.

Le release seguono spesso uno schema di numerazione chiamato **Semantic Versioning** (`MAJOR.MINOR.PATCH`, es. `2.3.1`): - **MAJOR:** cambiamenti grandi, potenzialmente non compatibili con le versioni precedenti (es. `1.x` → `2.0.0`); - **MINOR:** nuove funzionalità compatibili con le versioni precedenti (es. `2.2.x` → `2.3.0`); - **PATCH:** correzioni di bug, senza nuove funzionalità (es. `2.3.0` → `2.3.1`).

Su GitLab, una Release viene solitamente accompagnata da **note di rilascio** (release notes): un elenco leggibile di cosa è cambiato, utilissimo anche per un Project Manager che deve comunicare al cliente o agli stakeholder cosa contiene la nuova versione.

Esempio pratico: il team di ShopFacile rilascia la versione **2.3.0** del prodotto. Sulla pagina “Releases” di GitLab troveresti una voce con: - tag **v2.3.0** ; - titolo “Versione 2.3.0 — Gestione utenti”; - note di rilascio tipo: “ Nuove funzionalità: gestione ruoli utente. Fix: corretto un bug nel checkout. Nota: richiede un aggiornamento del database.”; - eventuali file scaricabili (es. un installer).

Un Project Manager potrebbe riprendere direttamente queste note di rilascio — magari semplificandole — in una comunicazione al cliente o in un aggiornamento agli stakeholder.

4.10 Tag: etichettare un punto preciso della storia

Ogni Release che abbiamo appena visto, in realtà, si appoggia su un meccanismo più elementare di Git per individuare quel punto preciso della storia: il tag. Un **Tag** è un’etichetta che punta a un commit specifico, per marcarlo come “punto di interesse” — quasi sempre usato insieme alle release.

Analogia: se la storia del progetto è un lungo rotolo di pellicola fotografica, un tag è un segnalibro adesivo attaccato a uno scatto preciso, con scritto sopra “v2.3.0” — così puoi ritrovarlo istantaneamente anche tra migliaia di altri scatti.

```
# Creo un tag "annotato" (con messaggio) sul commit corrente
git tag -a v2.3.0 -m "Versione 2.3.0: aggiunta gestione utenti"

# Invio il tag anche al repository remoto (altrimenti resta solo locale)
git push origin v2.3.0
```

Differenza pratica tra Tag e Release: il **tag** è un concetto di Git (un puntatore a un commit), mentre la **Release** è un concetto di GitLab costruito sopra un tag, con in più note descrittive, file scaricabili (binari, installer) e visibilità nella pagina del progetto.

Esempio pratico: dopo aver taggato e pubblicato la versione **v2.3.0**, se sei sul repository e lanci **git tag**, vedresti l'elenco di tutti i tag creati nel tempo:

```
v1.0.0
v1.1.0
v2.0.0
v2.1.0
v2.2.0
v2.3.0
```

Se un giorno serve tornare esattamente alla versione rilasciata qualche mese prima, basta fare `git checkout v2.1.0` (se quello era il tag di quella release) per vedere il progetto esattamente come era in quel momento, senza dover cercare “a occhio” nella cronologia dei commit.

4.11 Git Flow: un workflow strutturato a più branch

Finora abbiamo visto i singoli “mattoncini” (branch, merge, MR, tag, release): ma come si combinano insieme in una strategia coerente, giorno per giorno, per tutto il team? Un primo modello molto diffuso è Git Flow. **Git Flow** è un modello di organizzazione dei branch molto diffuso, pensato per progetti con **cicli di rilascio pianificati** (es. una nuova versione ogni mese) e la necessità di gestire più cose in parallelo: nuove funzionalità, preparazione di una release, correzioni urgenti su produzione.

Git Flow prevede branch con ruoli specifici:

- **main** : contiene sempre il codice in produzione, stabile al 100%.
- **develop** : il branch di “integrazione”, dove confluiscono le funzionalità completate, in attesa della prossima release.
- **feature/...** : branch temporanei per sviluppare singole funzionalità, che partono da **develop** e ci ritornano a lavoro finito.
- **release/...** : branch temporanei usati per finalizzare una versione (test, piccoli fix, aggiornamento numero di versione) prima di pubblicarla.
- **hotfix/...** : branch temporanei per correzioni urgenti direttamente su **main**, per problemi critici in produzione che non possono aspettare il prossimo ciclo di release.

Diagramma 4

Diagramma 4

Quando usare Git Flow: è utile quando il progetto ha rilasci pianificati e non continui (es. release mensili), quando serve mantenere in parallelo più versioni in produzione, o quando il processo di approvazione/rilascio è complesso e richiede una fase di stabilizzazione prima di ogni release. Lo svantaggio è che è un processo più pesante, con più branch da gestire e più passaggi di merge.

Esempio pratico: immaginiamo che ShopFacile rilasci una nuova versione ogni mese. Lo scenario tipico con Git Flow: - Marco, Giulia e Ahmed lavorano tutto il mese su vari branch **feature/...** che partono da **develop** e ci tornano quando pronti; - una settimana prima del rilascio si crea **release/1.4.0** da **develop**: qui si fanno solo piccoli fix e test finali, niente nuove funzionalità; - quando è tutto stabile, **release/1.4.0** viene unito sia in **main** (con tag **v1.4.0**, che diventa la produzione) sia di nuovo in **develop**; - se due giorni dopo il rilascio emerge un bug critico in produzione, si crea **hotfix/bug-pagamenti** direttamente da **main**, si corregge, si unisce in **main** (nuovo tag **v1.4.1**) e anche in **develop**, così il fix non si perde nel prossimo rilascio.

4.12 Trunk Based Development: il workflow semplificato

Git Flow, con i suoi cinque tipi di branch, funziona bene per rilasci pianificati, ma è un processo pesante per chi rilascia molto più spesso: esiste un'alternativa decisamente più snella. Il **Trunk Based Development** (TBD) è un approccio molto più semplice: tutti i membri del team lavorano il più possibile direttamente su **un unico branch principale** (il “trunk”, cioè `main`), con **commit frequenti e piccoli**, integrando il proprio lavoro più volte al giorno.

Analogia: se Git Flow è come costruire un edificio con tanti cantieri separati che vengono collegati alla struttura principale solo a lavori ultimati, il Trunk Based Development è come aggiungere un mattone alla volta direttamente sulla struttura principale, controllando ogni singolo mattone prima di posarlo, così l'edificio resta sempre in piedi e visitabile.

I branch di feature, quando si usano, sono **molto brevi** (durano ore o al massimo un paio di giorni, non settimane), e le funzionalità non ancora pronte per gli utenti finali vengono spesso “nascoste” tramite **feature flag** (interruttori che attivano/disattivano una funzionalità) invece che tenute isolate su un branch a lungo termine.

Diagramma 5

Diagramma 5

Quando usare Trunk Based Development: funziona molto bene quando il team ha una **suite di test automatici solida** e una pipeline di CI/CD matura, capace di verificare rapidamente che ogni piccolo cambiamento non rompa nulla. È il modello preferito dai team che fanno rilasci **continui** (più volte al giorno o alla settimana), tipico della cultura DevOps che vedremo più avanti nel corso. Lo svantaggio è che richiede molta disciplina e automazione: senza test solidi, integrare spesso su un unico branch diventa rischioso.

Esempio pratico: immaginiamo che il team di ShopFacile lavori con Trunk Based Development e una pipeline CI/CD solida. Giulia deve aggiungere un nuovo filtro di ricerca: - crea un branch `feature/filtro-ricerca` la mattina; - fa 2-3 commit piccoli nel corso della giornata, ciascuno verificato automaticamente dalla pipeline; - il filtro non è ancora pronto per tutti gli utenti, quindi lo nasconde dietro un feature flag chiamato `nuovo_filtro_ricerca_attivo` (inizialmente spento); - entro sera apre una MR piccola e veloce da revisionare, e la unisce in `main`; - il codice è ora in produzione ma invisibile agli utenti (flag spento); quando il team è pronto, attiva il flag per tutti (o per un gruppo di test) senza dover rilasciare nulla di nuovo.

Confronto rapido

	Git Flow	Trunk Based Development
Struttura branch	Molti branch a lungo termine (<code>main</code> , <code>develop</code> , <code>feature</code> , <code>release</code> , <code>hotfix</code>)	Un solo branch principale, branch di feature brevissimi
Frequenza integrazione	Bassa (le feature restano isolate a lungo)	Alta (più commit al giorno su <code>main</code>)
Adatto a	Rilasci pianificati, versioni multiple in produzione	Rilasci continui, cultura DevOps, forte automazione
Serve	Processo chiaro e disciplina sui merge	Test automatici solidi, feature flag
Rischio principale	Conflitti di merge grandi e complessi	Rischio di “rompere” main se manca automazione

Non esiste un modello “migliore in assoluto”: la scelta dipende dalla maturità del team, dal tipo di prodotto e dalla frequenza di rilascio desiderata. Molti team reali usano versioni semplificate o ibride di questi due modelli, adattandole al proprio contesto.

4.13 Riepilogo dei comandi visti in questa sezione

Abbiamo visto molti comandi sparsi nei vari esempi di questa sezione: raccogliamoli qui in un unico prontuario, utile come riferimento veloce quando ti troverai a seguire una conversazione tecnica del team.

```
git init                # crea un nuovo repository
git clone <url>         # scarica un repository esistente
git status              # mostra i file modificati
git add <file>          # prepara le modifiche per il commit
git commit -m "messaggio" # salva uno "scatto" delle modifiche
git branch <nome>       # crea un nuovo branch
git checkout <nome>     # mi sposto su un branch
git checkout -b <nome>  # crea e mi sposto su un nuovo branch
git merge <branch>      # unisce un branch in quello corrente
git tag -a <nome> -m "messaggio" # crea un tag annotato
git push origin <branch|tag> # invia branch/tag al repository remoto
```

Ricorda: non serve che tu memorizzi tutti questi comandi a memoria. Ciò che conta, nel tuo ruolo, è capire **cosa rappresentano concettualmente** (uno scatto, un ramo, un'unione) per poter seguire con consapevolezza le conversazioni tecniche del team, leggere lo stato di una Merge Request, e capire perché il team ha scelto un certo workflow.

Esercizi pratici

Gli esercizi che seguono ti servono a consolidare i concetti visti in questa sezione con le mani sulla tastiera. Non serve nessun progetto reale: puoi farli tutti con un account GitLab gratuito e un repository di prova che poi puoi anche eliminare.

- 1. Crea un repository e fai il tuo primo commit.** Crea un repository di prova su GitLab (pubblico o privato, non importa), clonalo sul tuo computer con `git clone`, crea un file `note.txt` con dentro una riga di testo a piacere, poi lancia `git add note.txt` e `git commit -m "Primo commit di prova"`. **Come verificare:** lanciando `git log`, deve apparire il tuo commit con il messaggio che hai scritto e un identificativo esadecimale univoco.
- 2. Crea un branch, modificalo e uniscilo.** Nel repository di prova, crea un branch con `git checkout -b feature/prova`, modifica `note.txt` aggiungendo una seconda riga, fai un commit, poi torna su `main` con `git checkout main` e lancia `git merge feature/prova`. **Come verificare:** dopo il merge, `note.txt` su `main` deve contenere anche la riga che hai aggiunto sul branch, e `git log` deve mostrare il commit del branch unito nella storia di `main`.
- 3. Genera un conflitto di merge e risolvi.** Sempre su `main`, modifica la prima riga di `note.txt` e fai un commit. Poi crea un nuovo branch da un punto *precedente* a quella modifica (o modifica la stessa riga su un secondo branch prima del merge) e prova a fare il merge: dovresti ottenere un `CONFLICT (content)`. Apri il file, scegli quale testo mantenere, rimuovi i marcatori `<<<<<<`, `=====`, `>>>>>>` che Git inserisce, e completa il merge con `git add note.txt` e `git commit`. **Come verificare:** `git status` non deve più segnalare conflitti, e `note.txt` deve contenere solo il testo finale che hai scelto, senza marcatori residui.
- 4. Apri una Merge Request fittizia su GitLab.** Fai il push del branch `feature/prova` (o uno nuovo) con `git push origin feature/prova`, poi su GitLab apri una Merge Request verso `main`, scrivendo un titolo e una breve descrizione di cosa cambia. Se hai un amico o un collega disponibile, chiedigli di lasciarti un commento; altrimenti lasciatelo da solo per vedere come funziona l'interfaccia. **Come verificare:** la Merge Request compare nella sezione "Merge requests" del repository con stato "Open", e mostra correttamente i file modificati nel tab "Changes".
- 5. Crea un tag e una Release.** Sul repository di prova, crea un tag annotato con `git tag -a v0.1.0 -m "Prima versione di prova"`, invialo con `git push origin v0.1.0`, poi su GitLab vai nella sezione "Releases" e pubblica una Release basata su quel tag, con qualche nota di rilascio scritta da te. **Come verificare:** il tag `v0.1.0` compare lanciando `git tag`, e la Release compare nella pagina "Releases" del repository con le note che hai scritto.
- 6. Disegna il workflow del tuo progetto reale.** Chiedi a un collega developer quale workflow (Git Flow, Trunk Based Development, o una via di mezzo) usa il progetto su cui sei inserito/a, e provate insieme a disegnare — anche su un foglio — la sequenza di branch che si crea per una feature "tipica" del progetto, dalla creazione del branch fino al rilascio in produzione. **Come verificare:** sai indicare, senza guardare gli appunti, se il progetto usa (o si avvicina a) Git Flow o a Trunk Based Development, e sai spiegare perché quella scelta si adatta al ritmo di rilascio del progetto.

Collegamenti

- [3. Come nasce un software](#) — dove si inserisce Git nel ciclo di vita di un progetto software
- [5. Agile](#) — il mindset dietro le pratiche di integrazione frequente che abbiamo visto con il Trunk Based Development
- [9. DevOps](#) — la cultura di automazione e rilascio continuo che rende possibile il Trunk Based Development
- [10. Azure DevOps](#) — Repos, l'equivalente di GitLab nella suite Microsoft
- [11. CI/CD](#) — le pipeline automatiche che si attivano su commit e Merge Request
- [16. Glossario](#) — per ripassare velocemente i termini di questa sezione

Risorse

- [Documentazione ufficiale di Git](#)
- [Git — Book \(Pro Git, gratuito e in italiano\)](#)
- [GitLab Docs](#)
- [Learn Git Branching \(esercizi interattivi nel browser\)](#)
- [GitLab Docs — Tutorials](#)
- [About merge requests — GitLab Docs](#)
- [Conventional Commits](#)
- [Semantic Versioning](#)
- [Trunk Based Development](#)
- [A successful Git branching model \(Git Flow, Vincent Driessen\)](#)

5. Agile

Nelle sezioni precedenti hai visto come nasce un software (requisiti, analisi, sviluppo, testing, rilascio, manutenzione) e come Git e GitLab permettano a più persone di lavorare insieme sullo stesso codice. Ora facciamo un passo diverso, ma altrettanto importante: non uno strumento, non una tecnologia, ma un **modo di pensare** e di organizzare il lavoro.

Si chiama **Agile**, e se lavori (o lavorerai) in un team di sviluppo software, è quasi certo che lo sentirai nominare fin dal primo giorno: “il nostro team lavora in modo agile”, “facciamo Scrum”, “usiamo una board Kanban”. Questa sezione ti dà le basi per capire da dove viene questo termine, cosa significa davvero, e perché ha cambiato il modo in cui si costruisce il software negli ultimi 20 anni.

Per rendere i concetti meno astratti, in questa sezione e nelle prossime due (Scrum e Kanban) useremo sempre lo stesso progetto di riferimento: **ShopFacile**, una piattaforma e-commerce (catalogo prodotti, carrello, ordini, pagamenti, sconti e promozioni) sviluppata da un piccolo team interno. Il team è composto da **Marco** e **Giulia** (developer, lei molto attenta a test e qualità), **Ahmed** (developer junior, in crescita), **Sara** (Product Owner, si occupa di business e priorità) e **Luca** (Scrum Master, facilita il lavoro del team). Li ritroverai in quasi tutti gli esempi pratici che seguono.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- spiegare perché è nato l'Agile e da cosa si differenzia rispetto ai modelli precedenti;
- elencare e spiegare con parole tue i 4 valori del Manifesto Agile;
- riconoscere i temi principali dei 12 principi Agile, senza doverli imparare a memoria;
- descrivere cosa significa avere un “mindset agile” nel lavoro quotidiano;
- distinguere concettualmente un approccio Waterfall da un approccio Agile;
- sapere che Scrum e Kanban sono due modi concreti (framework) per applicare l'Agile.

5.1 Perché nasce l'Agile: il problema del modello a “cascata”

Per capire perché è nato l'Agile, dobbiamo prima capire cosa c'era **prima**.

Per decenni, il modo più comune di gestire un progetto software è stato il modello **Waterfall** (in italiano “a cascata”). L'idea è semplice: il progetto si divide in fasi rigide, una dopo l'altra, e si passa alla fase successiva solo quando quella precedente è **completamente** finita. Prima si raccolgono *tutti* i requisiti, poi si fa *tutta* l'analisi, poi si scrive *tutto* il codice, poi si testa *tutto*, e solo alla fine si rilascia il prodotto finito.

Diagramma 1

Diagramma 1

Analogia: il Waterfall è come costruire una casa seguendo un progetto architettonico immutabile, firmato e bloccato fin dal primo giorno. Solo alla fine dei lavori, dopo mesi o anni, la famiglia entra per la prima volta nella casa finita — e solo in quel momento scopre se le misure della cucina erano davvero quelle giuste per le sue esigenze.

Immagina se il team di ShopFacile avesse dovuto raccogliere e bloccare *tutti* i requisiti su catalogo, carrello e pagamenti prima di scrivere una sola riga di codice: qualsiasi promozione lanciata da un concorrente nel frattempo avrebbe reso il progetto già in ritardo sul mercato ancora prima del rilascio.

Il problema di questo approccio è diventato via via più evidente, soprattutto a partire dagli anni '90, quando il software ha iniziato a diventare sempre più complesso e i mercati sempre più rapidi a cambiare:

- **Il cliente vede il risultato troppo tardi.** Se il progetto dura un anno, il cliente scopre il prodotto finito dopo un anno — e nel frattempo le sue esigenze possono essere cambiate, o si accorge che aveva descritto male ciò di cui aveva davvero bisogno.
- **Cambiare idea a metà strada è costosissimo.** In un modello a cascata, tornare indietro (es. modificare un requisito durante lo sviluppo) significa spesso rifare da capo intere fasi già “chiuse”.
- **I rischi si scoprono tardi.** Un problema tecnico grave, o un errore nei requisiti iniziali, viene spesso a galla solo nella fase di testing o, peggio, dopo il rilascio — quando correggerlo costa molto di più che se fosse stato scoperto all'inizio.

Negli anni '90 e nei primi anni 2000, diversi gruppi di sviluppatori (indipendenti tra loro) iniziarono a sperimentare approcci alternativi, più flessibili e incentrati sul consegnare valore al cliente in modo rapido e frequente, piuttosto che pianificare tutto in anticipo nei minimi dettagli. Nacquero così metodi con nomi come Scrum, Extreme Programming (XP), Crystal, e altri — tutti con filosofie simili ma pratiche diverse.

Nel **febbraio 2001**, 17 di questi sviluppatori si incontrarono in una località di montagna (Snowbird, Utah, USA) per discutere cosa avessero in comune i loro approcci. Da quell'incontro nacque un documento breve e diretto: il **Manifesto per lo Sviluppo Agile di Software** (*Manifesto Agile*). Non era un metodo, non era una checklist di regole: era un insieme di **valori e principi condivisi**, un vero e proprio cambio di mentalità. Vediamo subito questi 4 valori nel concreto, applicandoli al team di ShopFacile che ci accompagnerà per il resto della sezione.

5.2 Il Manifesto Agile: i 4 valori fondamentali

Il Manifesto Agile è composto, prima di tutto, da **4 valori**. Sono espressi con una struttura particolare: “preferiamo A rispetto a B”, il che **non significa che B non conti**, ma che, quando bisogna scegliere, A ha la priorità.

Testo originale (in inglese) di riferimento: agilemanifesto.org — esiste anche la versione ufficiale in italiano.

1. Persone e interazioni, più che processi e strumenti

Non significa “i processi e gli strumenti non servono” — Git, Jira, le pipeline di CI/CD sono tutti strumenti utilissimi che vedrai in questo corso. Significa che uno strumento perfetto usato da un team che non comunica bene produce risultati peggiori di un team che comunica bene con strumenti semplici.

Esempio pratico: quando il carrello di ShopFacile mostra un totale sbagliato dopo l'applicazione di un codice sconto, Marco e Giulia preferiscono chiarirsi con una videochiamata di 10 minuti piuttosto che scambiarsi 15 email formali su un sistema di ticket, sperando che il messaggio arrivi comunque chiaro.

2. Software funzionante, più che documentazione esaustiva

Non significa “non scriviamo documentazione” — la documentazione utile (come questo stesso corso!) ha valore. Significa che l'obiettivo primario di un progetto software è **produrre qualcosa che funziona e che le persone possono usare**, non produrre migliaia di pagine di specifiche che nessuno leggerà mai o che diventeranno obsolete in due settimane.

Esempio pratico: a metà di uno sprint, il team di ShopFacile può scegliere di dedicare tempo a completare il flusso di checkout che Sara può già provare sul carrello, invece di scrivere un documento di 40 pagine che descrive nel dettaglio come funzionerà quel flusso.

3. Collaborazione con il cliente, più che negoziazione dei contratti

Non significa “i contratti non servono” — restano necessari, soprattutto in ambito aziendale. Significa che il rapporto con il cliente non dovrebbe limitarsi a “firmiamo un contratto all'inizio e ci rivediamo alla consegna finale”, ma dovrebbe essere una **collaborazione continua**, con feedback frequenti durante tutto il progetto.

Esempio pratico: invece di consegnare il modulo pagamenti di ShopFacile finito dopo 6 mesi e scoprire solo allora che “non era esattamente quello che intendevamo”, il team mostra a Sara (e al cliente, tramite lei) una versione parziale ma funzionante del carrello ogni 2 settimane, raccogliendo feedback che permettono di correggere la direzione strada per strada.

4. Rispondere al cambiamento, più che seguire un piano

Non significa “non pianifichiamo nulla” — pianificare resta fondamentale, e lo vedrai bene nella sezione sul Project Management. Significa che un piano fatto a inizio progetto non può prevedere tutto, e quando emergono nuove informazioni (il mercato cambia, il cliente capisce meglio cosa vuole, si scopre un vincolo tecnico imprevisto), è più intelligente **adattare il piano** che seguirlo alla lettera solo perché “era scritto così”.

Esempio pratico: durante lo sviluppo, un concorrente lancia una funzionalità di sconti aggressivi che cambia le priorità del mercato. Sara può rivedere il backlog di ShopFacile (l'elenco delle attività da fare) e riorganizzare le priorità già alla prossima iterazione, invece di aspettare la fine di un piano annuale immutabile.

Diagramma 2

Diagramma 2

5.3 I 12 principi Agile: raggruppati per temi

I 4 valori che abbiamo appena visto restano un po' astratti se non si traducono in comportamenti concreti: è esattamente questo che fanno i **12 principi** più operativi elencati nel Manifesto. Non serve impararli a memoria uno per uno: è molto più utile raggrupparli per temi, perché in fondo raccontano poche idee ripetute con angolazioni diverse.

Tema 1 — Soddisfazione del cliente al centro

I principi Agile insistono sulla **consegna frequente** di valore reale al cliente, preferendo cicli di poche settimane piuttosto che consegne rare e lontane nel tempo. La priorità più alta è soddisfare il cliente attraverso consegne di valore continue e anticipate.

Esempio pratico: il team di ShopFacile non aspetta di avere “tutto il sito perfetto” per farlo vedere a Sara e al cliente. Consegna prima la funzionalità di ricerca prodotti, poi quella del carrello, poi quella dei pagamenti — ogni pezzo utilizzabile e verificabile appena pronto.

Tema 2 — Collaborazione quotidiana tra le persone coinvolte

I principi sottolineano che le persone di business (chi rappresenta gli interessi del cliente/progetto) e gli sviluppatori devono lavorare **insieme ogni giorno**, non in compartimenti stagni che si parlano solo a inizio e fine progetto. Si valorizzano anche gli incontri di persona (o in videochiamata) come modo più efficace di comunicare rispetto a lunghe catene di documenti scritti.

Esempio pratico: è per questo motivo che la maggior parte dei team Agile fa un breve incontro giornaliero (il “daily standup”, che vedrai nella sezione su Scrum): 15 minuti in cui ognuno racconta cosa ha fatto, cosa farà, e se ha ostacoli — invece di scoprire un problema solo a fine settimana leggendo un report.

Tema 3 — Adattamento al cambiamento

Un principio fondamentale afferma che il cambiamento dei requisiti va **accolto con favore**, anche a stadi avanzati dello sviluppo, perché i processi agili permettono di sfruttarlo a vantaggio del cliente, offrendo un vantaggio competitivo. Questo è un ribaltamento radicale rispetto alla mentalità Waterfall, dove il cambiamento era visto come un problema da evitare.

Esempio pratico: se durante il progetto il cliente si rende conto che una funzionalità pianificata non serve più, e ne serve invece un'altra più urgente (come i famosi sconti aggressivi del concorrente visti in 5.1), il team di ShopFacile riorganizza le priorità del proprio lavoro futuro senza dover “riaprire” un intero progetto formalmente chiuso.

Tema 4 — Semplicità ed efficienza

Un principio recita che “la semplicità — l'arte di massimizzare la quantità di lavoro non svolto — è essenziale”. In altre parole: fare solo il lavoro necessario per raggiungere l'obiettivo, evitando di costruire funzionalità complesse “che potrebbero servire un giorno” ma che nessuno ha davvero richiesto.

Esempio pratico: se un cliente chiede a ShopFacile un modulo per esportare in PDF lo storico dei propri ordini, il team costruisce prima la funzione più semplice che risolve il problema, invece di progettare da subito un sistema di esportazione universale con 20 formati diversi che nessuno userà.

Tema 5 — Team auto-organizzati e motivati

I principi insistono sul costruire progetti attorno a **persone motivate**, dando loro l'ambiente e il supporto di cui hanno bisogno, e fidandosi che sapranno portare a termine il lavoro. Le architetture, i requisiti e i progetti migliori emergono da **team che si auto-organizzano**, cioè che decidono internamente come dividersi il lavoro, piuttosto che ricevere ogni compito imposto dall'alto nei minimi dettagli.

Esempio pratico: invece che Luca assegni manualmente ogni singolo task a Marco, Giulia e Ahmed, il team stesso, durante la pianificazione dello sprint, decide chi si occuperà di cosa in base a competenze e disponibilità — Luca facilita il processo, non lo comanda dall'alto.

Tema 6 — Miglioramento continuo

Un ultimo tema, altrettanto importante: a intervalli regolari, il team deve **riflettere su come diventare più efficace**, per poi mettere in pratica quanto imparato. Questo principio è il seme di quella che nella sezione su Scrum chiamerai “retrospettiva”: una pratica dedicata proprio a imparare dagli errori e dai successi, iterazione dopo iterazione.

Esempio pratico: alla fine di ogni sprint di ShopFacile (ad esempio ogni 2 settimane), Luca facilita una retrospettiva di 30-60 minuti in cui il team si chiede: cosa è andato bene? Cosa possiamo migliorare? Il team prova concretamente a cambiare qualcosa nel ciclo successivo, senza aspettare la fine del progetto per correggere le proprie abitudini.

Diagramma 3

Diagramma 3

5.4 Il “mindset” Agile: più di un metodo, un modo di pensare

Valori e principi sono scritti su carta, ma conoscerli a memoria non basta: quello che conta davvero è come si comportano le persone quando nessuno controlla la checklist. Una delle cose più difficili da capire per chi arriva da fuori (te compreso, oggi) è che **Agile non è una checklist di regole da seguire**. È soprattutto un **mindset**, cioè un atteggiamento mentale che guida le decisioni quotidiane del team.

Cosa significa davvero avere un mindset agile, in pratica?

- **Accettare che il piano iniziale non sarà perfetto, e va bene così.** Un team agile non cerca di prevedere ogni dettaglio a inizio progetto: sa che imparerà cose nuove strada facendo, e costruisce processi che permettono di **adattarsi** invece che resistere al cambiamento.
- **Lavorare per piccoli passi verificabili (iterare).** Invece di lavorare per mesi su qualcosa senza mai mostrarlo a nessuno, il team consegna piccoli pezzi funzionanti spesso, verifica se sono nella direzione giusta, e corregge la rotta rapidamente se serve.
- **Collaborare in modo trasparente, anche quando qualcosa va storto.** Un mindset agile preferisce dire presto “questa cosa non funziona come pensavamo” piuttosto che nascondere fino alla scadenza finale.
- **Puntare all’adattamento continuo, non alla perfezione del piano.** Non è un fallimento se il piano cambia: è un segnale che il team sta imparando e reagendo bene alla realtà, invece di ignorarla per “restare fedele” a un documento scritto mesi prima.
- **Concentrarsi sul valore per l’utente/cliente, non sulla semplice esecuzione di task.** La domanda guida non è solo “abbiamo completato l’attività X?”, ma “questa attività ha davvero portato valore al cliente?”.

Analogia: pensa alla differenza tra seguire una ricetta di cucina alla lettera, ignorando che gli ospiti sono arrivati con un’ora di ritardo e i tempi di cottura non funzionano più, oppure adattarsi in corsa — magari cambiando l’ordine dei piatti — mantenendo l’obiettivo (una cena buona e godibile) più importante del rispetto rigido della procedura scritta. Il mindset agile è proprio questo: l’obiettivo (valore per il cliente) conta più della fedeltà al piano originale.

Esempio pratico: due team lavorano entrambi con “sprint di 2 settimane” e “daily standup” — le stesse pratiche, sulla carta identiche. Nel primo, quando a metà sprint emerge un problema imprevisto, il Product Owner e il team si fermano, ne discutono e ridefiniscono insieme le priorità. Nel secondo, il piano dello sprint resta “intoccabile” fino alla fine, anche quando è ormai chiaro che non porterà valore reale: si segue il rituale alla lettera, ma si è perso il mindset. Solo il primo dei due sta davvero applicando l’Agile — il secondo fa solo “teatro Agile”.

Un errore comune, anche in aziende che si definiscono “agili”, è applicare le **pratiche** (i daily standup, le board Kanban, gli sprint) senza davvero adottare il **mindset** sottostante. Il risultato è quello che nel settore si chiama scherzosamente “fare Agile senza essere agili”: si seguono i rituali nella forma, ma le decisioni continuano a essere prese come in un progetto rigido e tradizionale. Il tuo compito, da futuro Project Manager, sarà anche capire questa differenza e favorire davvero il mindset, non solo il rituale.

5.5 Waterfall vs Agile: il confronto visivo

Dopo aver visto valori, principi e mindset uno per uno, è utile fare un passo indietro e guardare il quadro d'insieme, confrontando direttamente il modello Agile con quello Waterfall visto nella sezione 5.1. Riassumiamo visivamente la differenza principale tra i due approcci: il Waterfall procede in linea retta, fase dopo fase, con un unico rilascio alla fine; l'Agile procede per **cicli brevi e ripetuti** (spesso chiamati iterazioni o sprint), ognuno dei quali produce qualcosa di utilizzabile.

Diagramma 4

Diagramma 4

Alcune differenze chiave riassunte in tabella:

Aspetto	Waterfall	Agile
Struttura	Fasi sequenziali, una alla volta	Cicli brevi e ripetuti (iterazioni)
Quando si vede il risultato	Solo alla fine del progetto	Ad ogni iterazione (es. ogni 2 settimane)
Gestione del cambiamento	Difficile e costosa a metà progetto	Prevista e “benvenuta” come opportunità
Coinvolgimento del cliente	Soprattutto a inizio e fine progetto	Continuo, durante tutto il progetto
Documentazione	Spesso estesa e dettagliata a priori	Leggera, quanto basta, aggiornata nel tempo
Rischio scoperto	Tardi (in fase di testing o dopo il rilascio)	Presto, a ogni iterazione
Adatto a	Progetti con requisiti molto stabili e ben noti (es. costruzioni fisiche, normative rigide)	Progetti con requisiti che possono evolvere, mercati dinamici, software

Non significa che il Waterfall sia “sbagliato” in assoluto: in contesti dove i requisiti sono davvero stabili e immutabili (pensa a certi progetti di ingegneria civile con vincoli normativi fissi), un approccio più sequenziale può avere senso. Ma nella maggior parte dei progetti software moderni — dove i requisiti evolvono, il mercato cambia rapidamente e il feedback degli utenti è preziosissimo — l'approccio Agile si è dimostrato, negli ultimi 20 anni, molto più efficace.

5.6 Dall'Agile ai framework concreti: Scrum e Kanban

Sappiamo ora perché l'Agile è nato e cosa lo distingue dal Waterfall: resta da capire come il team di ShopFacile applica concretamente questi valori nel lavoro di ogni giorno. Un punto importante da chiarire prima di procedere: **Agile non è un metodo con regole precise da seguire passo passo**. È un insieme di valori e principi — quello che abbiamo visto finora. Per applicarlo concretamente nel lavoro di tutti i giorni, negli anni sono nati diversi **framework** (cornici operative, con ruoli, riti e artefatti specifici) che *implementano* la filosofia Agile in modo pratico.

I due framework più diffusi nel mondo dello sviluppo software — e quelli che approfondirai nelle prossime due sezioni di questo corso — sono:

- **Scrum**: un framework strutturato, basato su cicli di lavoro a tempo fisso chiamati **sprint** (tipicamente 1-4 settimane), con ruoli precisi (Scrum Master, Product Owner, Development Team) e riti definiti (sprint planning, daily standup, sprint review, retrospettiva).
- **Kanban**: un framework più fluido, basato su un flusso continuo di lavoro visualizzato su una **board** (bacheca) con colonne come “Da fare”, “In corso”, “Fatto”, con un forte focus sulla limitazione del lavoro in corso (*Work In Progress*) per evitare colli di bottiglia.

Diagramma 5

Diagramma 5

Non sono in competizione tra loro in modo netto: molti team, in realtà, usano versioni ibride (a volte chiamate “Scrumban”) che mescolano elementi di entrambi. Ciò che conta è che **entrambi nascono per applicare gli stessi valori Agile** che hai visto in questa sezione: consegnare valore frequentemente, adattarsi al cambiamento, collaborare in modo trasparente, migliorare continuamente.

Esempio pratico: un team che sviluppa un nuovo prodotto da zero, con requisiti che cambiano spesso e la necessità di pianificare con un certo anticipo il lavoro delle prossime due settimane, troverà probabilmente più utile Scrum, con la sua cadenza regolare a sprint. Un team che si occupa soprattutto di supporto e correzione di problemi — richieste che arrivano in modo imprevedibile, una alla volta, senza un ritmo fisso — troverà invece più naturale Kanban, che non impone di pianificare in blocco cosa fare nelle prossime due settimane.

Nelle prossime due sezioni vedrai nel dettaglio come funzionano davvero, con ruoli, artefatti, riti e strumenti concreti che userai (o osserverai) nel tuo lavoro quotidiano da Junior Project Manager.

Esercizi pratici

Gli esercizi che seguono ti aiutano a consolidare i concetti *teorici* di questa sezione (valori, principi, mindset, Waterfall vs Agile, scelta del framework) prima di passare alla pratica operativa di Scrum e Kanban, che approfondirai nelle prossime due sezioni.

1. **Riscrivi i 4 valori del Manifesto con un esempio tuo.** Per ciascuno dei 4 valori (5.2), scrivi un esempio concreto diverso da quelli di questa pagina — puoi prenderlo dal progetto che osservi, dalla vita universitaria o anche da un’esperienza personale (es. organizzare un evento con altre persone). **Come verificare:** fatti leggere i tuoi 4 esempi da un collega o dalla tua collega Scrum Master/PM e chiedi se colgono davvero il “preferiamo A rispetto a B, ma B resta importante” — non un aut-aut assoluto.
2. **Scegli 3 principi su 12 e trova un esempio “sul campo”.** Tra i 6 temi visti in 5.3, scegline 3 e osserva (o chiedi a un collega di raccontarti) un episodio reale, anche piccolo, del progetto che li illustra — diverso dagli esempi di questa pagina. **Come verificare:** riesci a raccontare a voce, in meno di un minuto per principio e senza guardare gli appunti, sia il principio che l’esempio reale collegato?

3. **Costruisci la tua tabella Waterfall vs Agile su un caso non software.** Prendi un progetto qualsiasi che conosci bene, anche non informatico (organizzare un trasferimento, pianificare un evento, scrivere una tesi) e compila una mini-tabella con almeno 3 righe della tabella di 5.5, valutando se in quel caso specifico converrebbe un approccio più Waterfall o più Agile. **Come verificare:** per ogni riga hai scritto un motivo concreto (non solo “è più veloce” o “è più moderno”) che giustifica la scelta in quel contesto specifico?
4. **Riconosci il “teatro Agile”.** Scrivi 3-4 righe che descrivono una situazione (immaginaria o osservata) in cui un team segue le pratiche agili (sprint, daily, board) ma non il mindset — ispirandoti all'esempio pratico di 5.4 ma con un caso tuo. **Come verificare:** la tua descrizione indica chiaramente sia *quale pratica* viene seguita sia *quale decisione concreta* rivela che il mindset agile in realtà manca.
5. **Scrum o Kanban? Decidi per due scenari diversi.** Immagina due contesti di lavoro molto diversi tra loro (ad esempio: un team che costruisce da zero una nuova funzionalità complessa vs un team che gestisce solo richieste di supporto che arrivano senza un ritmo prevedibile) e per ciascuno indica se propenderesti più per Scrum o per Kanban, motivando con almeno 2 argomenti presi da 5.6. **Come verificare:** la tua motivazione fa riferimento esplicito alle caratteristiche di Scrum e Kanban descritte in questa sezione (cadenza fissa vs flusso continuo, pianificazione a blocchi vs limite di WIP), non solo a una preferenza personale.
6. **Prepara un “elevator pitch” dell’Agile.** Scrivi (o registra a voce) una spiegazione di massimo 5 frasi dell’Agile per una persona che non ha mai sentito questo termine, senza usare la parola “framework” e senza elencare i 12 principi uno per uno. **Come verificare:** fatti ascoltare da qualcuno non tecnico (un amico, un familiare, un collega di un altro reparto) — se in meno di 2 minuti capisce la differenza principale rispetto al “fare tutto e consegnare solo alla fine”, il pitch funziona.

Collegamenti

- [6. Scrum](#) — il framework Agile più diffuso, con sprint, ruoli e riti concreti
- [7. Kanban](#) — il framework Agile basato su flusso continuo e board visuale
- [8. Project Management](#) — come il tuo ruolo di Project Manager si inserisce concretamente in un contesto Agile

Risorse

- [Manifesto Agile \(versione ufficiale in italiano\)](#)
- [I 12 principi del Manifesto Agile \(italiano\)](#)
- [Atlassian — What is Agile?](#)
- [Scrum.org — What is Scrum?](#)
- [Atlassian — Agile vs Waterfall](#)
- [Project Management Institute — What is Agile?](#)

6. Scrum

Nella sezione precedente hai scoperto l'Agile come **filosofia**: un insieme di valori e principi (persone prima dei processi, collaborazione con il cliente, adattamento al cambiamento, consegne frequenti). Ma una filosofia, da sola, non ti dice cosa fare il lunedì mattina quando entri in ufficio.

Serve un **framework**, cioè un insieme di regole concrete, ruoli definiti e riunioni con un nome e uno scopo preciso, che traduca la filosofia Agile in pratica quotidiana. Il framework più usato al mondo per farlo si chiama **Scrum**, ed è quello che il progetto a cui sarai affiancato utilizza (o utilizzerà) ogni giorno.

Questa è la sezione più importante del corso per te, perché **Scrum Master** è il ruolo che andrai a occupare. Ogni concetto che leggerai qui non è teoria astratta: è il tuo futuro lavoro quotidiano.

Analogia iniziale: se l'Agile è la filosofia “cucina leggera, con ingredienti freschi, adattandosi a quello che il mercato offre oggi”, Scrum è la **ricetta strutturata** che dice esattamente quando fare la spesa, quanti piatti preparare, con quale squadra in cucina e quando assaggiare per correggere il sapore prima di servire in tavola.

Per non restare sul piano astratto, in questa sezione seguiremo lo stesso progetto di riferimento delle altre sezioni del corso: **ShopFacile**, la piattaforma di e-commerce (catalogo prodotti, carrello, ordini, pagamenti, sconti) su cui lavora un piccolo team interno. È proprio in questa sezione che i membri di quel team assumono, per la prima volta in modo formale, i tre ruoli di Scrum: **Sara** è la Product Owner, **Luca** è lo Scrum Master — il ruolo che anche tu andrai a occupare — e **Marco, Giulia e Ahmed** compongono il team di sviluppo. Li ritroverai in ogni paragrafo che segue, mentre applicano concretamente Scrum al lavoro quotidiano su ShopFacile.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- spiegare cos'è Scrum e perché è diventato il framework Agile più diffuso;
- distinguere i tre ruoli di Scrum (Product Owner, Scrum Master, Team di sviluppo) e in particolare il ruolo che occuperai tu;
- descrivere i cinque eventi di Scrum (Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective): durata, partecipanti, obiettivo;
- distinguere i tre artefatti (Product Backlog, Sprint Backlog, Increment);
- scrivere e riconoscere una User Story;
- capire cosa sono Story Point e Velocity e perché si usano al posto delle ore/giorni;
- applicare i concetti di Definition of Ready e Definition of Done;
- avere un quadro concreto delle attività quotidiane di uno Scrum Master.

6.1 Cos'è Scrum e perché è ovunque

Scrum è un framework leggero per gestire il lavoro di sviluppo di un prodotto in modo iterativo e incrementale. È stato formalizzato negli anni '90 da Ken Schwaber e Jeff Sutherland (due dei firmatari del Manifesto Agile che hai visto nella sezione precedente) e oggi è, di gran lunga, il framework Agile più utilizzato nel mondo del software — e sempre più anche fuori dal software (marketing, HR, persino organizzazione di eventi).

Il nome “Scrum” viene dal rugby: indica la **mischia**, la fase di gioco in cui l'intera squadra si compatta e avanza insieme, come un blocco unico, spingendo nella stessa direzione. Non è un caso: Scrum nasce proprio dall'idea che un progetto complesso si affronta meglio con una squadra compatta e auto-organizzata, piuttosto che con singoli individui che lavorano ognuno per proprio conto secondo un piano rigido stabilito a tavolino mesi prima.

Perché Scrum ha avuto così tanto successo? Alcuni motivi concreti:

- è **semplice da descrivere** (la Guida ufficiale a Scrum, la “Scrum Guide”, è lunga poche decine di pagine) ma **profondo da padroneggiare**;
- fornisce una **cadenza regolare** (lo Sprint) che dà al team e agli stakeholder un ritmo prevedibile di consegne e verifiche;
- rende **visibile** lo stato del lavoro, riducendo le sorprese a fine progetto;
- si adatta bene a contesti dove i requisiti **non sono chiari al 100% fin dall'inizio** — che, nella pratica, è quasi sempre il caso di un progetto software reale.

Analogia: pensa a una squadra di calcio che prepara la stagione. Non pianifica ogni singola azione di ogni singola partita dell'anno prima che inizi il campionato: pianifica una partita alla volta, si allena, gioca, guarda i risultati, corregge la tattica per la partita successiva. Scrum applica la stessa logica al lavoro di un team: pianifica “una partita alla volta” (lo Sprint), gioca, osserva il risultato, si adatta.

Un punto importante da ricordare: **Scrum non è un metodo per fare tutto e subito più velocemente**. È un modo per rendere visibile e gestibile la complessità, imparando dall'esperienza reale sprint dopo sprint, invece di affidarsi a un piano perfetto scritto a inizio progetto che quasi certamente si rivelerà sbagliato su qualche punto.

6.2 I tre ruoli di Scrum

Scrum definisce esattamente **tre ruoli** (nella terminologia ufficiale più recente si chiamano “accountability”, cioè “responsabilità”, ma nella pratica quotidiana si continua a parlare di ruoli). Non uno di più, non uno di meno. Vediamoli uno per uno.

Diagramma 1

Diagramma 1

6.2.1 Product Owner: la voce del business

Il **Product Owner** (spesso abbreviato **PO**) è la persona responsabile di **massimizzare il valore** del prodotto che il team sta costruendo. In pratica, decide **cosa** il team deve realizzare e **in che ordine di priorità**, rappresentando gli interessi del cliente, degli utenti finali e del business.

Analogia: il Product Owner è come il **capo cuoco che decide il menù** di un ristorante. Non è lui che cucina personalmente ogni piatto (questo lo fa la brigata di cucina, cioè il team di sviluppo), ma decide quali piatti proporre, in quale ordine introdurli nel menù, e quali ingredienti sono davvero prioritari da avere in cucina questa settimana.

Le responsabilità principali del Product Owner:

- gestisce e ordina per priorità il **Product Backlog** (lo vedremo tra poco): decide cosa viene fatto prima e cosa dopo;
- si assicura che il team capisca **perché** una funzionalità è importante, non solo “cosa” va costruito;
- è il punto di contatto principale con il cliente/gli stakeholder per le domande su requisiti e priorità;
- accetta o rifiuta il lavoro completato a fine sprint, verificando che soddisfi quanto richiesto.

Esempio pratico: Sara, Product Owner di ShopFacile, ha in cima al Product Backlog tre voci: “Aggiungere il pagamento con carta di credito”, “Correggere un bug che blocca il login su mobile” e “Aggiungere un filtro di ricerca avanzata”. Nella prossima Sprint Planning decide di dare priorità massima al bug di login (blocca utenti reali ogni giorno), poi al pagamento con carta (richiesto da molti clienti), lasciando il filtro di ricerca più in basso: non è meno utile, ma nessun cliente lo ha ancora richiesto con urgenza.

Un errore comune da evitare: il Product Owner **non è un manager che dà ordini al team** su come lavorare, e non decide **come** tecnicamente realizzare qualcosa — quella è una responsabilità del team di sviluppo. Il PO decide le priorità di business, non l'implementazione tecnica.

6.2.2 Scrum Master: il facilitatore (il TUO ruolo)

Il **Scrum Master** è la persona responsabile di **far funzionare bene il processo Scrum**: si assicura che il team capisca e applichi correttamente la teoria, le pratiche e le regole di Scrum, e lavora attivamente per rimuovere tutto ciò che ostacola il team.

Questo è il ruolo che tu andrai a occupare, quindi vale la pena spenderci più tempo degli altri due.

Analogia: se il Product Owner è il capo cuoco che decide il menù e il team di sviluppo è la brigata che cucina, lo **Scrum Master è il direttore di sala** — o meglio, un allenatore/coach della squadra di cucina. Non cucina lui i piatti, non decide il menù, ma si assicura che: la cucina abbia

tutto il necessario per lavorare senza intoppi, che i tempi tra un piatto e l'altro siano rispettati, che eventuali problemi (un fornitore in ritardo, un forno che non funziona) vengano risolti rapidamente, e che la squadra migliori il proprio modo di lavorare service dopo service.

Punti fondamentali da capire su questo ruolo, perché generano spesso confusione in chi inizia:

- **Lo Scrum Master non è un capo del team.** Non assegna task, non valuta le prestazioni individuali, non decide chi fa cosa. È un ruolo di **servizio**, non di autorità gerarchica.
- **Lo Scrum Master non è un Project Manager tradizionale.** Non tiene un piano di progetto dettagliato con scadenze imposte dall'alto, non fa micro-management delle attività. Il team di sviluppo si auto-organizza: decide da solo come distribuire il lavoro tra i suoi membri.
- **Lo Scrum Master è un servant leader** ("leader al servizio"): il suo potere non viene dal comandare, ma dal facilitare, proteggere e migliorare continuamente le condizioni di lavoro del team.

Le responsabilità concrete di uno Scrum Master, secondo la Scrum Guide, si dividono in tre direzioni:

1. **Verso il team di sviluppo:** aiuta il team ad auto-organizzarsi, a restare focalizzato, a rimuovere gli impedimenti (ostacoli) che incontra, e facilita tutti gli eventi Scrum.
2. **Verso il Product Owner:** aiuta a gestire e comunicare in modo efficace il Product Backlog, aiuta a trovare tecniche per ordinare le priorità in modo chiaro.
3. **Verso l'organizzazione:** guida e aiuta l'azienda ad adottare Scrum correttamente, rimuovendo barriere organizzative che ostacolano il team (es. troppi meeting esterni, dipendenze non chiare da altri team, processi burocratici lenti).

Un impedimento (in inglese *impediment* o *blocker*) è **qualsiasi cosa che rallenta o blocca il team**: un ambiente di test che non funziona, l'attesa di una risposta dal cliente, un accesso non ancora concesso, un conflitto tra due colleghi, un requisito ambiguo. Parte enorme del lavoro quotidiano di uno Scrum Master è **individuare questi ostacoli e lavorare per rimuoverli**, anche quando la soluzione non dipende direttamente da lui/lei (in quel caso, il compito diventa "scalare" il problema alla persona giusta e seguirne la risoluzione).

Esempio pratico: durante il Daily Scrum, uno sviluppatore di ShopFacile dice "sono bloccato da due giorni: aspetto le credenziali di accesso a un ambiente di test". **Luca**, lo Scrum Master, annota l'impedimento, non lo discute lì (per non allungare il daily), e subito dopo scrive al responsabile IT per sollecitare l'accesso, aggiornando il team il giorno seguente. Non risolve lui stesso il problema tecnico: si assicura che chi può risolverlo lo faccia in tempi rapidi.

6.2.3 Team di sviluppo: chi realizza il prodotto

Il **Team di sviluppo** (in inglese *Developers*, termine che nella Scrum Guide più recente indica chiunque contribuisca a realizzare l'Increment: programmatori, ma anche designer, tester, analisti, a seconda del contesto) è il gruppo di persone che **trasforma le voci del Product Backlog in un prodotto funzionante**, sprint dopo sprint.

Analogia: continuando il paragone gastronomico, il team di sviluppo è la **brigata di cucina**: chi taglia le verdure, chi cucina la carne, chi impiatta. Decidono loro, internamente, come organizzarsi per preparare i piatti richiesti nei tempi previsti — nessuno dall'esterno gli dice esattamente “tu tagli la cipolla per primo, tu impiatti per secondo”.

Caratteristiche chiave del team di sviluppo in Scrum:

- è **auto-organizzato**: decide internamente come distribuire il lavoro, nessuno (nemmeno lo Scrum Master) glielo impone dall'esterno;
- è **cross-funzionale**: ha, al suo interno, tutte le competenze necessarie per portare a termine il lavoro senza dover dipendere costantemente da persone esterne al team;
- è tipicamente composto da **3 a 9 persone** (indicazione tipica, non una regola rigida): squadre più piccole comunicano meglio, squadre troppo grandi diventano difficili da coordinare;
- è collettivamente responsabile della qualità del lavoro consegnato — non esiste “il problema è di quel singolo sviluppatore”, esiste “il problema è del team”.

Esempio pratico: lo Sprint Backlog di ShopFacile contiene una User Story che richiede sia lavoro di back-end che di front-end. Nessuno assegna i task dall'esterno: durante lo Sprint Planning **Marco** e **Ahmed** si dividono spontaneamente il lavoro (“io mi occupo dell'API, tu del componente grafico”) e si aggiornano a vicenda nei Daily Scrum successivi, senza bisogno che il Product Owner o lo Scrum Master decidano chi fa cosa.

6.3 Gli eventi di Scrum

Scrum organizza il lavoro attorno a **cinque eventi** con una cadenza fissa e ripetuta. Il contenitore che li racchiude tutti è lo **Sprint**.

Diagramma 2

Diagramma 2

6.3.1 Sprint: il contenitore di tutto

Lo **Sprint** è un periodo di tempo fisso (in inglese si dice *time-boxed*, cioè con una durata massima non superabile) durante il quale il team lavora per produrre un incremento di prodotto potenzialmente utilizzabile.

- **Durata tipica:** da 1 a 4 settimane, più comunemente **2 settimane**. La durata resta **costante** sprint dopo sprint (non si allunga “solo questa volta” perché il lavoro non è finito).
- **Partecipanti:** tutto il team Scrum (Product Owner, Scrum Master, Team di sviluppo).
- **Obiettivo:** consegnare un incremento di valore, utilizzabile e di qualità sufficiente, entro la fine del periodo stabilito.

Analogia: lo Sprint è come una **puntata di una serie TV**: ha una durata fissa, racconta un pezzo di storia autoconclusivo (l'incremento di prodotto), e si inserisce in una stagione più ampia (il progetto). Ogni puntata ha un suo mini-arco narrativo (lo Sprint Goal, l'obiettivo dello sprint) anche se la storia complessiva continua nella puntata successiva.

Dentro ogni Sprint si svolgono, in ordine, gli altri quattro eventi: la Sprint Planning all'inizio, i Daily Scrum ogni giorno, e la Sprint Review e la Sprint Retrospective alla fine.

Esempio pratico: il team di ShopFacile lavora a sprint di 2 settimane, che iniziano sempre di lunedì. Lunedì mattina si tiene la Sprint Planning, ogni mattina alle 9:30 il Daily Scrum, e il venerdì della seconda settimana si tengono, in sequenza, Sprint Review e Sprint Retrospective. Il lunedì successivo inizia già il nuovo sprint.

Visto il contenitore nel suo complesso, entriamo ora nel primo evento che lo apre concretamente: come il team decide, sprint per sprint, cosa finirà davvero dentro quelle due settimane.

6.3.2 Sprint Planning: cosa faremo in questo sprint

La **Sprint Planning** è la riunione che apre ogni Sprint, in cui il team decide **cosa** verrà realizzato nello sprint e, a grandi linee, **come**.

- **Durata tipica:** per uno sprint di 2 settimane, circa **2-4 ore** (in generale, si stima circa un'ora per ogni settimana di durata dello sprint).
- **Partecipanti:** tutto il team Scrum.
- **Obiettivo:** selezionare le voci più prioritarie del Product Backlog che il team ritiene di poter completare nello sprint, e definire lo **Sprint Goal** (l'obiettivo/tema principale dello sprint, in una frase).

Come si svolge in pratica: il Product Owner presenta le voci più in alto nel Product Backlog (quelle a maggiore priorità) e spiega il “perché” di ciascuna. Il team di sviluppo discute, fa domande, e stima quante di quelle voci può realisticamente completare nello sprint, sulla base della propria capacità (spesso guidata dalla Velocity, che vedremo più avanti). Il risultato finale è lo **Sprint Backlog**: l'elenco delle voci scelte per questo sprint, spesso scomposte in task più piccoli e operativi.

Analogia: è come la riunione della squadra prima della partita in cui l'allenatore, insieme ai giocatori, decide la formazione e la tattica di gioco per la partita di questa settimana — non per tutto il campionato, solo per la prossima partita.

Esempio pratico: il team di ShopFacile ha una Velocity media di 25 punti. In Sprint Planning, **Sara** presenta le prime 6 voci del Product Backlog (che valgono, in totale, 34 punti). Dopo discussione, il team si accorge che l'ultima voce (8 punti) è troppo rischiosa da completare

insieme alle altre e la rimanda al prossimo sprint, portando l'impegno totale a 26 punti — vicino alla propria Velocity storica. Lo Sprint Goal che ne esce è: "Permettere ai clienti di completare un ordine dall'inizio alla fine, incluso il pagamento".

Una volta chiuso lo Sprint Backlog, il team di ShopFacile non lo mette in un cassetto: lo esegue giorno per giorno, e il primo strumento con cui lo fa è proprio il Daily Scrum.

6.3.3 Daily Scrum: il check-in quotidiano

Il **Daily Scrum** (spesso chiamato semplicemente "il daily" o "lo stand-up") è una breve riunione quotidiana in cui il team di sviluppo sincronizza il proprio lavoro e pianifica le prossime 24 ore.

- **Durata tipica: massimo 15 minuti**, sempre alla stessa ora e nello stesso luogo (fisico o virtuale) per creare abitudine e ridurre attrito.
- **Partecipanti**: principalmente il team di sviluppo. Lo Scrum Master di solito partecipa per facilitare (soprattutto nelle prime fasi di vita di un team) ma non è tenuto a "guidare" la riunione: idealmente è il team a auto-gestirla. Il Product Owner può partecipare, ma non è obbligatorio.
- **Obiettivo**: ispezionare l'avanzamento verso lo Sprint Goal e adattare il piano di lavoro dei giorni successivi.

Come si svolge in pratica: un formato molto diffuso (anche se non è l'unico previsto) è che ogni persona risponda brevemente a tre domande: "cosa ho fatto ieri per lo Sprint Goal?", "cosa farò oggi?", "ci sono ostacoli che mi bloccano?". Se emerge un impedimento (es. "sono bloccato, aspetto una risposta dal reparto sicurezza"), **non si discute la soluzione durante il daily** (per non farlo diventare una riunione lunga): si segna, e se ne parla subito dopo, in un incontro più ristretto — spesso è esattamente qui che entra in gioco lo Scrum Master, per farsi carico di seguire e risolvere quell'ostacolo.

Esempio pratico — un mini Daily Scrum:

- **Marco**: "Ieri ho finito l'endpoint di login. Oggi lavoro sui test automatici. Nessun blocco."
- **Giulia**: "Ieri ho lavorato sulla pagina del profilo utente. Oggi continuo, ma sono bloccata: aspetto l'accesso all'ambiente di test da ieri."
- **Ahmed**: "Ieri ho aiutato Giulia a capire un errore di configurazione. Oggi riprendo la User Story sul carrello."

Tutto il daily dura meno di 10 minuti. Il blocco di Giulia viene segnato dallo Scrum Master, che se ne occupa subito dopo, fuori dal daily.

Analogia: è come il **check-in veloce dello spogliatoio a metà allenamento**: non si riprogetta la tattica da zero ogni giorno, ci si aggiorna velocemente su chi ha un problema (un infortunio, una difficoltà) e si aggiusta la rotta per il resto della sessione.

Daily dopo daily, il team di ShopFacile arriva così alla fine delle due settimane: è il momento di mostrare a chi non ha seguito il lavoro giorno per giorno cosa è stato davvero costruito.

6.3.4 Sprint Review: cosa abbiamo costruito

La **Sprint Review** è la riunione di chiusura in cui il team mostra il lavoro realizzato durante lo sprint e raccoglie feedback.

- **Durata tipica:** per uno sprint di 2 settimane, circa **1-2 ore** (proporzionale alla durata dello sprint, come la Planning).
- **Partecipanti:** tutto il team Scrum, più — ed è importante — gli **stakeholder** invitati: il cliente, altri reparti, chiunque abbia interesse a vedere l'avanzamento del prodotto.
- **Obiettivo:** ispezionare l'incremento realizzato, discutere cosa è cambiato nel contesto (mercato, priorità, vincoli) e adattare il Product Backlog di conseguenza.

Come si svolge in pratica: il team fa una **demo dal vivo** di ciò che ha realizzato (non slide che descrivono il lavoro: il prodotto funzionante, mostrato in azione). Gli stakeholder fanno domande, danno feedback, a volte propongono nuove idee o cambi di priorità. Il Product Owner tiene traccia di tutto questo per aggiornare il Product Backlog.

Analogia: è come l'**assaggio finale prima di servire il piatto ai clienti del ristorante**: la cucina porta il piatto (l'incremento) al tavolo dei giudici (gli stakeholder), che lo assaggiano, commentano, e magari chiedono "la prossima volta, un po' meno piccante" — feedback che entra direttamente nel menù (Product Backlog) della prossima settimana.

Esempio pratico: il team di ShopFacile mostra dal vivo, sull'ambiente di test, il nuovo flusso di checkout appena completato: uno sviluppatore inserisce un ordine di prova davanti a tutti, mostrando ogni passaggio fino alla conferma. Uno stakeholder nota che manca un messaggio di errore chiaro se la carta di credito viene rifiutata: **Sara** annota la richiesta e la aggiunge, come nuova voce, al Product Backlog per una prossima Sprint Planning.

Un errore comune da evitare come Scrum Master: la Sprint Review **non è una presentazione formale con slide** preparata a parte. È una demo pratica e informale del prodotto, pensata per generare conversazione e feedback reale, non per "fare bella figura".

Con il prodotto mostrato e il backlog aggiornato, resta un'ultima domanda prima di chiudere lo sprint: non cosa è stato costruito, ma come il team ci ha lavorato — ed è esattamente il tema della Sprint Retrospective.

6.3.5 Sprint Retrospective: come possiamo lavorare meglio

La **Sprint Retrospective** (spesso chiamata solo “retro”) è l'ultimo evento dello sprint: il momento in cui il team riflette su **come** ha lavorato (non su cosa ha costruito, quello è già stato discusso nella Review) per trovare modi di migliorare nel prossimo sprint.

- **Durata tipica:** per uno sprint di 2 settimane, circa **1-1,5 ore**.
- **Partecipanti:** solo il team Scrum (Product Owner, Scrum Master, team di sviluppo) — niente stakeholder esterni, per permettere un dialogo aperto e sincero.
- **Obiettivo:** identificare cosa ha funzionato bene, cosa no, e definire **almeno un'azione concreta di miglioramento** da provare nel prossimo sprint.

Come si svolge in pratica: esistono decine di formati per condurre una retrospettiva, ma uno dei più semplici e diffusi si chiama “Start / Stop / Continue”: ogni persona scrive su dei post-it (fisici o su una lavagna digitale) cosa il team dovrebbe **iniziare** a fare, cosa dovrebbe **smettere** di fare, e cosa dovrebbe **continuare** a fare così com'è. Si raggruppano le idee simili, si discutono le più votate, e si scelgono 1-2 azioni concrete da provare nel prossimo sprint (non 15 azioni: meglio poche e realizzabili).

Analogia: è la **riunione tecnica dello staff dopo la partita**, a bocce ferme: non si riguarda solo il punteggio finale, si analizza come si è giocato, cosa ha funzionato nello schema tattico e cosa va corretto per la partita successiva. È il momento in cui una squadra migliora davvero nel tempo, partita dopo partita.

Esempio pratico — risultato di una retrospettiva con il formato Start/Stop/Continue:

- **Start** (iniziare a fare): scrivere una breve nota nella descrizione di ogni Merge Request per spiegare “perché”, non solo “cosa” cambia.
- **Stop** (smettere di fare): accettare nuove richieste urgenti a metà sprint senza discuterne prima con il Product Owner.
- **Continue** (continuare a fare): il pairing tra un developer senior e uno junior sulle User Story più complesse, che ha ridotto gli errori nell'ultimo sprint.

Il team vota le idee più sentite e sceglie una sola azione concreta da provare nel prossimo sprint: introdurre la nota “perché” nelle Pull Request.

Facilitare bene la Retrospective è una delle competenze più importanti di uno Scrum Master: è il principale motore del **miglioramento continuo** del team, uno dei pilastri di Scrum e dell'Agile in generale.

Con la Retrospective si chiude il ciclo degli eventi: il team di ShopFacile riparte da capo con una nuova Sprint Planning, ma quel ciclo, ad ogni giro, produce e trasforma sempre gli stessi tre elementi tangibili. Vediamoli nel dettaglio.

6.4 Gli artefatti di Scrum

Scrum definisce anche **tre artefatti**: elementi tangibili (documenti, liste, prodotti) che rappresentano lavoro o valore, e che rendono trasparente lo stato del progetto a chiunque li guardi.

Diagramma 3

Diagramma 3

6.4.1 Product Backlog: la lista completa dei desideri

Il **Product Backlog** è l'elenco completo, ordinato per priorità, di **tutto ciò che potrebbe servire al prodotto**: nuove funzionalità, modifiche, correzioni di bug, miglioramenti tecnici. È di proprietà e responsabilità del **Product Owner**.

Analogia: è la **lista della spesa generale di casa**, quella che tieni sul frigorifero e continui ad aggiornare: non compri tutto oggi, ma sai cosa ti serve, in ordine di urgenza (il latte è finito, serve subito; le tovagliette nuove possono aspettare).

Esempio pratico: il Product Backlog di ShopFacile potrebbe contenere, tra le tante voci: “Come cliente, voglio salvare più indirizzi di spedizione” (priorità alta, già stimata), “Come cliente, voglio ricevere una notifica quando il mio ordine è spedito” (priorità media), “Come amministratore, voglio esportare un report vendite mensile” (priorità bassa, ancora da approfondire). Le prime sono in cima e ben dettagliate, l'ultima è più in basso e resta volutamente vaga per ora.

Caratteristiche importanti del Product Backlog:

- è **vivo e mai completamente finito**: si aggiornano ed evolve continuamente, man mano che si scopre di più sul prodotto, sul mercato, sui bisogni degli utenti;
- è **ordinato per priorità**: le voci più importanti (quelle che il team affronterà prima) stanno in alto;
- le voci in alto sono di solito più **detagliate e “pronte”** (ne parliamo con la Definition of Ready), mentre quelle in basso possono restare vaghe finché non si avvicina il momento di affrontarle — non ha senso dettagliare oggi qualcosa che verrà realizzato forse tra sei mesi e potrebbe cambiare completamente nel frattempo.

Le voci del Product Backlog sono tipicamente scritte in formato **User Story** (le vediamo nel prossimo paragrafo).

Il Product Backlog di ShopFacile, per quanto ben ordinato, resta però una lista di *possibilità*: solo una piccola parte di quelle voci diventa lavoro concreto in un dato sprint. È esattamente quel sottoinsieme a formare il secondo artefatto.

6.4.2 Sprint Backlog: il piano di questo sprint

Lo **Sprint Backlog** è il sottoinsieme di voci del Product Backlog che il team ha scelto di realizzare **in questo sprint**, insieme al piano per realizzarle (spesso scomposte in task operativi più piccoli) e allo Sprint Goal.

Analogia: se il Product Backlog è la lista della spesa generale di casa, lo Sprint Backlog è la **lista della spesa di oggi**: hai scelto cosa comprare in questo giro, in base a quanto tempo e quanti soldi hai disponibili adesso.

Esempio pratico — uno Sprint Backlog reale potrebbe contenere:

1. Come cliente, voglio salvare più indirizzi di spedizione nel mio profilo (5 punti) — task: creare tabella database, endpoint API, interfaccia utente.
2. Come cliente, voglio ricevere una email di conferma dopo l'acquisto (3 punti) — task: template email, integrazione con servizio di invio.
3. Correggere il bug che duplica il prodotto nel carrello in alcuni casi (2 punti) — task: riprodurre il bug, correggere, scrivere test di regressione.
4. Come amministratore, voglio poter disattivare temporaneamente un prodotto dal catalogo (3 punti) — task: nuovo campo nel database, pulsante in interfaccia admin.

Totale: 13 punti, coerente con la Velocity del team in quello sprint.

Di proprietà del **team di sviluppo** (non del Product Owner): è il team che decide come organizzare il lavoro dentro lo sprint, ed è il team che può aggiornarlo giorno per giorno (ad esempio aggiungendo un task che si scopre necessario durante lo sprint) — sempre restando fedele allo Sprint Goal stabilito in Planning.

Man mano che il team di ShopFacile porta a termine le voci dello Sprint Backlog, quelle righe scritte su una lista si trasformano in qualcosa di concreto e verificabile: è qui che entra in gioco il terzo artefatto, l'Increment.

6.4.3 Increment: il pezzo di prodotto realmente costruito

L'**Increment** è il pezzo di prodotto realmente completato durante lo sprint: funzionante, testato, e — questo è il punto chiave — **conforme alla Definition of Done** del team (ne parliamo tra poco).

Analogia: se il prodotto finale è una casa, ogni Increment è una stanza completamente finita e abitabile — non un muro a metà con i cavi elettrici scoperti. Ogni stanza che si aggiunge deve poter essere già vissuta, anche se la casa nel complesso non è ancora finita.

Esempio pratico: a fine sprint, la funzionalità “salvataggio di più indirizzi di spedizione” di ShopFacile è conforme alla Definition of Done: codice scritto e revisionato, test automatici che passano, verificata manualmente, distribuita in ambiente di test. Questo pezzo di prodotto è l'Increment di quello sprint — anche se **Sara** decide di non rilasciarlo subito agli utenti finali, aspettando di raggrupparlo con altre due funzionalità nella prossima release.

Ogni Increment si somma a tutti quelli realizzati negli sprint precedenti, costruendo progressivamente il prodotto completo. Un punto fondamentale: un Increment deve essere **potenzialmente utilizzabile**, anche se il Product Owner alla fine decide di non rilasciarlo subito agli utenti finali (magari perché si preferisce raggrupparlo con altre funzionalità in un'unica release). “Potenzialmente utilizzabile” significa che, dal punto di vista tecnico e qualitativo, **sarebbe già pronto per andare in produzione**.

Ruoli, eventi e artefatti danno la struttura; ma per capire davvero come una voce del Product Backlog di ShopFacile passa dall'essere un'idea a un compito che il team sa stimare e realizzare, serve guardare al formato con cui quelle voci vengono scritte concretamente.

6.5 User Story: come si descrive un requisito in Scrum

Una **User Story** (letteralmente “storia dell'utente”) è il formato più comune per descrivere una voce del Product Backlog, scritta dal punto di vista di chi utilizzerà quella funzionalità.

Il formato standard è:

Come [tipo di utente/ruolo], **voglio** [azione/funzionalità], **per** [beneficio/motivo].

Esempio concreto:

Come **cliente che sta effettuando un acquisto online**, voglio **poter salvare più indirizzi di spedizione nel mio profilo**, per **non dover reinserire l'indirizzo ogni volta che effettuo un nuovo ordine**.

Perché questo formato è così efficace ed è diventato uno standard:

- costringe a chiarire **chi** beneficia della funzionalità (non è la stessa cosa scrivere una funzionalità per un amministratore o per un cliente finale);
- costringe a esplicitare **il beneficio**, cioè il “perché” — che aiuta tutto il team a capire il valore reale, non solo il compito tecnico da eseguire, e permette anche soluzioni alternative se emergono durante lo sviluppo;
- è **breve e leggibile da chiunque**, anche da chi non ha background tecnico (perfetto per un contesto come il tuo).

Una User Story, di per sé, non è ancora sufficientemente dettagliata per essere sviluppata: quasi sempre si accompagna a **criteri di accettazione** (condizioni precise che permettono di verificare se la storia è stata implementata correttamente). Per la storia sopra, un criterio di accettazione potrebbe essere: *“Il cliente può salvare fino a 5 indirizzi diversi; al momento del checkout può selezionare uno degli indirizzi salvati da un menu a tendina.”*

Le User Story più grandi vengono spesso raggruppate in **Epic** (storie troppo grandi per essere completate in un solo sprint, che vanno scomposte in User Story più piccole prima di poter entrare in uno sprint).

Scrivere bene una User Story, però, risponde solo alla domanda “cosa serve fare e perché”. Resta da rispondere a un'altra domanda, altrettanto concreta per chi pianifica uno sprint: quanto lavoro richiede davvero.

6.6 Story Point: stimare senza usare ore o giorni

Un **Story Point** è un'unità di misura **relativa** che il team usa per stimare quanto “sforzo complessivo” richiede realizzare una User Story, combinando insieme complessità, quantità di lavoro e incertezza/rischio.

Il punto fondamentale da capire — e su cui i neofiti si confondono spesso — è: **i Story Point non sono ore o giorni**. Non esiste una formula fissa del tipo “1 Story Point = 4 ore”. Sono un numero **relativo**: dicono “questa storia è più grande/complessa di quest'altra”, non “questa storia richiederà esattamente questo tempo”.

Analogia: pensa a come stimeresti la difficoltà di alcune scalate in montagna, senza sapere esattamente quanto tempo impiegherai (dipende dal meteo, dalla forma del giorno, da imprevisti): diresti “questa è una scalata facile”, “questa è medio-difficile”, “questa è molto impegnativa e rischiosa”. Non stai dando un tempo in ore, stai dando una valutazione relativa di sforzo e difficoltà — ed è più facile e più affidabile essere d'accordo su “questa è più difficile di quella” che sul tempo esatto in minuti.

Perché il team preferisce i Story Point alle ore/giorni?

- **Le persone sono pessime a stimare il tempo assoluto** (quante volte hai stimato “questo esercizio mi prende 2 ore” e ne ha richieste 5?), ma sono molto più brave a **confrontare due cose** e dire quale è più grande;
- i Story Point si “liberano” dalla velocità specifica di ciascuna persona: una stima in ore cambia da persona a persona (un sviluppatore esperto ci mette 2 ore, un junior magari 6), mentre una stima in punti descrive la complessità del problema, indipendentemente da chi lo risolverà;
- riducono la falsa precisione: dire “questa storia vale 5 punti” comunica onestamente il livello di incertezza, mentre dire “questa storia richiede esattamente 13 ore” dà una falsa sensazione di precisione che quasi mai si rivela vera.

Per stimare, molti team usano la **sequenza di Fibonacci** (o una sua versione semplificata): **1, 2, 3, 5, 8, 13, 21...** I numeri crescono sempre più distanziati tra loro apposta: più una storia è grande, più diventa difficile essere precisi, quindi ha senso avere “scalini” più larghi (la differenza tra una storia da 1 e una da 2 punti deve essere chiara e netta, così come la differenza tra una da 13 e una da 21).

Esempio pratico di sessione di stima (tecnica molto diffusa chiamata *Planning Poker*): il team si riunisce, il Product Owner presenta una User Story, ogni membro del team di sviluppo sceglie in privato una carta con un numero della sequenza di Fibonacci che rappresenta la propria stima, poi tutti scoprono le carte insieme. Se le stime sono simili (es. 5 e 8), si prende una media o si discute brevemente e si converge. Se sono molto diverse (es. 2 e 21), è un segnale che le persone stanno immaginando scenari diversi: si discute finché non emerge un'intesa comune, e spesso proprio questa discussione fa emergere dettagli o rischi nascosti nella storia — che è, in realtà, uno dei benefici più grandi di questa tecnica, al di là del numero finale.

Una User Story troppo grande per essere stimata con sicurezza (es. “vale tra 40 e 100 punti, non sappiamo”) è quasi sempre un segnale che va **scomposta** in storie più piccole prima di poter entrare in uno sprint.

Stimare ogni singola storia è utile sprint dopo sprint, ma il vero valore di quei numeri emerge solo quando li si osserva nel tempo, aggregati: è proprio questo che fa la Velocity.

6.7 Velocity: quanto il team riesce a fare in uno sprint

La **Velocity** (velocità) è il numero medio di Story Point che un team completa in uno sprint, calcolato osservando gli sprint passati.

Esempio concreto: se negli ultimi 4 sprint il team ha completato rispettivamente 23, 27, 25 e 29 Story Point, la sua Velocity media è attorno a **26 punti a sprint** ($(23+27+25+29)/4 = 26$).

Analogia: è come il **ritmo medio di una maratona**: se sai che in allenamento riesci a coprire in media 10 km all'ora, puoi stimare con ragionevole affidabilità quanto tempo ti servirà per completare i 42 km della maratona — non con precisione assoluta (dipende dal giorno, dal meteo, dalla forma), ma con una stima utile per pianificare.

Perché la Velocity è utile, soprattutto per un ruolo come il tuo che dovrà poi parlare anche di pianificazione con il team e gli stakeholder:

- permette di fare **previsioni realistiche**: se il Product Backlog rimanente vale circa 200 punti e la Velocity media è 26 punti a sprint, ci si può aspettare che il lavoro richieda circa 8 sprint ($200/26 \approx 7,7$), un'informazione preziosa per rispondere alla classica domanda del cliente “quando sarà pronto?”;
- aiuta il team, durante la Sprint Planning, a non caricarsi di troppo lavoro in uno sprint (un errore molto comune nei team giovani): se la Velocity media è 26, pianificare uno sprint da 45 punti è un campanello d'allarme;

- diventa più stabile e affidabile nel tempo, man mano che il team accumula storia di sprint passati — nei primi sprint di un team nuovo è normale che la Velocity sia molto variabile.

Un avvertimento importante da Scrum Master: **la Velocity non va mai usata per confrontare team diversi** (“il team A fa 40 punti, il team B solo 20, quindi il team A lavora meglio”) né come metrica di performance individuale. I Story Point sono relativi e specifici di ogni singolo team: un punto per il team A non equivale a un punto per il team B, perché ogni team ha una propria scala interna di riferimento. Usare la Velocity per confrontare team, o peggio persone, è uno degli errori più dannosi che si possano fare con questa metrica, e può spingere i team a “gonfiare” artificialmente le stime — proteggere il team da questo uso scorretto della metrica è anch’esso un compito dello Scrum Master.

Sapere quanto il team riesce a fare non basta, però, se le storie che gli arrivano in Sprint Planning sono ancora ambigue o mal definite: serve un filtro condiviso prima ancora di iniziare a stimarle e pianificarle.

6.8 Definition of Ready: quando una storia può entrare in sprint

La **Definition of Ready** (DoR) è un elenco di criteri condivisi dal team che una User Story deve soddisfare **prima** di poter essere selezionata in una Sprint Planning ed entrare in uno sprint.

Analogia: è come il controllo dei documenti all'imbarco di un aereo: prima di salire a bordo (entrare nello sprint), il biglietto deve avere tutte le informazioni corrette, il documento d'identità deve essere valido, il bagaglio deve rispettare le regole. Se manca qualcosa, non sali a bordo — non perché il viaggio non sia importante, ma perché partire senza tutto il necessario crea problemi a metà volo.

Esempi tipici di criteri di Definition of Ready (variano da team a team, ma un insieme comune potrebbe essere):

- la User Story è scritta secondo il formato standard (Come/Voglio/Per) ed è comprensibile a tutto il team;
- ha criteri di accettazione chiari e verificabili;
- è stata stimata dal team (ha un valore in Story Point);
- le eventuali dipendenze da altri team o sistemi esterni sono note e gestibili;
- è sufficientemente piccola da poter essere completata in un singolo sprint.

Esempio pratico: nel progetto ShopFacile, la User Story “Come cliente, voglio pagare con carta di credito” arriva in Sprint Planning ma il team si accorge che non ha ancora criteri di accettazione chiari (cosa succede se la carta viene rifiutata? quali carte sono supportate?) né una stima. Non soddisfa la Definition of Ready: resta nel Product Backlog, **Sara** la raffina con il team in una sessione di refinement, e potrà entrare in uno sprint successivo, quando sarà davvero “pronta”.

Perché la Definition of Ready è utile: evita che il team scopra, a metà sprint, che una storia era troppo ambigua per essere realizzata bene, causando ritardi, rilavorazioni o discussioni infinite proprio quando ormai il tempo dello sprint è già stato impegnato. È molto meglio scoprire l'ambiguità **prima** che la storia entri nello sprint.

La Definition of Ready presidia l'ingresso di una storia nello sprint; la sua “gemella” presidia invece l'uscita, cioè il momento in cui il team può davvero dire di aver finito.

6.9 Definition of Done: quando una storia è davvero completata

La **Definition of Done** (DoD) è un elenco di criteri condivisi dal team che stabilisce quando un pezzo di lavoro (una User Story, e più in generale l'Increment) può essere considerato **realmente completato** — e non solo “quasi finito” o “finito sulla carta”.

Analogia: è come la checklist di un pilota prima del decollo: non basta che l'aereo “sembri” pronto a occhio. Ci sono controlli precisi e non negoziabili (carburante, strumentazione, comunicazioni) che devono essere tutti spuntati prima di poter dire davvero “pronti al decollo”. Saltare un controllo perché “abbiamo fretta” è esattamente il tipo di scorciatoia che, prima o poi, si paga cara.

Esempi tipici di criteri di Definition of Done (di nuovo, variano da team a team e spesso si arricchiscono nel tempo):

- il codice è stato scritto e sottoposto a code review (vedi la sezione su Git e GitLab — spesso corrisponde a una Merge Request approvata);
- i test automatici sono stati scritti e passano;
- la funzionalità è stata verificata manualmente secondo i criteri di accettazione della User Story;
- la documentazione (se necessaria) è stata aggiornata;
- il codice è stato distribuito almeno in un ambiente di test/staging;
- non introduce bug noti bloccanti.

Esempio pratico — la Definition of Done applicata alla User Story “Come cliente, voglio salvare più indirizzi di spedizione”:

- ☒ Codice scritto e Merge Request approvata da almeno un altro developer
- ☒ Test automatici scritti e superati con successo
- ☒ Funzionalità verificata manualmente rispetto ai criteri di accettazione (salvare fino a 5 indirizzi, selezionarne uno al checkout)
- ☒ Distribuita con successo in ambiente di test
- ☐

Nessun bug bloccante noto

Se anche un solo punto non è spuntato, la storia **non** può essere considerata completata: resta “in corso” fino a che tutti i controlli non sono superati.

Perché la Definition of Done è cruciale, e perché sarà una delle tue responsabilità quotidiane più importanti come Scrum Master: senza una DoD condivisa e rispettata, il team rischia di accumulare **debito tecnico nascosto** — storie segnate come “fatte” che in realtà mancano di test, di qualità, di rifiniture. Questo debito, prima o poi, esplode: bug in produzione, funzionalità che si rompono al primo cambiamento, sfiducia da parte del cliente. Uno dei compiti meno visibili ma più importanti dello Scrum Master è proprio **vigilare che il team non “bari” sulla Definition of Done** solo per dichiarare più storie completate in fretta.

Definition of Ready vs. Definition of Done: non confonderle

	Definition of Ready	Definition of Done
Quando si applica	Prima che una storia entri in sprint	Prima che una storia si consideri completata
Risponde alla domanda	“Siamo pronti a iniziare a lavorarci?”	“Abbiamo davvero finito di lavorarci?”
Se non soddisfatta	La storia resta nel Product Backlog, non entra in sprint	La storia resta “in corso”, non si conta come completata

Ruoli, eventi, artefatti, stime e criteri di qualità: abbiamo visto tutti i pezzi che il team di ShopFacile usa ogni sprint. Vale la pena, prima di chiudere la sezione, vederli riuniti in un unico schema.

6.10 Il flusso completo, in sintesi

Per fissare le idee, ecco come tutti i pezzi visti finora si incastrano insieme in un ciclo che si ripete sprint dopo sprint:

Diagramma 4

Diagramma 4

Questo schema è, in pratica, il lavoro che **Luca** ripete ogni due settimane su ShopFacile. Resta un'ultima domanda, la più concreta di tutte per te: cosa significa, giorno per giorno, essere lo Scrum Master che fa funzionare questo ciclo.

6.11 Il ruolo dello Scrum Master nella pratica quotidiana

Chiudiamo la sezione con la parte più concreta per te: **cosa farai davvero**, ogni settimana, una volta preso in mano questo ruolo. Ecco un elenco realistico di attività quotidiane e ricorrenti:

- **Facilitare gli eventi Scrum:** preparare e condurre Sprint Planning, Daily Scrum, Sprint Review e Sprint Retrospective, assicurandoti che abbiano uno scopo chiaro, rispettino i tempi previsti (*time-box*) e producano risultati concreti (non “riunioni per riunioni”).
- **Tracciare e seguire gli impedimenti:** mantenere una lista visibile degli ostacoli segnalati dal team (spesso emergono proprio nel Daily Scrum), capire chi può risolverli, seguirli fino alla chiusura, e segnalare a manager o altri reparti quelli che richiedono un intervento esterno al team.
- **Aggiornare e mantenere leggibile la board del team** (fisica o digitale, es. su Azure DevOps o Jira): verificare che lo stato delle User Story rifletta la realtà, che nulla resti “bloccato” senza che nessuno se ne accorga, che la board comunichi a colpo d’occhio lo stato reale del lavoro.
- **Vigilare sulla Definition of Done:** aiutare il team a non “accontentarsi” e a non dichiarare completato ciò che, in realtà, manca di qualche passaggio essenziale (test, review, documentazione).
- **Aiutare il Product Owner a preparare un Product Backlog “pronto”:** facilitare sessioni di raffinamento (*refinement*) del backlog, in cui le storie in alto vengono discusse, chiarite e stimate, così da essere pronte per la prossima Planning (Definition of Ready).
- **Proteggere il focus del team:** schermare il team da richieste esterne non pianificate, riunioni superflue, interruzioni che minano la concentrazione durante lo sprint — senza isolare il team dal mondo, ma facendo da filtro intelligente.
- **Monitorare metriche utili** (Velocity, numero di storie completate, eventuali “burndown chart” che mostrano l’avanzamento giorno per giorno dentro lo sprint), non per giudicare le persone, ma per aiutare il team a capire il proprio ritmo e migliorare le previsioni.
- **Coaching del team sull’Agile mindset:** aiutare, nel tempo, il team a interiorizzare i principi Agile visti nella sezione precedente, non solo a “eseguire” meccanicamente i rituali di Scrum.
- **Curare la comunicazione con gli stakeholder:** preparare, insieme al Product Owner, aggiornamenti chiari sull’avanzamento del progetto per il cliente e per il management, spesso appoggiandosi a strumenti come quelli che vedrai nella sezione sul Project Management.
- **Migliorare continuamente il processo del team:** dare seguito concreto alle azioni decise nelle Retrospective, verificando negli sprint successivi se hanno davvero funzionato, e aggiustando se non è così.

Un consiglio pratico per iniziare: nei primi sprint che seguirai, il tuo obiettivo principale non è “cambiare tutto e ottimizzare subito”. È **osservare, capire le dinamiche reali del team** (chi si blocca spesso su cosa, quali riunioni funzionano e quali sono percepite come inutili, dove nascono davvero gli impedimenti) e solo dopo iniziare a proporre piccoli miglioramenti, uno alla volta — esattamente come farebbe il team stesso in una Retrospective ben condotta.

È esattamente questo l’equilibrio che **Luca** ha imparato a trovare lavorando con **Sara, Marco, Giulia e Ahmed** su ShopFacile: lo stesso equilibrio che, da qui in avanti nel corso, dovrai imparare a trovare tu.

Esercizi pratici

Gli esercizi seguenti servono a consolidare quanto visto in questa sezione con qualcosa che puoi davvero fare, non solo leggere. Non serve un progetto reale per farli: puoi usare scenari immaginari, e diversi da quelli già proposti nel piano di studio (sezione 17) — qui l'obiettivo è approfondire con più dettaglio, proprio perché questa è la sezione più importante del corso per il tuo ruolo.

1. **Scrivi 3 User Story** per un'ipotetica app di noleggio di biciclette in condivisione (bike sharing), seguendo il formato Come/Voglio/Per, e aggiungi almeno un criterio di accettazione per ciascuna. **Come verificare:** ogni storia indica chiaramente un tipo di utente (non “come utente” generico, ma es. “come cliente che deve sbloccare una bici”), un beneficio concreto (non solo un'azione tecnica), e almeno un criterio di accettazione scritto in modo specifico e verificabile (non “deve funzionare bene”).
2. **Simula una sessione di Planning Poker** con 2-3 colleghi, amici o familiari (non serve che siano tecnici): scegli 5 attività quotidiane non lavorative (es. “cucinare la cena per 6 persone”, “organizzare una festa a sorpresa”, “traslocare in una nuova casa”) e stimale con la sequenza di Fibonacci (1, 2, 3, 5, 8, 13, 21), scoprendo le carte insieme e discutendo le differenze. **Come verificare:** hai almeno un caso in cui le stime iniziali erano molto diverse tra loro (es. 2 contro 13) e la discussione successiva ha fatto emergere un motivo concreto della differenza (non solo “abbiamo scelto un numero a caso”).
3. **Costruisci uno Sprint Backlog di esempio:** immagina un team con Velocity media di 20 punti e uno sprint di 2 settimane. Scegli 4-5 User Story (puoi riusare quelle scritte nell'esercizio 1 o inventarne altre), assegna a ciascuna una stima in punti, e scrivi anche uno Sprint Goal in una sola frase. **Come verificare:** il totale dei punti scelti è vicino alla Velocity indicata (né troppo sopra né troppo sotto, es. tra 17 e 22 punti), e lo Sprint Goal riassume in una frase il “tema” comune delle storie scelte, non un semplice elenco.
4. **Calcola una Velocity media:** un team fittizio ha completato, negli ultimi 5 sprint, rispettivamente 18, 22, 19, 24 e 21 Story Point. Usa questo numero per stimare quanti sprint servirebbero per completare un Product Backlog residuo di 150 punti. **Come verificare:** la Velocity media calcolata è 20,8 punti (arrotondabile a 21), e la stima è di circa $150/21 \approx 7$ sprint.
5. **Conduci una mini-retrospettiva** di circa 20 minuti con colleghi, amici o familiari su un'attività di gruppo recente qualsiasi (non lavorativa: un viaggio organizzato insieme, un progetto universitario di gruppo, l'organizzazione di un evento), usando il formato Start / Stop / Continue, e scegli una sola azione di miglioramento concreta da provare la prossima volta. **Come verificare:** alla fine hai una sola azione scritta, concreta e verificabile (non “comunicare meglio”, ma qualcosa come “condividere l'itinerario su un documento condiviso almeno 3 giorni prima”).
6. **Scrivi una Definition of Done di 5 punti** per un'attività non informatica (es. “organizzare una cena per 10 persone” o “preparare la presentazione per il primo giorno di lavoro”), poi verifica a fine attività quali punti hai davvero rispettato. **Come verificare:** la tua DoD ha esattamente 5 criteri verificabili con un sì/no (non vaghi come “tutto ok”), e a lavoro concluso sei in grado di dire con certezza quali hai soddisfatto e quali no.

7. **Ricostruisci un impedimento reale:** chiedi alla tua collega Scrum Master/PM di raccontarti un impedimento gestito di recente nel progetto (chi lo ha segnalato, in quale evento è emerso, chi lo ha risolto e in quanto tempo). Poi scrivi in 5 righe come ti saresti comportato tu, da Scrum Master, in quella stessa situazione. **Come verificare:** la tua ricostruzione indica chiaramente in quale evento Scrum è emerso l'impedimento (quasi sempre il Daily Scrum) e chi, concretamente, ha agito per rimuoverlo — non solo “il problema si è risolto da sé”.
-

Collegamenti

- [7. Kanban](#) — un altro framework Agile, spesso complementare o alternativo a Scrum
- [8. Project Management](#) — come gli strumenti di gestione progetto si integrano con la pratica quotidiana di uno Scrum Master
- [16. Glossario](#) — per ripassare velocemente i termini di questa sezione

Risorse

- [Scrum Guide ufficiale \(scrumguides.org\)](https://scrumguides.org)
- [Scrum.org](https://www.scrum.org) — What is Scrum?
- [Atlassian](#) — Agile Coach: cos'è Scrum

7. Kanban

Nella sezione precedente hai visto Scrum: sprint a tempo fisso, ruoli definiti, eventi ricorrenti (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective). Immagina il team di **ShopFacile** appena uscito da una Sprint Retrospective: tra i punti emersi c'è che i bug urgenti segnalati dagli utenti, arrivati a metà sprint senza preavviso, hanno continuato a “saltare la fila” e a scombinate la pianificazione fatta a inizio sprint. Sara e Luca si chiedono se, almeno per questo tipo di lavoro imprevedibile, un flusso continuo non sia più adatto di uno sprint a tempo fisso. È esattamente il problema che Kanban è pensato per risolvere.

Scrum è un framework molto “strutturato”. Kanban parte da un’idea diversa e più semplice: **rendere visibile il lavoro e limitare quanto ne facciamo contemporaneamente**, senza necessariamente imporre sprint o ruoli fissi.

Se Scrum è come organizzare il lavoro a “capitoli” di durata fissa, Kanban è più simile a un nastro trasportatore continuo: il lavoro scorre, una voce alla volta, dall’inizio alla fine, senza tappe temporali predefinite.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- da dove viene il metodo Kanban e perché si chiama così;
- come è strutturata una board Kanban e cosa rappresentano colonne e card;
- cos'è il WIP (Work In Progress) e perché limitarlo migliora il flusso di lavoro del team;
- la differenza tra Lead Time e Cycle Time, con un esempio numerico;
- quando conviene usare Kanban invece di Scrum (o entrambi insieme).

7.1 Da dove viene Kanban

La parola **Kanban** (看板) viene dal giapponese e significa più o meno “cartellino visivo” o “insegna”. Il metodo nasce **non nel software**, ma in fabbrica: negli anni '40-'50 **Toyota** lo introdusse nelle sue linee di produzione automobilistica come parte del sistema di produzione “lean” (cioè “snello”, senza sprechi).

L'idea originale era molto concreta: in una linea di montaggio, ogni reparto usava dei cartellini fisici per segnalare al reparto precedente “ho bisogno di altri pezzi” — invece di produrre pezzi in grandi quantità “a caso” e ammassarli in magazzino (con il rischio di produrne troppi o troppo pochi), si produceva **solo quello che serviva, quando serviva**. Questo riduceva sprechi, magazzini pieni di materiale inutilizzato, e colli di bottiglia nascosti.

Decenni dopo, a partire dagli anni 2000, questa filosofia è stata adattata al lavoro di team software (il nome più associato a questa trasposizione è David J. Anderson). L'idea di fondo resta identica: **rendere visibile il flusso di lavoro e non accumulare più lavoro “in corso” di quanto il team riesca effettivamente a gestire.**

7.2 La board Kanban: colonne e card

Il cuore pratico di Kanban è la **board** (bacheca), divisa in **colonne** che rappresentano le fasi che un elemento di lavoro attraversa, da quando viene richiesto a quando è completato.

La versione più semplice ha tre colonne:

- **To Do** (da fare): lavoro non ancora iniziato;
- **In Progress** (in corso): lavoro su cui qualcuno sta lavorando ora;
- **Done** (fatto): lavoro completato.

Nella pratica reale, quasi nessun team si ferma a tre colonne: le colonne vengono **personalizzate** per rispecchiare le fasi reali del lavoro del team. Un esempio molto comune in un contesto software:

To Do → In Analisi → In Sviluppo → In Test → Done

Ogni **card** (cartellino, che riprende proprio l'idea del cartellino giapponese originale) rappresenta un singolo elemento di lavoro: una user story, un bug da correggere, un task tecnico. La card si sposta da sinistra a destra sulla board, colonna dopo colonna, man mano che avanza. Ogni card contiene tipicamente: un titolo, una breve descrizione, chi ci sta lavorando (assegnatario), e a volte etichette o una stima di dimensione.

Diagramma 1

Diagramma 1

Analogia: pensa alla board come al monitor delle partenze in un aeroporto. Ogni volo (card) parte da “In attesa”, passa per “Imbarco in corso” e arriva a “Partito”. A colpo d'occhio, chiunque guardi il monitor capisce subito lo stato di ogni volo — senza dover chiedere a nessuno “a che punto siamo?”.

Le board Kanban possono essere fisiche (un muro con post-it) o digitali (strumenti come Azure DevOps Boards, Trello, Jira). Nella sezione 10 vedrai come si costruisce concretamente una board Kanban dentro Azure DevOps.

Esempio pratico: immagina la board Kanban del team di **ShopFacile**, con 4 colonne — **To Do, In Sviluppo, In Test, Done** — e in un dato lunedì mattina la situazione è questa:

- **To Do** (6 card): richieste appena arrivate, non ancora prese in carico — es. “Aggiungere validazione campo email”, “Bug: pagina lenta su mobile”.

- **In Sviluppo** (3 card): “Fix errore export PDF” (assegnata a Marco), “Aggiornare libreria di logging” (assegnata a Giulia), “Aggiungere filtro categoria al catalogo” (assegnata ad Ahmed).
- **In Test** (1 card): “Nuovo filtro ricerca prodotti”, in verifica dal tester.
- **Done** (3 card, completate questa settimana): “Fix bottone disabilitato”, “Aggiornamento pagina contatti”, “Correzione traduzione IT”.

Guardando solo questa riga di numeri (6 / 3 / 1 / 3), un Project Manager può già farsi una prima idea del flusso: c'è più lavoro in attesa (6) di quanto il team ne stia lavorando attivamente (3+1=4) — non è necessariamente un problema, ma è un segnale da tenere d'occhio se il numero in “To Do” continua solo a crescere.

Avere una board così, visibile a tutti, è già un grande passo avanti — ma non basta da solo: bisogna anche controllare **quanto lavoro il team si mette a fare contemporaneamente**, non solo renderlo visibile. È qui che entra in gioco il concetto di WIP.

7.3 WIP: il lavoro “in corso” e i suoi limiti

WIP è l'acronimo di **Work In Progress**, cioè “lavoro in corso”: il numero di elementi che, in un dato momento, sono già iniziati ma non ancora completati.

Uno dei principi cardine di Kanban è il **limite di WIP**: si stabilisce un numero massimo di card che possono trovarsi contemporaneamente in una determinata colonna (tipicamente le colonne di lavoro attivo, come “In Sviluppo” o “In Test”). Se una colonna ha già raggiunto il suo limite, **nessuno può iniziare una nuova card in quella colonna** finché non se ne libera una completandola e facendola avanzare.

Detto così può sembrare controintuitivo: perché limitare volontariamente quanto lavoro il team può iniziare? La risposta è che **fare tante cose contemporaneamente, in realtà, le fa fare tutte più lentamente**.

Analogia: pensa al casello di un'autostrada durante un weekend di esodo. Se tutte le macchine potessero entrare in autostrada contemporaneamente senza controllo, si formerebbe un ingorgo che blocca *tutti*, e alla fine le macchine arriverebbero più tardi di quanto arriverebbero se il casello facesse passare le auto in modo controllato, poche alla volta. È lo stesso principio di un **imbuto**: se versi il liquido troppo in fretta, l'imbuto si intasa e in realtà il liquido scende **più lentamente** rispetto a versarlo con un flusso costante e misurato.

Nel lavoro di un team succede la stessa cosa: se ognuno ha 6 task “aperti” contemporaneamente, continua a passare da uno all'altro (context switching), nessun task avanza davvero in fretta, e la sensazione di “essere sempre indaffarati” nasconde il fatto che **poco viene realmente completato**. Limitare il WIP forza il team a **finire prima di iniziare altro**, il che in pratica fa terminare più lavoro, più velocemente, con meno errori (perché ci si concentra su meno cose alla volta).

Diagramma 2

Diagramma 2

Un segnale tipico da Project Manager/Scrum Master: se vedi una colonna della board costantemente “esplosa” oltre il suo limite di WIP, è un sintomo concreto di un **collo di bottiglia** nel flusso di lavoro del team, su cui vale la pena indagare (manca una competenza? una fase richiede un’approvazione lenta? c’è troppa dipendenza da una sola persona?).

Esempio pratico: nel team di ShopFacile, la colonna “In Sviluppo” ha un limite di WIP fissato a **3**. Martedì la colonna ha già 3 card (A, B, C) e **Ahmed**, che ha appena finito un task, vorrebbe iniziarne uno nuovo, la card D. Con il limite di WIP attivo, **non può farlo**: deve prima aiutare a completare A, B o C (magari facendo code review a un collega, o testando manualmente una delle tre) e farla avanzare in “In Test”. Solo quando una delle tre card lascia la colonna, si “libera uno slot” e la card D può entrare. Il risultato pratico è che Ahmed, invece di aprire un quarto fronte, spinge a chiudere qualcosa che è già a metà — ed è esattamente l’effetto voluto.

Limitare il WIP non serve solo a evitare che il team apra troppi fronti contemporaneamente: come vedremo subito, avere poche card “in corso” alla volta rende anche possibile **misurare con precisione** quanto tempo impiega davvero una card ad attraversare la board, dall’ingresso in To Do al completamento.

7.4 Lead Time e Cycle Time

Kanban, essendo un metodo orientato al “flusso” continuo di lavoro, si misura soprattutto con due metriche di tempo, spesso confuse tra loro ma concettualmente diverse. Vediamole applicate alla board di ShopFacile che abbiamo appena descritto.

Lead Time

Il **Lead Time** è il tempo totale che passa **dal momento in cui una richiesta viene fatta** (la card entra nella colonna “To Do”, cioè nel backlog) **al momento in cui viene completata** (la card arriva in “Done”). È il tempo che interessa di più a **chi ha fatto la richiesta** (un cliente, uno stakeholder): “quanto tempo devo aspettare per avere questa cosa?”.

Cycle Time

Il **Cycle Time** è il tempo che passa **dal momento in cui il team inizia effettivamente a lavorarci** (la card entra in “In Progress” o “In Analisi”) **al momento in cui viene completata**. È il tempo che interessa di più al **team**, perché misura quanto efficiente è il lavoro una volta avviato, senza contare l’attesa in coda.

La differenza pratica è quindi il tempo di **attesa in coda**, prima che qualcuno inizi effettivamente a lavorare sulla richiesta.

Esempio numerico

Immagina questa card: “Aggiungere filtro di ricerca alla pagina prodotti”.

- **Lunedì 3**: la richiesta viene registrata e messa in “To Do”.
- **Giovedì 6**: uno sviluppatore inizia effettivamente a lavorarci (“In Progress”).
- **Lunedì 10**: il lavoro è completato e la card arriva in “Done”.

Calcoliamo:

- **Lead Time** = da lunedì 3 a lunedì 10 = **7 giorni**
- **Cycle Time** = da giovedì 6 a lunedì 10 = **4 giorni**

La differenza (3 giorni) è il tempo che la richiesta ha passato “in coda”, in attesa che qualcuno se ne occupasse.

Diagramma 3

Diagramma 3

Rappresentazione a barre, per confrontare visivamente i due intervalli:

Diagramma 4

Diagramma 4

Perché la distinzione conta per un Project Manager: se il cliente si lamenta che “le richieste ci mettono troppo ad arrivare”, devi capire **dove** si perde tempo. Se il Cycle Time è basso ma il Lead Time è alto, il problema non è che il team lavora lentamente: è che le richieste restano troppo tempo in coda prima che qualcuno le prenda in carico. Sono due problemi diversi, con soluzioni diverse (il primo si risolve migliorando l'efficienza del team, il secondo migliorando come si priorizza e si smista il lavoro in arrivo).

Esempio pratico: confrontiamo due card diverse della stessa settimana.

- Card “Bug critico: pagamento non va a buon fine” — entra in To Do **martedì 4** alle 9:00, un developer la prende in carico **subito** (martedì 4, 9:30) perché è urgente, ed è completata **mercoledì 5**. Lead Time $\approx$ Cycle Time $\approx$ **1 giorno**: nessuna attesa in coda.
- Card “Migliorare testo pagina ‘Chi siamo’” — entra in To Do **martedì 4**, ma essendo bassa priorità resta in coda per **12 giorni**; qualcuno la prende in carico **giovedì 16** e la completa lo stesso giorno in 2 ore. Lead Time = **12 giorni**, Cycle Time = **meno di 1 giorno**.

Stesso team, stessa efficienza di esecuzione (Cycle Time basso in entrambi i casi) — ma un Lead Time completamente diverso, perché dipende da **quanto la card ha aspettato in coda**, non da quanto ci ha messo il team a farla una volta iniziata.

Lead Time e Cycle Time sono le metriche con cui Kanban misura il flusso continuo di lavoro; Scrum, che lavora a sprint, misura invece l'avanzamento con una metrica diversa, la Velocity. Vale la pena confrontare i due approcci fianco a fianco, per capire quando ha senso usare l'uno o l'altro.

7.5 Kanban vs Scrum: quando usare cosa

Kanban e Scrum condividono lo stesso spirito Agile (visto nella sezione 5), ma organizzano il lavoro in modo diverso.

	Scrum	Kanban
Struttura del tempo	A sprint di durata fissa (es. 2 settimane)	Flusso continuo , senza intervalli di tempo fissi
Consegna del lavoro	Un “pacchetto” di elementi consegnato a fine sprint	Ogni elemento viene consegnato appena è pronto
Ruoli	Ruoli definiti (Scrum Master, Product Owner, Team)	Nessun ruolo obbligatorio specifico
Eventi ricorrenti	Planning, Daily, Review, Retrospectiva	Nessun evento obbligatorio (anche se molti team fanno comunque una daily)
Cosa si limita	La quantità di lavoro pianificata per lo sprint (lo Sprint Backlog)	Il WIP, cioè quante card possono essere “in corso” in ogni momento
Metriche tipiche	Velocity (punti completati per sprint)	Lead Time, Cycle Time, Throughput
Cambiamenti a metà lavoro	Scoraggiati a metà sprint (si aspetta il prossimo)	Benvenuti in qualsiasi momento, il flusso è continuo

Quando usare Scrum: quando il lavoro si presta a essere pianificato in blocchi (sprint), il team vuole momenti regolari di revisione e retrospettiva, e serve una cadenza predicibile per pianificare e comunicare con gli stakeholder (es. “a fine sprint mostriamo una demo”).

Quando usare Kanban: quando il lavoro arriva in modo **imprevedibile e continuo**, con priorità che cambiano spesso — è il caso tipico di team di **supporto, manutenzione, gestione incident** o di piattaforme DevOps, dove non ha molto senso “pianificare due settimane fisse” perché una richiesta urgente (es. un bug critico in produzione) può arrivare in qualsiasi momento e deve poter “saltare la fila”.

Scrumban: il meglio dei due mondi

Molti team reali non scelgono in modo rigido l'uno o l'altro, ma **combinano i due approcci**, in un ibrido informalmente chiamato **Scrumban**: si mantiene la cadenza degli sprint e degli eventi Scrum (planning, review, retrospettiva) per la pianificazione e la comunicazione, ma si gestisce il lavoro **dentro** lo sprint

con una board Kanban e limiti di WIP, per governare meglio il flusso quotidiano. È un approccio molto comune nei team che gestiscono sia nuovo sviluppo pianificato sia richieste impreviste (bug urgenti, richieste di supporto).

Diagramma 5

Diagramma 5

Chiudiamo qui il confronto con Scrum: riprendiamo in breve i punti chiave di questa sezione su Kanban.

7.6 Riepilogo

- Kanban nasce nella produzione industriale (Toyota, filosofia “lean”) e si è poi adattato al lavoro dei team software.
- La **board** con le sue colonne rende visibile lo stato di ogni elemento di lavoro (card); le colonne possono essere personalizzate per riflettere il processo reale del team.
- Limitare il **WIP** (lavoro in corso) migliora il flusso complessivo: meno cose iniziate contemporaneamente, più cose finite rapidamente.
- **Lead Time** misura l'attesa dal punto di vista di chi richiede; **Cycle Time** misura l'efficienza del team una volta che il lavoro è iniziato.
- Scrum lavora a **sprint**, Kanban lavora a **flusso continuo**: la scelta dipende dal tipo di lavoro del team, e i due approcci si possono anche combinare (**Scrumban**).

Esercizi pratici

1. **Disegna una board a 4 colonne.** Su carta o con un tool gratuito online, crea una board con le colonne **To Do** → **In Sviluppo** → **In Test** → **Done** e inventa 6 card realistiche per un progetto a tua scelta (anche immaginario, es. un sito di prenotazioni). Distribuisci le card nelle colonne come faresti se fossi tu a gestire il flusso in questo momento. **Come verificare:** se riesci a spiegare a voce, per ciascuna card, “perché è in quella colonna e non in un'altra”, l'esercizio è fatto bene.
2. **Applica un limite di WIP e osserva l'effetto.** Riprendi la board dell'esercizio 1 e imposta un limite di WIP di 2 sulla colonna “In Sviluppo”. Se hai messo più di 2 card in quella colonna, decidi quali spostare (indietro in To Do, o avanti se sono davvero pronte) per rispettare il limite. **Come verificare:** alla fine, la colonna “In Sviluppo” deve contenere esattamente 2 card (o meno), e devi saper indicare quale card verrebbe “sbloccata” per prima quando una delle due card attuali viene completata.
3. **Calcola Lead Time e Cycle Time con date a tua scelta.** Inventi una card con tre date: quando entra in “To Do”, quando qualcuno la prende in carico (“In Sviluppo”), quando viene completata (“Done”). Calcola a mano Lead Time e Cycle Time, ed evidenzia quanti giorni sono di “attesa in

coda”. **Come verificare:** Lead Time deve sempre essere **maggiore o uguale** al Cycle Time (mai il contrario) — se il tuo calcolo dà un Cycle Time più alto del Lead Time, hai invertito una data.

4. **Osserva la board reale del team.** Chiedi a un collega di farti vedere la board Kanban (o Scrum) usata realmente nel progetto: annota quante colonne ha, se ci sono limiti di WIP visibili e su quali colonne, e quante card ci sono in ciascuna colonna in questo momento. **Come verificare:** sei in grado di riportare, senza guardare di nuovo la board, il nome esatto di tutte le colonne e almeno un esempio di card per ciascuna.
5. **Individua un possibile collo di bottiglia.** Guardando la stessa board reale (o quella dell'esercizio 1), individua se c'è una colonna con più card delle altre, e formula un'ipotesi sul perché (manca una competenza? un'approvazione lenta? una sola persona sovraccarica?). **Come verificare:** prova la tua ipotesi con un collega o con la tua collega Scrum Master/PM — se conferma (anche parzialmente) la tua lettura, hai capito il concetto.
6. **Confronta Scrum e Kanban su un caso reale.** Pensa a un tipo di richiesta che arriva spesso nel progetto (es. un bug urgente in produzione, oppure una nuova feature pianificata) e scrivi 2-3 righe su quale dei due approcci (Scrum a sprint, o Kanban a flusso continuo) si adatterebbe meglio a gestirla, e perché. **Come verificare:** la tua risposta deve citare almeno uno dei criteri visti nella tabella della sezione 7.5 (es. prevedibilità del lavoro, necessità di “saltare la fila”, cadenza di consegna).

Collegamenti

- [8. Project Management](#) — come usare le metriche di flusso (Lead Time, Cycle Time, Throughput) per monitorare un progetto
- [10. Azure DevOps](#) — come costruire e configurare concretamente una board Kanban con Azure DevOps Boards

Risorse

- [Kanbanize — What is Kanban?](#)
- [Atlassian — Kanban](#)
- [Atlassian — Lead Time vs Cycle Time](#)
- [Atlassian — What is WIP limit?](#)
- [Kanban University — What is Kanban?](#)
- [Scrumban — Definizione ed esempi](#)

8. Project Management

Hai già visto Agile, Scrum e Kanban: sai quindi **come** un team organizza il lavoro giorno per giorno (sprint, board, colonne, cerimonie). Questa sezione affronta un livello diverso, complementare: quello del **Project Management**, cioè l'insieme di strumenti e pratiche che permettono di tenere sotto controllo un progetto nel suo complesso — chi è coinvolto, cosa può andare storto, a che punto siamo, e come lo comunichiamo a chi non è nella daily standup ogni giorno.

Attenzione a un equivoco comune: il Project Management **non è** la pianificazione rigida “a cascata” (Waterfall) che magari hai studiato all'università, con Gantt chart bloccati e fasi sequenziali immutabili. In un contesto Agile/DevOps, il project management convive con l'iterazione continua e l'incertezza: si pianifica, sì, ma a più livelli e con la consapevolezza che il piano cambierà. Il ruolo del Project Manager (o di chi ne fa le funzioni, come uno Scrum Master “aumentato” o un Delivery Manager) è più simile a un **navigatore che aggiorna la rotta ogni giorno** che a un ingegnere che disegna un progetto immutabile su carta.

Anche in questa sezione useremo come filo conduttore il team di **ShopFacile**, la piattaforma e-commerce che hai già incontrato: **Sara** (Product Owner) sarà la voce del business tra gli stakeholder, **Luca** (Scrum Master) seguirà da vicino processo e rischi — RAID Log, RACI, monitoraggio — mentre **Marco, Giulia e Ahmed**, il team di sviluppo, daranno concretezza agli esempi di KPI e reportistica.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- identificare gli **stakeholder** di un progetto software e capire perché vanno mappati;
- usare un **RAID Log** per tenere traccia di rischi, assunzioni, problemi e dipendenze;
- leggere e costruire una **matrice RACI** per chiarire le responsabilità;
- distinguere una **milestone** da una semplice scadenza;
- valutare un **rischio** in termini di probabilità e impatto, e capire come si mitiga;
- capire come funziona la **pianificazione a più livelli** in un contesto Agile (roadmap, release plan, sprint plan);
- riconoscere gli strumenti di **monitoraggio** dell'andamento di un team (burndown/burnup, stand-up, dashboard);
- conoscere i principali **KPI** usati in un team Agile/DevOps;
- distinguere i diversi livelli di **reportistica**, a seconda di chi la legge.

8.1 Stakeholder: chi ha interesse nel progetto

Uno **stakeholder** (letteralmente “portatore di interesse”) è **qualsiasi persona o gruppo che può influenzare il progetto, oppure che ne è influenzato**. Non sono solo “quelli che pagano”: sono tutti quelli che, in un modo o nell'altro, hanno voce in capitolo o subiscono le conseguenze delle decisioni prese.

Analogia: organizzare un progetto software senza mappare gli stakeholder è come organizzare un matrimonio pensando solo agli sposi. In realtà ci sono genitori che pagano parte del conto, invitati con esigenze alimentari particolari, il fotografo che deve essere informato sugli orari, il comune che deve validare i documenti. Se ignori qualcuno di questi soggetti, il giorno dell'evento emergono problemi che potevi prevedere con un minimo di mappatura.

In un progetto software tipico, gli stakeholder principali sono:

- **Il cliente** (o il committente): chi ha commissionato il progetto e ne finanzia lo sviluppo. Vuole vedere valore consegnato, rispetto dei tempi e dei costi concordati.
- **Il management** (interno all'azienda che sviluppa): responsabili di linea, direttori, chi deve rendere conto a sua volta di come vanno i progetti del team.
- **Gli utenti finali:** le persone che useranno concretamente il software ogni giorno. Spesso non partecipano alle decisioni, ma sono chi subisce più direttamente le conseguenze di scelte sbagliate (un'interfaccia scomoda, una funzionalità mancante).
- **Il team:** sviluppatori, tester, designer, chiunque contribuisca alla realizzazione. Sono stakeholder anche loro: un carico di lavoro insostenibile o requisiti ambigui li riguardano direttamente.
- **Altri team o reparti:** un team di sicurezza che deve validare il rilascio, un team di infrastruttura che gestisce i server, un ufficio legale che deve approvare termini contrattuali.

Una mappatura semplice, spesso usata nella pratica, classifica gli stakeholder in base a due dimensioni: **quanto potere/influenza hanno** sul progetto e **quanto interesse** hanno nel suo esito.

Diagramma 1

Diagramma 1

Esempio pratico: nel progetto ShopFacile per una nuova funzionalità di pagamento online, il cliente/committente ha alto potere e alto interesse (va coinvolto attivamente in ogni decisione importante) — nella pratica di ShopFacile è **Sara**, la Product Owner, a rappresentare questa voce del business nelle decisioni quotidiane; il team di sicurezza interno ha alto potere ma interesse più episodico (va coinvolto nei momenti critici, come la validazione prima del rilascio in produzione); gli utenti finali hanno alto interesse ma poco potere diretto sulle decisioni (vanno rappresentati tramite ricerche utente, feedback, test di usabilità).

Perché questo conta per un Junior Project Manager? Perché la causa più comune di problemi in un progetto non è “il codice non funziona”, ma **qualcuno importante non è stato informato o coinvolto al momento giusto**. Mappare gli stakeholder all'inizio ti evita sorprese a metà progetto — ma sapere *chi* coinvolgere non basta se non hai anche un posto dove tracciare cosa può andare storto: è il compito del RAID Log, che vediamo subito.

8.2 RAID Log: la mappa dei problemi (prima che diventino problemi)

RAID è un acronimo che raggruppa quattro categorie di informazioni che un team di progetto deve tracciare costantemente:

- **R — Risks** (Rischi): eventi che *potrebbero* accadere e che, se accadono, avrebbero un impatto negativo sul progetto.
- **A — Assumptions** (Assunzioni): cose che stiamo dando per scontate, senza averle verificate al 100%, per poter procedere con la pianificazione.
- **I — Issues** (Problemi): cose che *sono già accadute* e stanno attivamente impattando il progetto ora.
- **D — Dependencies** (Dipendenze): elementi esterni al controllo diretto del team, di cui il progetto ha bisogno per procedere.

Analogia: il RAID Log è come il cruscotto di un'automobile durante un lungo viaggio. Il rischio è la spia della benzina che si accende prima che tu resti a secco (un avviso preventivo). L'assunzione è il presupposto "ci sarà un distributore aperto lungo la strada" — non l'hai verificato, ma stai viaggiando come se fosse vero. Il problema è il pneumatico che si è già bucato: è già successo, va gestito subito. La dipendenza è il fatto che, per attraversare un ponte, devi aspettare che apra alle 8, che tu lo voglia o no: non lo controlli, ma ne dipendi.

Un esempio concreto di ciascuna categoria, per un progetto software reale:

Categoria	Esempio concreto
Risk	"C'è il rischio che il carico di traffico previsto per il Black Friday superi la capacità attuale dei server, causando lentezza o down del sito." (non è ancora successo, ma è possibile)
Assumption	"Stiamo pianificando la release assumendo che l'ambiente di test resti disponibile per tutto il mese; non abbiamo una conferma scritta dal team infrastruttura."
Issue	"Il servizio di invio email di terze parti è down da ieri: gli utenti non ricevono le email di conferma registrazione." (è già un problema attivo, in corso)
Dependency	"Il rilascio della nuova funzionalità di pagamento dipende dall'approvazione di sicurezza da parte di un team esterno al progetto, previsto per la settimana prossima."

Nella pratica, un RAID Log è semplicemente una tabella condivisa (spesso in un foglio, in una board dedicata, o in strumenti come Azure DevOps, che vedremo nella sezione 10), aggiornata regolarmente, con colonne tipo: descrizione, categoria, responsabile, impatto, stato, data di apertura, azione prevista.

Un aspetto importante da capire: le **assunzioni**, se non verificate per tempo, diventano facilmente **rischi**; e i **rischi**, se non mitigati per tempo, diventano **issue** attive. Il RAID Log serve proprio a intercettare questa “scala di gravità” prima che il problema esploda a valle.

Diagramma 2

Diagramma 2

Esempio pratico: un estratto reale (semplificato) di RAID Log a metà dello Sprint 6 di ShopFacile, così come potrebbe apparire in una board condivisa con il team. In ShopFacile non esiste un Project Manager dedicato: le responsabilità di “Tech Lead” e “Project Manager” viste in tabella sono spesso portate avanti da **Luca**, insieme al team, accanto al suo ruolo di Scrum Master:

ID	Categoria	Descrizione	Responsabile	Impatto (1-5)	Stato	Aperto il
R-12	Risk	Il fornitore di hosting ha comunicato una finestra di manutenzione non pianificata nella settimana del rilascio	Luca (Tech Lead)	4	Aperto	03/06
R-13	Risk	Se non testiamo il carico prima del Black Friday, rischio di rallentamenti sotto picco di traffico	Giulia (QA Lead)	5	In mitigazione	05/06
A-07	Assumption	Stiamo assumendo che l'ambiente di staging resti disponibile fino a fine mese	Luca (Project Manager)	3	Da verificare	01/06
A-08	Assumption	Assumiamo che Sara fornisca il feedback sul design entro 3 giorni lavorativi	Luca (Project Manager)	2	Da verificare	04/06
I-04	Issue	Il servizio di invio SMS di terze parti restituisce errori 500 dal 06/06	Marco (Sviluppatore backend)	4	In corso	06/06
D-05	Dependency	Il rilascio del modulo pagamenti dipende dall'approvazione del team sicurezza, prevista per il 12/06	Luca (Tech Lead)	5	In attesa	02/06

Con soli 6 righe, un Project Manager junior può già farsi una domanda utile: **quali elementi con impatto alto (4-5) non hanno ancora un'azione di mitigazione chiara?** In questo estratto sono tre (R-13, I-04, D-05): sono i primi tre di cui parlare nel prossimo check-in con il cliente, prima che diventino la causa di uno slittamento non comunicato per tempo.

8.3 RACI: chi fa cosa (e chi deve solo saperlo)

Una delle domande più comuni — e più fonte di attriti — in un progetto è: **“chi decide su questo?”** o **“a chi tocca fare questa cosa?”**. La matrice **RACI** è uno strumento semplice per chiarirlo prima che diventi un problema, assegnando a ogni attività uno dei quattro ruoli:

- **R — Responsible** (Esecutore): chi materialmente **fa il lavoro**.
- **A — Accountable** (Responsabile finale): chi **risponde del risultato** davanti a tutti — è la persona che, se qualcosa va storto, deve rispondere. Per ogni attività dovrebbe essercene **uno solo**, per evitare ambiguità.
- **C — Consulted** (Consultato): chi viene **coinvolto per un parere** prima che la decisione/attività sia completata, con una comunicazione a doppio senso (gli chiedi qualcosa, lui risponde).
- **I — Informed** (Informato): chi viene **aggiornato dopo il fatto**, con comunicazione a senso unico (non partecipa alla decisione, ma deve saperlo).

Analogia: pensa a un ristorante durante il servizio. Il cameriere che porta il piatto al tavolo è il *Responsible*: esegue l'azione. Lo chef di cucina è l'*Accountable*: risponde della qualità del piatto che esce dalla sua cucina. Il sommelier viene *Consulted* se il cliente chiede un abbinamento particolare di vino. Il titolare del ristorante è *Informed*: alla fine della serata gli viene detto quanti coperti sono stati serviti, ma non ha partecipato a nessun piatto.

Esempio pratico: matrice RACI per l'attività “rilascio di una nuova funzionalità in produzione” — lo stesso rilascio del modulo pagamenti visto nel RAID Log di ShopFacile qui sopra. I ruoli della tabella restano generici (così puoi riusarla per qualsiasi progetto), ma nel team ShopFacile sono coperti così: Sviluppatore → **Ahmed**, Tech Lead/Project Manager → **Luca**, QA/Tester → **Giulia**, Cliente → **Sara**, in quanto voce del business.

Attività	Sviluppatore	Tech Lead	Project Manager	QA / Tester	Cliente
Sviluppo del codice	R	C	I	I	—
Code review	C	A / R	I	I	—
Test della funzionalità	I	C	I	R	—
Decisione go/no-go al rilascio	I	C	A	C	I
Esecuzione del rilascio (deploy)	C	R	I	I	—
Comunicazione al cliente dell'avvenuto rilascio	I	I	R / A	I	I

Qualche osservazione utile da tenere a mente quando costruisci una RACI:

- Per ogni riga ci dovrebbe essere **esattamente una A** (Accountable): se nessuno è accountable, nessuno risponde davvero del risultato; se ce ne sono troppe, si diluisce la responsabilità e nelle emergenze “si scaricano” a vicenda.
- Una stessa persona può essere sia R che A sulla stessa attività (come il Tech Lead nella code review, sopra), specialmente in team piccoli.
- Troppi “C” su un’attività la rendono lenta: ogni consultazione è un potenziale collo di bottiglia. Usa “C” solo dove il parere serve davvero.

Una RACI ben fatta chiarisce chi fa cosa lungo il percorso di un rilascio, ma non dice ancora **quando** quel rilascio conta davvero per chi guarda il progetto da fuori. È qui che entra in gioco il concetto di milestone.

8.4 Milestone: un traguardo, non una scadenza qualsiasi

Una **milestone** (pietra miliare) è un **punto significativo** nel percorso del progetto: segna il raggiungimento di qualcosa di rilevante, non semplicemente il passaggio di una data sul calendario.

Analogia: pensa a una gara ciclistica a tappe. Ogni tappa ha ovviamente una data di arrivo, ma alcuni punti del percorso sono **traguardi volanti** o **cime di montagna**: superarli non è solo “un altro chilometro fatto”, è un momento che tutti notano, che dà punteggio a parte, che segna davvero un salto di fase della corsa. Una scadenza qualsiasi è come un cartello chilometrico: utile per orientarsi, ma non cambia la narrazione della corsa. Una milestone è il traguardo di tappa.

La differenza pratica:

	Scadenza semplice	Milestone
Cosa segna	Il completamento di un compito operativo	Il raggiungimento di un obiettivo significativo per il progetto
Visibilità	Interna al team, spesso operativa	Comunicata anche a stakeholder esterni
Esempio	“Completare la migrazione del database di test entro venerdì”	“Ambiente di test pronto e validato dal cliente”
Conseguenza se manca	Si riorganizza il lavoro della settimana	Può avere impatto su comunicazioni contrattuali, pianificazione di altri team, aspettative del cliente

Esempi concreti di milestone in un progetto software: “fine della fase di analisi dei requisiti”, “prima release disponibile per un gruppo pilota di utenti (beta)”, “completamento dei test di sicurezza”, “go-live in produzione”, “primo mese di produzione senza incidenti critici”. Non sono attività in sé, ma **traguardi** che raccontano un salto di stato del progetto — per ShopFacile, il go-live del nuovo modulo pagamenti che abbiamo visto nel RAID Log e nella RACI è esattamente questo tipo di milestone: non un task tra tanti, ma un momento che Sara comunicherà anche al di fuori del team.

In un contesto Agile, le milestone convivono bene con gli sprint: uno sprint può contribuire a una milestone senza esaurirla, e una milestone spesso coincide con la fine di una serie di sprint (ad esempio, la fine di una release che raggruppa più sprint). Ma un traguardo così visibile porta con sé anche più da perdere se qualcosa va storto lungo il percorso — ed è proprio quello che il prossimo paragrafo insegna a gestire.

8.5 Rischio: identificare, valutare, mitigare

Un **rischio** è un evento incerto che, se si verifica, ha un impatto (di solito negativo, anche se esistono rischi “positivi”, cioè opportunità) sul progetto. Gestire i rischi non significa eliminarli tutti — è impossibile — ma **conoscerli in anticipo** e decidere consapevolmente cosa fare.

Il processo tipico ha tre fasi:

1. **Identificazione**: elencare cosa potrebbe andare storto. Si fa in team, spesso in un momento dedicato (es. durante la pianificazione di una release), con tecniche semplici come il brainstorming o rivedendo i rischi materializzati in progetti simili passati.
2. **Valutazione**: per ogni rischio, stimare **probabilità** (quanto è plausibile che accada) e **impatto** (quanto danno farebbe se accadesse). Spesso si usa una scala semplice (basso/medio/alto) e si visualizza in una matrice.
3. **Mitigazione**: decidere una strategia. Le opzioni classiche sono:
 - **evitare** il rischio (cambiare approccio per non incontrarlo);
 - **ridurre** probabilità o impatto (azioni preventive);

- **trasferire** il rischio (es. a un fornitore, con un'assicurazione contrattuale);
- **accettare** il rischio consapevolmente, se il costo di mitigarlo supera il danno potenziale, tenendo comunque un piano B pronto.

Diagramma 3

Diagramma 3

Esempio concreto: “rischio di ritardo per dipendenza da un altro team” — lo stesso rischio D-05 già anticipato nel RAID Log di ShopFacile.

- **Descrizione del rischio:** la nuova funzionalità di pagamento di ShopFacile richiede che il team di infrastruttura fornisca un nuovo ambiente sicuro entro due settimane; quel team ha già altri due progetti in corso.
- **Probabilità:** media-alta (il team esterno è oggettivamente sovraccarico, lo si vede dal loro backlog).
- **Impatto:** alto (senza quell'ambiente, il rilascio non può partire, e la data comunicata al cliente è a rischio).
- **Mitigazione:** Luca decide di chiedere una conferma scritta della data al team infrastruttura appena possibile (per trasformare l'assunzione in un impegno verificato); valuta con il team un piano B con un ambiente temporaneo meno performante ma disponibile prima; e si accorda con Sara per comunicare per tempo al cliente che esiste un rischio, invece di farlo scoprire a due giorni dalla scadenza.

Da notare come questo esempio collega direttamente rischio, assunzione e dipendenza — gli stessi concetti visti nel RAID Log: non sono strumenti separati, ma **lenti diverse sullo stesso terreno**.

8.6 Pianificazione in un contesto Agile: livelli, non un unico piano

Nel project management “a cascata” tradizionale, l'idea è pianificare tutto in anticipo, in un unico grande piano dettagliato. In un contesto Agile/DevOps si pianifica invece **a più livelli**, con dettaglio crescente man mano che ci si avvicina all'esecuzione — perché sappiamo che i dettagli lontani nel tempo cambieranno comunque, mentre quelli vicini devono essere affidabili.

I livelli tipici, dal più generale al più operativo:

1. **Roadmap:** la visione a lungo termine (mesi/trimestri/anno), con i grandi temi e obiettivi di business. Non contiene dettagli tecnici, ma direzioni: nella roadmap di ShopFacile che Sara porta agli stakeholder potrebbe leggersi “nel primo trimestre miglioriamo l'esperienza di checkout”, “nel secondo trimestre lanciamo l'app mobile”.
2. **Release Plan:** un livello più concreto, che raggruppa più sprint per arrivare a un rilascio significativo per gli utenti o il cliente. Qui iniziano a comparire funzionalità specifiche e stime di massima.
3. **Sprint Plan:** il piano del singolo sprint (di solito 1-4 settimane, come hai visto nella sezione su Scrum), con gli elementi di backlog selezionati e scomposti in attività concrete.
4. **Daily:** l'allineamento quotidiano del team (la daily standup), dove si sincronizza il lavoro del singolo giorno rispetto all'obiettivo dello sprint.

Diagramma 4

Diagramma 4

Il punto chiave da capire, come futuro Project Manager: **più ti allontani nel tempo, meno il piano deve essere dettagliato — e va bene così**. Una roadmap troppo dettagliata è quasi sempre sbagliata (perché il futuro lontano è incerto), mentre uno sprint plan troppo vago è inutile (perché il team deve poter lavorare su qualcosa di concreto già domani mattina). Il tuo compito non è “prevedere tutto”, ma tenere questi livelli **coerenti tra loro** e aggiornarli quando la realtà li smentisce.

Avere un piano a più livelli, però, non ti dice ancora se ShopFacile lo sta effettivamente rispettando giorno per giorno: per quello serve osservare l’andamento reale del team, non solo il piano scritto.

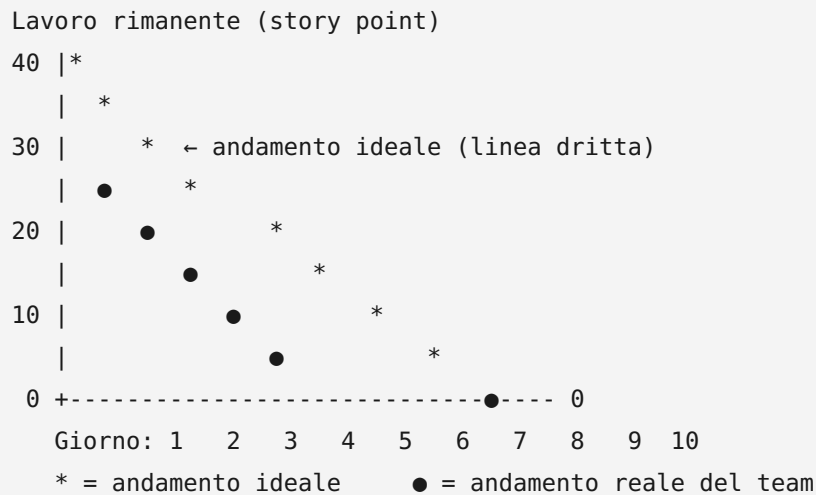
8.7 Monitoraggio: come si tiene traccia dell’andamento

Pianificare non basta: bisogna anche **verificare costantemente** se si sta procedendo come previsto, per poter reagire in tempo (non a fine progetto, quando è troppo tardi).

Gli strumenti principali di monitoraggio in un contesto Agile:

- **Daily stand-up**: il momento quotidiano (di solito 15 minuti, in piedi) in cui il team si sincronizza su cosa è stato fatto, cosa si farà e quali ostacoli ci sono. È monitoraggio “in tempo reale”, fatto dal team per il team.
- **Burndown chart**: mostra quanto lavoro **rimane** da fare in uno sprint (o in una release), giorno per giorno. La linea scende verso zero man mano che il lavoro si completa; una linea ideale tratteggiata mostra dove “dovremmo essere” per finire in tempo.
- **Burnup chart**: mostra quanto lavoro **è stato completato**, rispetto al totale previsto — utile soprattutto se lo scope cambia durante il progetto (con il burnup vedi anche se il “totale” sale, mentre il burndown da solo può confondere un aumento di scope con un rallentamento del team).
- **Dashboard**: pannelli visuali (spesso in Azure DevOps, Jira, o simili) che aggregano lo stato del backlog, delle Merge Request aperte, dei risultati delle pipeline di CI/CD, in un colpo d’occhio.

Un burndown chart “sano” tende ad avvicinarsi a una linea diagonale che scende gradualmente verso zero. Una rappresentazione testuale semplificata, per capire visivamente la forma:



In questo esempio, il team di ShopFacile parte più lento del previsto (la linea reale sta sopra quella ideale nei primi giorni — un possibile segnale di rischio da discutere in stand-up), ma recupera nella seconda metà dello sprint, chiudendo comunque a zero entro la fine. È tipicamente **Luca**, in quanto Scrum Master, a tenere d'occhio questo burndown e la dashboard del team ogni giorno: non guarda solo “se” la linea arriva a zero, ma **come** ci arriva — una discesa brusca solo nell'ultimo giorno è spesso il sintomo di lavoro “spuntato” tutto insieme a fine sprint, non di un reale progresso quotidiano.

Questi stessi grafici che Luca osserva ogni giorno sono anche la materia prima da cui si ricavano numeri di sintesi più stabili, utili per confrontare uno sprint con l'altro: i KPI.

8.8 KPI: gli indicatori chiave per un team Agile/DevOps

Un **KPI** (Key Performance Indicator, indicatore chiave di prestazione) è una misura quantitativa che aiuta a capire se le cose stanno andando bene, senza doversi fidare solo di impressioni soggettive (“mi pare che stiamo andando bene” non è un dato).

Alcuni KPI molto usati nei team Agile e DevOps:

KPI	Cosa misura	Perché è utile
Velocity	Quanti story point (o elementi di backlog) il team completa in media per sprint	Aiuta a pianificare quanto lavoro inserire negli sprint futuri, in modo realistico
Lead time	Tempo totale dalla richiesta di una funzionalità (apertura dell'elemento di backlog) al suo rilascio effettivo agli utenti	Misura quanto velocemente il team trasforma un'idea in valore consegnato
Cycle time	Tempo dall'inizio effettivo del lavoro su un elemento (non dalla richiesta) al suo completamento	Più mirato del lead time: misura l'efficienza del "fare", non dell'attesa in coda
Defect rate	Numero di bug rilevati (spesso in produzione) rispetto al volume di lavoro consegnato	Segnala la qualità di quanto viene rilasciato
Deployment frequency	Quante volte si rilascia in produzione in un dato periodo	KPI tipicamente DevOps: più alta è, di solito, più il processo di rilascio è maturo e automatizzato
Mean Time To Recovery (MTTR)	Tempo medio per ripristinare il servizio dopo un incidente in produzione	Misura la resilienza del team/ sistema, non solo la sua velocità
Sprint goal success rate	Percentuale di sprint in cui l'obiettivo dello sprint viene effettivamente raggiunto	Indica se la pianificazione degli sprint è realistica

Un avvertimento importante, da futuro Project Manager: i KPI vanno usati per **capire e migliorare il processo**, non per giudicare o "punire" le singole persone del team (es. confrontare la velocity di due sviluppatori diversi è quasi sempre un errore metodologico, perché la velocity dipende dal team nel suo insieme, dalla complessità del lavoro assegnato, e non è comparabile tra persone o addirittura tra team diversi). Usato male, un KPI distrugge la fiducia del team; usato bene, aiuta tutti a vedere dove migliorare.

Esempio pratico: confronto tra due sprint consecutivi del team di ShopFacile, con alcuni KPI di sintesi:

KPI	Sprint 8	Sprint 9	Cosa può significare
Velocity	32 story point	24 story point	Da solo non basta a dire se è un problema: va guardato insieme agli altri KPI
Lead time medio	6 giorni	9 giorni	Le richieste restano più tempo “in coda” prima di essere completate
Defect rate	2 bug su 15 elementi consegnati	5 bug su 11 elementi consegnati	Segnale di qualità in calo, non solo di velocità
Deployment frequency	4 rilasci in produzione	1 rilascio in produzione	Il processo di rilascio ha subito un rallentamento
Sprint goal success rate	Raggiunto	Non raggiunto	Conferma che qualcosa, nello sprint, non ha funzionato come previsto

Guardando la sola velocity, si potrebbe pensare “il team ha lavorato di meno”. Ma leggendo la riga del defect rate e quella della deployment frequency insieme, l'ipotesi più plausibile è diversa: il team ha probabilmente **rallentato per gestire un numero maggiore di bug**, riducendo sia la velocity che la frequenza dei rilasci. Un Project Manager che si fermasse al primo numero (velocity in calo = “il team lavora peggio”) trarrebbe la conclusione sbagliata; guardare i KPI **come insieme coerente**, non uno per uno isolato, è quello che permette di fare la domanda giusta in retrospettiva: “cosa ha causato l'aumento dei bug nello Sprint 9?”, invece di “perché avete lavorato di meno?”.

Questi KPI, però, restano numeri grezzi finché qualcuno non li traduce in un messaggio comprensibile per chi non li guarda ogni giorno: è il compito della reportistica.

8.9 Reportistica: cosa riportare, a chi, e con quale livello di dettaglio

Riportare lo stato di avanzamento è una delle attività più quotidiane di un Project Manager, ma non tutti i report sono uguali: **il livello di dettaglio deve adattarsi a chi lo legge**.

Analogia: pensa a come racconti la tua giornata a persone diverse. A un collega stretto racconti ogni dettaglio tecnico di un problema che hai risolto. A tuo padre, la sera, dici semplicemente “giornata impegnativa ma è andata bene”. Nessuna delle due versioni è “più vera” dell'altra: sono adatte a interlocutori diversi, con bisogni informativi diversi.

Due macro-categorie di report, con caratteristiche molto diverse:

	Report per il management/ cliente	Report operativi per il team
Frequenza tipica	Settimanale o mensile	Quotidiana o per sprint
Livello di dettaglio	Alto livello: stato generale, rischi principali, milestone raggiunte o a rischio	Dettagliato: singoli task, blocchi tecnici, chi sta facendo cosa
Linguaggio	Orientato al business, senza gergo tecnico	Tecnico, con riferimenti diretti a task, PR, ambienti
Contenuto tipico	Avanzamento % rispetto alla roadmap, KPI di sintesi, rischi da RAID Log con impatto su tempi/costi	Burndown/burnup dello sprint, stato delle Merge Request, esito delle pipeline CI/CD
Obiettivo	Dare fiducia e visibilità decisionale a chi non segue i dettagli ogni giorno	Coordinare il lavoro quotidiano e risolvere blocchi in tempo reale

Esempio pratico: la stessa informazione — “il rilascio della nuova funzionalità di pagamento di ShopFacile slitta di una settimana per un problema di sicurezza emerso nei test” — viene comunicata in due modi molto diversi:

- **Al cliente/management, a cura di Sara:** “Il rilascio della funzionalità di pagamento è posticipato di una settimana per garantire il rispetto degli standard di sicurezza richiesti. Non ci sono impatti sul budget. Nuova data prevista: [data].”
- **Al team, in stand-up o nella dashboard operativa, a cura di Luca:** “Il test di sicurezza automatizzato ha rilevato una vulnerabilità nella gestione dei token di sessione nel modulo di pagamento (vedi issue #128); serve un fix nel branch `fix/token-sicurezza` prima di riaprire la Merge Request e far ripartire la pipeline.”

Stessa realtà, due livelli di risoluzione informativa. Un errore comune dei Project Manager junior è portare al cliente il livello di dettaglio tecnico del team (che genera confusione e ansia inutile), oppure, al contrario, dare al team solo informazioni di alto livello (che non bastano per lavorare). Imparare a “tradurre” tra questi due livelli è una delle competenze più preziose del ruolo.

8.10 Riepilogo

In questa sezione hai visto come i concetti di project management si inseriscano in un contesto Agile/DevOps, senza contraddire ciò che hai imparato su Scrum e Kanban, ma completandolo con una vista più ampia sul progetto nel suo insieme:

- gli **stakeholder** ti dicono chi coinvolgere e come;
- il **RAID Log** ti dà un metodo per non farti sorprendere da rischi, assunzioni non verificate, problemi e dipendenze;
- la **RACI** chiarisce chi fa cosa, evitando ambiguità e conflitti;

- le **milestone** segnano i traguardi che contano davvero, distinti dalle scadenze operative quotidiane;
- la gestione del **rischio** ti allena a pensare in anticipo, non solo a reagire;
- la **pianificazione a più livelli** (roadmap, release plan, sprint plan, daily) convive naturalmente con l'incertezza tipica di un progetto Agile;
- il **monitoraggio** (stand-up, burndown/burnup, dashboard) ti permette di intercettare deviazioni mentre sono ancora piccole;
- i **KPI** ti danno numeri oggettivi su cui basare le decisioni, se usati con criterio;
- la **reportistica** ti insegna che comunicare bene significa adattare il dettaglio a chi ascolta, non ripetere sempre lo stesso messaggio.

Nelle prossime sezioni vedrai come molti di questi concetti si traducano in strumenti concreti: la cultura DevOps (sezione 9) e la piattaforma Azure DevOps (sezione 10) offrono board, dashboard, work item e pipeline che mettono in pratica esattamente ciò che hai visto qui — RAID Log, RACI, burndown chart e KPI diventano funzionalità cliccabili in uno strumento reale.

Esercizi pratici

Gli esercizi che seguono ti aiutano a passare dal “ho capito il concetto leggendolo” al “saprei applicarlo su un progetto reale”. Puoi farli con carta e penna, un foglio di calcolo, o uno strumento online gratuito: lo strumento conta meno del ragionamento che ci metti dietro.

1. **Mappa gli stakeholder di un progetto immaginario.** Scegli un progetto software semplice (es. “un’app per prenotare turni in palestra”) ed elenca almeno 6 stakeholder diversi. Per ciascuno, assegna una posizione sulla matrice potere/interesse (come quella della sezione 8.1) e scrivi in una riga **come** li terresti aggiornati (es. “email mensile”, “invito al Sprint Review”, “coinvolgimento diretto nelle decisioni”). **Come verificare:** se per almeno uno stakeholder non sai dire “cosa vuole” e “come lo tieni informato” in una frase, la mappatura non è ancora abbastanza concreta — torna a rileggere la sezione 8.1.
2. **Costruisci un RAID Log di 8 righe per un progetto reale o immaginario.** Deve contenere almeno 2 rischi, 2 assunzioni, 2 issue e 2 dipendenze, ciascuna con una stima di impatto (scala 1-5) e un responsabile assegnato. **Come verificare:** prova a ordinare le righe per impatto decrescente. Se le prime tre righe della lista non hanno ancora un’azione di mitigazione scritta, il RAID Log è incompleto — un RAID Log senza azioni è solo un elenco di preoccupazioni, non uno strumento di gestione.
3. **Disegna una matrice RACI per un’attività che conosci.** Scegli un’attività diversa da quella già usata come esempio nella sezione 8.3 (ad esempio: “pianificazione di uno sprint”, “gestione di un incidente in produzione”, “onboarding di un nuovo membro del team”) e costruisci una tabella RACI con almeno 5 righe/attività e almeno 4 ruoli. **Come verificare:** conta le “A” in ogni riga. Se una riga ha zero A o più di una A, correggila: è l’errore più comune di chi costruisce una RACI per la prima volta.

4. **Calcola e interpreta due KPI su dati inventati.** Immagina un team che, in 4 sprint consecutivi, completa rispettivamente 28, 25, 30 e 18 story point, con 1, 2, 1 e 6 bug rilevati in produzione nello stesso periodo. Calcola la velocity media dei primi tre sprint e confrontala con il quarto; scrivi in 3-4 righe quale ipotesi formularesti su cosa è successo nel quarto sprint, motivando la risposta con i dati (non solo con un'impressione). **Come verificare:** la tua ipotesi deve citare **entrambi** i KPI (velocity e defect rate) insieme, non uno isolato — se la tua conclusione si basa solo sul calo della velocity, rileggi l'avvertimento sui KPI della sezione 8.8.
5. **Scrivi due versioni dello stesso aggiornamento di stato.** Pensa a un contrattempo plausibile in un progetto software (un bug bloccante trovato tardi, un ritardo di un fornitore, un test di sicurezza fallito) e scrivi due comunicazioni brevi (massimo 3-4 righe ciascuna): una per il cliente/management, una per il team in stand-up. **Come verificare:** fai leggere la versione “per il cliente” a qualcuno senza background tecnico (anche un amico non del settore): se non capisce cosa sta succedendo e cosa aspettarsi, è ancora troppo tecnica.
6. **Simula una retrospettiva sui rischi di un progetto concluso (anche immaginario).** Elenca 3 rischi che, con il senno di poi, si sono effettivamente concretizzati in issue, e per ciascuno scrivi: come lo avresti potuto identificare prima, e quale delle quattro strategie di mitigazione (evitare, ridurre, trasferire, accettare) avrebbe avuto più senso applicare in anticipo. **Come verificare:** per ogni rischio, la strategia di mitigazione che scegli deve essere diversa da un generico “fare più attenzione” — deve essere un'azione concreta e verificabile (es. “richiedere conferma scritta entro una data”, non “stare più attenti alle dipendenze”).

Collegamenti

- [9. DevOps](#) — la cultura e le pratiche che rendono possibile un monitoraggio continuo e KPI come deployment frequency e MTTR
- [10. Azure DevOps](#) — dove RACI, RAID Log, burndown chart e dashboard diventano strumenti concreti da usare ogni giorno

Risorse

- [PMI — Project Management Institute](#)
- [PMBOK Guide — panoramica](#)
- [Atlassian — Guida ai KPI Agile](#)
- [Atlassian — Cos'è un RACI Chart](#)
- [Atlassian — Stakeholder analysis](#)
- [Atlassian — Burndown chart](#)
- [Scrum.org — Risk Management in Scrum](#)
- [Axelos — PRINCE2 e RAID Log](#)

9. DevOps

Nelle sezioni precedenti hai imparato **come un team organizza il lavoro** (Agile, Scrum, Kanban) e **come si tiene sotto controllo un progetto** nel suo complesso (Project Management, con stakeholder, RAID Log, RACI, KPI). Questa sezione affronta un tema diverso, e altrettanto centrale per il lavoro che farai ogni giorno: **come il software, una volta scritto, arriva davvero nelle mani degli utenti — e come ci resta, funzionando bene, nel tempo.**

Questo è il mondo di **DevOps**. È una delle parole che sentirai pronunciare più spesso nel tuo team, quasi sempre associata a strumenti molto concreti (pipeline, ambienti, dashboard) che vedrai nel dettaglio nella prossima sezione, dedicata ad Azure DevOps. Ma prima di usare gli strumenti, devi capire l'**idea** che c'è dietro — perché DevOps, prima di essere una piattaforma o un insieme di pratiche tecniche, è **un modo di pensare al lavoro del team**. Se capisci bene questa sezione, tutto quello che vedrai dopo (pipeline, ambienti, monitoraggio, sicurezza) ti sembrerà la conseguenza naturale di un'idea semplice, non un elenco di strumenti scollegati tra loro.

Per rendere tutto più concreto, in questa sezione useremo un unico progetto di riferimento, che ritroverai anche nelle sezioni successive: **ShopFacile**, una piattaforma e-commerce (catalogo prodotti, carrello, ordini, pagamenti, sconti) sviluppata da un piccolo team interno composto, tra gli altri, da **Marco** e **Giulia** (developer), **Ahmed** (developer junior, in crescita), **Sara** (Product Owner) e **Luca** (Scrum Master). Vedrai questi nomi tornare più volte: non sono persone diverse in ogni esempio, ma lo stesso team che affronta, di volta in volta, un problema diverso.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- spiegare **cos'è DevOps** e perché non è “uno strumento” ma una cultura;
 - raccontare **perché è nato** DevOps, partendo dal problema storico del “muro della confusione” tra sviluppo e infrastruttura;
 - riconoscere gli elementi chiave della **cultura DevOps** (collaborazione, responsabilità condivisa, blameless culture) e il modello **CALMS**;
 - capire perché l'**automazione** è al centro di DevOps, con esempi concreti;
 - distinguere con precisione **Continuous Integration**, **Continuous Delivery** e **Continuous Deployment** — tre termini che si confondono facilmente, ma che indicano cose diverse;
 - capire cos'è l'**Infrastructure as Code** e perché ha cambiato il modo di gestire server e reti;
 - distinguere **monitoring**, **observability** e **logging**, tre attività complementari per capire lo stato di un sistema in produzione.
-

9.1 Cos'è DevOps: non è un tool, è una cultura

La parola **DevOps** nasce dalla fusione di due parole inglesi:

- **Development** — lo **sviluppo** del software: scrivere codice, creare nuove funzionalità, correggere bug.
- **Operations** — le **operazioni**: tutto ciò che serve per far funzionare quel software in un ambiente reale, con utenti reali — server, reti, database, sicurezza, backup, monitoraggio.

DevOps è quindi, letteralmente, **l'unione di questi due mondi**. Ma qui sta il punto più importante da capire, e su cui vale la pena insistere subito: **DevOps non è un software che si installa, non è un ruolo con la targhetta “DevOps Engineer” sulla porta, e non è nemmeno un insieme fisso di pipeline**. DevOps è prima di tutto **una cultura**: un modo di organizzare le persone, i processi e (solo successivamente) gli strumenti, in modo che chi scrive il codice e chi lo fa funzionare in produzione lavorino **insieme**, verso lo stesso obiettivo, invece che come due squadre separate con interessi contrapposti.

Analogia: pensa a un ristorante. Se la cucina (chi cucina i piatti, lo “sviluppo”) e la sala (chi li serve al cliente e gestisce i reclami, le “operazioni”) lavorano come due mondi separati — la cucina si disinteressa di cosa succede quando il piatto arriva al tavolo, la sala non sa mai perché un piatto è in ritardo o come è stato preparato — il risultato è un servizio lento, con errori che nessuno riesce a spiegare e tutti scaricano sull'altro reparto. Un ristorante che funziona bene ha invece cucina e sala che comunicano costantemente: sanno cosa succede dall'altra parte, si scambiano informazioni in tempo reale, e se un piatto torna indietro non si accusano a vicenda, ma capiscono insieme cosa è andato storto per non farlo succedere di nuovo. DevOps è esattamente questo, applicato allo sviluppo software: fare in modo che “chi cucina il codice” e “chi lo serve agli utenti” lavorino come **una sola squadra**, non come due squadre che si passano un pacco.

Vale la pena essere ancora più precisi su un equivoco molto comune, perché lo sentirai spesso sul lavoro: quando qualcuno dice “usiamo Azure DevOps” o “abbiamo automatizzato con strumenti DevOps”, si riferisce agli **strumenti** che rendono più facile *applicare* la cultura DevOps (ad esempio le pipeline di cui parleremo più avanti). Ma installare quegli strumenti **non rende automaticamente un team “DevOps”**, così come comprare attrezzi da falegname non fa di te un falegname. Gli strumenti aiutano, e sono importanti — li vedrai nel dettaglio nella prossima sezione — ma senza il cambiamento di cultura e di collaborazione, restano scatole vuote.

Ma se DevOps è “solo” una cultura, perché è diventata così centrale nel lavoro quotidiano di un team come quello di ShopFacile? Per rispondere bisogna guardare al problema concreto che ha spinto a inventarla: torniamo indietro a come lavoravano i team **prima** che DevOps esistesse.

9.2 Perché nasce DevOps: il muro della confusione

Per capire perché DevOps è diventato così importante, bisogna capire il problema che risolve. E per capirlo, bisogna tornare a come funzionavano (e in molte aziende funzionano ancora) i team di sviluppo e di infrastruttura **prima** di DevOps.

Il mondo prima di DevOps: due team, due obiettivi opposti

Tradizionalmente, in molte organizzazioni, esistevano due reparti separati, spesso con manager diversi, obiettivi diversi e persino uffici diversi:

- Il team di **sviluppo (Dev)**, il cui obiettivo — spesso l'unico su cui veniva valutato — era **consegnare nuove funzionalità velocemente**. Più release, più funzionalità, più velocità: questo veniva premiato.
- Il team di **operazioni/infrastruttura (Ops)**, il cui obiettivo era **mantenere i sistemi stabili, sicuri e sempre disponibili**. Ogni cambiamento nei sistemi in produzione è un potenziale rischio di down: il loro obiettivo, spesso l'unico su cui venivano valutati, era **la stabilità**, non la velocità.

Il problema è evidente non appena lo si scrive esplicitamente: **questi due obiettivi sono in tensione tra loro**. Il team di sviluppo vuole rilasciare spesso e velocemente; il team di operazioni vuole cambiare le cose il meno possibile, perché ogni cambiamento è un rischio per la stabilità che deve garantire. Non è che uno dei due team avesse torto: **ciascuno stava ottimizzando esattamente per quello su cui veniva giudicato**, ma il risultato complessivo era un conflitto strutturale.

Questa frizione, con il tempo, ha preso un nome molto azzeccato nella comunità tecnica: **“il muro della confusione” (wall of confusion)**. Il team di sviluppo “lanciava” il software oltre un muro metaforico verso il team di operazioni, che doveva farlo funzionare in produzione senza averlo scritto, spesso senza documentazione adeguata, e senza aver preso parte alle decisioni di progettazione. Quando qualcosa si rompeva in produzione, ciascun team tendeva ad accusare l'altro: “il codice ha un bug” contro “l'infrastruttura non era configurata bene” — e nessuno aveva una visione completa per capire davvero cosa fosse successo.

Diagramma 1

Diagramma 1

Perché questo era un problema reale (non solo “scomodo”)

Questo modello creava conseguenze molto concrete, che vale la pena elencare perché le riconoscerai facilmente in aziende che non hanno ancora adottato una cultura DevOps:

- **Rilasci rari e rischiosi**: se rilasciare è complicato, doloroso e richiede il coinvolgimento manuale di tante persone, i team tendono a rilasciare raramente (una volta al mese, o anche meno). Ma rilasci rari significano rilasci **grandi**, che accumulano tanti cambiamenti insieme — e più cambiamenti insieme, più è probabile che qualcosa si rompa, e più è difficile capire *quale* cambiamento specifico ha causato il problema.

- **Colpa reciproca invece di soluzioni:** quando qualcosa andava storto in produzione, il tempo veniva spesso speso a capire “di chi è la colpa” invece che a risolvere il problema e capire come evitarlo in futuro.
- **Comunicazione tardiva:** gli sviluppatori progettavano soluzioni senza sapere come sarebbero state effettivamente eseguite in produzione (quante risorse servono, quali vincoli di sicurezza esistono); il team operazioni riceveva richieste dell'ultimo minuto, senza tempo per prepararsi.
- **Lentezza percepita dal cliente:** il risultato finale, quello che conta davvero per chi ha commissionato il progetto, era che le nuove funzionalità richiedevano troppo tempo per arrivare davvero in mano agli utenti, anche quando il codice era pronto da settimane.

Nota per un futuro Project Manager: molti dei rischi e delle issue che hai imparato a tracciare nel RAID Log (sezione 8) — ritardi nei rilasci, problemi di comunicazione tra team, dipendenze che si scoprono troppo tardi — sono spesso **sintomi diretti** dell'assenza di una cultura DevOps. Non è un caso che questi due argomenti siano collegati.

La soluzione: eliminare il muro, non spostarlo

DevOps nasce esattamente per rispondere a questo problema, e lo fa non “velocizzando” il lancio oltre il muro, ma **eliminando il muro stesso**. L'idea è che sviluppo e operazioni non siano più due team separati con obiettivi opposti, ma **un'unica squadra, con un obiettivo condiviso**: consegnare valore agli utenti **velocemente e in modo stabile**, non l'uno a scapito dell'altro.

Diagramma 2

Diagramma 2

Il punto chiave visualizzato nel diagramma sopra è che il flusso non è più una linea a senso unico che “lancia” il lavoro da un team all'altro, ma un **ciclo continuo di collaborazione e feedback**: lo stesso team che scrive il codice si occupa (insieme, non da solo) anche di come funziona in produzione, usa l'automazione per ridurre il rischio dei rilasci frequenti, e riceve informazioni immediate su come il software si comporta nel mondo reale — informazioni che tornano utili per il prossimo ciclo di sviluppo. Vedremo nella prossima sezione (9.3) come questo ciclo si rappresenta in modo ancora più esplicito con il celebre “simbolo a infinito” di DevOps.

9.3 Il ciclo DevOps: un percorso senza fine

Uno dei modi più famosi per rappresentare DevOps visivamente è un simbolo a forma di **infinito (∞)**, diviso in otto fasi che si susseguono e non finiscono mai: appena termina “Operate/Monitor”, si riparte immediatamente con “Plan” per il ciclo successivo. Questo simbolo comunica un messaggio molto preciso: **DevOps non è un progetto con un inizio e una fine, ma un flusso continuo**.

Le otto fasi tipiche del ciclo sono:

Fase	Cosa succede	Chi è coinvolto principalmente
Plan	Si decide cosa costruire: requisiti, priorità, backlog (collegamento diretto con quanto visto in Agile/Scrum)	Product Owner, team, stakeholder
Code	Si scrive il codice della nuova funzionalità o della correzione	Sviluppatori
Build	Il codice viene compilato/assemblato in un pacchetto eseguibile, in modo automatico	Automazione (pipeline)
Test	Il pacchetto viene testato automaticamente (test unitari, di integrazione, talvolta di sicurezza)	Automazione + QA
Release	Il pacchetto testato viene preparato e approvato per il rilascio	Team + approvazioni
Deploy	Il pacchetto viene effettivamente installato/attivato nell'ambiente di produzione (o di test)	Automazione (pipeline)
Operate	Il sistema funziona in produzione, gestito e mantenuto	Operations / SRE
Monitor	Si osserva come si comporta il sistema in produzione, raccogliendo dati utili per il prossimo ciclo di pianificazione	Tutti, grazie a strumenti di monitoraggio

Diagramma 3

Diagramma 3

Nota una cosa importante nella tabella e nel diagramma: le prime due fasi (Plan, Code) corrispondono grosso modo al mondo **Dev** “classico”; le ultime due (Operate, Monitor) al mondo **Ops** “classico”. Le fasi centrali (Build, Test, Release, Deploy) sono la “zona di confine” dove, storicamente, si trovava il muro della confusione — ed è proprio lì che l'**automazione** gioca il ruolo più importante, come vedremo nella prossima sezione, perché è lì che il lavoro passa fisicamente da un mondo all'altro.

Il dettaglio da ricordare: **Monitor non è la fine del ciclo, è l'inizio del ciclo successivo**. I dati raccolti osservando il sistema in produzione (quanti utenti usano una funzionalità, dove si verificano errori, quali parti sono lente) diventano input diretto per la prossima fase di Plan. Questo è il motivo per cui il simbolo è un infinito e non una linea con un punto di arrivo.

Conoscere le otto fasi del ciclo, però, non basta a farle funzionare bene insieme: perché quel ciclo scorra senza attriti, serve che le persone che lo attraversano — chi scrive il codice, chi lo gestisce in produzione — si comportino secondo alcuni principi condivisi. È proprio a questi principi, cioè alla cultura DevOps vera e propria, che dedichiamo il prossimo paragrafo.

9.4 La cultura DevOps: collaborazione, responsabilità, fiducia

Abbiamo detto che DevOps è prima di tutto una cultura. Ma cosa significa concretamente? Significa un insieme di comportamenti e valori condivisi dal team, che si traducono in scelte quotidiane precise.

Collaborazione, non “passaggio di consegne”

Nel modello vecchio, lo sviluppo “consegnava” al team operazioni, e la comunicazione era spesso a senso unico e tardiva (un po’ come il modello “Informed” della RACI che hai visto nella sezione 8, applicato però a qualcosa che avrebbe avuto bisogno di molto più coinvolgimento). Nella cultura DevOps, sviluppo e operazioni **collaborano dall’inizio**: chi scrive il codice pensa già a come sarà eseguito in produzione (quante risorse servono, come si comporta sotto carico, come si può monitorare); chi gestisce l’infrastruttura è coinvolto già nelle fasi di progettazione, non solo quando il software è “pronto” e deve solo essere installato.

Responsabilità condivisa: “you build it, you run it”

Una delle frasi più citate nella cultura DevOps è “**you build it, you run it**” (“chi lo costruisce, lo gestisce anche”). Significa che il team che scrive una funzionalità non se ne “disinteressa” una volta rilasciata: resta coinvolto (spesso tramite turni di reperibilità, i cosiddetti *on-call*) anche quando quella funzionalità è in produzione e qualcosa va storto.

Analogia: è la differenza tra un ristorante dove il cuoco cucina e se ne va, lasciando alla sala il compito di gestire ogni reclamo su un piatto che non conosce nei dettagli, e un ristorante dove — se un cliente segnala un problema con un piatto — quel piatto torna direttamente al cuoco che lo ha preparato, perché è lui che ha davvero le informazioni e gli strumenti per capire cosa è andato storto e correggerlo.

Questo cambia gli incentivi in modo molto concreto: se sai che sarai tu (o il tuo team) a dover gestire un problema alle tre di notte, scriverai codice più robusto, penserai di più a come monitorarlo, e sarai più motivato a collaborare con chi gestisce l’infrastruttura, perché non è più “un problema di qualcun altro”.

Blameless culture: imparare dagli errori senza cercare un colpevole

Uno dei pilastri più importanti — e spesso più difficili da costruire davvero — della cultura DevOps è la cosiddetta **blameless culture** (cultura “senza colpa”). L’idea è semplice da enunciare ma richiede maturità organizzativa per essere applicata davvero: **quando qualcosa va storto (un bug critico, un’interruzione del servizio), l’obiettivo dell’analisi che segue non è trovare “di chi è la colpa” per punirlo, ma capire le cause reali per evitare che accada di nuovo.**

Analogia: pensa alla differenza tra un'aviazione civile e un ambiente dove ogni errore viene punito severamente. Quando un aereo ha un incidente o un "quasi incidente", gli investigatori non cercano primariamente "chi punire": cercano di ricostruire la catena di eventi (un errore umano, ma anche una procedura poco chiara, uno strumento mal progettato, una comunicazione ambigua) per correggere il **sistema**, non solo la persona. Questo è uno dei motivi per cui il trasporto aereo è diventato via via più sicuro nel tempo: chi commette un errore in buona fede lo segnala subito, perché sa che verrà usato per migliorare, non per essere licenziato.

Nel contesto DevOps, questo si traduce in pratiche concrete come il **post-mortem** (o *retrospettiva sull'incidente*): dopo un problema importante in produzione, il team si riunisce per ricostruire cosa è successo, con una linea temporale precisa, e per proporre azioni concrete di miglioramento — senza che il documento serva a puntare il dito su una persona specifica. Se le persone temono di essere colpevolizzate, tendono a **nascondere i problemi o segnalarli in ritardo** — esattamente il contrario di quello che un team DevOps efficace ha bisogno di fare.

Il modello CALMS: cinque pilastri per riconoscere la cultura DevOps

Un modo diffuso per ricordare gli elementi fondamentali della cultura DevOps è l'acronimo **CALMS**:

Lettera	Pilastro	Significato
C	Culture (Cultura)	Collaborazione, fiducia reciproca, responsabilità condivisa tra Dev e Ops, blameless culture
A	Automation (Automazione)	Automatizzare compiti ripetitivi e a rischio di errore umano: build, test, deploy, provisioning dell'infrastruttura
L	Lean (Snellezza)	Eliminare gli sprechi nel processo: passaggi inutili, attese, lavoro in eccesso non necessario (lo stesso principio che hai visto in Kanban)
M	Measurement (Misurazione)	Misurare tutto ciò che conta con dati oggettivi: quanto spesso si rilascia, quanto tempo serve per riparare un guasto, quanti errori arrivano in produzione (i KPI DevOps visti nella sezione 8, come deployment frequency e MTTR)
S	Sharing (Condivisione)	Condividere conoscenza, strumenti, successi e insuccessi tra i team, per non ripetere gli stessi errori e non “reinventare la ruota”

Esempio pratico: immagina che il team di **ShopFacile** rilasci ancora manualmente, una volta al mese, seguendo una checklist su carta, senza sapere quanto tempo richieda in media risolvere un guasto. Applicando CALMS:

- **Culture:** **Marco** (developer) e chi si occupa dell'infrastruttura iniziano a partecipare alla stessa retrospettiva, invece di due riunioni separate che non si parlano.
- **Automation:** la checklist manuale viene trasformata in una pipeline che esegue build, test e deploy in automatico, riducendo gli errori.
- **Lean:** il team si rende conto che 3 dei 12 passaggi della vecchia checklist non servivano più a nulla, ed erano solo un'abitudine mai rimessa in discussione.
- **Measurement:** **Luca** (Scrum Master) inizia a far misurare al team la *deployment frequency* (quante volte si rilascia) e l'*MTTR* (quanto tempo serve per risolvere un guasto), passando da decisioni “a sensazione” a decisioni basate su dati concreti.
- **Sharing:** la pipeline creata per il catalogo prodotti di ShopFacile viene documentata da **Giulia** e riutilizzata anche per il servizio ordini, invece di essere reinventata da zero.

Vale la pena notare che **la Cultura è la prima lettera, non l'ultima**: non è un caso. Molte aziende provano a “fare DevOps” partendo dall'automazione o dagli strumenti, saltando il lavoro culturale — e spesso falliscono, perché installano pipeline sofisticate sopra un'organizzazione che continua a comportarsi con la vecchia logica del “muro della confusione”. Gli strumenti amplificano una buona cultura; non la creano da soli.

Aver visto CALMS ci porta dritti a una delle sue lettere in particolare: la **A** di Automation. È il pilastro più “tecnico” e visibile dei cinque, ed è talmente centrale nel lavoro quotidiano del team di ShopFacile che merita un paragrafo tutto suo.

9.5 Automazione: il motore pratico di DevOps

Se la cultura è il “perché” di DevOps, l'**automazione** è il “come” con cui quella cultura si traduce in pratica quotidiana. L'idea di base è semplice: **ogni compito ripetitivo, manuale e prevedibile che un umano farebbe sempre nello stesso modo è un candidato ideale per essere automatizzato**.

Perché è così importante automatizzare? Per tre ragioni concrete:

1. **Riduce gli errori umani.** Un essere umano che ripete lo stesso compito manuale centinaia di volte, prima o poi si distrae, salta un passaggio, digita qualcosa di sbagliato. Una macchina, se lo script è scritto correttamente, esegue lo stesso identico procedimento ogni singola volta, senza stanchezza e senza distrazioni.
2. **Velocizza il lavoro.** Un compito che a mano richiede 40 minuti (con passaggi da ricordare, comandi da digitare, controlli da fare manualmente) può essere eseguito automaticamente in pochi minuti, o anche secondi, liberando le persone per lavoro che richiede davvero giudizio umano.
3. **Rende il lavoro ripetibile e verificabile.** Uno script di automazione è, esso stesso, codice: può essere letto, revisionato, versionato con Git (esattamente come hai visto nella sezione 4), e quindi **si sa esattamente cosa fa**, senza doversi fidare della memoria di una persona su “come si faceva quella cosa”.

Analogia: pensa alla differenza tra lavare i piatti a mano, uno per uno, ogni giorno, e usare una lavastoviglie. La lavastoviglie non è “più intelligente” di una persona: esegue sempre la stessa sequenza di passaggi (acqua, detersivo, risciacquo, asciugatura), ma lo fa in modo costante, senza mai saltare il risciacquo per stanchezza dopo una lunga giornata, e libera le persone per fare altro nel frattempo. L'automazione nello sviluppo software è la lavastoviglie del team: non sostituisce il giudizio umano dove serve davvero (decidere cosa costruire, valutare un compromesso tra soluzioni), ma elimina la fatica ripetitiva dove non serve giudizio, solo costanza.

Esempi molto concreti di automazione che incontrerai nel team di ShopFacile, e che approfondiremo nelle prossime sezioni (specialmente nella 11, dedicata proprio a CI/CD):

- **Build automatiche:** ogni volta che **Ahmed** propone un cambiamento al codice del carrello, un sistema compila automaticamente il progetto, senza che nessuno debba farlo manualmente sul proprio computer.
- **Test automatici:** una suite di test viene eseguita automaticamente ad ogni cambiamento, verificando che il codice funzioni come previsto e che non abbia rotto nulla che funzionava prima.
- **Deploy automatici:** il pacchetto software viene installato automaticamente nell'ambiente giusto (test, staging, produzione), senza che una persona debba collegarsi manualmente a un server e copiare file a mano.
- **Provisioning automatico dell'infrastruttura:** la creazione di server, reti, database avviene tramite file di configurazione eseguiti automaticamente (lo vedremo nel dettaglio parlando di Infrastructure as Code, sezione 9.9).

Tra tutte queste forme di automazione, una in particolare merita un paragrafo dedicato, perché è quella con cui uno sviluppatore di ShopFacile ha a che fare più spesso di tutte: l'integrazione automatica del codice di più persone che lavorano insieme sullo stesso progetto.

9.6 Continuous Integration (CI): integrare spesso, non alla fine

Continuous Integration, spesso abbreviata **CI**, è la pratica di **integrare il codice di più sviluppatori nel ramo principale del progetto molto frequentemente** (più volte al giorno, tipicamente), invece di lavorare separati per settimane e unire tutto solo alla fine.

Ricorda cosa hai visto nella sezione su Git e GitLab: quando più persone lavorano sullo stesso codice in branch diversi, unire i cambiamenti (il *merge*) può generare conflitti. Più tempo passa tra un'integrazione e la successiva, più il codice dei diversi sviluppatori si allontana l'uno dall'altro, e più i conflitti che emergono al momento dell'unione sono grandi, complicati da risolvere e rischiosi.

CI risolve questo problema imponendo una disciplina precisa: **ogni volta che uno sviluppatore propone un cambiamento (tipicamente con un commit o una Merge Request), un sistema automatico esegue immediatamente la build del progetto e i test automatici**, per verificare che quel cambiamento si integri correttamente con tutto il resto del codice, senza romperlo.

Analogia: pensa alla differenza tra scrivere un libro a più mani aggiornando ogni giorno un documento condiviso online (dove ogni piccola modifica di ciascun autore viene subito vista dagli altri, e le incoerenze si notano immediatamente), e scrivere ciascuno il proprio capitolo separatamente per sei mesi, per poi provare a incollare tutto insieme alla fine. Nel secondo caso, è quasi garantito che i personaggi abbiano nomi diversi in capitoli diversi, che la timeline non torni, che il tono sia incoerente — e correggere tutto questo alla fine è molto più difficile che correggere piccole incoerenze giorno per giorno.

Il ciclo tipico di CI, ogni volta che qualcuno propone un cambiamento:

Esempio pratico: Ahmed apre una Merge Request che modifica la funzione di calcolo dello sconto su un ordine di ShopFacile. Ecco cosa succede, passo per passo, in una pipeline di CI tipica (li vedrai nel dettaglio nella sezione 11):

1. Il codice modificato viene “caricato” (push) sul repository condiviso.
2. La pipeline si attiva automaticamente (il cosiddetto *trigger*), senza che nessuno debba lanciarla a mano.
3. Viene eseguita la **build**: il codice viene compilato/assemblato. Se c'è un errore di sintassi, la pipeline si ferma già qui.
4. Vengono eseguiti i **test automatici**: ad esempio, un test verifica che “uno sconto del 10% su un ordine da 100€ dia esattamente 90€”.
5. Se un test fallisce (magari Ahmed ha sbagliato un calcolo), la pipeline segnala l'errore in pochi minuti, direttamente nella Pull Request, prima che il codice venga integrato. Sarà **Giulia**, che revisiona spesso queste Merge Request, ad aiutarlo a individuare il problema.
6. Solo se build e test passano, il codice viene integrato nel ramo principale — pronto per le fasi successive (Delivery o Deployment).

Il beneficio più importante di CI è il **feedback rapido**: se qualcosa non funziona, lo sviluppatore lo scopre in pochi minuti, mentre ha ancora ben in mente cosa ha appena scritto — non settimane dopo, quando ha già dimenticato i dettagli di quella modifica e deve “ricordarsi” cosa aveva fatto.

Sapere che il codice è stato integrato correttamente, però, non dice ancora nulla su **quando e come** quella modifica arriverà davanti agli utenti di ShopFacile: è qui che entrano in gioco Delivery e Deployment, i due concetti che completiamo nel prossimo paragrafo.

9.7 Continuous Delivery e Continuous Deployment: la differenza che confonde tutti

Arriviamo a uno dei punti più importanti — e più spesso confusi da chi è alle prime armi — di tutto il vocabolario DevOps: la differenza tra **Continuous Delivery** e **Continuous Deployment**. Le due espressioni si abbreviano entrambe con **CD**, iniziano con le stesse lettere, e vengono spesso usate (erroneamente) come sinonimi. Non lo sono. Vale la pena fissare bene questa differenza, perché la incontrerai continuamente nel lavoro quotidiano e nella prossima sezione dedicata proprio a CI/CD.

Continuous Delivery: sempre pronto, ma il rilascio è una decisione manuale

Continuous Delivery significa che, grazie all'automazione (build, test, e tutto ciò che serve per preparare il pacchetto software), **il software è sempre in uno stato pronto per essere rilasciato in produzione in qualsiasi momento** — ma il rilascio effettivo in produzione richiede **un'azione manuale, di solito un'approvazione o un click da parte di una persona**.

Continuous Deployment: il rilascio è automatico, senza intervento umano

Continuous Deployment va un passo oltre: qui **anche il rilascio in produzione avviene automaticamente**, senza bisogno di un'approvazione manuale, non appena il codice ha superato con successo tutti i controlli automatici (build, test, eventuali controlli di sicurezza). Nessuna persona deve premere un bottone: se i controlli automatici passano, il codice va in produzione da solo.

Analogia molto concreta: pensa a una catena di montaggio che produce automaticamente dei pacchi pronti per la spedizione, testati e verificati.

- Con **Continuous Delivery**, ogni pacco pronto viene messo su uno scaffale, etichettato “pronto per la spedizione” — ma è sempre una persona che decide quando e quale pacco spedire effettivamente al cliente, magari aspettando il momento giusto, o un'ultima verifica manuale.
- Con **Continuous Deployment**, non c'è nessuno scaffale di attesa: non appena un pacco supera i controlli di qualità automatici, un corriere lo spedisce **automaticamente** al cliente, senza che nessuno debba decidere nulla in quel momento.

Un'altra analogia, forse ancora più diretta per il contesto software: pensa a un aereo pronto al decollo. Con Continuous Delivery, l'aereo è rifornito, controllato, con l'equipaggio a bordo e pronto a partire in qualsiasi momento — ma serve sempre il comando esplicito della torre di controllo (“via libera al decollo”) prima che parta davvero. Con Continuous Deployment, non appena tutti i controlli pre-volo automatici sono superati, l'aereo decolla da solo, senza aspettare quel comando umano.

Esempio pratico: immagina che in ShopFacile una nuova funzionalità (“aggiungi un filtro di ricerca per data nel catalogo”) abbia appena superato build e test automatici.

- Con **Continuous Delivery**, il pacchetto pronto resta “in attesa” e **Sara** (Product Owner), insieme a un responsabile del rilascio, apre la pipeline, guarda che tutto sia verde, e clicca manualmente su “Deploy in produzione” — magari aspettando il momento di minor traffico sul sito.
- Con **Continuous Deployment**, non appena i test automatici passano, la stessa funzionalità viene installata in produzione da sola, nel giro di pochi minuti dal commit originale, senza che nessuno clicchi nulla.

La differenza pratica che noterai osservando una pipeline reale (lo farai nella sezione 11): nel primo caso c'è uno step chiamato tipicamente “Approvazione” o “Gate” che aspetta un umano; nel secondo caso quello step semplicemente non esiste.

Perché questa distinzione conta davvero

La scelta tra Continuous Delivery e Continuous Deployment non è solo tecnica: è una **decisione di business e di rischio**. Alcuni contesti (sistemi bancari, sanitari, o semplicemente prodotti dove un errore in produzione ha un costo molto alto) preferiscono quasi sempre Continuous Delivery, per

mantenere un punto di controllo umano finale prima di esporre gli utenti a un cambiamento. Altri contesti (ad esempio prodotti web con rilasci molto frequenti e un basso costo di un eventuale rollback rapido) possono permettersi Continuous Deployment, guadagnando velocità.

Nota terminologica importante: quando qualcuno usa genericamente l'espressione **"CI/CD"**, di solito si riferisce all'intero flusso automatizzato che va dall'integrazione del codice fino alla sua consegna o al suo rilascio — e la "CD" in questo contesto può voler dire *Delivery oppure Deployment*, a seconda del team e del progetto specifico. Per questo, quando la precisione conta (ad esempio in una discussione tecnica o in un documento di progetto), è sempre meglio specificare esplicitamente quale delle due si intende.

Uno sguardo d'insieme: CI vs Continuous Delivery vs Continuous Deployment

	Continuous Integration (CI)	Continuous Delivery	Continuous Deployment
Cosa succede	Il codice viene integrato, compilato e testato automaticamente ad ogni commit/PR	Il pacchetto software è sempre pronto per il rilascio, dopo build e test automatici	Il pacchetto software viene rilasciato automaticamente in produzione
Il rilascio in produzione è...	Non è ancora in discussione a questo livello	Manuale (richiede un'azione/approvazione umana)	Automatico (nessun intervento umano)
Dove avviene	Ambiente di build/test, prima della produzione	Fino "alla porta" della produzione	Direttamente in produzione
Livello di rischio percepito	Basso: non si tocca ancora la produzione	Medio: c'è un controllo umano finale	Più alto: richiede grande fiducia nei test automatici e nel processo
Frase riassuntiva	"Ci assicuriamo che il codice di tutti funzioni insieme"	"Il software è sempre pronto, ma decidiamo noi quando"	"Il software si rilascia da solo, se supera i controlli"

Diagramma 5

Diagramma 5

Nota come, nel diagramma, **CI è sempre presente** in entrambi gli scenari: è la base comune. La differenza tra Continuous Delivery e Continuous Deployment si gioca esclusivamente sull'ultimo passaggio, quello dell'ingresso in produzione: **presenza o assenza di un'approvazione umana**. Se ricordi solo una cosa di questa sezione, ricorda questa: CI riguarda l'integrazione del codice; Delivery e Deployment riguardano entrambi il rilascio, e si distinguono solo per **chi (o cosa) preme l'ultimo bottone**.

Sia con Delivery che con Deployment, però, il codice di ShopFacile deve comunque “atterrare” da qualche parte: su server, reti e database che qualcuno deve aver preparato in anticipo. Il prossimo paragrafo racconta come anche questa parte — l'infrastruttura, non solo il codice applicativo — sia diventata “automazione”, esattamente come build e test.

9.8 Infrastructure as Code (IaC): l'infrastruttura scritta come codice

Fino a qui abbiamo parlato soprattutto di codice applicativo (quello che implementa le funzionalità). Ma un software, per funzionare, ha bisogno anche di **infrastruttura**: server su cui essere eseguito, reti che permettano le comunicazioni, database per salvare i dati, regole di sicurezza che decidano chi può accedere a cosa.

Storicamente, questa infrastruttura veniva creata e configurata **a mano**: una persona si collegava a un server, installava software, modificava file di configurazione, cliccava su pannelli di amministrazione — un lavoro lento, difficile da ripetere in modo identico, e soggetto a errori umani (proprio i problemi che l'automazione, vista nella sezione 9.5, cerca di risolvere).

Infrastructure as Code (IaC) applica esattamente la stessa logica del controllo di versione del codice (Git, che hai visto nella sezione 4) all'infrastruttura: invece di configurare server e reti manualmente, **si scrivono file di configurazione che descrivono esattamente l'infrastruttura desiderata**, e uno strumento automatico legge quei file e crea (o modifica) l'infrastruttura reale in modo coerente con quanto descritto.

Analogia: pensa alla differenza tra costruire una casa “a memoria”, decidendo ogni misura sul momento parlando a voce con gli operai, e costruirla seguendo un progetto architettonico scritto, preciso, misurabile, che può essere consultato, corretto, e — soprattutto — **rieseguito identico** per costruire una seconda casa uguale alla prima in un altro terreno. Il progetto scritto (il “codice” dell'infrastruttura) permette a chiunque di capire esattamente cosa è stato costruito, di modificarlo in modo controllato, e di ricostruirlo da zero in caso di disastro, senza dover “ricordare a memoria” cosa era stato fatto.

Esempio pratico: ecco un estratto semplificato (sintassi approssimativa, solo per farti capire l'idea, non un file da eseguire davvero) di come potrebbe apparire un file IaC che descrive il server web e il database di ShopFacile:

```

resource "server_web" {
  nome          = "server-shopfacile-produzione"
  dimensione    = "media" # 2 CPU, 4 GB RAM
  sistema       = "Linux"
  porta_aperta  = 443      # HTTPS
}

resource "database" {
  nome          = "db-ordini-shopfacile-produzione"
  tipo          = "PostgreSQL"
  backup_auto   = true
  accesso       = "solo da server_web"
}

```

Questo file descrive **cosa deve esistere**, non i singoli comandi per crearlo. Uno strumento IaC (es. Terraform) legge questo file e si occupa lui di creare davvero il server e il database nel cloud, con quelle caratteristiche esatte. Se domani serve un secondo ambiente identico per il collaudo, basta eseguire lo stesso file puntandolo a un ambiente diverso: niente configurazione manuale ripetuta a mano, niente rischio di “dimenticare un dettaglio” rispetto all’ambiente di produzione.

I vantaggi concreti di questo approccio:

- **Ripetibilità:** lo stesso file di configurazione produce sempre la stessa infrastruttura, sia che lo esegua un ingegnere esperto sia che lo esegua un collega alle prime armi.
- **Versionamento:** i file IaC si salvano in Git, come il codice applicativo — si può vedere la storia di ogni modifica, chi l’ha fatta, perché, e tornare indietro se necessario.
- **Revisione:** una modifica all’infrastruttura può passare per una Pull Request e una code review, esattamente come una modifica al codice — invece di essere una modifica manuale invisibile fatta da una persona su un server, di cui nessun altro sa nulla.
- **Velocità e scalabilità:** creare dieci ambienti di test identici, o ricreare rapidamente un intero ambiente dopo un disastro, diventa una questione di minuti, eseguendo lo stesso file, invece di settimane di lavoro manuale ripetitivo.

Alcuni strumenti diffusi di IaC che potresti incontrare (li vedrai citati anche nella sezione 13 sul Cloud):

- **Terraform:** uno degli strumenti IaC più diffusi in assoluto, utilizzabile con diversi fornitori cloud (AWS, Azure, Google Cloud e altri) con un linguaggio di configurazione comune.
- **ARM Template / Bicep:** gli strumenti IaC “nativi” di Microsoft Azure, con Bicep pensato come evoluzione più semplice da scrivere e leggere rispetto agli ARM Template originali (basati su JSON).

Diagramma 6

Diagramma 6

Una volta che server e database di ShopFacile sono stati creati — a mano o, meglio, tramite IaC — il lavoro non finisce lì: bisogna sapere, giorno per giorno, se quell'infrastruttura e il software che ci gira sopra stanno funzionando bene. È il compito del monitoring, che vediamo subito.

9.9 Monitoring: sapere se le cose stanno andando bene

Una volta che il software è in produzione (fase “Operate” del ciclo DevOps), non basta “sperare che vada tutto bene”: bisogna **osservarlo attivamente**. Questo è il compito del **monitoring** (monitoraggio).

Il monitoring è l'attività di **raccogliere e osservare dati sullo stato di un sistema in tempo reale**, per rispondere a una domanda semplice ma fondamentale: **“è tutto ok, o c'è un problema?”**.

Concretamente, significa tenere sotto controllo indicatori come: il sito web risponde? Quanto tempo impiega a rispondere? Il server ha ancora memoria disponibile? Ci sono errori nei log?

Analogia: il monitoring è come il cruscotto di un'automobile mentre guidi: la spia della temperatura del motore, l'indicatore di velocità, la spia della pressione delle gomme. Ti dicono, in modo semplice e immediato, **se qualcosa è fuori dai parametri normali** — ma non ti spiegano *perché* il motore si sta scaldando troppo. Per quello serve altro (lo vedremo con l'observability, nella prossima sezione).

Strumenti tipici di monitoring includono **dashboard** con grafici in tempo reale, e **alert** (avvisi automatici, spesso via email, SMS o strumenti come Slack/Teams) che si attivano quando un indicatore supera una soglia critica — ad esempio, “il tempo di risposta del sito ha superato i 3 secondi” o “meno del 5% di spazio disco disponibile”. In questo modo, il team di ShopFacile viene avvisato **prima** che un problema diventi visibile agli utenti finali, o quasi contemporaneamente al primo utente che lo nota — non ore o giorni dopo.

Il monitoring, però, ha un limite che vale la pena anticipare subito: ti dice *che* qualcosa non va (il sito è lento), ma non sempre ti dice *perché*. Per quel “perché” serve un concetto più ampio, che vediamo nel prossimo paragrafo: l'observability.

9.10 Observability: capire non solo il “cosa”, ma il “perché”

Observability (osservabilità) è un concetto più ampio del monitoring, ed è importante distinguerlo bene, perché nel lavoro quotidiano i due termini vengono spesso confusi.

Il monitoring, come abbiamo visto, risponde principalmente alla domanda **“cosa sta succedendo”**: il sistema è su o giù, la CPU è alta o bassa, il tempo di risposta è nella norma o no. È molto utile, ma ha un limite: di solito richiede di aver **già previsto in anticipo** cosa monitorare (hai bisogno di sapere già quale spia mettere sul cruscotto).

L'**observability** va oltre: è la **capacità di capire perché un sistema si comporta in un certo modo**, anche di fronte a un problema che non era stato previsto in anticipo, senza dover aggiungere nuovo codice o nuovi strumenti di misurazione *dopo* che il problema si è già verificato. Un sistema è “osservabile” quando, guardando i dati che già produce, un ingegnere è in grado di **ricostruire e diagnosticare** un comportamento anomalo, anche complesso o mai visto prima.

Analogia: se il monitoring è il cruscotto dell'auto (spie predefinite su parametri già noti), l'observability è come avere a disposizione **tutti i dati del motore, del sistema elettrico e dei sensori**, più uno strumento diagnostico da meccanico, che ti permette — anche di fronte a un problema mai visto prima, per cui non esisteva una spia dedicata — di indagare a fondo e capire esattamente cosa sta succedendo e perché, ricostruendo la catena di eventi. Non ti basta sapere che “il motore scalda troppo” (questo te lo dice già la spia): ti serve poter scoprire *perché* scalda, magari per la prima volta in assoluto, senza dover smontare il motore alla cieca.

I tre pilastri dell'observability

L'observability si costruisce tipicamente su tre tipi di dati complementari, spesso chiamati i suoi “tre pilastri”:

1. **Metriche (metrics):** numeri aggregati nel tempo, come il numero di richieste al secondo, il tempo medio di risposta, la percentuale di CPU utilizzata. Sono leggere da raccogliere e ottime per grafici e alert (il “cosa” del monitoring).
2. **Log:** registrazioni testuali di eventi specifici avvenuti nel sistema (li vedremo nel dettaglio nella prossima sezione). Sono più dettagliati delle metriche: raccontano *cosa è successo esattamente*, in un dato momento, spesso con informazioni di contesto (un errore, un messaggio, un identificativo di richiesta).
3. **Tracce (traces):** seguono il percorso completo di una singola richiesta di un utente **attraverso i diversi componenti di un sistema** (ad esempio: il sito web, poi un servizio di autenticazione, poi un database, poi un servizio di pagamento). Sono particolarmente utili nei sistemi complessi, composti da tanti servizi diversi che collaborano tra loro, per capire **in quale punto esatto della catena** si è verificato un ritardo o un errore.

Diagramma 7

Diagramma 7

Un modo semplice per ricordare la relazione tra i due concetti: **il monitoring è un'attività (osservare dashboard e alert); l'observability è una proprietà del sistema** (la sua capacità intrinseca di essere comprensibile e diagnosticabile quando serve). Puoi fare ottimo monitoring di un sistema poco osservabile (hai le spie giuste, ma non riesci a capire il “perché” quando succede qualcosa di nuovo e inatteso); un sistema davvero osservabile, invece, ti dà gli strumenti per indagare in profondità anche su problemi che nessuno aveva previsto in anticipo.

Tra i tre pilastri dell'observability appena visti — metriche, log e tracce — uno in particolare è così radicato nel lavoro quotidiano di un team come quello di ShopFacile da meritare un paragrafo a sé: i log. Vediamoli più da vicino.

9.11 Logging: la memoria degli eventi del sistema

Abbiamo già citato i log come uno dei tre pilastri dell'observability; vale la pena approfondirli separatamente, perché il **logging** (la pratica di produrre e gestire log) è probabilmente lo strumento di diagnosi più antico, più diffuso e più immediato nel lavoro quotidiano di un team tecnico.

Un **log** è, molto semplicemente, **una registrazione testuale di un evento accaduto in un sistema**, con solitamente un timestamp (quando è successo), un livello di gravità (informazione, avviso, errore, errore critico) e un messaggio che descrive cosa è successo.

Analogia: pensa al log come al **diario di bordo di una nave**. Il capitano non scrive solo “oggi tutto bene”: annota eventi specifici, con data e ora precisa — “14:32, avvistata tempesta a nord-ovest”, “18:05, motore sinistro rallentato per manutenzione”, “22:10, rotta corretta di 5 gradi”. Se qualcosa va storto durante il viaggio, il diario di bordo è il primo posto dove si va a cercare *cosa è successo, esattamente, e quando* — non a memoria, ma con un record scritto e preciso di ogni evento rilevante.

Esempio semplificato di alcune righe di log di un'applicazione web, per darti un'idea concreta del formato:

```
2026-07-25 09:14:02 [INFO] Utente 4821 ha effettuato il login
2026-07-25 09:14:05 [INFO] Richiesta GET /catalogo completata in 120ms
2026-07-25 09:15:33 [WARN] Tempo di risposta al database superiore a 500ms
2026-07-25 09:16:01 [ERROR] Pagamento fallito per ordine #98213: timeout verso il
servizio esterno
2026-07-25 09:16:02 [ERROR] Impossibile inviare email di conferma: servizio SMTP non
risponde
```

Da questo semplice estratto, un ingegnere può già iniziare a ricostruire una storia: qualcosa ha iniziato a rallentare (il WARN sul database), poco dopo un pagamento è fallito per timeout verso un servizio esterno, e subito dopo anche l'invio email ha iniziato a fallire — indizi che potrebbero indicare un problema di rete o di un servizio esterno condiviso, non necessariamente un bug nel codice dell'applicazione stessa.

Esempio pratico: monitoring, observability e logging insieme in un incidente reale su ShopFacile. Per capire davvero come questi tre concetti lavorano insieme (e non sono tre cose scollegate), segui questo scenario passo per passo:

1. **Monitoring** (il “cosa”): alle 09:16 un **alert** automatico avvisa il team su Slack/Teams: “tempo di risposta medio del sito superiore a 3 secondi”. La dashboard mostra una linea che sale bruscamente. Il team sa *che* qualcosa non va, ma non ancora *perché*.

2. **Observability** (il “perché”): **Marco** guarda le **tracce** delle richieste più lente e nota che il ritardo si concentra sempre nello stesso punto della catena: la chiamata dal servizio ordini verso un servizio esterno di pagamento. Le **metriche** confermano che è proprio quel servizio esterno ad avere un tempo di risposta anomalo, non il database o il resto dell'applicazione.
3. **Logging** (il dettaglio esatto): Marco apre i **log** dello stesso intervallo di tempo — proprio quelli visti sopra — e trova la riga `[ERROR] Pagamento fallito per ordine #98213: timeout verso il servizio esterno`, che conferma con precisione l'orario, l'ordine coinvolto e la causa tecnica esatta (un timeout).

Risultato: in pochi minuti il team capisce non solo che c'era un problema (monitoring), ma anche dove si trovava nel sistema (observability) e qual era il dettaglio preciso da riportare a chi gestisce quel servizio esterno (logging) — senza dover indovinare nulla.

Perché il logging è così importante nel contesto DevOps:

- **Diagnosi post-incidente**: quando qualcosa va storto (collegandoci direttamente alla blameless culture vista nella sezione 9.4), i log sono spesso la prima fonte di informazione per ricostruire la sequenza esatta degli eventi, senza dover indovinare o fidarsi solo della memoria delle persone.
- **Tracciabilità**: sapere non solo *che* qualcosa è successo, ma *quando* esattamente, e in quale ordine rispetto ad altri eventi.
- **Base per l'automazione**: molti sistemi di alert (visti nel monitoring) si basano proprio sull'analisi automatica dei log — ad esempio, un alert che si attiva se il numero di righe con livello `ERROR` supera una certa soglia in pochi minuti.

Un aspetto pratico da conoscere: nei sistemi moderni, composti da tanti servizi diversi (specialmente nei sistemi che vedrai nella sezione 12 sulle architetture software), i log di ciascun servizio vengono spesso raccolti in un **unico sistema centralizzato**, per poterli cercare e correlare tutti insieme, invece di dover collegarsi manualmente a decine di server diversi per leggere ciascuno il proprio file di log locale.

Con questo, il team di ShopFacile — e tu insieme a loro — ha attraversato tutti i concetti chiave di DevOps, dalla cultura agli strumenti concreti. Prima di passare ad Azure DevOps, la piattaforma dove questi concetti diventano schermate cliccabili, vale la pena fermarsi un momento a ricapitolare il percorso fatto.

9.12 Riepilogo

Questa sezione ha introdotto DevOps non come uno strumento, ma come una **cultura e un insieme di pratiche** che nascono per risolvere un problema storico molto concreto: la separazione, spesso conflittuale, tra chi sviluppa il software e chi lo gestisce in produzione.

- **DevOps** è l'unione (culturale, prima ancora che tecnica) di Development e Operations, per superare “il muro della confusione” tra i due mondi.

- Il **ciclo DevOps** (Plan-Code-Build-Test-Release-Deploy-Operate-Monitor) è rappresentato come un infinito, perché è un flusso continuo, non un progetto con inizio e fine.
- La **cultura DevOps** si basa su collaborazione, responsabilità condivisa (“you build it, you run it”) e **blameless culture**; il modello **CALMS** (Culture, Automation, Lean, Measurement, Sharing) riassume i suoi pilastri fondamentali, con la cultura sempre al primo posto.
- L'**automazione** riduce errori umani e velocizza il lavoro ripetitivo: build, test e deploy automatici sono gli esempi più concreti.
- **Continuous Integration (CI)** integra il codice frequentemente, con build e test automatici ad ogni commit/PR.
- **Continuous Delivery** rende il software sempre pronto al rilascio, ma richiede un'approvazione umana finale; **Continuous Deployment** rilascia in produzione in modo completamente automatico — la differenza sta esattamente in quell'ultimo passaggio.
- **Infrastructure as Code** applica la logica del controllo di versione (Git) all'infrastruttura, rendendola ripetibile, versionata e revisionabile.
- **Monitoring** risponde al “cosa” (è tutto ok?); **observability** va oltre e permette di capire il “perché”, basandosi su metriche, log e tracce; il **logging** è la registrazione dettagliata degli eventi, fondamentale per la diagnosi e la tracciabilità.

Nella prossima sezione vedrai come tutti questi concetti — pipeline CI/CD, board di lavoro, dashboard di monitoraggio — si traducano in funzionalità concrete e cliccabili all'interno di **Azure DevOps**, la piattaforma che il team usa ogni giorno per metterli in pratica.

Esercizi pratici

Questa è una delle sezioni più dense del corso: la teoria da sola non basta a fissarla davvero. Prova a completare questi esercizi con calma, magari uno al giorno, prendendoti il tempo per collegarli a esempi reali del progetto ogni volta che puoi.

1. **Racconta il “muro della confusione” con parole tue.** Scrivi (o registra a voce, se preferisci) una spiegazione di 5-6 righe di cosa fosse il “muro della confusione” tra Dev e Ops, come se dovessi spiegarlo a un amico che non ha mai lavorato in ambito tecnico, usando un esempio diverso da quello del ristorante già visto in questa sezione (puoi ispirarti a qualsiasi altro contesto: un ufficio postale, una catena di montaggio, uno studio medico). **Come verificare:** fatti ascoltare/leggere la spiegazione da un collega non tecnico (o da un familiare): se la capisce senza fare domande di chiarimento, hai centrato il punto.
2. **Applica CALMS al progetto reale.** Per ciascuna delle 5 lettere di CALMS (Culture, Automation, Lean, Measurement, Sharing), trova un esempio concreto nel progetto su cui lavori — anche piccolo — che dimostri quel pilastro in azione. Se per una lettera non trovi nessun esempio, annotalo: potrebbe essere un'area di miglioramento da discutere con la tua collega. **Come verificare:** hai scritto 5 esempi (uno per lettera), ciascuno con un fatto specifico del progetto, non una definizione generica presa dal libro.
3. **Ricostruisci una pipeline CI su carta.** Prendi carta e penna e disegna, passo per passo (come nel diagramma della sezione 9.6), cosa succede da quando uno sviluppatore propone un cambiamento a quando il codice viene integrato nel ramo principale. Poi aggiungi, con un colore

diverso, cosa cambierebbe se il team usasse Continuous Delivery invece che Continuous Deployment nello step finale. **Come verificare:** nel tuo schema devi avere almeno 5 step distinti (proposta del cambiamento, build, test, integrazione, e lo step finale di rilascio) e riuscire a indicare con precisione dove si trova “l’ultimo bottone” premuto da un umano nel caso di Continuous Delivery.

4. **Trova (o fatti mostrare) un file di Infrastructure as Code reale.** Chiedi a un collega developer o operations se il progetto usa IaC (ad esempio Terraform, Bicep o ARM Template) e fatti mostrare un file reale, anche solo per pochi minuti sullo schermo. Non serve capire ogni riga: cerca di individuare almeno una “risorsa” descritta nel file (un server, una rete, un database) e confrontala con l’esempio semplificato visto nella sezione 9.8. **Come verificare:** sai indicare a voce, guardando il file reale, almeno un pezzo di infrastruttura che descrive e a cosa serve nel progetto — anche con parole semplici e imprecise dal punto di vista tecnico.
5. **Scrivi tre righe di log realistiche.** Immagina una funzionalità qualsiasi del progetto (ad esempio “un utente carica un documento”) e scrivi tre righe di log plausibili che quella funzionalità potrebbe generare: una di livello **INFO** per il caso normale, una di livello **WARN** per un caso limite (es. file grande, connessione lenta), e una di livello **ERROR** per un caso di fallimento. Segui il formato visto nella sezione 9.11 (timestamp, livello, messaggio). **Come verificare:** le tue tre righe hanno timestamp, livello di gravità corretto rispetto alla situazione descritta, e un messaggio che un collega — leggendolo senza altro contesto — capirebbe cosa è successo.
6. **Distingui monitoring da observability su un caso reale.** Chiedi a un collega di raccontarti un incidente reale (anche piccolo) capitato sul progetto. Per quell’episodio, identifica separatamente: cosa ha fatto scattare l’allarme iniziale (monitoring — la spia che si è accesa) e cosa ha permesso di capire la causa reale (observability — metriche, log o tracce usate per indagare). Se il collega non ricorda una distinzione così netta, va benissimo: è normale, e ti dà comunque materiale utile. **Come verificare:** riesci a scrivere due frasi separate — una che inizia con “il team si è accorto del problema perché...” (monitoring) e una che inizia con “il team ha capito la causa guardando...” (observability) — riferite allo stesso episodio reale.
7. **Simula un mini post-mortem blameless.** Pensa a un piccolo errore che hai fatto tu stesso di recente, anche non legato al lavoro (es. hai dimenticato un appuntamento, hai fatto un acquisto sbagliato online). Scrivi un breve “post-mortem” in stile blameless culture: cosa è successo (fatti, senza giudizi), quale causa reale ha portato all’errore (una procedura poco chiara? un’informazione mancante?), e un’azione concreta per evitare che si ripeta. Evita ogni frase del tipo “ho sbagliato perché sono stato distratto”: cerca la causa nel “sistema” (promemoria assenti, processo poco chiaro), non solo nella persona. **Come verificare:** il tuo testo non contiene nessuna frase colpevolizzante verso te stesso o altri, e termina con un’azione concreta e verificabile (non un generico “starò più attento”).

Collegamenti

- [10. Azure DevOps](#) — la piattaforma dove le pratiche DevOps viste qui diventano board, pipeline e dashboard concrete

- [11. CI/CD](#) — approfondimento tecnico su pipeline, Continuous Integration, Delivery e Deployment
- [13. Cloud](#) — dove gira davvero l'infrastruttura gestita con Infrastructure as Code

Risorse

- [AWS — Cos'è DevOps](#)
- [Atlassian — DevOps: guida completa](#)
- [Microsoft Learn — Introduzione a DevOps](#)
- [Microsoft Learn — Infrastructure as Code](#)
- [Atlassian — CI/CD: guida completa](#)

10. Azure DevOps

Nelle sezioni precedenti hai visto tanti concetti “in teoria”: cos’è una board Kanban, come funziona uno sprint Scrum, cosa sono un repository, un commit, una Merge Request, e cos’è la cultura DevOps. Tutti questi concetti, nel lavoro reale, non restano sulla lavagna: vivono dentro uno **strumento** che il team usa ogni giorno per pianificare, scrivere codice, testare e rilasciare.

Azure DevOps è uno di questi strumenti — anzi, è una **suite** di strumenti, prodotta da Microsoft, che copre l’intero ciclo di vita di un progetto software in un unico posto. Nel progetto a cui sarai affiancato è molto probabile che tu veda Azure DevOps aperto in una scheda del browser per tutta la giornata: è lì che troverai i work item da seguire, il codice del progetto, lo stato delle pipeline e i grafici da mostrare al cliente.

Questa sezione ti porta “dentro” lo strumento, collegando ogni sua parte a ciò che hai già imparato: non sono concetti nuovi da zero, sono le stesse idee (backlog, board, repository, pipeline) che ora vedrai **incarnate in un prodotto specifico**.

Per rendere tutto concreto, seguiremo lo stesso progetto già incontrato nella sezione DevOps:

• **ShopFacile**, la piattaforma e-commerce del team. In questa sezione la vedremo gestita proprio dentro Azure DevOps: **Sara** (Product Owner) inserisce requisiti e work item su Azure Boards, mentre **Marco**, **Giulia** e **Ahmed** (i tre developer del team) li implementano usando Repos, Pipelines e Test Plans.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- cos’è Azure DevOps e perché è definito una suite “end-to-end”;
- cosa sono i cinque moduli principali (Boards, Repos, Pipelines, Test Plans, Artifacts) e a cosa servono nella pratica;
- collegare ogni modulo a un concetto già visto (Scrum, Kanban, Git, CI/CD);
- leggere una Dashboard di progetto e capire cosa rappresentano i suoi widget;
- seguire il percorso end-to-end di una user story, dalla sua creazione su Boards fino al rilascio in produzione.

10.1 Cos’è Azure DevOps

Azure DevOps è una suite di strumenti Microsoft, disponibile sia come servizio cloud (**Azure DevOps Services**) sia installabile sui server dell’azienda (**Azure DevOps Server**, la versione “on-premises”). Il nome non è casuale: è pensata per supportare esattamente la cultura **DevOps** che hai visto nella sezione precedente, cioè l’unione tra sviluppo (Dev) e operazioni (Ops) in un flusso continuo, senza muri tra “chi scrive il codice” e “chi lo porta in produzione”.

Analogia: pensa a un cantiere edile in cui, invece di avere l'architetto in uno studio separato, i muratori in un altro cantiere, e chi consegna i materiali in un magazzino scollegato da tutti, esiste **un unico centro operativo**: lì trovi le planimetrie aggiornate (i requisiti), la lista dei materiali da ordinare (i pacchetti software), lo stato di avanzamento di ogni stanza (le board), e il registro di collaudo di ogni impianto (i test). Tutti guardano lo stesso posto, con la stessa versione delle informazioni.

Il punto di forza principale di Azure DevOps è proprio questo: **integrazione**. Invece di usare uno strumento diverso per gestire il backlog, un altro per il codice, un altro per le pipeline di build e un altro ancora per i test, tutto vive in un solo posto, con collegamenti diretti tra le parti (es. un commit può referenziare direttamente un work item, una pipeline può aggiornare automaticamente lo stato di una user story).

Azure DevOps è organizzato in **cinque moduli principali**, ognuno accessibile da un menu laterale all'interno di un progetto:

Modulo	A cosa serve, in una frase
Boards	Gestire il lavoro: backlog, sprint, board Kanban
Repos	Ospitare il codice sorgente (repository Git)
Pipelines	Automatizzare build, test e rilascio (CI/CD)
Test Plans	Pianificare ed eseguire test manuali e automatizzati
Artifacts	Conservare e distribuire pacchetti software

Nei prossimi paragrafi vediamo ciascuno di questi moduli, uno per uno, sempre collegandoli a concetti che hai già incontrato nel corso — e sempre seguendo lo stesso progetto ShopFacile, per vedere come si muove concretamente da un modulo all'altro.

Cominciamo dal punto da cui parte sempre il lavoro, prima ancora che qualcuno scriva una riga di codice: **cosa c'è da fare**. È il modulo Boards, dove Sara inserisce le richieste del cliente.

10.2 Boards: la gestione del lavoro

Boards è il modulo che gestisce **tutto il lavoro** del progetto: cosa c'è da fare, chi ci sta lavorando, a che punto è. Se hai letto le sezioni su Scrum e Kanban, qui non trovi nulla di concettualmente nuovo — trovi esattamente quegli stessi concetti, ma implementati come schermate concrete di uno strumento.

Work item: l'unità base del lavoro

In Azure DevOps, ogni “elemento di lavoro” — che nelle sezioni precedenti abbiamo chiamato genericamente “card” o “elemento del backlog” — si chiama **work item**. Esistono diversi **tipi** di work item, organizzati in una gerarchia che va dal più generale al più specifico:

- **Epic**: un obiettivo molto grande, che richiede mesi di lavoro e si scompone in più Feature (es. “Area riservata clienti”).
- **Feature**: un pezzo di funzionalità consistente, che si scompone in più User Story (es. “Login e gestione profilo utente”).
- **User Story**: una singola richiesta dal punto di vista dell'utente, esattamente come l'hai vista nella sezione su Scrum (es. “Come utente, voglio recuperare la password se la dimentico”).
- **Task**: un'attività tecnica concreta necessaria per completare una User Story (es. “Creare l'endpoint API per il reset password”).
- **Bug**: un malfunzionamento da correggere (es. “Il link di reset password scade dopo 1 minuto invece di 24 ore”).

Diagramma 1

Diagramma 1

Analogia: pensa a una gerarchia come quella di un libro. L'Epic è l'intero **libro** (es. “Storia dell'Europa”), la Feature è un **capitolo** (es. “Il Rinascimento”), la User Story è un **paragrafo** che racconta un evento specifico, e il Task è una singola **frase** che devi scrivere per completare quel paragrafo. Il Bug è come un errore di stampa scoperto dopo la pubblicazione, da correggere in una ristampa.

Esempio pratico: immagina che **Sara**, la Product Owner di ShopFacile, riceva dal cliente la richiesta “vogliamo permettere ai clienti di scaricare la fattura del proprio ordine in PDF”. Sara crea una User Story con titolo “Come cliente di ShopFacile, voglio scaricare la fattura del mio ordine in PDF, così da poterla archiviare”, ID **#341**, stato **New** e una stima di 5 Story Point. Durante lo Sprint Planning il team aggiunge due Task collegati: **#342 Creare l'endpoint di generazione PDF**, assegnato a **Marco**, e **#343 Aggiungere il bottone "Scarica fattura" nell'interfaccia**, assegnato ad **Ahmed**. Se durante il test **Giulia** nota che il PDF esce senza il logo di ShopFacile, apre un nuovo Bug **#350**, collegato alla User Story **#341**: il collegamento tra i work item permette di ricostruire in ogni momento “da dove nasce” quella correzione, senza doverlo chiedere a voce a chi l'ha scritta.

Backlog, Sprint Board e board Kanban

Boards offre diverse **viste** sullo stesso lavoro, a seconda di cosa ti serve vedere:

- **Backlog**: la lista ordinata per priorità di tutte le User Story (o Feature) ancora da fare — è l'implementazione concreta del Product Backlog che hai visto nella sezione su Scrum.

- **Sprint Board:** la board che mostra il lavoro dello sprint corrente, organizzato in colonne di stato (es. To Do, In Progress, Done) — è dove il team guarda ogni giorno durante la Daily Scrum.
- **Board Kanban:** una board a flusso continuo, con colonne personalizzabili e limiti di WIP configurabili — è esattamente la board Kanban che hai visto nella sezione 7, ma cliccabile e configurabile direttamente nello strumento (puoi impostare il limite di WIP di una colonna direttamente nelle sue impostazioni).

Diagramma 2

Diagramma 2

Esempio pratico: durante lo Sprint Planning (visto nella sezione Scrum), il team di ShopFacile seleziona alcune User Story dal Backlog — tra cui la **#341** sulla fattura PDF — e le “porta” nello sprint. Da quel momento, quelle stesse User Story appaiono sulla Sprint Board, e ogni Task collegato può essere spostato di colonna man mano che avanza — esattamente il movimento delle card che hai visto nel capitolo su Kanban, solo che qui avviene con un drag-and-drop del mouse.

Una volta che la User Story **#341** è entrata nello sprint, qualcuno deve effettivamente scrivere il codice che la implementa. È il momento in cui Marco e Ahmed passano dal guardare la board ad aprire il modulo successivo: Repos.

10.3 Repos: il codice sorgente

Repos è il modulo che ospita i **repository Git** del progetto — se hai letto la sezione 4 su Git e GitLab, qui il concetto ti sarà già familiare: Azure DevOps Repos è, dal punto di vista di Git, praticamente equivalente a GitLab. Sotto il cofano c'è sempre lo stesso motore (Git), cambia solo la “concessionaria” che lo ospita.

Tutto quello che hai imparato su Git resta identico:

- si lavora con **branch**, **commit**, **merge**;
- la collaborazione passa per una richiesta di unione, che in Azure DevOps si chiama **Pull Request** — un nome diverso da quello usato su GitLab (**Merge Request**), ma esattamente lo stesso concetto;
- si possono collegare i commit e le Pull Request ai work item di Boards (es. scrivendo **Fixes #123** in un commit, la User Story numero 123 si aggiorna automaticamente).

Le differenze pratiche rispetto a GitLab sono minime e soprattutto di **interfaccia** e di **integrazione**:

	GitLab	Azure DevOps Repos
Motore	Git	Git
Merge/Pull Request	Sì (si chiama Merge Request)	Sì (si chiama Pull Request — stesso concetto)
Issue	Sì (modulo Issues)	Sostituito dai work item di Boards
Pipeline collegata	GitLab CI/CD	Azure Pipelines
Punto di forza	Molto diffusa anche self-hosted, forte integrazione CI/CD nativa	Integrazione nativa con Boards, Pipelines, Test Plans nello stesso prodotto

Esempio pratico: in Azure DevOps, quando **Marco** apre una Pull Request per unire il branch `feature/fattura-pdf` dentro `main`, la collega direttamente alla User Story **#341** “Fattura PDF” di ShopFacile: chi guarda quella User Story su Boards vede subito il link alla Pull Request corrispondente, e quando **Giulia** completa la revisione e la PR viene completata, lo stato della User Story può aggiornarsi automaticamente. È la stessa idea del “Closes #42” di GitLab che hai visto nella sezione 4, ma ancora più integrata perché Boards e Repos vivono nello stesso prodotto.

Il codice di Marco è ora unito nel branch principale — ma unito non significa ancora “pronto per gli utenti”. Perché quel codice diventi davvero parte di ShopFacile in produzione, deve prima passare per build e test automatici: è il modulo Pipelines, il prossimo che vediamo.

10.4 Pipelines: build e rilascio automatizzati

Pipelines è il modulo di **CI/CD** (Continuous Integration / Continuous Delivery) di Azure DevOps: automatizza tutto ciò che deve succedere al codice dopo che è stato scritto — compilarlo, testarlo, e distribuirlo negli ambienti di destinazione. Approfondirai il concetto di CI/CD in dettaglio nella sezione 11; qui ci concentriamo su come si concretizza dentro Azure DevOps.

Una pipeline si compone tipicamente di due fasi concettuali:

- **Build:** compila il codice, esegue i test automatici, e produce un “pacchetto” pronto per essere distribuito (chiamato spesso *artifact* di build — da non confondere con il modulo Artifacts che vedremo dopo, anche se il concetto è collegato).
- **Release:** prende quel pacchetto e lo distribuisce nei vari ambienti (es. prima in un ambiente di Test, poi in Staging, infine in Produzione), spesso con approvazioni manuali tra un ambiente e il successivo.

Diagramma 3

Diagramma 3

YAML pipeline vs Classic: “pipeline as code”

Azure DevOps offre due modi per definire una pipeline:

- **Classic pipeline:** si costruisce visivamente, con un editor grafico a “trascina e configura” i passaggi, senza scrivere codice.
- **YAML pipeline:** si definisce scrivendo un file di testo in formato YAML (un formato di testo semplice e leggibile, simile a un elenco annidato), che descrive passo per passo cosa la pipeline deve fare.

La modalità YAML è oggi quella raccomandata e più usata, perché applica un principio chiamato **“pipeline as code”**: la definizione della pipeline non vive in una configurazione grafica separata e “invisibile”, ma è un **file di testo salvato nel repository stesso**, insieme al codice.

Analogia: pensa alla differenza tra assemblare un mobile IKEA seguendo solo dei disegni a mano libera che tieni in testa (pipeline classica: funziona, ma solo tu sai esattamente come l'hai fatto e non è facile ripeterlo identico) oppure seguendo il **libretto di istruzioni scritto**, numerato, che chiunque può leggere, modificare, versionare e ripetere esattamente allo stesso modo (pipeline YAML): il libretto stesso diventa un documento che puoi salvare, confrontare tra versioni diverse, e su cui puoi anche aprire una Pull Request se vuoi cambiarlo.

Un piccolo esempio illustrativo di come appare un file YAML di pipeline (non serve che tu sappia scriverlo, solo che tu lo riconosca se lo vedi):

```
trigger:
  - main

steps:
  - script: npm install
    displayName: "Installa le dipendenze"
  - script: npm test
    displayName: "Esegue i test automatici"
  - script: npm run build
    displayName: "Compila l'applicazione"
```

Il vantaggio pratico più importante di avere la pipeline “come codice”: se qualcosa nella pipeline si rompe o cambia, **si vede nella storia di Git**, esattamente come per qualsiasi altra modifica al codice — puoi capire chi ha cambiato cosa e quando, e volendo tornare indietro a una versione precedente.

Esempio pratico: Marco fa il push del commit con la sua modifica sul branch `main` di ShopFacile. In automatico, la pipeline si attiva (trigger) ed esegue in sequenza: installazione delle dipendenze, esecuzione dei test automatici, compilazione. Se un test fallisce, la pipeline si ferma e risulta “rossa” (failed): Marco riceve una notifica, corregge il problema e fa un nuovo commit, che fa ripartire tutta la sequenza da capo. Se invece tutti i passaggi vanno a buon fine, la pipeline

è “verde” (succeeded) e il pacchetto compilato passa alla fase di Release: prima viene distribuito automaticamente nell'ambiente di Test, poi resta in attesa dell'approvazione di un responsabile prima di passare in Staging, e infine in Produzione — con un'ulteriore approvazione, perché il rilascio in Produzione è quasi sempre un passaggio “manuale per scelta”, anche se la build è completamente automatica.

Prima che il pacchetto arrivi davvero in Produzione, però, c'è ancora un controllo da fare, distinto da quello automatico appena visto: verificare che la fattura PDF si comporti bene anche agli occhi di una persona che la prova manualmente. È il compito del modulo Test Plans.

10.5 Test Plans: qualità e collaudo

Test Plans è il modulo dedicato alla gestione dei **test**, sia manuali che automatizzati. Mentre le pipeline (visto sopra) eseguono automaticamente test già scritti nel codice, Test Plans serve soprattutto a organizzare e tracciare **test manuali** — cioè test eseguiti a mano da una persona, seguendo dei passaggi definiti — insieme a una vista d'insieme anche sui test automatizzati collegati alle pipeline.

I concetti chiave sono:

- **Test Plan**: un contenitore che raggruppa i test da eseguire per un determinato obiettivo (es. “Test Plan per la Release 2.3.0”).
- **Test Suite**: un sotto-raggruppamento di casi di test all'interno di un Test Plan (es. “Suite: Login e autenticazione”).
- **Test Case**: il singolo test, con **passaggi precisi** da seguire e il **risultato atteso** di ognuno.

Esempio pratico: un Test Case per la User Story **#341** “Fattura PDF” di ShopFacile potrebbe essere: 1. Vai alla pagina “I miei ordini” e clicca “Scarica fattura”. **Risultato atteso**: si scarica un file PDF. 2. Apri il PDF scaricato. **Risultato atteso**: il PDF contiene il logo di ShopFacile, i dati dell'ordine e il totale corretto. 3. Prova a scaricare la fattura di un ordine di un altro cliente modificando l'URL a mano. **Risultato atteso**: l'accesso viene negato.

Giulia, che in questo esempio esegue il test, segue questi passaggi uno per uno, e per ciascuno segna se il risultato osservato corrisponde a quello atteso (**Passed**) o no (**Failed** — che tipicamente genera automaticamente un nuovo work item di tipo Bug, collegato al test fallito, come il Bug **#350** visto al paragrafo 10.2).

Analogia: pensa a un Test Plan come alla **checklist di collaudo** di un'automobile prima che esca dalla fabbrica: freni, luci, climatizzatore, ognuno con un passaggio preciso da verificare e un esito da segnare. Alcuni controlli li fa una macchina automaticamente (i test automatizzati eseguiti

nella pipeline), altri richiedono ancora l'occhio di una persona (i test manuali di Test Plans) — ad esempio, valutare se un'interfaccia “si sente giusta” da usare è qualcosa che un test automatico difficilmente può giudicare bene.

Perché serve un modulo dedicato e non basta “provare a mano” senza tracciare nulla? Perché tracciare i test permette di rispondere con sicurezza a domande fondamentali per un Project Manager, come: “questa funzionalità è stata testata prima del rilascio?”, “quali test sono falliti nell'ultima release?”, “quanta copertura di test abbiamo su questa Feature?”.

Il codice della fattura PDF, ormai compilato e testato, non nasce dal nulla: come qualsiasi altra funzionalità di ShopFacile, si appoggia a librerie esterne e, a sua volta, può diventare una libreria riutilizzabile da altre parti del progetto. È qui che entra in gioco il modulo Artifacts.

10.6 Artifacts: i pacchetti software

Artifacts è il modulo che funge da **repository di pacchetti software**: un magazzino centrale dove si conservano librerie e componenti di codice riutilizzabile, pronti per essere scaricati e usati da altri progetti o da altre parti dello stesso progetto.

Per capire perché serve, fai un passo indietro: quasi nessun software si scrive completamente da zero. Ogni progetto si appoggia a centinaia di **pacchetti** già scritti da altri (librerie che gestiscono, ad esempio, l'invio di email, la formattazione delle date, la validazione di form) — esistono formati standard di pacchetto per ogni linguaggio, come **NuGet** (per .NET), **npm** (per JavaScript) o **Maven** (per Java). Un repository di pacchetti è il “magazzino” da cui questi pacchetti vengono scaricati e in cui, se il team ne scrive uno riutilizzabile per uso interno, può essere pubblicato per gli altri progetti dell'azienda.

Analogia: pensa a un repository di pacchetti come alla **dispensa condivisa** di un ristorante con più cucine (più progetti). Invece che ogni cuoco (ogni team) debba coltivare da solo il proprio basilico o macinare la propria farina, esiste una dispensa comune con ingredienti pronti, etichettati con un nome e un **numero di versione** (es. “farina-00, versione 2.1.0”): quando serve, si preleva l'ingrediente giusto nella versione giusta, senza doverlo rifare da zero ogni volta.

Perché serve un repository di pacchetti **privato** (come quello offerto da Artifacts), invece di usare solo repository pubblici come npmjs.com o NuGet.org? Alcuni motivi molto concreti:

- pubblicare pacchetti scritti internamente (es. una libreria comune con le funzioni condivise tra più progetti dello stesso cliente), **senza** renderli pubblici su internet;
- avere il controllo su **quali versioni** dei pacchetti pubblici il team è autorizzato a usare (per motivi di sicurezza e stabilità);
- avere un unico posto dove tracciare **quali pacchetti e quali versioni** sono effettivamente in uso nel progetto.

Esempio pratico: il team di ShopFacile scrive una libreria interna che gestisce la formattazione degli importi in euro secondo le regole del cliente (es. “1.234,56 €”), usata sia dal sito web sia dall’app mobile. Invece di copiare e incollare lo stesso codice in due repository diversi (con il rischio che, dopo una correzione, uno dei due resti “vecchio”), **Marco** pubblica la libreria come pacchetto su Artifacts con nome `shopfacile-formattazione-valuta`, versione `1.0.0`. Sia il sito che l’app la scaricano da lì come dipendenza. Quando **Ahmed** corregge un bug di arrotondamento, pubblica la versione `1.0.1`: ogni progetto che dipende dalla libreria può aggiornarsi a quella versione quando è pronto, senza che nessuno debba “andare a modificare a mano” il codice in più posti.

Diagramma 4

Diagramma 4

Boards, Repos, Pipelines, Test Plans e Artifacts producono, ogni giorno, una quantità enorme di piccoli segnali: sprint in corso, pipeline che passano o falliscono, bug aperti. Nessuno vuole aprire cinque moduli diversi ogni mattina solo per farsi un’idea generale: è qui che entra in gioco l’ultimo modulo, la Dashboard.

10.7 Dashboard: lo stato del progetto a colpo d’occhio

Dashboard è il modulo di **visualizzazione**: una pagina, personalizzabile, composta da **widget** (piccoli riquadri, ciascuno con un grafico o un numero), che mostra in tempo reale lo stato del progetto.

Alcuni widget tipici che troverai in un progetto reale:

- un grafico a **burndown** dello sprint corrente (quanto lavoro resta rispetto al tempo rimasto — utile per uno Scrum Master);
- il numero di **Pull Request aperte** in attesa di revisione;
- lo stato dell’ultima **pipeline** eseguita (verde = passata, rossa = fallita);
- un elenco dei **Bug attivi**, magari filtrati per priorità;
- il numero di **Test Case** passati/falliti nell’ultima esecuzione.

Analogia: pensa alla Dashboard come al **cruscotto di un’auto**: non guidi guardando il motore smontato pezzo per pezzo, guardi pochi indicatori sintetici (velocità, livello benzina, spia dell’olio) che ti dicono subito se tutto va bene o se c’è qualcosa da controllare. Una Dashboard di Azure DevOps fa lo stesso con lo stato del progetto: non devi aprire ogni singolo modulo per capire come vanno le cose, un’occhiata alla Dashboard basta per uno stato generale.

Le Dashboard sono particolarmente utili per un **Project Manager** proprio per questo motivo: puoi costruire (o farti costruire) una Dashboard riassuntiva da mostrare in una riunione con il cliente, senza dover spiegare ogni singolo dettaglio tecnico — i numeri e i grafici parlano da soli.

Esempio pratico: è lunedì mattina e devi preparare in 5 minuti un aggiornamento rapido per il cliente. Apri la Dashboard del progetto e vedi: il burndown dello sprint mostra che il team è leggermente indietro rispetto alla linea ideale; ci sono 3 Pull Request aperte in attesa di revisione da più di un giorno; l'ultima pipeline eseguita è verde (passata); ci sono 2 Bug attivi con priorità alta. Da questi soli quattro numeri, senza aprire Boards, Repos o Pipelines singolarmente, puoi già dire al cliente: “siamo leggermente in ritardo sullo sprint, probabilmente per le revisioni di codice in coda, ma la qualità del rilascio è sotto controllo e stiamo lavorando su due bug prioritari” — una sintesi corretta costruita in pochi secondi.

10.8 Il flusso end-to-end: dalla User Story alla produzione

Ecco come i cinque moduli si integrano concretamente, seguendo il percorso di una singola User Story dall'inizio alla fine:

Diagramma 5

Diagramma 5

Ogni freccia di questo diagramma è un collegamento **automatico o semi-automatico** tra moduli: è proprio questa catena di collegamenti che rende Azure DevOps una suite “integrata” e non solo una collezione di strumenti scollegati tra loro.

10.9 Mappa dei concetti: cosa hai già visto, con un altro nome

Una delle cose più utili che puoi fare, arrivato a questo punto del corso, è collegare ogni modulo di Azure DevOps a un concetto che hai già imparato nelle sezioni precedenti. Ecco una mappa riassuntiva:

Concetto Azure DevOps	Concetto già visto	Sezione del corso
Boards → Backlog, Sprint Board	Product Backlog, Sprint Backlog	6. Scrum
Boards → Board Kanban, limiti di WIP	Board Kanban, WIP	7. Kanban
Work item (Epic/Feature/User Story/Task/Bug)	User Story, task, backlog item	6. Scrum
Repos → repository, branch, commit, merge	Repository, branch, commit, merge	4. Git e GitLab
Repos → Pull Request	Merge Request	4. Git e GitLab
Pipelines	CI/CD, integrazione e distribuzione continua	9. DevOps, 11. CI/CD
Pipeline YAML (“pipeline as code”)	Trunk Based Development, automazione	4. Git e GitLab, 9. DevOps
Test Plans	Qualità e collaudo nel ciclo di vita del software	3. Come nasce un software
Artifacts	Gestione delle dipendenze/librerie di un progetto	3. Come nasce un software
Dashboard → burndown	Metriche di flusso e avanzamento	8. Project Management

Questa tabella non è solo un ripasso: è anche il motivo per cui, se hai capito bene le sezioni precedenti, **imparare Azure DevOps diventa molto più semplice** — non stai imparando concetti nuovi, stai imparando dove si trova, nello strumento, ciò che già conosci.

10.10 Riepilogo

- Azure DevOps è una suite Microsoft che copre l'intero ciclo di vita DevOps in un unico prodotto integrato, con cinque moduli principali.
- **Boards** gestisce il lavoro (backlog, sprint, board Kanban) attraverso i work item: Epic, Feature, User Story, Task, Bug.
- **Repos** ospita i repository Git del progetto, con lo stesso motore Git e lo stesso concetto di Pull Request (chiamato Merge Request su GitLab) visto nella sezione 4.
- **Pipelines** automatizza build e rilascio (CI/CD); la modalità YAML applica il principio di “pipeline as code”, cioè la pipeline stessa versionata come un file di testo nel repository.
- **Test Plans** organizza e traccia test manuali e automatizzati, tramite Test Plan, Test Suite e Test Case.
- **Artifacts** è il magazzino condiviso dei pacchetti software (NuGet, npm, Maven), utile per riusare codice senza riscriverlo da zero.
- **Dashboard** mostra lo stato del progetto tramite widget configurabili, utile sia al team che a chi deve comunicare l'avanzamento al cliente.

- I cinque moduli sono collegati tra loro in una catena continua, dalla User Story sul Backlog fino al rilascio in produzione e all'aggiornamento della Dashboard.

Esercizi pratici

1. **Mappa un work item reale.** Chiedi a un collega di mostrarti sullo schermo una User Story reale del progetto su Boards: annota il suo ID, il tipo (Epic/Feature/User Story/Task/Bug), i suoi Task collegati e il suo stato attuale. Poi disegna a mano la gerarchia completa (Epic → Feature → User Story → Task) a cui appartiene. **Come verificare:** se riesci a ridisegnare la gerarchia su un foglio senza guardare lo schermo, e a spiegare a voce cosa cambierebbe se quella User Story venisse spostata da “In Progress” a “Done”, hai capito il concetto.
2. **Segui una Pull Request dall'apertura al collegamento.** Osserva (o fatti raccontare) una Pull Request reale del progetto: qual è il branch di origine e quello di destinazione, quale User Story o Task referencia, chi la sta revisionando, quanti commenti ha ricevuto prima di essere completata. **Come verificare:** sai indicare, senza guardare gli appunti, dove su Boards si vede il collegamento a quella Pull Request e cosa succede allo stato della User Story quando la PR viene completata.
3. **Leggi un file YAML di pipeline reale.** Fatti mostrare (o trova tu stesso, se hai accesso) un file YAML di pipeline del progetto. Senza preoccuparti di capire ogni riga, individua: cosa fa scattare la pipeline (trigger), quali sono gli step principali, e se c'è un passaggio di test prima della build finale. **Come verificare:** riesci a indicare a un collega, puntando col dito sullo schermo, dove nel file YAML si trova il trigger e dove si trova lo step che esegue i test — senza dover chiedere aiuto.
4. **Ricostruisci il flusso end-to-end con un esempio a tua scelta.** Scegli (o inventa) una piccola funzionalità (es. “aggiungere un filtro di ricerca”) e scrivi, passo per passo, come attraverserebbe i cinque moduli di Azure DevOps: da quando viene creata la User Story su Boards, fino a quando la Dashboard mostra lo sprint aggiornato a rilascio avvenuto. Usa come riferimento il diagramma della sezione 10.8. **Come verificare:** confronta il tuo schema con quello della sezione 10.8 — se hai nominato tutti i moduli nell'ordine corretto e sai spiegare perché ogni passaggio avviene dopo il precedente, l'esercizio è riuscito.
5. **Costruisci una Dashboard “immaginaria”.** Su un foglio (o con un tool gratuito di disegno), disegna una Dashboard con 4 widget a tua scelta che vorresti mostrare in una riunione settimanale con il cliente (es. burndown, PR aperte, stato pipeline, bug attivi). Per ciascun widget, scrivi una frase che useresti per commentarlo a voce. **Come verificare:** fatti guardare lo schizzo da un collega o dalla tua tutor e chiedi se i 4 widget scelti sarebbero davvero utili in una riunione reale con il cliente, oppure se ne manca uno importante (es. lo stato dei rischi).
6. **Compila la tabella dei collegamenti a memoria.** Senza guardare la tabella della sezione 10.9, prova a scrivere su un foglio, per ognuno dei cinque moduli (Boards, Repos, Pipelines, Test Plans, Artifacts), il concetto già visto nelle sezioni precedenti a cui corrisponde. **Come**

 **verificare:** confronta il tuo elenco con la tabella della sezione 10.9 — se hai indovinato almeno 4 collegamenti su 5 senza guardare, la mappa concettuale è solida; altrimenti, ripassa i collegamenti mancanti prima di andare avanti.

Collegamenti

-  [11. CI/CD](#) — approfondimento sui concetti di integrazione e distribuzione continua che stanno dietro al modulo Pipelines
- [13. Cloud](#) — dove vengono effettivamente distribuite le applicazioni rilasciate dalle pipeline di Azure DevOps

Risorse

- [Documentazione ufficiale di Azure DevOps](#)
- [Azure Boards](#) — panoramica
- [Azure Repos](#) — panoramica
- [Azure Pipelines](#) — panoramica
- [Azure Test Plans](#) — panoramica
- [Azure Artifacts](#) — panoramica
- [Dashboard in Azure DevOps](#)
- [Pipeline YAML](#) — schema di riferimento

11. CI/CD

Nella sezione DevOps hai già incontrato i concetti di **Continuous Integration** e **Continuous Delivery/Deployment**, applicati al lavoro quotidiano del team di **ShopFacile**: sai che servono a integrare e rilasciare il codice in modo frequente e automatizzato, invece di accumulare mesi di lavoro in un unico rilascio rischioso. Questa sezione resta sullo stesso progetto, ma va un livello più in profondità: non “cosa sono CI/CD e perché servono” (quello lo trovi in [9. DevOps](#)), ma **come funziona davvero, passo per passo, la pipeline** che Marco, Giulia e Ahmed usano ogni giorno per far arrivare le loro modifiche in produzione — cioè lo strumento concreto che rende possibile la CI/CD.

Alla fine di questa sezione, quando in una daily standup qualcuno dirà “la pipeline è rossa sul quality gate della coverage” o “il deploy in staging è partito ma serve l’approvazione manuale per la produzione”, capirai esattamente di cosa si sta parlando.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- ripassare rapidamente la differenza tra Continuous Integration e Continuous Delivery/Deployment;
- descrivere le **fasi tipiche** di una pipeline, dal checkout del codice al deploy in produzione;
- riconoscere i diversi **trigger** che possono far scattare una pipeline;
- distinguere gli **ambienti** (Dev, Test/QA, Staging, Produzione) e capire cosa significa “promuovere” il codice tra di essi;
- capire cos’è un **artifact** e perché viaggia da una fase all’altra della pipeline senza essere ricostruito ogni volta;
- spiegare perché si scrive una pipeline come **codice versionato** (Pipeline as Code) invece di configurarla a mano da un’interfaccia grafica;
- leggere un semplice file YAML che definisce una pipeline;
- capire cos’è un **quality gate** e come blocca un rilascio problematico;
- descrivere cosa succede — e cosa dovrebbe succedere — quando un deploy va male (**rollback**);
- riconoscere alcuni **strumenti concreti** che implementano queste fasi nella realtà (Jenkins, Docker, SonarQube, Dynatrace, Jira Software).

11.1 Ripasso: cosa sono CI e CD

Giusto due righe di richiamo, perché il resto della sezione si basa su questi due concetti.

- **Continuous Integration (CI)**: ogni volta che qualcuno modifica il codice, quella modifica viene automaticamente **integrata** con il resto del progetto, **compilata** e **testata**. Lo scopo è scoprire i problemi di integrazione (il classico “sul mio computer funzionava”) nel giro di minuti, non di settimane.

- **Continuous Delivery / Continuous Deployment (CD)**: una volta che il codice ha superato la CI, viene automaticamente **preparato per il rilascio** (Delivery, con un click finale umano) oppure **rilasciato direttamente** (Deployment, senza intervento umano), fino a produzione.

Se questi concetti non ti sono ancora chiari al 100%, torna un momento alla sezione [9. DevOps](#), dove sono introdotti insieme al perché culturale che c'è dietro. Qui li diamo per acquisiti e ci concentriamo sul **meccanismo pratico**.

Il meccanismo che rende concreti questi due concetti si chiama **pipeline**: una sequenza automatizzata di fasi che il codice attraversa, una dopo l'altra, dal momento in cui viene scritto al momento in cui gira davanti agli utenti reali di ShopFacile. Vediamo ora, fase per fase, come è fatta davvero.

11.2 Anatomia di una pipeline: le fasi tipiche

Immagina una pipeline come una **catena di montaggio in una fabbrica**: il “pezzo” che entra da un lato è il codice appena scritto da uno sviluppatore, e ogni stazione della catena lo controlla, lo trasforma o lo sposta, finché non esce dall'altra parte come un prodotto pronto all'uso, installato e funzionante per l'utente finale. Se una stazione rileva un difetto, il pezzo **non passa alla stazione successiva**: si ferma lì, e qualcuno viene avvisato.

Le fasi (in inglese *stage*) più comuni, nell'ordine in cui compaiono quasi sempre:

1. **Checkout del codice**: la pipeline scarica dal repository Git (GitLab, Azure DevOps Repos...) esattamente la versione di codice che ha fatto scattare la pipeline — un commit specifico, un branch specifico.
2. **Build / Compilazione**: il codice sorgente viene trasformato in qualcosa di eseguibile (per i linguaggi compilati) o comunque preparato per l'esecuzione (installazione delle dipendenze, per linguaggi interpretati). Se il codice ha errori di sintassi o non compila, la pipeline si ferma già qui — è il controllo più veloce e più economico da fare.
3. **Test automatici**: vengono eseguiti i test scritti dal team (unit test, test di integrazione) per verificare che il comportamento del software sia quello atteso. Se anche un solo test fallisce, la pipeline normalmente si ferma.
4. **Analisi di qualità e sicurezza del codice**: strumenti automatici analizzano il codice sorgente (senza eseguirlo) per individuare problemi di leggibilità, complessità eccessiva, duplicazioni, e soprattutto **vulnerabilità di sicurezza** note (es. uso di librerie con falle documentate, pattern di codice pericolosi). Questa fase è spesso chiamata *static analysis* o *code scanning*.
5. **Packaging**: il risultato della build viene “impacchettato” in una forma pronta per essere distribuita e installata — un file eseguibile, un pacchetto installabile, un'immagine Docker (ne parliamo più avanti come *artifact*).
6. **Deploy in ambiente di test**: il pacchetto viene installato in un ambiente dedicato ai test (Test/QA), separato da quello che usano gli utenti reali.

7. **Test di accettazione:** test più vicini al punto di vista dell'utente finale (test end-to-end, test manuali di un QA, a volte test di performance), eseguiti sull'ambiente di test appena aggiornato, per confermare che il software si comporti bene in un ambiente “vero”, non solo nella teoria dei test automatici della fase
- 1.
8. **Deploy in produzione:** solo se tutte le fasi precedenti sono state superate, lo stesso identico pacchetto viene installato nell'ambiente che usano davvero gli utenti finali.

Diagramma 1

Diagramma 1

Un punto importante: **non tutte le pipeline hanno esattamente queste otto fasi**. Un progetto piccolo può avere una pipeline con solo build e test; un progetto più maturo, con requisiti di sicurezza stringenti, può avere fasi aggiuntive (scansione delle dipendenze, test di carico, firma digitale del pacchetto). Il principio, però, resta lo stesso: **fasi in sequenza, ciascuna con la possibilità di fermare tutto se qualcosa non va bene**.

Esempio pratico: Giulia apre alle 10:03 una Merge Request con una modifica al calcolo di uno sconto sul carrello di ShopFacile. Alle 10:04 la pipeline parte da sola: il checkout del codice impiega 5 secondi, la build 40 secondi. Al minuto 2 la fase di test automatici fallisce, perché uno dei test esistenti si aspettava un risultato diverso da quello prodotto dalla sua modifica. La pipeline si ferma **esattamente lì**: non arriva né alla fase di analisi di sicurezza né, ovviamente, al deploy. Giulia riceve una notifica automatica (email o messaggio nella chat del team) con il link al log del test fallito, corregge il codice, fa un nuovo commit sulla stessa MR e la pipeline riparte da capo, dal checkout.

Nell'esempio la pipeline è partita da sola, senza che Giulia dovesse lanciarla a mano: è successo perché ha aperto la Merge Request, uno degli eventi che possono far scattare una pipeline. Vediamo ora quali sono questi eventi, chiamati **trigger**.

11.3 Trigger: cosa fa scattare una pipeline

Una pipeline non gira “sempre”: scatta quando succede un evento specifico, chiamato **trigger**. I trigger più comuni sono:

- **Push su un branch:** ogni volta che qualcuno invia (push) nuovi commit su un branch (tipicamente `main`, o un branch di feature), la pipeline di CI parte automaticamente per validare quel codice.
- **Apertura (o aggiornamento) di una Merge Request:** quando si apre una MR per proporre l'unione di un branch in un altro, la pipeline gira sul codice della MR *prima* che venga approvata e unita — è uno dei modi più efficaci per impedire che codice difettoso entri nel branch principale. Se aggiungi altri commit alla MR, la pipeline riparte.

- **Pianificazione a orario (schedule):** la pipeline parte a intervalli regolari indipendentemente da modifiche al codice — ad esempio ogni notte alle 2:00, per eseguire una suite di test più lunga e completa che non ha senso far girare a ogni singolo commit, o per verificare che tutto continui a funzionare anche senza modifiche (es. dopo un aggiornamento di una libreria esterna).
- **Trigger manuale:** una persona avvia la pipeline a mano, cliccando un bottone “Run pipeline” nell’interfaccia (es. Azure DevOps, GitLab CI/CD). È tipico per il deploy in produzione, dove spesso si preferisce un’azione deliberata di una persona autorizzata, invece che un automatismo completo.

Diagramma 2

Diagramma 2

Esempio pratico: Marco fa un push di alcuni commit sul branch della sua feature per il servizio ordini di ShopFacile. Non deve avvisare nessuno né lanciare nulla a mano: il push stesso è il trigger, e la pipeline di CI (build + test) parte automaticamente, per dare a Marco un feedback rapido su quel codice. Il deploy in produzione, invece, ha un trigger manuale: anche se tutte le fasi precedenti sono andate a buon fine, è una persona (tipicamente il Tech Lead o un ruolo di Release Manager) che clicca “Deploy in produzione” in un momento concordato — ad esempio non di venerdì pomeriggio, per non rischiare un weekend di incidenti con poche persone disponibili a intervenire.

Una volta scattata la pipeline, però, il codice di Marco non resta fermo in un solo posto: attraversa una serie di ambienti diversi, ciascuno con un ruolo preciso. È il prossimo argomento.

11.4 Ambienti: dove “vive” il codice, fase per fase

Il codice, durante il suo percorso, non gira sempre nello stesso posto. Attraversa diversi **ambienti**, ciascuno con uno scopo preciso:

- **Dev (Sviluppo):** l’ambiente dove uno sviluppatore lavora e verifica le prime modifiche, spesso sul proprio computer o in un ambiente condiviso ma instabile. Qui è normale che qualcosa non funzioni: è lavoro in corso.
- **Test / QA:** un ambiente dedicato a verificare che il software funzioni correttamente, con dati simili (ma non identici) a quelli reali. Qui lavorano i tester (automatici e umani) prima che il codice vada oltre.
- **Staging:** un ambiente che riproduce **quanto più fedelmente possibile** la produzione (stessa configurazione, volumi di dati simili, stessa infrastruttura), usato come ultimo controllo prima del rilascio reale. Non tutti i progetti hanno uno staging separato dal Test/QA, ma nei progetti più maturi è una tappa distinta.
- **Produzione:** l’ambiente reale, usato dagli utenti finali. Qui ogni errore ha un impatto reale su persone vere.

Il concetto chiave è quello di **promozione**: lo stesso pacchetto di codice (lo stesso artifact, vedi paragrafo successivo) viene spostato in avanti, ambiente dopo ambiente, solo se supera i controlli previsti in quello attuale. Non si “ricostruisce” il software per ogni ambiente: si promuove **la stessa cosa**, testata identica in ogni tappa, per essere sicuri che quello che finisce in produzione sia esattamente ciò che è stato validato prima.

Analogia: pensa alla selezione per una squadra sportiva nazionale. Un giocatore passa dalle squadre giovanili (Dev), a un torneo regionale di qualificazione (Test/QA), a un'amichevole contro una nazionale di alto livello in condizioni quasi reali (Staging), fino alla partita ufficiale (Produzione). Non cambi giocatore a ogni tappa: **lo stesso giocatore** viene promosso, se supera ogni prova, fino al campo che conta davvero.

Diagramma 3

Diagramma 3

Approfondirai la configurazione pratica di questi ambienti — chi ci accede, come si isolano, come si gestiscono le credenziali diverse per ciascuno — nella sezione [15. Ambienti di sviluppo](#).

Abbiamo detto che il codice viene “promosso” da un ambiente all'altro **senza essere ricostruito**: ma cos'è, esattamente, questa cosa che si sposta identica da Dev a Test/QA a Staging a Produzione? È il momento di dare un nome preciso a quel “pacco”: l'artifact.

11.5 Artifact: il “pacco” che viaggia lungo la pipeline

Un **artifact** è l'output concreto e riutilizzabile prodotto da una fase della pipeline (tipicamente dalla build/packaging), che viene poi passato alle fasi successive **senza doverlo ricostruire ogni volta**.

Esempi concreti di artifact:

- un file eseguibile o una libreria compilata;
- un pacchetto installabile (es. uno `.zip`, un pacchetto per un gestore di dipendenze);
- un'**immagine Docker**, cioè un “contenitore” con dentro applicazione e tutto ciò che serve per farla girare, pronto per essere avviato identico su qualsiasi macchina;
- un report di test o di analisi di sicurezza, allegato come documentazione del rilascio.

Perché è importante non ricostruire il software a ogni fase? Perché se ogni ambiente compilasse il codice da zero, potresti finire per testare una cosa e rilasciarne un'altra leggermente diversa (magari compilata con una versione diversa di una libreria, scaricata in un momento diverso). L'artifact garantisce che **quello che hai testato in Test/QA è esattamente, byte per byte, quello che finisce in produzione** — questo principio si chiama a volte “build once, deploy many” (compila una volta sola, rilascia molte volte).

Diagramma 4

Diagramma 4

Sappiamo ora **cosa** attraversa la pipeline (l'artifact) e **dove** (gli ambienti). Manca un pezzo: **chi decide** l'ordine esatto di queste fasi, e come quella decisione viene scritta e mantenuta nel tempo dal team di ShopFacile. È qui che entra in gioco il concetto di Pipeline as Code.

11.6 Pipeline as Code: la pipeline è codice, non un click di troppo

Nei primi anni degli strumenti di automazione, le pipeline si configuravano quasi sempre **a mano**, da un'interfaccia grafica: si cliccava “aggiungi fase”, si compilavano campi, si salvava. Funzionava, ma aveva problemi seri:

- **non c'era storia delle modifiche**: se qualcuno cambiava una configurazione e qualcosa si rompeva, non c'era modo semplice di vedere “chi ha cambiato cosa e quando” (lo stesso problema che risolve Git per il codice, visto nella sezione 4);
- **non era riutilizzabile**: replicare la stessa pipeline su un altro progetto significava ricliccare tutto daccapo;
- **non era rivedibile in Merge Request**: un collega non poteva fare code review di una configurazione grafica come farebbe su del codice.

La soluzione, oggi lo standard del settore, si chiama **Pipeline as Code**: la definizione della pipeline (quali fasi, in che ordine, con quali condizioni) è scritta in un **file di testo** (quasi sempre in formato YAML), **versionato in Git insieme al codice dell'applicazione**.

I vantaggi pratici di questo approccio:

- la pipeline ha una **cronologia di modifiche** come qualsiasi altro file di codice (chi, quando, perché);
- le modifiche alla pipeline passano per **Merge Request e code review**, come qualsiasi altra modifica importante;
- la pipeline può essere **copiata e riutilizzata** facilmente su altri progetti simili;
- la pipeline **vive insieme al codice che testa**: se un branch richiede una fase diversa, quella definizione sta proprio in quel branch, non in una configurazione globale scollegata dal codice.

Parlare di “file YAML versionato in Git” resta astratto finché non lo si vede scritto per intero: ecco quindi come potrebbe apparire davvero il file che definisce la pipeline di ShopFacile.

11.7 Un esempio di pipeline in YAML

Ecco un esempio semplificato (pseudo-YAML leggibile, non legato a uno strumento specifico) di come potrebbe apparire il file che definisce una pipeline con le fasi principali viste sopra. In Azure DevOps un file di questo tipo si chiama in genere `azure-pipelines.yml`; in GitLab CI/CD si chiama invece `.gitlab-ci.yml` e vive nella radice del repository.

```
# azure-pipelines.yml (esempio semplificato)

trigger:
  branches:
    include:
      - main
      - "feature/*"

pr:
  branches:
    include:
      - main

stages:

  - stage: Build
    jobs:
      - job: CompilaEdEsegue
        steps:
          - checkout: self # 1. Checkout del codice
          - script: npm install
            displayName: "Installa dipendenze"
          - script: npm run build # 2. Build
            displayName: "Compilazione"

  - stage: Test
    dependsOn: Build
    jobs:
      - job: TestAutomatici
        steps:
          - script: npm run test:unit # 3. Test automatici
            displayName: "Unit test"
          - script: npm run lint # 4. Analisi qualità
            displayName: "Analisi statica del codice"
          - task: SecurityScan@1 # 4. Analisi sicurezza
            displayName: "Scansione vulnerabilità"

  - stage: Package
    dependsOn: Test
    jobs:
      - job: CreaArtifact
        steps:
          - script: docker build -t app:${(Build.BuildId)} . # 5. Packaging
            displayName: "Crea immagine Docker"
          - publish: $(Build.ArtifactStagingDirectory)
```

```

    artifact: app-image

- stage: DeployTest
  dependsOn: Package
  jobs:
    - deployment: DeployInTest          # 6. Deploy in Test/QA
      environment: test
      strategy:
        runOnce:
          deploy:
            steps:
              - script: ./deploy.sh test $(Build.BuildId)

- stage: DeployProduzione
  dependsOn: DeployTest
  condition: succeeded()
  jobs:
    - deployment: DeployInProduzione    # 8. Deploy in Produzione
      environment: produzione          # richiede approvazione manuale
      strategy:
        runOnce:
          deploy:
            steps:
              - script: ./deploy.sh produzione $(Build.BuildId)

```

Non preoccuparti di capire ogni singola parola chiave: quello che conta è la **struttura**. Nota come:

- il blocco `trigger` e `pr` in cima definisce **quando** la pipeline parte (i trigger visti al paragrafo 11.3);
- ogni `stage` corrisponde a una delle fasi della catena di montaggio vista al paragrafo 11.2;
- `dependsOn` collega le fasi in sequenza: uno stage non parte se quello precedente non è andato a buon fine;
- l'ambiente `produzione` è configurato (fuori da questo file, nelle impostazioni dello strumento) per richiedere un'**approvazione manuale** prima di procedere — il trigger manuale di cui parlavamo sopra, applicato a un singolo stage invece che a tutta la pipeline.

Guardando questo file, nota lo stage `Test`: contiene non solo gli unit test, ma anche un'analisi di qualità e sicurezza. È proprio lì, in quel punto della pipeline, che entra in gioco uno dei controlli più importanti per un team come quello di ShopFacile: il quality gate.

11.8 Quality Gate: i controlli che possono bloccare tutto

Un **quality gate** (letteralmente “cancello di qualità”) è un punto della pipeline in cui viene verificata una condizione precisa e misurabile: se la condizione non è soddisfatta, la pipeline **si ferma**, e il codice non avanza alla fase successiva — a prescindere da quanta fretta ci sia di rilasciare.

Esempi tipici di quality gate:

Controllo	Condizione tipica	Cosa succede se non è soddisfatta
Test falliti	Tutti i test automatici devono passare	La pipeline si ferma, il team viene notificato, il codice non prosegue
Code coverage	Es. almeno il 70-80% del codice deve essere coperto da test automatici	Se la coverage scende sotto la soglia, la pipeline si blocca (anche se i test esistenti passano)
Vulnerabilità di sicurezza	Nessuna vulnerabilità di livello "alto" o "critico" nelle dipendenze usate	Il rilascio viene bloccato finché la libreria vulnerabile non viene aggiornata o sostituita
Qualità del codice	Es. nessun nuovo problema critico introdotto (complessità eccessiva, duplicazioni)	Il rilascio viene bloccato o segnalato per revisione, secondo le regole del progetto

Esempio pratico: in ShopFacile, il quality gate sulla coverage è impostato all'80%. **Ahmed** aggiunge una nuova funzionalità di 120 righe per il carrello ma scrive test solo per 60 di quelle righe: la coverage complessiva del progetto scende sotto la soglia, e la pipeline si ferma con un messaggio tipo **Quality gate failed: coverage 76% (soglia 80%)**. Ahmed non può "convincere" la pipeline con una buona motivazione: deve aggiungere altri test finché la percentuale non torna sopra soglia. **Giulia**, che segue spesso queste segnalazioni, lo aiuta a capire quali casi mancano di copertura — un'occasione di apprendimento più che un rimprovero, in linea con la blameless culture vista nella sezione DevOps. Solo se davvero giustificato e con l'approvazione di un ruolo autorizzato, il team può decidere consapevolmente di alzare temporaneamente l'eccezione per quel singolo commit, lasciando comunque una traccia scritta del perché.

Analogia: un quality gate funziona come i controlli di sicurezza in aeroporto. Non importa quanta fretta hai di prendere il volo (di rilasciare in produzione): se il metal detector suona (un test fallisce, una vulnerabilità viene trovata), **non passi**, punto. Il gate non è lì per essere scortese: è lì perché le conseguenze di far salire qualcuno con un oggetto pericoloso a bordo (un bug critico in produzione) sono molto peggiori del ritardo di qualche minuto per il controllo.

Il valore dei quality gate non è tecnico in senso stretto: è **organizzativo**. Sostituiscono il giudizio soggettivo ("secondo me va bene così") con una **regola oggettiva e uguale per tutti**, applicata automaticamente a ogni singolo rilascio, senza eccezioni "perché stavolta ho fretta". Questo è uno dei motivi per cui i team DevOps maturi riescono a rilasciare spesso e con affidabilità: i quality gate tolgono dalle spalle delle persone la responsabilità di "ricordarsi di controllare".

Ma anche superando ogni quality gate immaginabile, resta un caso che nessun controllo automatico può eliminare del tutto: un problema che emerge solo quando il codice è già davanti agli utenti reali di ShopFacile. Cosa fa il team in quel momento è l'ultimo argomento di questa parte più "operativa" della pipeline.

11.9 Rollback: cosa succede se un deploy va male comunque

Anche con tutti i quality gate del mondo, un rilascio in produzione può comunque rivelare un problema che nessun test aveva previsto — magari per un comportamento reale degli utenti diverso da quello simulato nei test, o per un'interazione con un sistema esterno non riproducibile in Staging.

Il **rollback** è l'operazione di **tornare alla versione precedente**, funzionante, il più rapidamente possibile, quando ci si accorge che il nuovo rilascio ha introdotto un problema serio in produzione.

Alcune strategie comuni per rendere il rollback rapido e poco rischioso:

- **Mantenere l'artifact della versione precedente pronto:** proprio perché ogni deploy usa un artifact versionato (vedi paragrafo 11.5), tornare indietro spesso significa semplicemente "rilascia di nuovo l'artifact della versione N-1", un'operazione rapida quanto il deploy stesso. Ad esempio, per lo storico di rilasci `deploy.sh produzione 214` (versione attuale, con il problema), il rollback potrebbe essere semplicemente `deploy.sh produzione 213` (versione precedente, nota funzionante).
- **Deploy graduali (blue-green, canary):** invece di sostituire l'intero ambiente di produzione in un colpo, si rilascia la nuova versione solo a una piccola parte del traffico/utenti (canary) o si mantengono due ambienti identici e si sposta il traffico dall'uno all'altro solo dopo aver verificato che tutto funzioni (blue-green). Se qualcosa va storto, si torna a instradare il traffico sulla versione precedente, con impatto minimo sugli utenti.
- **Monitoraggio post-deploy:** metriche e allarmi automatici (es. tasso di errore, tempi di risposta) subito dopo un deploy, per accorgersi di un problema in minuti e non quando arrivano le segnalazioni degli utenti.

x

Diagramma 5

Diagramma 5

Esempio pratico: pochi minuti dopo il deploy della versione 214 del servizio pagamenti di ShopFacile, gli alert segnalano un tasso di errore anomalo sugli acquisti. **Luca**, come Scrum Master, convoca al volo Marco e Giulia per decidere insieme: il problema è abbastanza serio da giustificare un rollback immediato, oppure basta un hotfix rapido? In questo caso decidono per il rollback: si ri-rilascia la versione 213, nota e stabile, mentre Marco indaga con calma sulla causa reale, senza la pressione di dover risolvere tutto a caldo in produzione.

Un punto culturale importante da portare con te come futuro Project Manager: **un rollback non è un fallimento del team, è un successo del processo**. Un'organizzazione che rilascia frequentemente e ha un rollback rapido e testato è molto più sana di una che rilascia raramente sperando che “vada tutto bene”, perché quest'ultima, quando qualcosa va storto (e prima o poi succede), non ha un piano B pronto.

Fin qui abbiamo visto ogni fase, trigger e controllo della pipeline in modo concettuale, senza legarli a un prodotto specifico. Nella pratica, però, il team di ShopFacile usa strumenti reali per implementare ciascuna di queste fasi: è il momento di vederli con i loro nomi.

11.10 Strumenti concreti della pipeline

Finora abbiamo parlato di fasi, trigger, ambienti e quality gate in modo concettuale, senza legarci a un prodotto specifico. Nella pratica, ogni fase della pipeline è quasi sempre implementata da uno strumento reale, e sentirai questi nomi citati costantemente nelle conversazioni tecniche del team di ShopFacile. Vediamone cinque tra i più diffusi, seguendo lo stesso ordine in cui si passano il testimone lungo la pipeline.

Jenkins: l'orchestratore storico della pipeline

Jenkins è un motore di automazione CI/CD **open source**, tra i più storici e diffusi al mondo: orchestra l'esecuzione delle varie fasi della pipeline (build, test, packaging, deploy) tramite **job configurabili**, spesso definiti in un file chiamato **Jenkinsfile** — lo stesso principio di “Pipeline as Code” visto al paragrafo 11.6, applicato a Jenkins.

È un'alternativa **self-hosted** a strumenti di CI/CD nativi come GitLab CI/CD o Azure Pipelines: molte aziende lo scelgono quando vogliono mantenere il pieno controllo sull'infrastruttura di automazione, o quando hanno esigenze di integrazione con strumenti legacy che uno strumento nativo non copre.

Analogia: se GitLab CI/CD è la catena di montaggio già integrata nella fabbrica (lo stesso edificio che ospita anche il magazzino del codice), Jenkins è un impianto di automazione **indipendente**, che puoi installare dove vuoi e collegare a qualsiasi “magazzino” di codice (GitLab, ma anche altri), con la massima flessibilità di configurazione.

Una volta che Jenkins ha completato con successo build e test del servizio ordini di ShopFacile, il risultato non può restare un file sparso sul disco di un agente di build: serve un formato pronto per viaggiare identico fino in produzione. È qui che il testimone passa a Docker.

Docker: il packaging portatile

Abbiamo già incontrato **Docker** al paragrafo 11.5, parlando di artifact: è la tecnologia di **containerizzazione** più diffusa, che permette di “impacchettare” un'applicazione con tutte le sue dipendenze in un'**immagine** eseguibile identica ovunque venga avviata. Nella pipeline, Docker

interviene tipicamente nella fase di **packaging**: il codice compilato e testato viene trasformato in un'immagine versionata (es. `app:1.4.2`), pronta per essere distribuita e avviata identica in Test, Staging e Produzione — lo stesso principio “build once, deploy many” già visto.

Avere un'immagine Docker pronta, però, non significa ancora che sia **sicuro** farla avanzare verso il deploy: prima bisogna verificare che il codice al suo interno superi le soglie di qualità e sicurezza del progetto. È il compito di SonarQube.

SonarQube: il quality gate concreto

SonarQube è uno strumento di **analisi statica del codice** (*static analysis*): esamina il codice sorgente senza eseguirlo, misurando copertura dei test (coverage), complessità, duplicazioni e vulnerabilità di sicurezza note. È, in pratica, **l'implementazione concreta del quality gate** descritto al paragrafo 11.8: se il codice non supera le soglie configurate (es. coverage minima, zero vulnerabilità critiche), SonarQube segnala il fallimento e la pipeline si ferma, esattamente come nell'esempio della coverage all'80% visto in quel paragrafo.

Superato il quality gate di SonarQube e completato il deploy, il lavoro della pipeline non finisce lì: bisogna sapere come si comporta l'applicazione una volta davanti agli utenti reali di ShopFacile. Questo compito spetta a Dynatrace.

Dynatrace: il monitoraggio che informa il rollback

Dynatrace è una piattaforma di **observability e monitoring**: tiene sotto controllo l'applicazione **dopo** il deploy, in tempo reale, rilevando anomalie di performance, errori e comportamenti sospetti in produzione. È esattamente il tipo di “monitoraggio post-deploy” citato al paragrafo 11.9 sul rollback: quando Dynatrace rileva un tasso di errore anomalo dopo un rilascio, quel segnale è spesso l'innescò che porta il team a decidere un rollback, prima ancora che arrivino segnalazioni dagli utenti.

Tutto questo percorso — build, packaging, quality gate, monitoraggio — non nasce dal nulla: parte quasi sempre da un ticket che descrive cosa costruire, ed è a quel ticket che il lavoro tecnico torna a riferirsi. È il ruolo di Jira Software.

Jira Software: il tracking del lavoro, collegato alla pipeline

Jira Software è uno strumento di **work e project tracking**, concettualmente equivalente al modulo **Boards** di Azure DevOps visto nella sezione 10 (Epic, Story, Task che avanzano su una board). Molti team che usano GitLab per il codice continuano a usare Jira, invece delle Issue native di GitLab, per gestire il backlog: in questi casi è comune integrare i due strumenti, referenziando l'ID del ticket Jira (es. `PROJ-123`) nel nome del branch o nel messaggio di commit, così che Jira mostri automaticamente il collegamento alla Merge Request GitLab corrispondente, e chi guarda il ticket veda subito a che punto è il lavoro tecnico collegato.

Come si incastrano nella pipeline

Diagramma 6

Diagramma 6

Tool	A cosa serve	Fase della pipeline
Jenkins	Motore di automazione CI/CD, orchestra l'intera pipeline tramite job/Jenkinsfile	Orchestrazione end-to-end
Docker	Containerizzazione: impacchetta l'app e le dipendenze in un'immagine portatile	Packaging / artifact
SonarQube	Analisi statica del codice: coverage, complessità, vulnerabilità	Quality gate
Dynatrace	Observability/monitoring in tempo reale dell'applicazione in produzione	Monitoraggio post-deploy / rollback
Jira Software	Work e project tracking (Epic/Story/Task), integrato con MR e commit	Trasversale, collegato al codice

Non serve che tu sappia configurare nessuno di questi strumenti: quello che conta, nel tuo ruolo, è riconoscerli quando li senti nominare e sapere a quale fase della pipeline (o del lavoro del team) corrispondono, per seguire con consapevolezza le conversazioni tecniche e capire, ad esempio, perché “il rilascio è bloccato dal quality gate di SonarQube” o “Dynatrace ha segnalato un'anomalia, stiamo valutando il rollback”.

11.11 Riepilogo

In questa sezione hai visto, con più dettaglio pratico rispetto alla sezione DevOps, come funziona davvero una pipeline di CI/CD:

- una pipeline è una **sequenza di fasi** (checkout, build, test, analisi qualità/sicurezza, packaging, deploy in test, test di accettazione, deploy in produzione), ciascuna in grado di fermare tutto se qualcosa non va;
- scatta grazie a **trigger** diversi: push, Merge Request, pianificazione a orario, o avvio manuale;
- il codice si muove attraverso **ambienti** (Dev, Test/QA, Staging, Produzione) tramite un processo di **promozione**, senza essere ricostruito a ogni passaggio;
- l'**artifact** è il pacchetto concreto (un eseguibile, un'immagine Docker...) che viaggia identico da una fase all'altra;
- scrivere la pipeline come **Pipeline as Code** (un file YAML versionato) porta gli stessi benefici che Git porta al codice: storia, revisione, riuso;
- i **quality gate** trasformano il controllo di qualità da giudizio soggettivo a regola automatica e oggettiva;
- il **rollback** è il piano B necessario quando, nonostante tutti i controlli, qualcosa va comunque storto in produzione;

- strumenti concreti come **Jenkins**, **Docker**, **SonarQube**, **Dynatrace** e **Jira Software** sono le implementazioni reali di queste fasi che incontrerai nel lavoro quotidiano del team.

Nelle prossime sezioni vedrai come questi concetti si intreccino con le scelte di **architettura software** (sezione 12) e con l'infrastruttura **cloud** (sezione 13) su cui le pipeline effettivamente rilasciano il codice.

Esercizi pratici

Gli esercizi che segui qui sotto ti chiedono di **osservare, leggere e provare a ragionare** su una pipeline reale, non di scriverne una da zero: l'obiettivo di questa fase è riconoscere sul campo i concetti visti in questa sezione, non ancora configurare strumenti in autonomia.

1. **Disegna lo schema di una pipeline reale.** Chiedi a un developer o al Tech Lead di mostrarti (anche solo a schermo) l'ultima esecuzione di una pipeline nel progetto. Disegna su un foglio le fasi che vedi davvero (non quelle "da manuale"): quali ci sono, quali non ci sono, in che ordine.
Come verificare: mostra il tuo schizzo a chi te l'ha fatta vedere e chiedi "manca qualcosa o ho capito bene l'ordine?" — se la risposta è "esatto", hai capito la struttura.
2. **Trova i trigger reali del progetto.** Fatti mostrare (o trova da solo, se hai accesso) il file YAML della pipeline principale e individua il blocco che definisce i trigger (push, MR, schedule, manuale). Scrivi in una riga, con parole tue, quando scatta la pipeline di CI e quando invece scatta il deploy in produzione. **Come verificare:** chiedi a un collega di confermare che la tua descrizione corrisponde al comportamento reale che ha osservato lui nei giorni scorsi.
3. **Leggi un file YAML di pipeline senza aiuto.** Prendi l'esempio YAML del paragrafo 11.7 (o, meglio, un estratto reale e semplice del progetto, se riesci a farti mostrare i primi 20-30 righe di un file di pipeline) e, senza guardare le spiegazioni, prova a rispondere a voce alta: "quanti stage ci sono? qual è la dipendenza tra il primo e il secondo? cosa fa scattare l'intera pipeline?".
Come verificare: rileggi la tua risposta confrontandola con il file — se hai individuato correttamente `trigger / pr`, il numero di `stage` e almeno un `dependsOn`, hai capito la struttura di base.
4. **Simula un quality gate che blocca un rilascio.** Immagina (per iscritto, in 4-5 righe) uno scenario concreto — diverso da quelli già visti in questa sezione — in cui un quality gate blocca una pipeline: scegli tu il controllo (test falliti, coverage, vulnerabilità, qualità del codice), descrivi cosa succede e chi dovrebbe essere avvisato. **Come verificare:** fai leggere il tuo scenario a un collega developer o QA e chiedigli se, secondo la sua esperienza, è realistico e se la pipeline nel progetto reagirebbe davvero così.
5. **Ricostruisci un rollback a parole.** Chiedi a un collega se nel progetto è mai capitato un rollback (anche piccolo) e fatti raccontare cosa è successo: cosa ha fatto scattare l'allarme, quanto tempo è passato tra il deploy e l'accorgersi del problema, come si è tornati alla versione precedente. Scrivi la sequenza in 4-6 passaggi. **Come verificare:** la tua sequenza deve contenere almeno un momento di "rilevazione del problema" e un momento di "ripristino della versione precedente" — se manca uno dei due, richiedi il dettaglio mancante e completa lo schema.

6. **Distingui Delivery da Deployment nel progetto.** Osservando (o chiedendo) come funziona davvero il rilascio in produzione nel progetto, stabilisci se si tratta di Continuous Delivery (con approvazione manuale finale) o Continuous Deployment (automatico fino in produzione), e spiega in 2-3 righe **perché**, secondo te, il team ha scelto quell'approccio per quell'ambiente specifico. **Come verificare:** confronta la tua conclusione con quanto ti conferma la tua collega Scrum Master/PM o un Tech Lead — se coincide, hai capito la differenza pratica tra i due concetti, non solo la definizione teorica.
7. **Riconosci gli strumenti concreti del progetto.** Chiedi a un developer o al Tech Lead quali strumenti concreti, tra quelli visti al paragrafo 11.10 (o eventuali equivalenti), usa davvero il progetto per orchestrare la pipeline, fare code quality e monitorare la produzione. Per ciascuno che ti viene citato, scrivi a quale fase della pipeline corrisponde. **Come verificare:** confronta il tuo elenco con la tabella del paragrafo 11.10 — se hai assegnato correttamente ogni strumento citato alla fase giusta, hai capito il collegamento tra teoria e strumenti reali.

Collegamenti

- [12. Architetture software](#) — come è organizzato il software che la pipeline compila, testa e rilascia
- [13. Cloud](#) — dove “vivono” fisicamente gli ambienti di Dev, Test, Staging e Produzione su cui la pipeline effettua il deploy
- [10. Azure DevOps](#) — Boards, l'equivalente concettuale di Jira Software visto al paragrafo 11.10

Risorse

- [Microsoft Learn — Continuous integration e continuous delivery](#)
- [Microsoft Learn — Azure Pipelines, key concepts](#)
- [Microsoft Learn — YAML schema reference per Azure Pipelines](#)
- [Jenkins — Documentazione ufficiale](#)
- [Docker Docs — What is a container image](#)
- [SonarQube — Documentazione ufficiale](#)
- [Dynatrace — Cos'è l'observability](#)
- [Atlassian — Jira Software, panoramica](#)
- [GitLab Docs — CI/CD YAML syntax reference](#)
- [GitLab Docs — Get started with GitLab CI/CD](#)
- [Atlassian — Continuous integration vs continuous delivery vs continuous deployment](#)
- [Martin Fowler — Continuous Delivery](#)
- [Docker Docs — What is a container image](#)

12. Architetture software

Nella sezione 2 (Fondamenti di informatica) hai già incontrato i mattoncini di base: API, REST, database, container. In questa sezione facciamo un passo in più: vediamo come questi mattoncini vengono **combinati insieme** per costruire il software di cui il tuo team si occupa — e lo facciamo seguendo di nuovo **ShopFacile**, la piattaforma e-commerce già incontrata nelle sezioni DevOps, Azure DevOps e CI/CD, con **Marco** nel ruolo di chi discute spesso le scelte infrastrutturali e architetturali del progetto.

Non ti serve saper progettare un'architettura software — è il lavoro degli sviluppatori e degli architetti. Ti serve invece **capire a grandi linee come è “fatto” il software**, per poter:

- capire perché certe attività richiedono più tempo di altre (“aggiungere questa funzionalità è complicato perché tocca tre servizi diversi”);
- dialogare con il team usando le parole giuste (“frontend”, “backend”, “microservizio”, “database”);
- capire meglio certe scelte tecniche quando vengono discusse in retrospettiva o in fase di stima.

Obiettivi della sezione

Al termine di questa sezione saprai:

- distinguere un'architettura monolitica da un'architettura a microservizi, e spiegarne pro e contro;
- capire la differenza tra frontend e backend;
- capire cos'è il modello client-server e l'architettura a 3 livelli;
- avere un'idea di base di come comunicano i vari “pezzi” di un software (API/REST e code di messaggi);
- sapere cos'è il “serverless” e quando viene usato.

12.1 Cos'è un'architettura software

Quando si costruisce una casa, prima di posare un mattone si disegna una **pianta**: dove va la cucina, dove il bagno, come sono collegate le stanze, dove passano i tubi dell'acqua e i cavi elettrici. Quella pianta è l’“architettura” della casa: definisce come sono organizzati e collegati i suoi pezzi.

L'**architettura software** è la stessa cosa applicata a un programma: è il modo in cui le sue diverse parti (chi mostra le pagine, chi elabora i dati, chi li salva) sono organizzate e come comunicano tra loro.

Come per una casa, non esiste un'unica pianta giusta per tutti i casi: una villetta e un grattacielo hanno esigenze diverse e richiedono progetti diversi. Lo stesso vale per il software: la scelta dell'architettura dipende dalle dimensioni del progetto, dal numero di utenti, dal team disponibile e da quanto velocemente deve poter crescere.

Anche ShopFacile, come ogni software, ha dovuto affrontare questa scelta fin dall'inizio. Vediamo le due “piante” più comuni tra cui il team ha dovuto decidere: il monolite e i microservizi.

12.2 Architettura Monolitica

Un'architettura **monolitica** (dal greco “un unico blocco di pietra”) è quella in cui **tutto il software è scritto e distribuito come un unico blocco unito**: la parte che gestisce gli utenti, quella che gestisce gli ordini, quella che gestisce i pagamenti, quella che parla con il database... tutto vive nello stesso progetto, viene compilato insieme e viene eseguito come un unico programma.

Analogia: il grande magazzino unico

Pensa a un **grande magazzino** che vende di tutto sotto lo stesso tetto: abbigliamento, elettronica, alimentari, arredamento. C'è un'unica cassa centrale, un unico sistema di gestione del magazzino, un'unica squadra di addetti che, in teoria, potrebbe occuparsi di qualsiasi reparto. È comodo da costruire all'inizio (un solo edificio, una sola gestione), ma se un giorno vuoi ampliare solo il reparto elettronica, devi comunque intervenire sull'intero edificio, e se va in tilt il sistema di cassa centrale, si ferma la vendita in **tutti** i reparti contemporaneamente.

Diagramma 1

Diagramma 1

Pro del monolite

- **Semplice da avviare**: all'inizio di un progetto, con un team piccolo, è molto più rapido costruire e mandare in produzione un blocco unico.
- **Meno complessità operativa**: un solo “pacchetto” da distribuire, un solo ambiente da monitorare, meno pezzi che possono comunicare male tra loro.
- **Più facile da testare end-to-end**, perché tutto il flusso vive nello stesso posto.

Contro del monolite

- **Difficile da scalare in modo selettivo**: se solo una parte del software è sotto carico (es. tanti utenti che fanno ricerche), non puoi “potenziare” solo quella parte: devi duplicare l'intero blocco, anche le parti che non ne hanno bisogno.
- **Un problema in una parte può bloccare tutto**: un bug o un sovraccarico in un singolo componente rischia di far cadere l'intera applicazione, non solo quella funzionalità.
- **Rilasci più lenti e rischiosi al crescere delle dimensioni**: con tanti sviluppatori che lavorano sullo stesso blocco di codice, ogni rilascio richiede di testare e distribuire **tutto** insieme, anche se hai modificato solo un piccolo pezzo.
- **Difficile far crescere tanti team in parallelo** senza che si “pestino i piedi” a vicenda sullo stesso codice.

Esempio pratico: ShopFacile è nato esattamente così, come **monolite**: il catalogo prodotti, il carrello, la gestione degli ordini, i pagamenti e gli sconti vivevano tutti nello stesso progetto, compilato in un unico eseguibile. Era la scelta giusta per un team piccolo che doveva partire in fretta. Ma quando **Marco** deve correggere un piccolo bug nel calcolo di uno sconto, si trova a dover ricompilare, testare e rilasciare **l'intera applicazione**, comprese le parti (pagamenti, magazzino) che non ha nemmeno toccato — un costo che, all'inizio, il team accettava volentieri in cambio della semplicità.

Per confronto: non tutti i software devono necessariamente evolvere oltre il monolite. Un gestionale interno per l'ufficio commerciale, usato da poche decine di persone e senza picchi di traffico, può restare un monolite per anni senza alcun problema: la scelta dipende dal contesto, non è "il monolite è superato". Lo vedremo tra poco parlando di quando ha davvero senso cambiare.

Con il tempo, però, ShopFacile è cresciuto: più utenti, più developer nel team, più traffico concentrato su alcune funzionalità (il catalogo durante i saldi, i pagamenti durante il Black Friday). È qui che il monolite ha iniziato a mostrare i suoi limiti, ed è qui che entra in scena l'architettura alternativa: i microservizi.

12.3 Architettura a Microservizi

Un'architettura a **microservizi** divide il software in tanti **piccoli servizi indipendenti**, ciascuno responsabile di **una sola area** (es. "il servizio che gestisce gli utenti", "il servizio che gestisce i pagamenti", "il servizio che gestisce il magazzino"). Ogni servizio ha il suo codice, a volte il suo database, e viene distribuito e aggiornato **separatamente** dagli altri. I servizi comunicano tra loro tramite API (esattamente quelle che hai visto nella sezione 2) o tramite code di messaggi (le vediamo a breve).

Analogia: tanti piccoli negozi specializzati

Rispetto al grande magazzino unico, immagina ora una **via dello shopping** con tanti piccoli negozi specializzati: la libreria, il negozio di elettronica, il fruttivendolo. Ogni negozio ha la sua cassa, il suo personale, i suoi orari, la sua gestione del magazzino, ed è **indipendente** dagli altri: se il fruttivendolo chiude per ferie, la libreria continua a vendere libri senza problemi. Se un giorno la libreria diventa particolarmente famosa e affollata, il proprietario può assumere più commessi solo lì, senza dover potenziare anche gli altri negozi. In compenso, se un cliente vuole comprare un libro E della frutta, deve andare in due negozi diversi (cioè: serve più "coordinamento" tra i pezzi).

Diagramma 2

Diagramma 2

Pro dei microservizi

- **Scalabilità selettiva:** puoi potenziare solo il servizio sotto carico (es. più istanze del servizio Ordini durante il Black Friday), senza toccare gli altri.
- **Rilasci indipendenti:** puoi aggiornare il servizio Pagamenti senza dover rilasciare (e rischiare di rompere) il servizio Magazzino.
- **Team più autonomi:** team diversi possono lavorare su servizi diversi in parallelo, con meno conflitti e meno dipendenze reciproche.
- **Guasto isolato:** se un servizio ha un problema, in teoria gli altri possono continuare a funzionare (non sempre in pratica, ma è l'obiettivo).

- **Libertà tecnologica:** ogni servizio può, in teoria, usare il linguaggio o il database più adatto al suo compito specifico.

Contro dei microservizi

- **Complessità operativa molto più alta:** ora ci sono tanti “pezzi” da distribuire, monitorare, far comunicare correttamente tra loro. Serve un’infrastruttura DevOps solida (vedi sezione 9) per gestirla bene.
- **Comunicazione tra servizi da progettare con cura:** se il servizio A chiama il servizio B che chiama il servizio C, e uno di questi è lento o irraggiungibile, il problema si propaga.
- **Più difficile da debuggare:** un errore può nascere dall’interazione tra più servizi, e “seguire il filo” del problema è più complesso che in un monolite.
- **Richiede un team più maturo** su temi come CI/CD, containerizzazione e monitoraggio, perché la complessità operativa cresce parecchio.

Esempio pratico: Marco e il resto del team decidono di scomporre ShopFacile in microservizi: il catalogo prodotti diventa un servizio separato, con il suo repository e la sua pipeline di rilascio (quella vista nella sezione 11). Se il team vuole aggiungere un nuovo filtro di ricerca nel catalogo, rilascia **solo quel servizio**: i pagamenti e la gestione ordini continuano a funzionare esattamente come prima, senza bisogno di essere ritestati o rilasciati di nuovo.

Quando ha senso passare da monolite a microservizi

Non è vero che i microservizi sono “sempre meglio”. Anzi: molti progetti, soprattutto agli inizi, **partono volutamente come monolite**, proprio come ha fatto ShopFacile, perché è più rapido e il team è piccolo. Il passaggio a microservizi ha senso quando succede una o più di queste cose — ed è esattamente il ragionamento che Marco ha portato al team di ShopFacile:

- il team è cresciuto molto e più squadre lavorano sullo stesso codice, intralciandosi a vicenda;
- alcune parti del sistema hanno bisogno di scalare molto più di altre (es. il catalogo prodotti di ShopFacile riceve 100 volte più traffico della gestione fatture);
- i rilasci del monolite sono diventati lenti, rischiosi e “tutto o nulla”;
- il codice è diventato talmente grande e intrecciato che anche piccole modifiche richiedono di toccare (e ritestare) mezza applicazione.

Una buona regola pratica che sentirai spesso: **“non iniziare con i microservizi”**. Molte aziende, incluse alcune molto famose nel settore tech, sono partite da un monolite e sono passate ai microservizi solo quando la crescita lo ha reso necessario — non il contrario. Un gestionale interno per l’ufficio commerciale, usato da poche decine di persone senza picchi di traffico, è un controesempio utile: probabilmente non incontrerà mai nessuna di queste quattro condizioni, e restare monolite per lui è la scelta giusta, non un ritardo da recuperare.

Monolite vs Microservizi: la tabella di confronto

Aspetto	Monolite	Microservizi
Velocità di partenza di un progetto	Alta: si parte rapidamente	Più bassa: serve progettare i confini tra servizi e l'infrastruttura
Scalabilità	Si scala tutto insieme, anche le parti che non servono	Si scala solo il servizio che ne ha bisogno
Complessità operativa (deploy, monitoraggio)	Bassa: un solo blocco da gestire	Alta: tanti servizi da distribuire, far comunicare, monitorare
Rilasci	Un rilascio unico, tocca tutto il sistema	Rilasci indipendenti per singolo servizio
Isolamento dei guasti	Basso: un problema può bloccare tutto	Più alto: un servizio può fallire senza fermare tutti gli altri
Autonomia dei team	Bassa: più team sullo stesso codice si intralciano	Alta: ogni team può gestire i propri servizi
Adatto a	Progetti piccoli/medi, team piccoli, fasi iniziali (MVP)	Progetti grandi, molti team, esigenze di scalabilità differenziate

12.4 Come comunicano i componenti

Nel momento in cui ShopFacile è diventato un insieme di servizi separati (catalogo, ordini, pagamenti, magazzino), è emersa una domanda nuova: come fanno questi servizi a scambiarsi informazioni tra loro? Sia nel monolite (tra i suoi moduli interni) che, soprattutto, nei microservizi (tra servizi separati), i vari “pezzi” del software devono comunicare. Ci sono principalmente due modalità.

Comunicazione sincrona: API e REST

Il modo più comune è quello che hai già visto nella sezione 2: un componente fa una **richiesta** a un altro tramite un’**API**, tipicamente in stile **REST**, e resta in attesa della **risposta** prima di andare avanti — un po’ come una telefonata: fai la domanda e resti in linea aspettando la risposta, prima di riprendere quello che stavi facendo.

Questo funziona benissimo per molti casi (“dammi i dati di questo cliente”, “verifica se questo pagamento è andato a buon fine”), ma ha un limite: se il servizio che deve rispondere è lento o irraggiungibile, chi ha fatto la richiesta resta bloccato ad aspettare.

Esempio pratico: il frontend di ShopFacile vuole mostrare i dettagli di un ordine al cliente. Fa una richiesta HTTP di tipo GET a un’API REST del backend, e resta in attesa della risposta:

Richiesta:

```
GET /api/ordini/42
```

Risposta (dopo pochi millisecondi):

```
{
  "id": 42,
  "cliente": "ACME S.r.l.",
  "stato": "in consegna",
  "totale": 129.90
}
```

Finché il backend non risponde, il frontend resta “in attesa” — esattamente come nell’analogia della telefonata — prima di poter mostrare questi dati a schermo.

Questo tipo di comunicazione, però, non è adatto a tutto: se il servizio Ordini di ShopFacile dovesse restare “in attesa” ogni volta che avvisa il servizio Email di inviare una conferma, un rallentamento della posta elettronica bloccherebbe anche la conferma dell’ordine. Per questi casi serve un modo diverso di comunicare: la comunicazione asincrona.

Comunicazione asincrona: le code di messaggi

Per certi casi è più comodo **non restare in attesa** di una risposta immediata, ma semplicemente “lasciare un messaggio” che qualcun altro leggerà quando potrà. È qui che entrano in gioco le **code di messaggi** (*message queue*), uno strumento tipico della comunicazione **asincrona**.

Analogia: la cassetta della posta

Pensa alla differenza tra una telefonata e una **cassetta della posta**:

- con la telefonata (comunicazione sincrona/API) chiami qualcuno e resti in attesa che risponda subito, in tempo reale;
- con una lettera lasciata nella cassetta della posta (comunicazione asincrona/coda di messaggi) **scrivi il messaggio e vai avanti con le tue cose**: non resti lì ad aspettare. Il destinatario, quando ha tempo, apre la cassetta, legge il messaggio e agisce di conseguenza. Se il destinatario in questo momento non è in casa, il messaggio resta comunque nella cassetta ad aspettarlo, senza perdersi.

Esempio pratico: quando un cliente completa un ordine su ShopFacile, il servizio Ordini non chiama direttamente e “in diretta” il servizio Email per inviare la conferma. Invece, scrive un messaggio del tipo “ordine 42 completato” in una coda; il servizio Email (quando è pronto, magari qualche secondo dopo) legge quel messaggio dalla coda e invia l’email. Se il servizio Email in quel momento è sovraccarico o temporaneamente fermo, il messaggio resta comunque in coda, e verrà elaborato appena il servizio torna disponibile — non si perde nulla.

Diagramma 3

Diagramma 3

Questo approccio è molto usato quando un evento (es. “ordine completato”) deve essere notificato a **più** servizi interessati, senza che il servizio che genera l'evento debba conoscerli tutti o aspettare che rispondano tutti. Esempi di strumenti che sentirai citare per questo scopo: **RabbitMQ**, **Azure Service Bus**, **Kafka**.

	Comunicazione sincrona (API/ REST)	Comunicazione asincrona (coda di messaggi)
Analogia	Telefonata	Cassetta della posta
Chi chiama resta in attesa?	Sì, della risposta	No, va avanti
Adatta a	Richieste che servono “subito” (es. leggere dati)	Notifiche/eventi che possono essere elaborati “quando capita”
Rischio se il destinatario è lento/ offline	Chi chiama resta bloccato o riceve un errore	Il messaggio resta in coda, nessuna perdita

12.5 Frontend vs Backend

Finora abbiamo parlato di come i servizi di ShopFacile comunicano **tra loro**. Ma c'è un'altra comunicazione altrettanto importante: quella tra il sistema e la persona che lo usa, cioè il cliente che naviga il sito. Per capirla, serve distinguere due grandi parti presenti in ogni applicazione web o mobile (un sito di e-commerce come ShopFacile, un'app bancaria, un gestionale aziendale): **frontend** e **backend**.

- il **frontend** è tutto ciò che l'utente vede e con cui interagisce direttamente: pagine, bottoni, moduli da compilare, grafici. È la parte che “gira” nel browser o nell'app sul telefono dell'utente;
- il **backend** è la parte che l'utente non vede: elabora la logica di business (“questo sconto si applica solo se il carrello supera 50€”), gestisce l'accesso al database, si occupa della sicurezza, e risponde alle richieste che arrivano dal frontend.

Analogia: il ristorante, sala vs cucina

Pensa a un ristorante:

- la **sala** (i tavoli, i camerieri, il menù che vedi) è il **frontend**: è quello che tu, cliente, vedi e con cui interagisci direttamente. Ordini un piatto guardando il menù, il cameriere prende nota;
- la **cucina** è il **backend**: è dove avviene il vero lavoro (cucinare il piatto, gestire le scorte, applicare la ricetta), ma tu, seduto al tavolo, non la vedi. Interagisci con essa solo indirettamente, tramite il cameriere (che, come abbiamo visto nella sezione 2, è un po' come l'API).

Un frontend curato ma con un backend debole è come un ristorante con una sala elegantissima ma una cucina che sbaglia sempre gli ordini: l'esperienza finale ne risente comunque. Vale anche il contrario: un backend eccellente con un frontend confuso farà comunque fatica a piacere ai clienti.

Diagramma 4

Diagramma 4

Esempio pratico: quando clicchi il pulsante “Conferma ordine” su ShopFacile, il **frontend** (il bottone che hai cliccato) invia una richiesta al **backend** con i dati del carrello; il backend verifica che i prodotti siano disponibili, calcola il totale applicando eventuali sconti, salva l'ordine nel database e restituisce al frontend una risposta (“ordine confermato, numero 42”), che il frontend traduce in un messaggio di conferma a schermo. Tu, da cliente, vedi solo l'ultimo passaggio: tutto il resto avviene “in cucina”.

Nel team sentirai spesso parlare di “sviluppatori frontend” e “sviluppatori backend” (o “full-stack”, chi lavora su entrambi): sono specializzazioni diverse, con linguaggi e strumenti spesso diversi, anche se lavorano sullo stesso prodotto finale.

Detto così, “frontend” e “backend” sembrano semplicemente due metà dello stesso programma. In realtà, quasi sempre, girano fisicamente in due posti diversi e comunicano attraverso una rete: è il momento di guardare a questo rapporto con il nome più tecnico che porta, il modello client-server.

12.6 Client-Server: il concetto base

Hai già incontrato questo concetto nella sezione 2 parlando di HTTP: il modello **client-server** è l'idea di base secondo cui esiste un **client** (chi fa richieste, es. il tuo browser) e un **server** (chi riceve la richiesta, la elabora e risponde, es. il computer che ospita l'applicazione).

È utile ricordarlo qui perché frontend e backend, di fatto, **spesso corrispondono** a client e server: il frontend che gira nel tuo browser è il client, il backend che gira su un server (fisico, virtuale, o un container in cloud) è il server. Non è una regola matematica assoluta — esistono architetture più sofisticate — ma per il 90% dei casi che incontrerai questa corrispondenza è un buon punto di partenza mentale.

Esempio pratico: quando apri ShopFacile sul telefono, l'app stessa (che gira sul tuo telefono) è il **client**: fa una richiesta (“dammi i dettagli del mio ultimo ordine”) a un **server** remoto di ShopFacile, che elabora la richiesta e restituisce il dato. Se chiudi l'app, il server continua a funzionare tranquillamente per tutti gli altri clienti: il client è “usa e getta”, il server resta sempre lì, pronto a rispondere a nuove richieste.

Per confronto: lo stesso schema vale, identico, per un'app di home banking: l'app sul telefono è il client, il server remoto della banca è il server. Cambia il contesto (acquisti contro conto corrente), non la struttura client-server sottostante.

Sappiamo quindi che un client parla con un server, e che dentro quel server convivono frontend e backend. Ma **dentro il backend stesso**, come sono organizzati presentazione, logica di business e dati? Un modo diffuso di rispondere a questa domanda è l'architettura a 3 livelli.

12.7 Architettura a 3 livelli

Un modo molto comune, semplice e diffuso di organizzare un'applicazione, soprattutto nei sistemi gestionali "classici", è l'**architettura a 3 livelli** (in inglese *3-tier architecture*). Divide il software in tre strati, ciascuno con una responsabilità precisa:

1. **Presentation layer** (livello di presentazione): è il frontend, quello che l'utente vede e usa;
2. **Business logic layer** (livello di logica di business): è il backend, dove vengono applicate le regole ("uno sconto si applica solo se...", "un ordine non può essere spedito se non è stato pagato...");
3. **Data layer** (livello dei dati): è il database, dove i dati vengono effettivamente salvati e recuperati.

L'idea chiave è che **ogni livello parla solo con quello adiacente**: il livello di presentazione non accede mai direttamente al database, ma passa sempre dal livello di logica di business, che fa da "filtro" e applica le regole necessarie prima di leggere o scrivere i dati.

Analogia

Pensa di nuovo al ristorante, ma con un passaggio in più: tu (client) parli solo con il cameriere (presentation layer), il cameriere porta l'ordine allo chef (business logic layer), che decide come preparare il piatto secondo la ricetta e le regole della cucina, e solo lo chef accede alla **dispensa** (data layer) per prendere gli ingredienti. Tu non entri mai direttamente nella dispensa: passi sempre dallo chef, che sa quali regole rispettare (es. "questo piatto non si può preparare se manca un ingrediente fondamentale").

Diagramma 5

Diagramma 5

Questo schema a 3 livelli è alla base di moltissime applicazioni aziendali "tradizionali" (gestionali, portali interni, applicazioni web classiche), e puoi ritrovarlo sia dentro un monolite (i tre livelli vivono nello stesso blocco di codice) sia distribuito su più microservizi (ogni servizio ha comunque, internamente, una sua piccola struttura a livelli) — come nel caso del servizio Ordini di ShopFacile.

Esempio pratico: nel servizio Ordini di ShopFacile, che applica uno sconto del 10% agli ordini superiori a 100€, il livello di presentazione mostra semplicemente il totale finale nel carrello; il livello di logica di business è quello che **decide** se lo sconto si applica o no, facendo il calcolo; il livello dati è quello che, a fine ordine, salva il totale scontato nel database. Se un giorno la regola cambia (es. lo sconto diventa 15%), basta modificare il livello di logica di business: la presentazione e il database non vengono toccati. Lo stesso identico schema, del resto, è alla base anche di un gestionale aziendale "classico" per l'ufficio commerciale: cambia il dominio (sconti sugli ordini contro preventivi e fatture), non la struttura a tre livelli.

Dopo monolite, microservizi, comunicazione e 3 livelli, resta un ultimo modo di organizzare il codice, agli antipodi rispetto al server "sempre acceso" che abbiamo dato per scontato finora: il serverless.

12.8 Un accenno al Serverless

Finora abbiamo parlato di software che gira su server (fisici, virtuali, o container) che devono restare **sempre attivi**, pronti a rispondere in qualsiasi momento. Il modello **serverless** (“senza server”, anche se in realtà un server c’è sempre, solo che non te ne devi occupare tu) propone un’idea diversa: scrivi piccole **funzioni** che vengono eseguite dal fornitore cloud **solo quando servono**, e vengono “spente” subito dopo.

Analogia: il taxi vs l’auto di proprietà

Gestire un server tradizionale è un po’ come avere un’**auto di proprietà**: la paghi (e la mantieni) sempre, anche quando è parcheggiata e non la usi. Il serverless è più come prendere un **taxi**: lo chiami solo quando ti serve uno spostamento, paghi solo per quella corsa, e non devi preoccuparti di manutenzione, parcheggio o assicurazione quando non lo stai usando.

Esempio pratico: in ShopFacile, quando un cliente carica la foto di un prodotto reso da segnalare, una funzione serverless si attiva solo in quel momento, per ridimensionare l’immagine automaticamente in diverse dimensioni, e poi si “spegne” fino al prossimo caricamento. Non serve tenere un server acceso 24 ore su 24 in attesa che qualcuno carichi una foto — a differenza del servizio Ordini o Pagamenti, che devono restare sempre pronti a rispondere.

Vantaggi principali: si paga solo per l’uso effettivo (nessun costo per il tempo “di inattività”), e non serve gestire manualmente l’infrastruttura sottostante (il fornitore cloud se ne occupa). Lo svantaggio principale è che non è adatto a tutti i casi: per applicazioni che devono rispondere istantaneamente e in modo continuo, o che richiedono elaborazioni molto lunghe, il modello serverless può introdurre piccoli ritardi (“tempo di avvio a freddo”) o limiti di durata.

Esempi di servizi serverless che sentirai citare: **Azure Functions**, **AWS Lambda**. Ne ripareremo con più dettaglio nella sezione 13 (Cloud).

Con questo abbiamo attraversato tutte le scelte architetturali principali che il team di ShopFacile ha dovuto affrontare, da “un blocco unico o tanti servizi?” fino a “serve un server sempre acceso o basta una funzione al bisogno?”. Prima di passare al Cloud, fermiamoci un momento a riepilogare.

12.9 Riepilogo: cosa ti serve ricordare

Non devi memorizzare ogni dettaglio di questa sezione. Ti basta portarti via questi concetti chiave, che ti aiuteranno a capire meglio le conversazioni tecniche del team:

- **Monolite**: un unico blocco di codice. Semplice all’inizio, difficile da scalare selettivamente e da far crescere con tanti team.
- **Microservizi**: tanti servizi piccoli e indipendenti. Più scalabili e flessibili, ma più complessi da gestire operativamente.
- **API/REST** (comunicazione sincrona, “telefonata”) e **code di messaggi** (comunicazione asincrona, “cassetta della posta”) sono i due modi principali con cui i componenti di un software comunicano tra loro.

- **Frontend** (sala) e **Backend** (cucina): la parte visibile all'utente e quella che elabora la logica e i dati.
- **Client-Server**: chi fa la richiesta (client) e chi la elabora (server).
- **Architettura a 3 livelli**: Presentazione → Logica di business → Dati, ognuno responsabile del proprio compito.
- **Serverless**: funzioni eseguite solo quando servono, senza gestire un server sempre attivo — come un taxi rispetto a un'auto di proprietà.

Quando il team discute di “spacchettare il monolite”, “aggiungere un servizio”, “usare una coda per questo evento” o “spostare questa funzione su serverless”, ora avrai gli strumenti concettuali per seguire la conversazione e fare le domande giuste.

Checklist di autoverifica

- Sapresti spiegare la differenza tra monolite e microservizi con un'analogia tua?
 - Sai indicare almeno due vantaggi e due svantaggi dei microservizi?
 - Sapresti spiegare quando ha senso passare da monolite a microservizi?
 - Sai spiegare la differenza tra comunicazione sincrona e asincrona, con l'analogia della telefonata e della cassetta della posta?
 - Sai spiegare la differenza tra frontend e backend con l'analogia del ristorante?
 - Sai descrivere i tre livelli dell'architettura a 3 livelli?
 - Sai spiegare, a grandi linee, cos'è il serverless e quando può essere utile?
-

Esercizi pratici

1. **Disegna l'architettura di un'app che usi tutti i giorni.** Scegli un'app che conosci bene (es. un'app di food delivery, di messaggistica o di home banking) e disegna a mano uno schema con frontend, backend e database, provando a immaginare se sia più vicina a un monolite o a un sistema a microservizi. **Come verificare:** se riesci a indicare almeno tre “pezzi” separati (es. “app sul telefono”, “server che elabora i pagamenti”, “database degli utenti”) e a spiegare a voce come comunicano tra loro, hai centrato l'esercizio.
2. **Scrivi una richiesta e una risposta API “a mano”.** Scegli una funzionalità semplice (es. “ottieni il profilo di un utente”) e scrivi tu stesso, come nell'esempio pratico di questa sezione, una richiesta GET/POST semplificata e la relativa risposta in formato JSON, con almeno 3-4 campi. **Come verificare:** fai leggere quello che hai scritto a un developer del team e chiedi se il formato “assomiglia” a una vera richiesta/risposta API che usano nel progetto.
3. **Trasforma un'operazione sincrona in asincrona.** Pensa a un'operazione che oggi, nella tua esperienza quotidiana (non necessariamente software), richiede un'attesa “in linea” (es. una telefonata al call center) e descrivi come diventerebbe se fosse gestita con una coda di messaggi (es. lasciare un messaggio in segreteria che verrà richiamato quando c'è disponibilità). **Come verificare:** la tua descrizione deve contenere chiaramente chi “scrive” il messaggio, dove viene “depositato” e chi lo “legge” più tardi, senza che nessuno resti bloccato ad aspettare.

4. **Intervista un developer sull'architettura del progetto.** Chiedi a un collega developer se il progetto su cui lavori è organizzato come monolite, come microservizi, o come una via di mezzo, e fatti indicare un esempio concreto di comunicazione tra due componenti (API o coda di messaggi). **Come verificare:** dopo la chiacchierata, sapresti riassumere in 3-4 frasi, senza guardare appunti, "come è fatto" il software del progetto e quali pezzi lo compongono.
5. **Compila la tabella di confronto con un esempio reale.** Riprendi la tabella "Monolite vs Microservizi" della sezione 12.3 e, per ogni riga, scrivi a fianco un esempio concreto (reale o immaginario) legato al contesto del tuo progetto. **Come verificare:** condividi la tabella compilata con la tua collega Scrum Master/PM e chiedile se gli esempi che hai scelto sono plausibili per un progetto reale.
6. **Individua un caso d'uso serverless.** Pensa a una funzionalità che, nel progetto o in un'app che usi, viene eseguita "una tantum" e su richiesta (es. generazione di un PDF, invio di una notifica, ridimensionamento di un'immagine) e spiega perché potrebbe essere un buon candidato per il modello serverless, oppure perché no. **Come verificare:** la tua spiegazione deve menzionare esplicitamente il criterio "si attiva solo quando serve, poi si spegne" e collegarlo al caso d'uso scelto.

Collegamenti

- [13. Cloud](#) — dove vedremo come queste architetture vengono effettivamente eseguite su infrastrutture cloud come Azure o AWS
- [14. Sicurezza](#) — dove vedremo come proteggere questi componenti e le comunicazioni tra di essi

Risorse

- [Microsoft Learn – Architetture dei microservizi](#) — guida approfondita (in italiano) su monolite vs microservizi
- [Martin Fowler – Microservices](#) — l'articolo di riferimento sul tema, uno dei più citati nel settore
- [Martin Fowler – MonolithFirst](#) — perché spesso ha senso partire da un monolite prima di passare ai microservizi
- [Microsoft Learn – Cos'è il serverless computing](#) — introduzione al modello serverless
- [Microsoft Learn – Message queue e comunicazione asincrona](#) — approfondimento sui pattern di comunicazione asincrona
- [Documentazione ufficiale RabbitMQ – Concetti base](#) — introduzione pratica alle code di messaggi

13. Cloud

Nella sezione precedente hai visto come è “fatto” un software: monolite o microservizi, frontend e backend, database, code di messaggi. Ora facciamo un passo ulteriore e ci chiediamo: **dove vive fisicamente tutto questo?** Su quali macchine viene eseguito il codice? Dove sono salvati i dati?

La risposta, oggi, nella grande maggioranza dei casi è: **nel cloud**. In questa sezione capirai cosa significa davvero questa parola (che spesso viene usata in modo vago, quasi magico), quali sono i principali modelli con cui si “affitta” il cloud, e come si chiamano i pezzi principali di Azure e AWS, i due grandi fornitori che incontrerai più spesso lavorando in ambito DevOps.

Obiettivi della sezione

Al termine di questa sezione saprai:

- spiegare cos'è il cloud computing con parole tue, senza tecnicismi;
- distinguere IaaS, PaaS e SaaS e sapere chi gestisce cosa in ciascun modello;
- riconoscere i nomi dei principali servizi Azure e i loro equivalenti AWS;
- capire cosa significano scalabilità verticale/orizzontale, pay-as-you-go, regioni e alta disponibilità.

13.1 Cos'è il cloud computing

Immagina di dover organizzare una cena per 50 persone. Hai due opzioni:

1. Comprare pentole, forni, piatti, tavoli e sedie per 50 persone, tenerli in un magazzino tutto l'anno e usarli una volta ogni tanto.
2. Affittare una sala da un catering: paghi solo per l'evento, usi le loro attrezzature, e quando hai finito non devi più pensarci.

Per decenni, le aziende che avevano bisogno di far girare un software hanno fatto la scelta (1) — ed è esattamente quello che ha fatto per anni **ShopFacile**, la piattaforma di e-commerce (catalogo prodotti, carrello, ordini, pagamenti) che useremo come filo conduttore di questa sezione: compravano server fisici, li installavano in una stanza climatizzata (la “sala server”), pagavano tecnici per mantenerli, e se il software cresceva dovevano comprare altri server. Se il software calava di utilizzo, quei server restavano lì, comprati e pagati, a fare poco.

Il **cloud computing** è la scelta (2) applicata all'informatica: invece di possedere e gestire tu i server fisici, **usi via internet i server e le risorse informatiche di qualcun altro** (un “provider cloud” come Microsoft, Amazon o Google), pagando solo per quello che usi e quando lo usi.

Analogia: possedere un data center è come costruirsi una casa da zero — devi comprare il terreno, costruire le fondamenta, tirare su i muri, allacciare le utenze. Usare il cloud è come **affittare un appartamento già arredato**: entri, usi quello che c'è, e quando ti serve una stanza in più la aggiungi (o la togli) senza dover ristrutturare un edificio.

In pratica, quando ShopFacile “mette il software sul cloud” significa che il codice, il database degli ordini e i file (immagini dei prodotti, fatture) non girano più su un server fisico di proprietà dell'azienda dentro il suo ufficio, ma su server che appartengono a un provider cloud, distribuiti in enormi data center in giro per il mondo, ai quali si accede via internet.

Questo cambia radicalmente il modo di lavorare:

- non serve comprare hardware in anticipo “per sicurezza”: si aggiungono risorse quando servono, in pochi minuti;
- non serve occuparsi di manutenzione fisica (cambiare un disco rotto, sostituire un alimentatore): è il provider che ci pensa;
- si paga in base al consumo, non un costo fisso indipendentemente dall'uso.

Esempio pratico: Marco, che in ShopFacile si occupa spesso di infrastruttura, guida la migrazione: il vecchio server fisico che ospitava il catalogo prodotti e il database degli ordini viene spento, e lo stesso software torna a girare — senza essere riscritto — su risorse affittate da un provider cloud. Il giorno dei saldi, quando il traffico su ShopFacile quadruplica, il team non deve più sperare che il server fisico regga: basta aggiungere risorse a richiesta e poi rilasciarle.

13.2 I tre modelli principali: IaaS, PaaS, SaaS

Migrare ShopFacile sul cloud non è stata una scelta “tutto o niente”: Marco ha dovuto decidere, pezzo per pezzo dell'infrastruttura, **quanta responsabilità di gestione lasciare al provider e quanta tenersi**. Non tutto il cloud è uguale: quando “affitti” qualcosa dal cloud, puoi affittare **più o meno responsabilità**. Il modo più semplice per capirlo è pensare a come si può abitare in un posto: puoi comprare un terreno e costruire tu la casa, puoi affittare un appartamento arredato, oppure puoi prenotare una stanza d'hotel con servizio completo.

Nel mondo cloud questi tre livelli si chiamano **IaaS**, **PaaS** e **SaaS**.

13.2.1 IaaS — Infrastructure as Service

IaaS significa “infrastruttura come servizio”. Il provider ti affitta l'**infrastruttura grezza**: server virtuali, spazio di archiviazione (dischi virtuali) e rete. Tutto il resto — sistema operativo, aggiornamenti, programmi installati, sicurezza a livello di software — è **compito tuo**.

Analogia: è come affittare un **terreno con già lo scheletro di un edificio** (fondamenta, struttura portante, allacci alla rete elettrica e idrica). Il proprietario ti garantisce che lo scheletro sta in piedi e che l'elettricità arriva. Ma le pareti interne, gli impianti, l'arredamento e le tinteggiature te li fai tu.

Esempio concreto: una **macchina virtuale** su Azure o AWS. Il provider ti dà un computer virtuale con una certa potenza di calcolo, ma sei tu a installare il sistema operativo, gli aggiornamenti di sicurezza, il software applicativo e a occuparti della configurazione.

Chi usa IaaS: chi ha bisogno di massimo controllo e flessibilità, o deve migrare software esistente pensato per girare su server “tradizionali” senza riscriverlo.

Esempio pratico: Marco deve spostare sul cloud il vecchio motore di generazione delle fatture di ShopFacile, pensato per girare su un server Windows in ufficio, senza riscriverlo. La soluzione è creare una macchina virtuale IaaS: si installa lo stesso sistema operativo, si copiano gli stessi file, si configurano le stesse porte di rete. Non cambia una riga di codice. Il vantaggio non è “meno lavoro da fare” ma “niente hardware da comprare e mantenere”: se la macchina virtuale si rivela sottodimensionata durante un picco di ordini, in pochi minuti Marco ne aumenta la potenza (CPU, RAM) dal pannello di controllo del provider, invece di aspettare settimane per un nuovo server fisico.

13.2.2 PaaS — Platform as Service

Se con la fattura Marco ha scelto di portarsi dietro anche il sistema operativo da gestire, per il catalogo prodotti di ShopFacile — la parte dell'applicazione che cambia più spesso — il team preferisce liberarsi anche di quel livello: entra in scena il PaaS.

PaaS significa “piattaforma come servizio”. Il provider ti affitta anche **la piattaforma** su cui far girare il software: sistema operativo, ambiente di esecuzione (runtime), database già pronti e gestiti. Tu ti devi occupare solo di scrivere e distribuire il tuo codice.

Analogia: è l'**appartamento già arredato**. Non devi costruire i muri né comprare i mobili: trovi tutto pronto, incluse le utenze allacciate. Ti devi solo preoccupare di viverci, cioè — nel caso del software — di scrivere e pubblicare il codice della tua applicazione.

Esempio concreto: **Azure App Service**, dove carichi il codice della tua applicazione web e il provider si occupa di far girare il sistema operativo, il runtime (per esempio .NET o Node.js), gli aggiornamenti di sicurezza e la scalabilità di base. Oppure un database gestito come **Azure SQL Database**, dove non devi installare né amministrare tu il motore del database: esiste già, pronto all'uso, e il provider si occupa di backup, patch e affidabilità.

Chi usa PaaS: la maggior parte dei team che sviluppano applicazioni moderne, perché permette di concentrarsi sul codice senza perdere tempo a gestire server e sistemi operativi.

Esempio pratico: Ahmed finisce di scrivere una nuova funzionalità per il catalogo prodotti di ShopFacile (i filtri di ricerca per categoria). Con un servizio PaaS, pubblica il codice (spesso con un comando o con la pipeline vista nella sezione 11) e in pochi minuti la nuova versione è online, raggiungibile via browser, senza che nessuno debba configurare un sistema operativo, aprire porte di rete o installare un runtime a mano. In parallelo, i dati del catalogo vivono in un database gestito: nessuno del team deve occuparsi di installare il motore del database, programmare i backup notturni o applicare le patch di sicurezza — il provider lo fa in automatico, e se qualcosa va storto è possibile ripristinare un backup recente con pochi clic.

13.2.3 SaaS — Software as Service

Con IaaS e PaaS, il team di ShopFacile ha comunque scritto e distribuito codice proprio. C'è però un terzo livello, ancora più delegato, che il team usa ogni giorno senza nemmeno pensarci: quello del software già pronto.

SaaS significa “software come servizio”. Qui non affitti infrastruttura né piattaforma: **usi direttamente un software già finito**, di solito via browser, senza installare nulla e senza sapere (né doverti preoccupare di) dove e come girano i server dietro.

Analogia: è l'**hotel a servizio completo**. Non ti preoccupi di struttura, arredamento, pulizie o lenzuola: prenoti la stanza, entri, e tutto è già pronto per essere usato. Il tuo unico compito è vivere la tua vacanza (cioè usare il software per il tuo lavoro).

Esempi concreti che usi già ogni giorno: **Gmail** o **Outlook** (posta elettronica), **Office 365** o **Google Workspace** (documenti e foglio di calcolo), **Salesforce** (gestione clienti), la stessa **Azure DevOps** che hai visto nella sezione 10, o **Slack/Teams** per la comunicazione. In tutti questi casi apri un browser, accedi con le tue credenziali, e il software è già lì, funzionante, aggiornato, senza che tu debba installare o gestire nulla.

Chi usa SaaS: chiunque abbia bisogno di una funzionalità (email, CRM, gestione progetti) senza voler sviluppare o mantenere il software che la fornisce.

Esempio pratico: il team di ShopFacile ha bisogno di gestire il backlog, le pipeline e i repository del progetto. Nessuno “installa” Azure DevOps su un server: Sara e Luca si registrano, aprono il browser, accedono con le proprie credenziali e il servizio è già lì, pronto all'uso e aggiornato dal provider senza che il team debba fare nulla. Lo stesso vale per la posta elettronica aziendale o per lo strumento di videochiamata usato nei daily: sono tutti software “già finiti”, usati così come sono, senza che qualcuno in ShopFacile ne debba gestire l'infrastruttura sottostante.

13.3 Confronto: chi gestisce cosa

Ora che Marco ha usato tutti e tre i livelli su pezzi diversi dell'infrastruttura di ShopFacile, conviene vedere in un colpo d'occhio come si posizionano l'uno rispetto all'altro. La differenza chiave tra IaaS, PaaS e SaaS è **quanta responsabilità di gestione resta al cliente e quanta passa al provider cloud**. Più si sale da IaaS verso SaaS, meno cose deve gestire il cliente (ma meno controllo ha).

Diagramma 1

Diagramma 1

La tabella seguente è il modo più chiaro per vedere la “torta delle responsabilità”: ogni riga è uno strato tecnico, e la cella indica **chi** se ne occupa.

Livello / Strato	On-Premise (server proprio)	IaaS	PaaS	SaaS
Applicazioni	Cliente	Cliente	Cliente	Provider
Dati	Cliente	Cliente	Cliente	Cliente*
Runtime (linguaggio, librerie)	Cliente	Cliente	Provider	Provider
Middleware	Cliente	Cliente	Provider	Provider
Sistema operativo	Cliente	Cliente	Provider	Provider
Virtualizzazione	Cliente	Provider	Provider	Provider
Server (hardware fisico)	Cliente	Provider	Provider	Provider
Storage (fisico)	Cliente	Provider	Provider	Provider
Rete (fisica)	Cliente	Provider	Provider	Provider

** anche in SaaS i dati che inserisci nel software (es. le tue email, i tuoi documenti) restano “tuoi”: il provider li custodisce e li protegge, ma il contenuto è di tua responsabilità (es. non condividere per errore un file sensibile).*

In sintesi, salendo da IaaS a SaaS **deleghi progressivamente più lavoro operativo al provider**, in cambio di **meno controllo diretto** su come le cose sono configurate. Non esiste un livello “migliore” in assoluto: la scelta dipende da quanto controllo serve rispetto a quanta comodità si vuole.

13.4 Azure: panoramica generale

Visti i tre modelli in astratto, è utile sapere anche su quale provider concreto ShopFacile li ha effettivamente implementati. **Microsoft Azure** è la piattaforma cloud di Microsoft, ed è quella che incontrerai più spesso in questo corso e nel lavoro quotidiano, dato che il team usa già Azure DevOps (sezione 10) e vi ha migrato la propria infrastruttura. Azure offre centinaia di servizi; qui vediamo solo i concetti principali, giusto per riconoscerli quando li sentirai nominare.

- **Macchine virtuali (Virtual Machines)** — il servizio IaaS di base: un server virtuale su cui installare qualsiasi sistema operativo e software, esattamente come faresti su un computer fisico.
- **Azure App Service** — il servizio PaaS per pubblicare applicazioni web e API senza gestire server o sistemi operativi: carichi il codice, Azure fa girare tutto il resto.
- **Database gestiti** (es. **Azure SQL Database**, **Cosmos DB**) — database pronti all'uso, con backup automatici, aggiornamenti e scalabilità gestiti dal provider: non devi installare né amministrare tu il motore del database.
- **Storage** (es. **Azure Blob Storage**) — spazio per salvare file, backup, immagini, documenti: una specie di “grande armadio” accessibile via internet, pensato per grandi quantità di dati.
- **Azure Kubernetes Service (AKS)** — servizio gestito per eseguire container orchestrati con Kubernetes (concetto già visto nella sezione 2), senza dover installare e configurare Kubernetes da zero.

Non serve memorizzare questi nomi: basta sapere che esistono e a cosa servono a grandi linee, così quando il team dice “l’abbiamo messo su App Service” o “i dati sono su Blob Storage” saprai di cosa si sta parlando.

13.5 AWS: la principale alternativa

ShopFacile ha scelto Azure, ma non è l'unico provider possibile: se un giorno il progetto dovesse integrarsi con un partner che usa un altro fornitore, o valutare un cambio, è utile riconoscere subito i nomi equivalenti. **Amazon Web Services (AWS)** è il principale concorrente di Azure (ed è storicamente il pioniere del cloud computing su larga scala). Alcuni progetti o clienti usano AWS invece di (o insieme ad) Azure, quindi è utile riconoscere i nomi equivalenti:

Concetto	Azure	AWS
Macchine virtuali (IaaS)	Virtual Machines	EC2
Piattaforma per applicazioni web (PaaS)	App Service	Elastic Beanstalk
Storage di file/oggetti	Blob Storage	S3
Database relazionale gestito	Azure SQL Database	RDS
Container orchestrati	AKS	EKS
Funzioni serverless	Azure Functions	Lambda

Il concetto importante da portarti via non è il nome preciso di ogni servizio, ma il fatto che **i grandi provider cloud offrono tutti gli stessi tipi di servizio**, solo con nomi diversi. Una volta capito il concetto (es. “storage per file”), riconoscere l'equivalente su un altro provider è semplice.

13.6 Scalabilità: verticale vs orizzontale

Che si scelga Azure o AWS, il motivo per cui Marco ha spinto per il cloud fin dall'inizio ha un nome preciso: la scalabilità. Uno dei grandi vantaggi del cloud è la **scalabilità**: la capacità di adattare le risorse informatiche disponibili in base a quanto ne serve in un dato momento, in modo rapido.

Esistono due modi per scalare:

- **Scalabilità verticale** (“scale up”): dare **più risorse alla stessa macchina** (più potenza di calcolo, più memoria). È come sostituire il motore di un'auto con uno più potente: la stessa auto va più veloce, ma c'è un limite a quanto puoi potenziarla.
- **Scalabilità orizzontale** (“scale out”): aggiungere **più macchine** che lavorano in parallelo, distribuendo il carico tra loro. È come aggiungere più corsie a un'autostrada: non rendi ogni auto più veloce, ma fai passare più traffico complessivamente.

Diagramma 3

Diagramma 3

Nel cloud, la scalabilità orizzontale è particolarmente potente perché si possono aggiungere (o togliere) macchine in pochi minuti, spesso in modo **automatico** in base al traffico reale — esattamente il caso di ShopFacile durante i saldi, con più macchine durante il picco e meno durante la notte. Questo sarebbe quasi impossibile con server fisici di proprietà, dove comprare un nuovo server richiede settimane.

13.7 Pay-as-you-go: paghi solo quello che usi

Poter scalare su e giù in pochi minuti sarebbe un vantaggio a metà se si continuasse comunque a pagare come prima, a prescindere dall'uso: il pezzo che completa il quadro è il modo in cui il cloud fa pagare quelle risorse. Il modello di pagamento tipico del cloud si chiama **pay-as-you-go** ("paga in base a quanto usi"), ed è probabilmente il cambiamento più concreto rispetto al vecchio modo di gestire l'infrastruttura.

Analogia: è come la bolletta della luce o dell'acqua: non paghi una cifra fissa indipendentemente da quanto consumi, ma in base ai consumi effettivi (kWh usati, litri consumati). Se un mese consumi meno, paghi meno.

Nel cloud questo significa, per esempio, che se hai una macchina virtuale attiva 3 ore al giorno, paghi solo per quelle 3 ore (e non per le 24). Se il sito di ShopFacile ha un picco di traffico per una settimana di saldi e poi torna normale, il team può scalare su e poi tornare giù, pagando solo per il periodo di picco.

Questo modello è molto diverso dall'acquisto di un server fisico, dove il costo è quasi tutto "anticipato" (compri il server, che poi resta tuo per anni, usato o no).

13.8 Regioni, data center e alta disponibilità

Pagare solo per quello che si usa presuppone che ci sia sempre qualcosa da usare: se ShopFacile fosse ospitata in un solo data center e quello avesse un guasto, tutto il pay-as-you-go del mondo non servirebbe a nulla mentre il sito è offline. È qui che entrano in gioco regioni e data center multipli. I provider cloud non hanno un solo, enorme data center: ne hanno **decine**, sparsi in tutto il mondo, organizzati in **regioni** (per esempio "Europa occidentale", "Stati Uniti orientali", "Asia sud-orientale"). Ogni regione contiene a sua volta più data center fisicamente separati.

Questo ha due vantaggi principali:

- **Vicinanza geografica:** puoi far girare il tuo software in una regione vicina ai tuoi utenti, così le pagine si caricano più velocemente (meno distanza = meno tempo di viaggio dei dati) — per questo ShopFacile, che vende soprattutto in Italia, gira su una regione Azure in Europa occidentale.
- **Alta disponibilità:** se un data center ha un problema (es. un guasto elettrico), il software può continuare a funzionare perché è distribuito su più data center o più regioni. È come avere due farmacie in città invece di una sola: se una chiude per un imprevisto, l'altra resta aperta e il servizio continua.

Non serve approfondire i dettagli tecnici dell'alta disponibilità in questa fase: basta sapere che è uno dei motivi per cui le grandi aziende scelgono il cloud invece di un singolo server fisico in un singolo posto — meno rischio di interruzioni del servizio.

Con questo, il percorso di ShopFacile dal server in ufficio a un'infrastruttura cloud distribuita su più data center è completo: riassumiamo i concetti principali visti lungo la strada.

13.9 Riepilogo: cosa ti serve ricordare

- **Cloud computing**: usare risorse informatiche (server, storage, rete) di qualcun altro via internet, invece di possedere e gestire hardware proprio — come affittare un appartamento invece di costruirsi una casa.
 - **IaaS**: affitti l'infrastruttura grezza (server virtuali, storage, rete); gestisci tu sistema operativo e software.
 - **PaaS**: affitti anche la piattaforma (OS, runtime, database gestiti); ti concentri solo sul codice.
 - **SaaS**: usi un software finito via browser (Gmail, Office 365, Salesforce); non gestisci nulla sotto.
 - Più sali da IaaS a SaaS, meno responsabilità di gestione hai (e meno controllo).
 - **Azure** (Microsoft) e **AWS** (Amazon) sono i due principali provider cloud: offrono gli stessi tipi di servizi con nomi diversi (es. Virtual Machines/EC2, App Service/Elastic Beanstalk, Blob Storage/S3).
 - **Scalabilità verticale**: più risorse alla stessa macchina. **Scalabilità orizzontale**: più macchine in parallelo.
 - **Pay-as-you-go**: paghi in base al consumo reale, come una bolletta.
 - **Regioni e data center**: i provider distribuiscono le loro infrastrutture in tutto il mondo, garantendo vicinanza agli utenti e **alta disponibilità** (continuità del servizio anche in caso di guasti locali).
-

Checklist di autoverifica

- Sapresti spiegare cos'è il cloud computing con un'analogia tua?
 - Sai distinguere IaaS, PaaS e SaaS e dire, per ciascuno, cosa gestisce il cliente e cosa il provider?
 - Sapresti nominare almeno due servizi Azure e i loro equivalenti AWS?
 - Sai spiegare la differenza tra scalabilità verticale e orizzontale?
 - Sai spiegare cosa significa "pay-as-you-go" con un esempio concreto?
 - Sai spiegare perché avere più regioni/data center aiuta l'alta disponibilità?
-

Esercizi pratici

1. **Mappa il progetto sul modello IaaS/PaaS/SaaS.** Chiedi a un collega developer o operations quali servizi cloud usa il progetto e prova a classificarli come IaaS, PaaS o SaaS (es. "usiamo una macchina virtuale per X" è IaaS, "il sito web gira su un servizio di hosting gestito" è PaaS). Scrivi la lista su un foglio o in un documento. **Come verificare:** mostra la tua classificazione alla tua collega Scrum Master/PM o al developer che hai intervistato e fatti confermare se hai classificato correttamente almeno 3 servizi su 4.

2. **Costruisci la tua “torta delle responsabilità”.** Senza guardare la tabella della sezione 13.3, ridisegna a mano su un foglio le righe Applicazioni/Dati/Runtime/Sistema operativo/Server e prova a compilare le colonne On-Premise, IaaS, PaaS, SaaS indicando chi gestisce cosa. **Come verificare:** confronta il tuo schema con la tabella originale nella sezione 13.3; se hai più di due celle diverse, rileggi la sezione 13.2 prima di andare avanti.
3. **Traduci Azure in AWS (e viceversa).** Prendi 4 servizi Azure a caso tra quelli citati nella sezione 13.4 (es. Virtual Machines, App Service, Blob Storage, AKS) e scrivi a memoria il loro equivalente AWS, senza guardare la tabella della sezione 13.5. Poi controlla. **Come verificare:** hai indovinato almeno 3 corrispondenze su 4 senza guardare la tabella.
4. **Scalabilità verticale o orizzontale?** Pensa a tre scenari reali (es. “il sito del progetto ha un picco di traffico durante una campagna marketing”, “un singolo processo di calcolo è troppo lento”, “un servizio deve restare disponibile anche se una macchina si guasta”) e per ciascuno decidi se serve scalabilità verticale, orizzontale, o entrambe, motivando la scelta a voce alta o per scritto. **Come verificare:** chiedi a un collega tecnico di ascoltare/leggere le tue tre risposte e dirti se il ragionamento (non necessariamente la soluzione tecnica esatta) è sensato.
5. **Calcola un pay-as-you-go semplificato.** Immagina una macchina virtuale che costa 0,10 € all'ora. Calcola quanto costerebbe tenerla attiva 24 ore su 24 per un mese, e quanto costerebbe invece tenerla attiva solo 8 ore al giorno nei giorni lavorativi. Confronta i due numeri. **Come verificare:** il secondo scenario deve costare meno di un terzo del primo; se il conto non torna, rileggi la sezione 13.7 sul concetto di pagamento a consumo.
6. **Individua le regioni del progetto reale.** Chiedi a un collega developer o a chi si occupa dell'infrastruttura in quale regione (o regioni) cloud gira il progetto, e se è previsto un meccanismo di ripristino in caso di guasto di un data center (anche solo “sappiamo che esiste un piano B” è una risposta valida a questo livello). **Come verificare:** sai rispondere, in una frase, alla domanda “cosa succederebbe se il data center principale del progetto avesse un problema per un giorno?” basandoti su quello che hai scoperto.

Collegamenti

- [14. Sicurezza](#) — dove vedremo come proteggere i dati e i sistemi che girano su queste infrastrutture cloud
- [15. Ambienti di sviluppo](#) — dove vedremo come sviluppo, test e produzione si organizzano concretamente, spesso proprio su infrastrutture cloud come quelle viste qui

Risorse

- [Microsoft Azure – Cos'è il cloud computing](#) — introduzione ufficiale ai concetti base del cloud
- [Microsoft Learn – Modelli di cloud computing: IaaS, PaaS, SaaS](#) — panoramica dei modelli di servizio cloud
- [AWS – What is Cloud Computing?](#) — la spiegazione ufficiale di Amazon Web Services
- [AWS – Tipi di cloud computing](#) — approfondimento su IaaS, PaaS e SaaS lato AWS

- [Microsoft Learn – Panoramica dei servizi Azure](#) — elenco dei servizi Azure più usati

14. Sicurezza

Fin qui hai visto come il codice viene scritto, integrato, testato e rilasciato attraverso una pipeline (sezione 11), come viene organizzato in architetture e servizi (sezione 12), e dove “vive” fisicamente nell’infrastruttura cloud (sezione 13). Manca un ultimo ingrediente, che non è un passaggio in più nel processo ma una **lente che attraversa tutti i passaggi precedenti**: la sicurezza informatica.

Non ti aspettare in questa sezione un corso da esperto di cybersecurity: non è il tuo ruolo, e non deve diventarlo. L’obiettivo è più modesto e insieme molto concreto: darti il vocabolario e i concetti di base per **capire di cosa parla il team** quando dice “abbiamo trovato una vulnerabilità critica, il rilascio è bloccato” o “dobbiamo attivare l’MFA per l’accesso all’ambiente di produzione”, e per **valutare correttamente il peso** di questi temi quando pianifichi il lavoro, parli con il cliente o scrivi un contratto.

Obiettivi della sezione

Al termine di questa sezione saprai:

- spiegare perché la sicurezza informatica non è “un problema tecnico” ma riguarda anche costi, reputazione, contratti e conformità legale;
- distinguere **autenticazione** e **autorizzazione**;
- descrivere le buone pratiche di base su password e **autenticazione a più fattori (MFA)**;
- ricollegare HTTPS e crittografia a quanto già visto nei Fondamenti di informatica, con un’idea semplice di cosa significhi “dato cifrato”;
- riconoscere, a grandi linee, cosa sono vulnerabilità comuni come SQL Injection e Cross-Site Scripting (XSS), e perché esistono;
- sapere cos’è la **OWASP Top 10** e a cosa serve come riferimento di settore;
- capire cos’è il **DevSecOps** e il principio dello “**shift left**” della sicurezza;
- capire come gli scan di sicurezza automatici si inseriscono nella pipeline CI/CD già vista nella sezione 11;
- spiegare, a livello base, perché il **GDPR** e la protezione dei dati personali riguardano anche il tuo lavoro di PM;
- capire perché **backup e disaster recovery** sono parte della sicurezza, non un tema separato.

14.1 Perché la sicurezza riguarda anche un PM, non solo i tecnici

Prova a pensarla così: la sicurezza informatica è come l’**impianto antincendio di un edificio**. Non lo vedi quasi mai in azione, ci speri di non doverlo mai usare, e nessuno pianifica una giornata di lavoro pensando all’impianto antincendio. Ma se manca, o è mal progettato, il giorno in cui succede qualcosa **le conseguenze sono enormemente più gravi** di quanto sarebbe costato prevenirle.

Per un Project Manager o uno Scrum Master, un incidente di sicurezza non è “un bug in più da segnare sul backlog”: ha impatti che riconoscerai immediatamente come “roba tua”:

- **Costi diretti:** fermare un servizio compromesso, investigare cosa è successo, correggere la falla, spesso richiede ore o giorni di lavoro imprevisto di più persone del team, sottratte a quanto era pianificato nello sprint.
- **Reputazione:** un cliente che scopre che i dati dei suoi utenti sono stati esposti perde fiducia nel progetto e in chi lo ha realizzato — una fiducia che si ricostruisce molto più lentamente di quanto si perda.
- **Contratti:** molti contratti con i clienti includono clausole precise su sicurezza e gestione dei dati (a volte chiamate SLA di sicurezza o requisiti di compliance). Non rispettarle può voler dire penali economiche o, nei casi più seri, la fine della collaborazione.
- **Conformità legale:** normative come il GDPR (che vedremo al paragrafo 14.8) impongono obblighi precisi sulla protezione dei dati personali, con sanzioni concrete in caso di violazione.

Il punto chiave da portare con te: **la sicurezza è un requisito del progetto, esattamente come una funzionalità richiesta dal cliente** — non un “optional” che si aggiunge se resta tempo a fine sprint. Un PM che tratta la sicurezza come priorità fin dalla pianificazione fa risparmiare tempo, denaro e problemi seri più avanti.

Esempio pratico: durante la pianificazione di una nuova funzionalità di ShopFacile che permette ai clienti di caricare la foto di un prodotto da restituire, il team si accorge che serve anche configurare i permessi di accesso a quei file e cifrare il canale di upload. Se Sara, come PO, inserisce questo lavoro nella stima dello sprint fin dall'inizio (come farebbe con qualsiasi altro requisito), il rilascio resta nei tempi. Se invece la sicurezza viene “ricordata” solo a fine sprint, il rilascio slitta e il team lavora sotto pressione per rimettersi in pari — lo stesso lavoro, fatto più tardi, costa di più.

14.2 Autenticazione vs Autorizzazione

Il caricamento delle foto di resa solleva subito una domanda concreta: chi può fare cosa, su quei file e su ShopFacile in generale? È il momento di distinguere con precisione due concetti che si sentono nominare quasi sempre insieme, e che vengono facilmente confusi — ma rispondono a due domande diverse:

- **Autenticazione** risponde alla domanda: “**Chi sei?**”. È il processo con cui un sistema verifica che tu sia davvero chi dici di essere (tipicamente inserendo username e password, o mostrando un altro tipo di “prova d'identità”).
- **Autorizzazione** risponde alla domanda: “**Cosa puoi fare, ora che sappiamo chi sei?**”. È il processo con cui il sistema decide a quali risorse o funzionalità hai accesso.

Analogia: il cinema

Immagina di andare al cinema:

- Alla biglietteria mostri il tuo **biglietto**: la persona alla cassa verifica che sia un biglietto valido per lo spettacolo di oggi. Questa è l'**autenticazione**: hai dimostrato di essere una persona con diritto a entrare.
- Una volta dentro, il biglietto indica anche **quale posto** puoi occupare — Sala 3, fila D, posto 12. Non puoi sederti dove vuoi, né entrare nella sala riservata a un altro film. Questa è l'**autorizzazione**: definisce esattamente cosa puoi fare, anche se sei già stato ammesso.

Puoi essere autenticato correttamente (il tuo biglietto è vero) ma non autorizzato a fare una certa cosa (entrare nella Sala 5, dove è in programmazione un altro film). Sono due controlli **distinti e sequenziali**: prima si stabilisce chi sei, poi cosa puoi fare.

Diagramma 1

Diagramma 1

Esempio pratico nel progetto: Ahmed può autenticarsi correttamente sulla piattaforma Azure DevOps del progetto ShopFacile (sa la password, entra), ma essere autorizzato solo a **leggere** il codice del repository del catalogo prodotti e non a **modificare** le impostazioni della pipeline di produzione — quel permesso è riservato, ad esempio, a Marco o a Luca. Due sviluppatori autenticati sulla stessa piattaforma possono avere autorizzazioni completamente diverse.

14.3 Password, credenziali e MFA: le buone pratiche di base

Autenticazione e autorizzazione presuppongono entrambe una cosa: che sia davvero difficile “spacciarsi” per qualcun altro. Ed è proprio qui, sulle credenziali usate per autenticarsi, che si concentra la maggior parte degli incidenti reali. La stragrande maggioranza degli incidenti di sicurezza non nasce da tecniche sofisticate da film di spionaggio, ma da **credenziali debitamente rubate o indovinate**: password troppo semplici, riutilizzate ovunque, o condivise con troppa disinvoltura.

Qualche buona pratica base, valida per te e per chiunque nel team:

- **Password forti e uniche**: una password lunga (idealmente 12+ caratteri), non riconducibile a informazioni personali facilmente reperibili (data di nascita, nome del cane), e **diversa per ogni servizio**. Se un servizio viene compromesso e la password è riusata altrove, il danno si moltiplica.
- **Gestori di password (password manager)**: strumenti come quelli aziendali forniti dall'IT (o soluzioni diffuse come Bitwarden, 1Password) generano e ricordano password complesse per ogni servizio, così non devi memorizzarle tu né riusarle. È molto più sicuro affidarsi a uno di questi strumenti che scrivere le password su un file di testo o, peggio, su un post-it.
- **Autenticazione a più fattori (MFA - Multi-Factor Authentication)**: oltre alla password (qualcosa che sai), il sistema richiede una seconda “prova”, ad esempio un codice generato da un'app sul telefono o un'impronta digitale (qualcosa che hai o che sei). Anche se qualcuno scoprisse la tua password, senza il secondo fattore non potrebbe entrare.

Analogia: la password è come la chiave di casa. L'MFA è come avere, oltre alla chiave, anche un citofono con telecamera che chiede conferma prima di far entrare qualcuno — anche se ha copiato la chiave, senza quella seconda verifica non entra.

Diagramma 2

Diagramma 2

Nel progetto, è molto probabile che l'accesso a strumenti sensibili (Azure DevOps, ambiente cloud, VPN aziendale) richieda l'MFA per policy aziendale: se ti viene chiesto di installare un'app di autenticazione sul telefono per lavoro, ora sai perché.

Esempio pratico: immagina che la password di un membro del team venga scoperta perché usata anche su un altro servizio online violato in passato (è più comune di quanto sembri: succede quando la stessa password viene riusata su più siti). Chi ha ottenuto quella password prova ad accedere ad Azure DevOps con quelle credenziali. Senza MFA, entrerebbe subito. Con l'MFA attivo, il sistema chiede anche il codice generato dall'app sul telefono di quella persona: l'attaccante non lo ha, e l'accesso viene bloccato. Questo è esattamente il motivo per cui, dopo un data breach di un servizio terzo, la prima raccomandazione è sempre “cambia la password e verifica che l'MFA sia attivo”, non solo la prima.

14.4 HTTPS e crittografia: un richiamo veloce

Password e MFA proteggono l'accesso a un sistema, ma non bastano da sole: anche quando i dati viaggiano in rete o restano salvati da qualche parte, serve un'altra forma di protezione. Nella sezione [2. Fondamenti di informatica](#) hai già visto la differenza tra HTTP e HTTPS: HTTPS **cifra** i dati scambiati tra client e server, in modo che chi li intercettasse lungo il tragitto non possa leggerli.

Qui aggiungiamo solo un tassello: cosa significa concretamente “dato cifrato”?

Cifrare un dato significa trasformarlo, tramite un algoritmo matematico e una “chiave” segreta, in una sequenza illeggibile per chiunque non possieda la chiave giusta per invertire la trasformazione (**decifrare**). Un esempio semplicissimo, giocattolo, solo per l'idea:

Testo originale: CIAO

Regola di cifratura: sposta ogni lettera di 3 posizioni nell'alfabeto

Testo cifrato: FLDR

Chi intercetta “FLDR” senza conoscere la regola (“sposta di 3 posizioni”) non ha modo semplice di capire che il messaggio originale era “CIAO”. Gli algoritmi usati davvero da HTTPS sono incomparabilmente più complessi e robusti di questo esempio — è solo per rendere intuitiva l'idea di “trasformare un dato leggibile in uno illeggibile, con una chiave che serve per tornare indietro”.

Due momenti in cui i dati vanno protetti, entrambi importanti:

- **Dati “in transito”**: mentre viaggiano in rete tra client e server — è il ruolo di HTTPS, già visto.
- **Dati “a riposo”** (*at rest*): mentre sono salvati su un disco, in un database, in un backup. Anche questi vanno spesso cifrati, perché se qualcuno accede fisicamente o illegittimamente a quel disco, senza la chiave i dati restano illeggibili.

Non avrai bisogno di implementare crittografia, ma è utile sapere che quando un cliente chiede “i nostri dati sono cifrati?”, la risposta completa riguarda **sia** il transito (HTTPS) **sia** il salvataggio (a riposo).

Esempio pratico: un cliente compila il modulo di pagamento al checkout di ShopFacile e invia i propri dati (indirizzo di spedizione, dati della carta). Se il sito usa HTTPS, quei dati viaggiano cifrati dal browser del cliente al server — anche se qualcuno intercettasse il traffico di rete (ad esempio su un Wi-Fi pubblico non sicuro), leggerebbe solo una sequenza illeggibile. Una volta arrivati al server, quei dati vengono salvati nel database ordini: se anche il database è configurato per cifrare i dati “a riposo”, chi eventualmente accedesse senza autorizzazione al disco fisico del server (un furto, una copia illegittima del backup) troverebbe comunque dati illeggibili senza la chiave giusta. Le due protezioni sono indipendenti: un sito può avere HTTPS ma un database non cifrato, o viceversa — un cliente attento chiede entrambe.

14.5 Vulnerabilità comuni: perché il codice va “testato” anche per la sicurezza

Cifrare i dati non serve a nulla se il codice stesso ha una falla che permette di aggirare i controlli a monte: è il momento di guardare da dove nascono davvero questi problemi. Una **vulnerabilità** è un errore o una debolezza nel software che qualcuno con intenzioni malevole (un *attaccante*) può sfruttare per fare qualcosa che non dovrebbe poter fare: leggere dati che non gli appartengono, modificarli, bloccare un servizio.

Non serve conoscere i dettagli tecnici di come si sfruttano, ma è utile sapere che **esistono**, che hanno nomi ricorrenti, e che è per questo che il codice va controllato anche dal punto di vista della sicurezza, non solo della correttezza funzionale. Due esempi molto citati:

- **SQL Injection**: succede quando un'applicazione inserisce, senza i controlli giusti, un testo scritto dall'utente direttamente dentro una query verso il database (quelle query SQL viste nella sezione 2). Se l'utente scrive, in un campo di ricerca, non un nome ma un pezzo di comando SQL “camuffato”, e l'applicazione non lo riconosce come pericoloso, quel comando può finire per essere eseguito sul database — ad esempio per leggere dati che non dovrebbe vedere.
- **Cross-Site Scripting (XSS)**: succede quando un'applicazione web mostra, senza i controlli giusti, un testo scritto da un utente (es. un commento, un messaggio) direttamente dentro la pagina vista da altri utenti. Se quel testo contiene in realtà un piccolo “programma” camuffato da testo normale, il browser di chi legge quella pagina può finire per eseguirlo — ad esempio per rubare dati della sessione di quell'altro utente.

Analogia semplice: pensa a un modulo di iscrizione dove il campo “Nome” dovrebbe contenere solo... un nome. Se il sistema si fida ciecamente di quello che scrivi e non controlla mai cosa c'è davvero scritto lì dentro, qualcuno potrebbe scrivere, invece del suo nome, un'istruzione nascosta — e se il sistema la esegue senza accorgersene, il “modulo” è diventato una porta d'accesso indesiderata.

Il punto per te, come PM, non è saper riconoscere queste vulnerabilità nel codice — è sapere che **esistono categorie note e ricorrenti** di errori di questo tipo, e che per questo il codice va **testato anche per la sicurezza**, con strumenti dedicati, esattamente come si testa la correttezza funzionale con i test automatici visti nella sezione 11.

Esempio pratico: Ahmed, ancora alle prime armi, scrive una nuova funzionalità per applicare i codici promozionali al carrello e, per fare in fretta, costruisce la query verso il database concatenando direttamente il testo digitato dal cliente nel campo “codice promozionale”, senza validarlo. Durante la code review, Giulia — che segue da vicino test e qualità del codice — nota il problema e segnala la Merge Request come da correggere prima ancora che arrivi a un test funzionale: è esattamente il tipo di errore (un classico caso di SQL Injection) che un developer alle prime esperienze può commettere senza malizia, e che una revisione attenta deve intercettare prima del rilascio, non dopo.

14.6 OWASP Top 10: il riferimento standard del settore

Riconoscere una singola query pericolosa in una code review è utile, ma il team non può fare affidamento solo sulla memoria di chi legge il codice quel giorno: serve un riferimento condiviso e sistematico. Con il tempo, la community internazionale della sicurezza informatica ha raccolto e catalogato le vulnerabilità più comuni e più pericolose che colpiscono le applicazioni web, mantenendo una lista aggiornata periodicamente: la **OWASP Top 10**.

OWASP (Open Worldwide Application Security Project) è un'organizzazione internazionale, non a scopo di lucro, dedicata a migliorare la sicurezza del software. La sua **Top 10** è probabilmente il documento più citato al mondo in questo ambito: una classifica delle categorie di vulnerabilità più critiche e più frequenti nelle applicazioni web (di cui SQL Injection e XSS, visti sopra, sono esempi storici).

Non devi memorizzare l'elenco né conoscerne i dettagli tecnici. Quello che ti serve sapere è:

- è un **riferimento riconosciuto a livello mondiale**: quando un cliente, in un contratto o in un audit, chiede che l'applicazione sia “conforme alla OWASP Top 10” o che sia stato eseguito un “test basato sulla OWASP Top 10”, sta chiedendo che il software sia stato verificato contro questo elenco di rischi noti;
- gli strumenti automatici di analisi di sicurezza che vedrai nella pipeline (paragrafo 14.8) fanno spesso riferimento esplicito a queste categorie;
- è aggiornata periodicamente, perché le tecniche di attacco e le tecnologie usate nel software cambiano nel tempo.

Sapere che questo standard esiste ti permette di seguire una conversazione tecnica sulla sicurezza senza perderti, e di capire perché in una gara d'appalto o in un capitolato può comparire esplicitamente il riferimento a OWASP.

Esempio pratico: il cliente di un progetto chiede, in fase di capitolato, che prima del rilascio in produzione venga eseguito un “penetration test basato sulla OWASP Top 10”. In pratica, un team di sicurezza (interno o esterno) prova a verificare, categoria per categoria di quella lista, se l'applicazione presenta debolezze note — ad esempio, controlla se è vulnerabile a SQL Injection o XSS (visti sopra), se gestisce correttamente l'autenticazione, se espone informazioni che non dovrebbe. Il risultato è un report con eventuali criticità trovate, classificate per gravità: è esattamente il tipo di documento che un PM può dover presentare al cliente come prova che il software è stato verificato secondo uno standard riconosciuto, non “a occhio”.

14.7 DevSecOps: la sicurezza fin dall'inizio (“shift left”)

La OWASP Top 10 dice cosa controllare; resta da capire *quando* farlo nel ciclo di vita del progetto — ed è qui che entra in gioco un principio già visto altrove in una forma diversa. Nella sezione [9. DevOps](#) hai visto come la cultura DevOps abbatta il muro tra chi scrive il software e chi lo mette in produzione, integrando i due mondi in un unico flusso continuo. Il **DevSecOps** estende esattamente la stessa logica alla sicurezza: la “Sec” (Security) non è più una fase separata, affidata a un team diverso, che interviene solo alla fine, poco prima del rilascio — è **integrata in ogni fase** del processo di sviluppo, fin dall'inizio.

Questo principio si chiama “**shift left**” (letteralmente “spostare a sinistra”): se disegni il processo di sviluppo come una linea temporale che va da sinistra (progettazione) a destra (produzione), storicamente i controlli di sicurezza avvenivano quasi solo a destra, poco prima del rilascio — quando un problema trovato è già molto costoso da correggere, perché magari richiede di rivedere scelte di progettazione fatte mesi prima. Il DevSecOps **sposta quei controlli verso sinistra**, il più presto possibile nel processo.

Diagramma 3

Diagramma 3

Diagramma 4

Diagramma 4

Perché conviene? Perché **correggere un problema di sicurezza costa sempre meno, più presto lo si scopre** — esattamente come per i bug funzionali: un errore di progettazione trovato durante la stesura dei requisiti costa una discussione; lo stesso errore scoperto in produzione, con dati reali di utenti reali già esposti, può costare giorni di lavoro d'emergenza, comunicazioni al cliente e, potenzialmente, obblighi legali di notifica (ne parliamo al paragrafo 14.9).

Per un PM/Scrum Master, il DevSecOps si traduce concretamente in domande da porsi già in fase di pianificazione, non solo a rilascio imminente: “questa nuova funzionalità gestisce dati sensibili?”, “abbiamo previsto tempo per la revisione di sicurezza?”, “il quality gate di sicurezza è configurato sulla pipeline di questo progetto?”.

14.8 Scan di sicurezza nella pipeline CI/CD

Il “shift left” resta un principio astratto finché non si traduce in qualcosa che gira automaticamente a ogni commit: vediamo concretamente come. Nella sezione [11. CI/CD](#) hai già incontrato, nella fase “Analisi di qualità e sicurezza del codice” e nella tabella dei quality gate, l’idea che la pipeline includa controlli automatici di sicurezza. Qui rendiamo quel concetto un po’ più concreto.

Gli strumenti di scan automatico più comuni, integrati direttamente nella pipeline, verificano cose diverse:

- **Analisi statica del codice (SAST - Static Application Security Testing)**: analizza il codice sorgente, senza eseguirlo, cercando pattern noti di vulnerabilità (es. un punto dove un dato dell’utente finisce, senza controlli, dentro una query SQL — il rischio di SQL Injection visto al paragrafo 14.5).
- **Analisi delle dipendenze (SCA - Software Composition Analysis)**: quasi nessun software è scritto completamente da zero — si usano librerie esterne (open source o commerciali). Questi strumenti controllano se una delle librerie usate ha vulnerabilità **note e pubblicamente documentate**, e segnalano se serve aggiornarla.
- **Analisi dinamica (DAST - Dynamic Application Security Testing)**: a differenza del SAST, prova ad “attaccare” l’applicazione realmente in esecuzione (tipicamente in un ambiente di test), simulando alcune delle tecniche di un vero attaccante, per vedere come si comporta davvero.

Il risultato di questi scan si traduce, come già visto nella sezione 11, in un **quality gate**: se viene trovata una vulnerabilità sopra una certa soglia di gravità (tipicamente “alta” o “critica”), la pipeline si ferma e il rilascio viene bloccato finché il problema non viene risolto — non è diverso, nel meccanismo, da un test funzionale che fallisce.

Diagramma 5

Diagramma 5

Il vantaggio di questi scan automatici, rispetto a una revisione manuale occasionale, è lo stesso visto per i test automatici in generale: **ripetibilità e velocità**. Ogni singolo commit viene controllato, sempre, senza dipendere dalla memoria o dalla disponibilità di una persona che “se ne ricordi”.

Esempio pratico: Marco, che segue spesso la parte infrastrutturale di ShopFacile, configura lo scan SCA sulla pipeline del servizio pagamenti e scopre che una libreria usata per generare i PDF delle fatture ha una vulnerabilità nota, già pubblicata. Il quality gate blocca il rilascio finché la libreria non viene aggiornata: senza quello scan automatico, la falla sarebbe rimasta in produzione fino a quando (e se) qualcuno se ne fosse accorto da solo, magari troppo tardi.

14.9 GDPR e privacy dei dati: un accenno essenziale

Gli scan automatici proteggono il codice, ma c'è un'altra dimensione della sicurezza che riguarda direttamente le persone i cui dati ShopFacile raccoglie ogni giorno — clienti che comprano, si registrano, chiedono un rimborso. Il **GDPR** (General Data Protection Regulation, “Regolamento Generale sulla Protezione dei Dati”) è la normativa europea che disciplina come le aziende devono trattare i **dati personali** delle persone (nomi, email, indirizzi, dati di navigazione, e molto altro).

Non è un tema puramente etico (“è giusto proteggere i dati delle persone”), è un **obbligo legale**, con conseguenze concrete in caso di violazione: sanzioni economiche (che possono essere molto rilevanti), obbligo di comunicare la violazione alle autorità competenti e, spesso, agli stessi utenti coinvolti entro tempi stretti, oltre al danno reputazionale già visto al paragrafo 14.1.

Per il tuo ruolo, alcuni concetti base sufficienti per orientarti:

- se un progetto raccoglie o gestisce dati di persone (utenti, dipendenti, clienti), quei dati vanno protetti con misure tecniche adeguate (accessi controllati, cifratura, backup sicuri — tutti concetti già visti in questa sezione);
- va raccolto solo il dato **necessario** allo scopo dichiarato (principio di “minimizzazione”), non “tutto quello che si può raccogliere”;
- una violazione di dati personali (un cosiddetto **data breach**) ha obblighi di notifica precisi, con scadenze temporali stringenti;
- molti progetti prevedono un ruolo o un referente per la protezione dei dati (in alcuni contesti, un DPO - Data Protection Officer) da coinvolgere quando una funzionalità tratta dati personali in modo nuovo.

Non ti verrà chiesto di diventare un esperto legale di GDPR, ma è utile riconoscere quando una richiesta del cliente o una funzionalità del progetto **tocca dati personali**, per sapere che in quei casi il tema sicurezza/privacy va coinvolto fin dalla pianificazione, non aggiunto a posteriori.

Esempio pratico: Sara, come PO, riceve dal cliente una richiesta di funzionalità che prevede di salvare, per ogni cliente di ShopFacile, anche il numero di telefono e la posizione geografica approssimativa (per proporre le consegne più rapide vicino a lui). Sono entrambi dati personali secondo il GDPR. Sara, in fase di pianificazione, si pone (e pone al team) alcune domande prima di stimare lo sprint: “questo dato è davvero necessario per la funzionalità, o stiamo raccogliendo più del dovuto?”, “chi avrà accesso a questo dato una volta salvato?”, “per quanto tempo lo conserviamo?”. Se invece la funzionalità viene sviluppata e rilasciata senza porsi queste domande, e in seguito si scopre un accesso non autorizzato ai dati dei clienti, ShopFacile si trova davanti a un data breach da notificare formalmente, con tempi stretti e conseguenze reali — non un semplice bug da correggere alla prossima release.

14.10 Backup e disaster recovery: il piano B quando qualcosa va storto

Anche il progetto più attento a GDPR, scan e code review non è immune da un imprevisto puro e semplice: un tema che a volte viene percepito come “amministrativo” ma è parte integrante della sicurezza, ed è l'ultimo ingrediente di questa sezione. Cosa succede se, nonostante tutte le precauzioni, qualcosa va comunque storto — un guasto hardware, un errore umano che cancella dati per sbaglio, o un attacco che compromette un sistema.

- **Backup:** una copia dei dati, salvata separatamente (spesso in un luogo fisicamente diverso), che permette di recuperare le informazioni se l'originale viene perso o danneggiato. Un backup che non viene mai testato — cioè non si è mai provato davvero a ripristinarlo — è un falso senso di sicurezza: l'unico backup di cui ci si può davvero fidare è quello che si è verificato essere effettivamente ripristinabile.
- **Disaster Recovery (DR):** il piano più ampio, non limitato ai soli dati, per far ripartire un intero sistema/servizio dopo un evento grave (un guasto importante, un disastro naturale che colpisce un data center, un attacco su larga scala), nel modo più rapido possibile.

Analogia: il backup è come la copia delle chiavi di casa lasciata a un vicino di fiducia: se perdi le tue, non resti fuori. Il disaster recovery è come avere un piano intero per la tua famiglia in caso la casa diventi inabitabile — dove andare, cosa portare, chi contattare — non basta la copia delle chiavi se la casa stessa non esiste più.

Due metriche che il team può citare parlando di DR, utili da riconoscere:

- **RTO (Recovery Time Objective):** quanto tempo, al massimo, ci si può permettere che il servizio resti fermo prima di essere ripristinato.
- **RPO (Recovery Point Objective):** quanti dati, al massimo, ci si può permettere di perdere (misurato in tempo — es. “non più di 1 ora di dati persi”) in caso di ripristino da backup.

Per un PM, sapere che backup e DR non sono un dettaglio tecnico invisibile ma **una scelta con un costo e un livello di rischio accettato** è importante: un cliente che chiede “in quanto tempo torniamo operativi se tutto si ferma?” sta facendo una domanda di business legittima, a cui il team tecnico deve poter rispondere con numeri concreti (gli RTO/RPO concordati), non con un generico “dovremmo farcela”.

Esempio pratico: il database ordini di ShopFacile subisce un guasto e diventa inaccessibile alle 14:00 di un giorno lavorativo, proprio durante i saldi. Luca, come Scrum Master, coordina la risposta all'incidente: raccoglie chi serve, tiene aggiornato il resto del team e fa da punto di contatto con Sara per le comunicazioni verso il cliente, mentre Marco lavora al ripristino tecnico. ShopFacile ha concordato un RTO di 2 ore, quindi il team ha l'obiettivo di ripristinare il servizio entro le 16:00. Se l'RPO concordato è di 1 ora, e l'ultimo backup automatico risale alle 13:30, il team sa già, prima ancora di iniziare il ripristino, che andranno persi al massimo 30 minuti di ordini recenti — un'informazione che

Luca può comunicare subito, invece di scoprirla a ripristino concluso. Senza questi numeri concordati in anticipo, la stessa situazione genera solo incertezza e domande a cui nessuno sa rispondere con precisione.

14.11 Riepilogo

In questa sezione hai visto i concetti di base della sicurezza informatica utili al tuo ruolo di PM/Scrum Master:

- la sicurezza non è solo un tema tecnico: ha impatti diretti su costi, reputazione, contratti e conformità legale;
- **autenticazione** (chi sei) e **autorizzazione** (cosa puoi fare) sono due controlli distinti e sequenziali;
- password forti, gestori di password e **MFA** sono le difese di base contro il tipo di incidente più comune: il furto di credenziali;
- **HTTPS** cifra i dati in transito; i dati “a riposo” (salvati) vanno cifrati a loro volta;
- vulnerabilità come **SQL Injection** e **XSS** sono categorie note e ricorrenti di errori nel codice, motivo per cui il software va testato anche dal punto di vista della sicurezza;
- la **OWASP Top 10** è il riferimento standard del settore per le vulnerabilità più critiche nelle applicazioni web;
- il **DevSecOps** integra la sicurezza fin dall'inizio del processo di sviluppo, con il principio dello “**shift left**”;
- gli **scan di sicurezza automatici** (SAST, SCA, DAST) si inseriscono nella pipeline CI/CD come quality gate, esattamente come i test funzionali;
- il **GDPR** impone obblighi legali precisi sulla protezione dei dati personali, con conseguenze concrete in caso di violazione;
- **backup e disaster recovery** sono il piano B necessario quando, nonostante tutte le precauzioni, qualcosa va comunque storto.

✓ Nella prossima sezione vedrai come questi concetti si traducono in scelte concrete di configurazione degli ambienti di sviluppo, test, staging e produzione.

Checklist di autoverifica

Prima di passare alla sezione successiva, prova a rispondere (anche solo a voce, o scrivendo due righe) a queste domande:

- Sai spiegare la differenza tra autenticazione e autorizzazione con l'analogia del cinema?
- Sai spiegare a cosa serve l'MFA e perché rafforza la sola password?
- Sai spiegare, con parole tue, cosa significa “dato cifrato”?
- Sapresti spiegare a un collega non tecnico perché il codice va testato anche per la sicurezza, non solo per la correttezza funzionale?
- Sai spiegare cos'è la OWASP Top 10 e perché viene citata nei contratti?
- Sai spiegare il principio dello “shift left” nel DevSecOps?

- Sai collegare gli scan SAST/SCA/DAST al concetto di quality gate visto nella sezione CI/CD?
- Sai spiegare perché il GDPR riguarda anche le scelte di un PM, non solo i legali?
- Sai spiegare la differenza tra backup e disaster recovery?

Esercizi pratici

1. **Mappa autenticazione/autorizzazione sul progetto:** chiedi a un collega developer come funziona il login nell'applicazione del progetto e quali ruoli/permessi esistono (es. "utente base", "amministratore"). Disegna uno schema semplice (anche a mano) con almeno due ruoli diversi e cosa ciascuno può o non può fare. **Come verificare:** sei in grado di indicare almeno un'azione che un ruolo può fare e un altro no, usando esempi reali del progetto (non inventati) e senza confondere "chi è" con "cosa può fare".
2. **Controlla l'MFA sui tuoi strumenti di lavoro:** verifica se hai l'autenticazione a più fattori attiva su tutti gli strumenti che usi per lavoro (email aziendale, Azure DevOps, VPN). Se manca su qualcuno, attivala (o chiedi all'IT come fare). **Come verificare:** puoi elencare, per ogni strumento di lavoro che usi, se l'MFA è attiva sì/no, e sai spiegare a un collega perché è importante che lo sia anche se la password è "forte".
3. **Riconosci una vulnerabilità in un caso pratico:** leggi la descrizione di una vulnerabilità reale (puoi cercarne una recente sul sito OWASP o in una news di settore, senza bisogno di capirne i dettagli tecnici) e prova a classificarla: assomiglia più a una SQL Injection, a una XSS, o a un'altra categoria? Scrivi due righe su cosa avrebbe potuto fare un attaccante se quella vulnerabilità non fosse stata corretta. **Come verificare:** sai spiegare, con parole tue e senza gergo tecnico eccessivo, cosa rischiava di succedere e perché "testare anche per la sicurezza" avrebbe potuto prevenirlo.
4. **Trova gli scan di sicurezza nella pipeline reale:** chiedi a un collega (developer o DevOps engineer) di farti vedere, nella pipeline CI/CD del progetto, dove sono configurati gli scan di sicurezza (SAST, SCA, o eventualmente DAST) e cosa succede quando trovano una vulnerabilità critica. **Come verificare:** sai indicare in quale fase della pipeline si trova ciascun tipo di scan, e sai descrivere concretamente cosa vede il team quando un quality gate di sicurezza blocca un rilascio (una notifica? un'email? il rilascio che semplicemente non parte?).
5. **Simula una richiesta del cliente sul GDPR:** immagina che il cliente chieda "per quanto tempo conservate i dati personali degli utenti dopo che chiudono l'account, e chi può accedervi?". Prova a scrivere, in 5-6 righe, come struttureresti la risposta (anche senza conoscere i dettagli tecnici esatti del progetto — l'obiettivo è la struttura del ragionamento, non il dato preciso). **Come verificare:** la tua risposta tocca sia l'aspetto tecnico (dove e come sono protetti i dati) sia l'aspetto di processo (chi decide i tempi di conservazione, chi ha accesso), e non promette nulla che non puoi verificare con il team.
6. **Chiedi RTO e RPO reali del progetto:** chiedi alla tua collega Scrum Master/PM o a un membro del team operations quali sono, se definiti, l'RTO e l'RPO concordati per l'ambiente di produzione del progetto (anche se la risposta è "non sono ancora stati definiti formalmente" — è comunque

un'informazione utile). **Come verificare:** sai riportare a un collega, con le tue parole, cosa significano quei due numeri specifici (non la definizione generica del libro) applicati al contesto reale del progetto.

Collegamenti

- [15. Ambienti di sviluppo](#) — come si configurano in pratica accessi, credenziali e isolamento tra Dev, Test, Staging e Produzione
- [16. Glossario](#) — per ripassare rapidamente ogni termine visto in questa sezione

Risorse

- [OWASP Top 10](#) — la classifica ufficiale delle vulnerabilità più critiche per le applicazioni web
- [OWASP Foundation — sito ufficiale](#) — panoramica su progetti, guide e risorse gratuite in ambito sicurezza applicativa
- [NCSC \(National Cyber Security Centre, UK\) — Cyber security basics](#) — guide pratiche e accessibili su password, MFA e igiene informatica di base
- [Microsoft Learn — Security concepts](#) — introduzione ai principi moderni di sicurezza (Zero Trust) applicati al cloud
- [Microsoft Learn — Cos'è il DevSecOps](#) — approfondimento su come la sicurezza si integra nel ciclo DevOps
- [Garante per la protezione dei dati personali — Cos'è il GDPR](#) — riferimento istituzionale italiano sul regolamento europeo

15. Ambienti di sviluppo

Nella sezione [11. CI/CD](#) hai già incontrato il concetto di **ambiente**: sai che il codice non passa direttamente dal computer di uno sviluppatore agli utenti finali, ma attraversa diverse “tappe” (Dev, Test/QA, Staging, Produzione) prima di arrivare al traguardo. In questa sezione ci fermiamo su ciascuna di queste tappe, una per una, per capire **cosa succede davvero dentro ognuna di esse**, chi ci lavora, che dati si usano e perché la disciplina attorno a questo tema è tutt'altro che un dettaglio tecnico secondario. Seguiremo, tappa per tappa, un unico bug reale del carrello di ShopFacile: vedrai lo stesso identico problema attraversare le quattro tappe, dalla scrivania di Marco fino alla produzione.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- spiegare perché non si testa mai direttamente in produzione;
- descrivere lo scopo specifico di ognuno dei quattro ambienti tipici (Dev, Test/QA, Staging, Produzione);
- capire perché gli ambienti devono essere quanto più simili possibile tra loro, ed evitare il classico “funziona sul mio computer”;
- distinguere dati di test da dati reali, e sapere perché non si usano mai dati sensibili reali fuori da produzione;
- capire cos'è una configurazione per ambiente (es. l'indirizzo di un database diverso per ogni ambiente);
- descrivere perché l'accesso alla produzione è, di norma, molto più restrittivo di quello agli altri ambienti.

15.1 Perché servire diversi ambienti: la “prova costume”

Immagina uno spettacolo teatrale che debutta davanti al pubblico pagante la prima serata, senza mai aver fatto una prova. Nessun regista serio lo farebbe: prima ci sono le **provo generali**, poi la **prova costume** (con luci, scenografia e costumi veri, ma senza pubblico), e solo alla fine la **prima** davanti agli spettatori. Ogni tappa serve a scoprire un problema diverso — un attore che sbaglia una battuta, un cambio scena troppo lento, un microfono che non funziona — in un contesto dove sbagliare **non costa nulla**, invece di scoprirlo in diretta con il pubblico in sala.

Il software funziona esattamente allo stesso modo. **Non si testa mai direttamente in produzione** per un motivo semplicissimo: in produzione ci sono utenti reali, che stanno usando il software per lavorare, comprare, prenotare, pagare. Se un test fallisce lì, l’“errore” non è un rigo rosso in un log: è un utente che perde dei dati, una transazione che non va a buon fine, un cliente che chiama il servizio assistenza arrabbiato.

Per questo si costruisce una **catena di ambienti**, ciascuno con un livello crescente di somiglianza alla produzione e un livello crescente di cautela richiesta, esattamente come le prove di uno spettacolo prima del debutto.

15.2 Ambiente di Sviluppo (Dev)

L'ambiente di **Sviluppo** è dove nasce il codice. Ogni sviluppatore scrive e prova le proprie modifiche, quasi sempre in due modalità complementari:

- **In locale**, sul proprio computer: l'applicazione gira dentro il suo portatile, con dati finti creati al momento, senza toccare nulla di condiviso con il resto del team.
- **In un ambiente Dev condiviso**, in cloud: una versione dell'applicazione sempre aggiornata con l'ultimo codice del branch principale, usata per verificare che le proprie modifiche funzionino anche insieme a quelle degli altri, prima ancora di arrivare al Test/QA.

Chi ci accede: gli sviluppatori, in modo ampio e informale.

Cosa succede qui: è normale — anzi previsto — che qualcosa non funzioni. È lavoro in corso, come un' scena provata da capo più volte prima di essere pronta. Non c'è alcun tipo di garanzia di stabilità: l'ambiente Dev può “rompersi” più volte al giorno senza che questo sia un problema per nessuno fuori dal team di sviluppo.

Esempio pratico: Marco deve correggere un bug del carrello di ShopFacile: quando un cliente applica un codice sconto, il totale mostrato a video non tiene conto dell'IVA ricalcolata, e risulta più basso di quanto dovrebbe. Scrive il codice della correzione e lo prova prima **in locale**, sul suo computer, con dati completamente inventati (un carrello finto con prodotti fittizi e un codice sconto di prova). Quando è abbastanza sicuro del risultato, pusha il proprio branch: la stessa modifica viene automaticamente distribuita anche nell'**ambiente Dev condiviso** in cloud, dove i colleghi possono verificare che funzioni insieme al resto dell'applicazione, ancora prima che Marco apra la Merge Request.

15.3 Ambiente di Test / QA

Una volta che la correzione di Marco supera i controlli automatici della pipeline (build, unit test, analisi di qualità — visti nella sezione CI/CD), viene **promossa** nell'ambiente di **Test/QA** (Quality Assurance, ovvero “garanzia di qualità”).

Qui il software viene verificato in modo più realistico:

- test automatici più ampi (test di integrazione, test end-to-end);
- verifiche manuali da parte del team di test, che prova il software come farebbe un utente, cercando deliberatamente di romperlo;

- prime verifiche di casi limite: cosa succede se inserisco un valore strano, se clicco due volte sullo stesso bottone, se la connessione si interrompe a metà di un'operazione.

Chi ci accede: il team di test/QA, gli sviluppatori per investigare eventuali segnalazioni, occasionalmente il Project Manager per una demo interna.

Cosa succede qui: si scopre e si corregge la maggior parte dei problemi funzionali, **prima** che possano avvicinarsi anche solo lontanamente agli utenti reali. Se qualcosa non funziona, il codice **non viene promosso** oltre: torna in Sviluppo per essere corretto.

Esempio pratico: la correzione di Marco al totale del carrello supera la build e i test automatici della pipeline, e viene promossa in Test/QA. Qui Giulia prova deliberatamente a “romperla”: applica due codici sconto diversi allo stesso carrello, prova uno sconto su un prodotto già scontato di suo, svuota il carrello a metà del ricalcolo. Scopre che, combinando due sconti insieme, il totale torna a essere sbagliato in un modo nuovo: apre una segnalazione, e il codice **torna** in Sviluppo per la correzione, prima di poter proseguire oltre.

15.4 Ambiente di Staging (Pre-produzione)

Superato anche il secondo giro di test, la correzione del carrello è pronta per un controllo ancora più realistico, in un ambiente che assomiglia quanto più possibile alla produzione vera. Lo **Staging** (a volte chiamato Pre-produzione) è l'ambiente pensato per essere **quanto più fedele possibile alla produzione**: stessa configurazione di infrastruttura, stesso tipo di database, volumi di dati paragonabili, stesse integrazioni con sistemi esterni (per quanto possibile in modo sicuro). È la “prova costume”: tutto è allestito come sarà la sera del debutto, ma senza pubblico pagante in sala.

Qui si eseguono gli ultimi controlli prima del rilascio reale:

- test di accettazione finali, spesso con il coinvolgimento di chi ha richiesto la funzionalità (il Project Manager, un referente del progetto);
- a volte test di performance/carico, per verificare che l'applicazione regga un numero di utenti realistico;
- verifica che il processo di deploy stesso funzioni senza sorprese, proprio perché l'ambiente è configurato come la produzione.

Chi ci accede: team di test/QA, sviluppatori senior o Tech Lead, Project Manager per l'ultima validazione, in modo più controllato rispetto al Test/QA.

Cosa succede qui: se qualcosa emerge in Staging che non era emerso prima, è un segnale importante — spesso significa che Test/QA non era sufficientemente simile alla produzione, ed è un'occasione per migliorare quell'ambiente. Non tutti i progetti hanno uno Staging separato dal Test/QA: nei progetti più piccoli le due cose a volte coincidono, ma nei progetti maturi restano due tappe distinte, con scopi diversi.

Esempio pratico: la correzione al totale del carrello, dopo aver superato il Test/QA, viene promossa in Staging. Qui il volume di dati è paragonabile a quello reale (migliaia di ordini storici anonimizzati, non le tre righe di prova usate in Test), e l'infrastruttura è configurata esattamente come in produzione. Sara, come Product Owner che aveva segnalato il problema arrivato dai clienti, prova il flusso completo end-to-end — carrello, sconto, checkout — prima di dare l'ultima validazione. Ahmed segue il test insieme a lei: è il primo bug che vede attraversare tutte le tappe fino a qui, ed è un buon modo per imparare cosa cambia da un ambiente all'altro. Solo a questo punto il rilascio in produzione può essere pianificato.

15.5 Ambiente di Produzione

Dopo Dev, Test/QA e Staging, resta un'ultima tappa, quella in cui il bug del carrello incontra finalmente i clienti veri di ShopFacile. La **Produzione** è l'ambiente reale, quello che usano davvero gli utenti finali — che siano dipendenti del cliente, clienti finali di un servizio, o cittadini che usano un servizio pubblico. È la “prima” davanti al pubblico pagante.

Qui la parola d'ordine è **massima cautela**:

- ogni deploy segue i quality gate e, tipicamente, un'approvazione manuale (vista nella sezione CI/CD);
- ogni modifica è monitorata attentamente subito dopo il rilascio (metriche di errore, tempi di risposta);
- esiste sempre un piano di rollback pronto, nel caso qualcosa vada storto nonostante tutti i controlli precedenti.

Chi ci accede: un numero molto ristretto di persone, quasi sempre solo tramite strumenti e processi automatizzati (la pipeline stessa), raramente con accesso manuale diretto — approfondiremo il perché nel paragrafo 15.10.

Cosa succede qui: ogni errore ha un impatto reale su persone vere. Non è più “lavoro in corso”: è il prodotto finito, in uso.

Esempio pratico — il percorso completo di una modifica: seguiamo l'intero viaggio della correzione al totale del carrello, dal commit alla produzione:

1. **Dev:** Marco corregge il calcolo dell'IVA con lo sconto applicato e lo prova in locale, poi pusha il branch: la pipeline fa build e unit test con esito positivo.
2. **Test/QA:** la modifica viene promossa automaticamente. Giulia riprova esattamente lo scenario dei due sconti combinati, più altri casi limite: questa volta tutto funziona come previsto. Promozione approvata.
3. **Staging:** la stessa modifica (lo stesso identico pacchetto, non una ricompilazione) viene distribuita in un ambiente identico alla produzione. Sara verifica il flusso end-to-end con dati realistici e dà l'ultima validazione.

4. **Produzione:** dopo un'approvazione manuale del Release Manager, la pipeline distribuisce la modifica ai clienti reali, fuori dall'orario di punta. Nei minuti successivi, il team monitora le metriche di errore: se qualcosa andasse storto, è pronto un piano di rollback.

Da notare: è **lo stesso pacchetto** che attraversa tutte le tappe, non una versione “ricostruita” ambiente per ambiente — esattamente il principio “build once, deploy many” che vedrai nel paragrafo 15.7.

15.6 I quattro ambienti, fase per fase

Il viaggio della correzione al carrello, visto passo per passo nel box precedente, si può anche osservare “da fermo”, guardando in un unico schema chi lavora in ciascuna tappa e cosa cambia tra l'una e l'altra. Il diagramma seguente riprende e amplia quello già visto nella sezione CI/CD, aggiungendo per ciascun ambiente **chi vi accede** e **cosa succede concretamente**.

Diagramma 2

Diagramma 2

15.7 “Funziona sul mio computer”: perché gli ambienti devono somigliarsi

Il percorso della correzione al carrello ha funzionato senza sorprese proprio perché Dev, Test/QA, Staging e Produzione si somigliano il più possibile. Vediamo perché questa somiglianza non è un dettaglio, ma il punto centrale di tutta la catena. Hai forse già sentito la battuta “funziona sul mio computer” — è la scusa (semi-seria) di uno sviluppatore quando qualcosa non funziona in un altro ambiente. Il problema reale dietro la battuta è che **se gli ambienti sono troppo diversi tra loro**, un test superato in uno di essi non garantisce nulla sugli altri:

- una versione diversa di una libreria installata;
- un sistema operativo diverso;
- variabili di configurazione mancanti o diverse;
- un database con una struttura leggermente diversa.

La soluzione moderna è duplice: da un lato l'uso di **container** (visti nella sezione DevOps/CI-CD, es. immagini Docker), che pacchettizzano l'applicazione con tutto ciò che le serve per girare identica ovunque; dall'altro, il principio già incontrato nella sezione CI/CD di “**build once, deploy many**” — si compila e pacchettizza il software **una sola volta**, e lo stesso identico artifact viene promosso, senza modifiche, da un ambiente all'altro. In questo modo, se qualcosa funziona in Staging, è ragionevole aspettarsi che funzioni anche in Produzione, perché tecnicamente **è lo stesso identico pacchetto**, non una ricostruzione “simile”.

Più gli ambienti sono simili, meno sorprese si trovano man mano che si avanza nella catena — ed è per questo che lo Staging, in particolare, è tenuto quanto più possibile identico alla Produzione.

Esempio pratico: in ShopFacile, ogni volta che una modifica passa la fase di build in pipeline, viene generato un **unico** artifact (es. un'immagine container con un numero di versione, tipo `shopfacile-carrello:1.42.0`). Quello stesso artifact — non uno ricostruito da capo — viene distribuito in Test/QA, poi in Staging, poi in Produzione. Se in Staging l'immagine `1.42.0` funziona correttamente, il team sa che in Produzione girerà **esattamente lo stesso codice, con le stesse dipendenze**: non resta da chiedersi “avranno usato la stessa versione di libreria X?”, perché la domanda non ha nemmeno senso — è letteralmente lo stesso file.

15.8 Dati di test vs dati reali

Sapere che l'artifact è sempre lo stesso risolve il problema del codice, ma resta un'altra domanda: se il codice è identico, i dati che usa devono esserlo altrettanto? Qui la risposta cambia radicalmente. Un punto spesso sottovalutato da chi arriva da fuori dal settore: **gli ambienti di Sviluppo e Test/QA non usano mai dati reali degli utenti**.

Perché? Perché i dati reali possono contenere informazioni sensibili: nomi, indirizzi, numeri di documento, dati di pagamento, informazioni sanitarie. Usarli fuori da un ambiente protetto come la Produzione significherebbe esporli a un numero molto più ampio di persone (tutto il team di sviluppo e test) e a controlli di sicurezza tipicamente meno stringenti — un rischio enorme di violazione della privacy, oltre che, in molti contesti, una vera e propria violazione di legge.

Per questo si usano invece:

- **dati sintetici:** generati appositamente, plausibili ma inventati (nomi, indirizzi ed email finti ma verosimili);
- **dati anonimizzati o pseudonimizzati:** partono da dati reali, ma privati di ogni elemento che permetterebbe di risalire a una persona specifica.

Questo tema è collegato a doppio filo con quanto visto nella sezione [14. Sicurezza](#): la protezione dei dati personali non è solo una questione di “chi può leggerli in produzione”, ma anche di “dove questi dati non devono mai nemmeno arrivare”.

15.9 Configurazioni per ambiente

Dati diversi non bastano da soli: se lo stesso artifact deve girare su dati e infrastrutture diverse in ogni ambiente, deve anche sapere in quale ambiente si trova in un dato momento. Lo stesso identico artifact (lo stesso pacchetto di codice) deve comportarsi in modo leggermente diverso a seconda dell'ambiente in cui gira: in Test deve collegarsi al database di test, in Produzione al database di produzione; in Test può usare un servizio di invio email “finto” che non spedisce nulla, in Produzione deve usare il servizio vero.

La soluzione **non è modificare il codice** per ogni ambiente (violerebbe il principio “build once, deploy many” visto sopra), ma usare delle **variabili di configurazione** (o variabili d'ambiente): valori esterni al codice, forniti al momento dell'avvio, che dicono all'applicazione “in questo momento stai girando in Test, quindi usa questo indirizzo di database” oppure “stai girando in Produzione, quindi usa quest'altro”.

Diagramma 3

Diagramma 3

Le credenziali (password, chiavi di accesso) che fanno parte di queste configurazioni non vengono mai scritte nel codice: sono gestite tramite strumenti dedicati (es. un “vault” di segreti), collegandosi ancora al tema della sicurezza approfondito nella sezione dedicata.

Esempio pratico: ShopFacile deve inviare un'email di conferma quando un ordine viene completato. In Test/QA, la variabile di configurazione `EMAIL_SERVICE_URL` punta a un servizio “finto” che si limita a registrare in un log “avrei inviato questa email”, senza spedire nulla per davvero — utile per verificare che il codice tenti l'invio, senza intasare le caselle di posta di nessuno con conferme d'ordine finte. In Produzione, la stessa identica variabile punta invece al servizio email reale, quello che avvisa davvero il cliente. Il codice che gestisce l'invio non cambia di una riga tra i due ambienti: cambia solo il valore di quella variabile.

15.10 Chi ha accesso a quali ambienti

Hai visto chi lavora in ciascun ambiente lungo il percorso del bug del carrello: Marco in Dev, Giulia in Test/QA, Sara e Ahmed in Staging, un numero ristretto di persone in Produzione. Questo schema non è casuale, ma segue una regola precisa. L'accesso agli ambienti segue un principio semplice: **più un ambiente è vicino agli utenti reali, più l'accesso è restrittivo**. Non è burocrazia fine a se stessa: è la stessa logica per cui non chiunque può entrare backstage la sera della prima, mentre durante le prove chiunque del cast può girare liberamente per il teatro.

- **Sviluppo:** accesso ampio, quasi tutto il team tecnico.
- **Test/QA:** accesso al team di test/QA e agli sviluppatori, più limitato di Dev.
- **Staging:** accesso più ristretto, tipicamente ruoli senior (Tech Lead, QA lead) e il Project Manager per le validazioni finali.
- **Produzione:** accesso **molto** ristretto — spesso solo tramite la pipeline automatizzata stessa, con un numero minimo di persone autorizzate ad accedere manualmente in casi eccezionali (es. un'emergenza), e ogni accesso manuale tipicamente registrato e tracciato.

Confronto tra i quattro ambienti

Ambiente	Scopo	Chi lo usa	Tipo di dati	Stabilità richiesta	Frequenza di aggiornamento
Sviluppo (Dev)	Scrivere e provare codice nuovo	Sviluppatori	Dati finti, creati al momento	Bassa: è normale che si rompa	Molto alta, più volte al giorno
Test / QA	Verificare funzionalmente il software	Team di test/ QA, sviluppatori	Dati sintetici o anonimizzati, simili ai reali	Media: deve funzionare per essere utile	Alta, ad ogni promozione dalla CI
Staging (Pre-produzione)	Ultimo controllo prima del rilascio reale	QA senior, Tech Lead, Project Manager	Dati anonimizzati, volumi simili al reale	Alta: quasi come la produzione	Media, prima di ogni rilascio
Produzione	Uso reale da parte degli utenti finali	Utenti finali; accesso tecnico molto limitato	Dati reali e sensibili	Massima: ogni errore ha impatto reale	Bassa/ controllata, solo rilasci approvati

15.11 Riepilogo

Il bug del carrello di ShopFacile ci ha accompagnato da Marco in Dev fino al rilascio in Produzione: riassumiamo i concetti principali visti lungo il percorso. In questa sezione hai approfondito il concetto di ambiente già incontrato nella sezione CI/CD:

- non si testa mai direttamente in produzione: si passa attraverso Sviluppo, Test/QA e Staging, come le prove prima di uno spettacolo dal vivo;
- ogni ambiente ha un pubblico e uno scopo diversi, con un livello di cautela e stabilità crescente man mano che ci si avvicina alla produzione;
- gli ambienti devono somigliarsi quanto più possibile tra loro, per evitare il problema del “funziona sul mio computer”;
- non si usano mai dati reali e sensibili fuori dall’ambiente di produzione, per motivi di sicurezza e privacy;
- lo stesso pacchetto di codice cambia comportamento tra ambienti grazie a **variabili di configurazione**, non a modifiche del codice stesso;
- l’accesso agli ambienti è tanto più restrittivo quanto più ci si avvicina alla produzione.

Esercizi pratici

1. **Mappa gli ambienti del progetto.** Con l'aiuto di un collega developer o del Tech Lead, scopri quanti e quali ambienti esistono davvero nel progetto (Dev, Test, Staging, Prod, o eventuali nomi diversi usati internamente) e scrivilo in ordine, con una riga di descrizione per ciascuno. **Come verificare:** sai disegnare, senza guardare gli appunti, la sequenza di ambienti del progetto con una freccia di "promozione" tra l'uno e l'altro, proprio come nei diagrammi di questa sezione.
2. **Segui una modifica reale attraverso gli ambienti.** Chiedi a uno sviluppatore di mostrarti (anche solo a schermo, senza dover intervenire) l'ultima modifica che ha promosso da Dev fino a Produzione: in quale ambiente si trova adesso, cosa ha superato per arrivare lì, cosa manca prima della produzione (se non è già arrivata). **Come verificare:** sai raccontare a voce, in meno di due minuti, il percorso di quella specifica modifica citando ogni ambiente attraversato e cosa è stato verificato in ciascuno.
3. **Trova le variabili di configurazione per ambiente.** Chiedi a un developer di farti vedere (anche solo a schermo) dove sono definite le variabili di configurazione che cambiano tra un ambiente e l'altro nel progetto (es. l'indirizzo del database, l'URL di un servizio esterno). Non serve capire ogni dettaglio tecnico: l'obiettivo è vedere con i tuoi occhi che esistono e dove vivono. **Come verificare:** sai indicare almeno due valori di configurazione che cambiano tra Test/QA e Produzione nel progetto, senza però conoscere (né dover conoscere) le password o i segreti veri.
4. **Distingui i dati usati nei diversi ambienti.** Chiedi al team se e come vengono generati o anonimizzati i dati usati in Test/QA e in Staging nel progetto (dati sintetici generati da uno script? dati reali anonimizzati? un mix?). **Come verificare:** sai spiegare a un collega non tecnico perché non si può semplicemente "copiare" il database di produzione dentro l'ambiente di Test così com'è, citando almeno un rischio concreto.
5. **Osserva chi può fare cosa, dove.** Confronta la tabella di accesso agli ambienti vista nel paragrafo 15.10 con la situazione reale del progetto: chi ha accesso diretto a Dev? Chi può promuovere codice in Staging? Chi (o cosa, es. solo la pipeline) può rilasciare in Produzione? **Come verificare:** sai indicare, per ciascuno dei quattro ambienti, almeno un ruolo del progetto che vi ha accesso e uno che invece **non** ce l'ha, spiegando perché.
6. **Simula un rollback.** Chiedi a un collega operations o Tech Lead di raccontarti (anche solo a parole, con un esempio ipotetico o reale se ce n'è uno) cosa succederebbe se, subito dopo un rilascio in Produzione, le metriche di errore iniziassero a salire in modo anomalo: chi se ne accorge, chi decide il rollback, quanto tempo richiede tipicamente tornare alla versione precedente. **Come verificare:** sai descrivere, passo per passo, la sequenza di azioni tra "qualcosa va storto in produzione" e "il sistema torna stabile", indicando almeno un ruolo coinvolto in ogni passo.

Collegamenti

- [11. CI/CD](#) — come la pipeline promuove il codice tra questi stessi ambienti

- [16. Glossario](#) — definizioni rapide di Dev, Staging, Produzione e termini correlati

Risorse

- [Microsoft Learn](#) — Panoramica sugli ambienti di rilascio
- [Atlassian](#) — What is a staging environment
- [Martin Fowler](#) — Bliki: DeploymentEnvironment
- [OWASP](#) — Data Protection e ambienti non di produzione
- [GitLab Docs](#) — Manage deployment environments
- [12 Factor App](#) — Config

16. Glossario

Questo glossario raccoglie in un unico posto tutti i termini incontrati nel corso, dai concetti base di informatica fino alla sicurezza e agli ambienti di produzione. Le voci sono in **ordine alfabetico rigoroso** (gli acronimi come CI, CPU o KPI sono ordinati per le lettere dell'acronimo stesso, non per il nome per esteso), così puoi usarlo come un vero dizionario: se leggi un termine che non ricordi mentre studi una sezione, torna qui, cercalo e poi eventualmente segui il rimando “→ approfondito nella sezione X” per tornare al capitolo che lo spiega con esempi e contesto completo.

Non serve leggerlo tutto d'un fiato: è pensato per essere consultato, non studiato in ordine. Buona consultazione.

Mapa dei concetti: come si collegano

Questo glossario ha due viste, da usare per scopi diversi: la mappa qui sotto ti mostra **come i termini si concatenano tra loro**, raggruppati per famiglia tematica, mentre l'elenco alfabetico che segue resta il dizionario da consultare voce per voce quando ti serve una definizione precisa. Le definizioni non si ripetono qui: sotto trovi solo i **collegamenti**.

Le basi tecniche

Tutto parte dal **Computer**: un insieme di componenti hardware in cui la **CPU** esegue le istruzioni usando la **RAM** come memoria di lavoro temporanea e il **Disco** come memoria permanente, organizzata in cartelle e file grazie al **File System**. Il **Sistema Operativo** fa da intermediario tra questo hardware e i programmi, e ogni programma in esecuzione diventa un **Processo**, a sua volta eventualmente suddiviso in più **Thread** che lavorano in parallelo. Su questa base tecnica gira tutto il resto: dal browser di un cliente di ShopFacile al server che elabora i suoi ordini.

Come le applicazioni comunicano

Il modello **Client-Server** descrive chi chiede (client) e chi risponde (server); per farlo, i due capi si scambiano dati attraverso una **Rete**, seguendo le regole del **TCP/IP** e, nello specifico del web, del protocollo **HTTP** (o della sua versione cifrata **HTTPS**). Prima ancora di parlarsi, il client deve sapere *dove* si trova il server: è compito del **DNS** tradurre un nome leggibile in un indirizzo. Una volta stabilita la comunicazione, i programmi si scambiano informazioni tramite una **API**, spesso progettata secondo lo stile **REST** e con i dati serializzati in **JSON** (o, più raramente oggi, **XML**).

Come si struttura un'applicazione

Un'applicazione divide le proprie responsabilità tra **Frontend** (quello che l'utente vede) e **Backend** (la logica e i dati che l'utente non vede); quest'ultimo può essere organizzato come un unico **Monolite** oppure spezzato in **Microservizi** indipendenti che comunicano tra loro, a volte tramite una **Message Queue** invece che con chiamate dirette. Un'alternativa a gestire server propri è il modello **Serverless**, dove il codice viene eseguito solo su richiesta.

Dove vivono i dati

I dati di ShopFacile si salvano in un **Database relazionale**, interrogabile con **SQL**, oppure in un **Database NoSQL** quando la struttura dei dati è più flessibile o i volumi molto grandi: la scelta tra i due dipende dalla forma dei dati (es. il catalogo prodotti rispetto ai log delle sessioni carrello).

Come si scrive e si versiona il codice

Tutto il codice vive in un **Repository**, la cui storia è fatta di **Commit** successivi, ciascuno registrato su un **Branch** dedicato per non toccare il codice principale durante lo sviluppo. Quando una modifica è pronta, si apre una **Merge Request**: qui il team fa **Code Review** prima di eseguire il **Merge** che la unisce definitivamente. Un punto del repository può poi essere marcato con un **Tag**, tipicamente per identificare una **Release**, seguendo una convenzione di **Versioning** come il **Semantic Versioning**. Il modo in cui i branch vengono organizzati nel tempo segue un modello come il **Git Flow** oppure, all'opposto, il **Trunk Based Development**; un problema scoperto lungo il percorso viene tracciato come **Issue**.

Come nasce e cresce un prodotto

Tutto comincia dai **Requisiti (funzionali e non funzionali)** che descrivono cosa deve fare il software: da lì nasce una **User Story** che lo racconta dal punto di vista dell'utente, oppure, se il lavoro è troppo grande per uno sprint, un **Epic** che raggruppa più User Story. Ogni User Story diventa una **Feature** una volta realizzata, oppure genera un **Bug** se qualcosa non funziona come previsto. Tutto questo percorso, dalla richiesta iniziale alla manutenzione, è descritto dal **Ciclo di vita del software**.

Come si organizza il lavoro

Il **Manifesto Agile** e il **Mindset Agile** superano il **Waterfall** tradizionale con due framework operativi concreti. Nello Scrum, il **Product Owner** cura il **Product Backlog**, da cui il team seleziona in **Sprint Planning** le voci dello **Sprint Backlog** per ogni **Sprint**, sincronizzandosi nel **Daily Scrum** e chiudendo con la **Sprint Review** e la **Sprint Retrospective**; il risultato è un **Increment** che rispetta la **Definition of Done (DoD)** (ed è entrato in sprint solo se soddisfaceva la **Definition of Ready (DoR)**), mentre lo **Scrum Master** facilita tutto questo. Nel Kanban lo stesso lavoro scorre su una **Board Kanban** con il **WIP (Work In Progress)** limitato per colonna, misurato in **Lead Time** e **Cycle Time** (lo **Scrumban** mescola i due mondi); gli **Story Point** e la **Velocity** quantificano la capacità del team di Luca, mentre **KPI**, **Milestone**, **RACI**, **RAID Log**, il **Rischio** e gli **Stakeholder**, insieme a **Burndown Chart** e **Burnup Chart**, tengono sotto controllo l'andamento del progetto nel suo complesso.

Come il codice arriva agli utenti

Quando una Merge Request supera la Code Review ed entra nel branch principale, un **Trigger** avvia la **Pipeline**: build, test e poi il **Quality Gate**, il controllo automatico che blocca l'avanzamento se la qualità non basta. Se tutto va bene, la pipeline produce un **Artifact (build)** che viene promosso dall'**Ambiente di Sviluppo** all'**Ambiente di Test/QA**, poi allo **Staging** e infine alla **Produzione** tramite il **Deploy/Rilascio**; le pratiche di **CI (Continuous Integration)** e **Continuous Delivery/Continuous Deployment** sono ciò che rende questo percorso frequente e automatico invece che manuale e raro. Se qualcosa va storto dopo il rilascio, il **Rollback** riporta rapidamente alla versione precedente.

Dove gira il software

Un Artifact, per essere eseguito ovunque allo stesso modo, viene spesso pacchettizzato in un **Container** tramite **Docker**, e quando i container sono troppi da gestire a mano entra in gioco **Kubernetes**, che li orchestra automaticamente. Un'alternativa più tradizionale è la **Virtual Machine (VM)**, che simula un intero computer; la capacità di questi ambienti di gestire più carico segue i due modi descritti da **Scalabilità verticale e orizzontale**. Tutto questo gira su infrastrutture cloud offerte secondo tre livelli di responsabilità crescente per il fornitore — **IaaS**, **PaaS** e **SaaS** — quasi sempre pagate a consumo con il modello **Pay-as-you-go**.

Come si tiene tutto in piedi e sicuro

Una volta in produzione, il **Monitoring** osserva lo stato del sistema e il **Logging** ne registra gli eventi; l'**Observability** va oltre entrambi, aiutando a capire *perché* qualcosa non funziona, non solo *cosa*. Sul fronte sicurezza, l'**Autenticazione** verifica chi sei e l'**Autorizzazione** stabilisce cosa puoi fare, spesso rafforzate da **MFA (autenticazione a più fattori)**; le vulnerabilità più comuni sono catalogate da **OWASP**, mentre il **GDPR** impone regole precise sul trattamento dei dati personali degli utenti di ShopFacile. Il **DevSecOps** integra questi controlli di sicurezza direttamente nella pipeline, invece di lasciarli come verifica finale isolata.

La cultura che tiene insieme tutto

Il **DevOps** è la cornice culturale che tiene insieme tutti i fili visti sopra: il **CALMS** ne riassume i cinque pilastri, mentre la **Cultura DevOps** descrive concretamente cosa cambia nel modo di lavorare quotidiano di un team come quello di ShopFacile, fino a pratiche come l'**IaC (Infrastructure as Code)** che trattano l'infrastruttura come se fosse codice, versionabile e revisionabile come qualsiasi Merge Request.

Azure DevOps: stessi concetti, altri nomi

Se lavori con GitLab, **Merge Request**, **Issue** e **Pipeline** sono i nomi che conosci; su Azure DevOps lo stesso lavoro passa attraverso **Repos (Azure DevOps)** (dove la Merge Request si chiama "Pull Request"), **Boards (Azure DevOps)** (dove ogni **Work Item** — una **User Story**, un **Bug** o un **Epic** — avanza su una board) e **Pipelines (Azure DevOps)**. Gli **Artifacts (Azure DevOps)** sono il servizio che conserva i pacchetti software, da non confondere con l'**Artifact (build)** generico prodotto da qualsiasi pipeline; i **Test Plans (Azure DevOps)** infine gestiscono i test manuali ed esplorativi. Conoscere questa corrispondenza ti evita di pensare che siano concetti diversi solo perché cambia lo strumento.

Il percorso end-to-end in un unico schema

Diagramma 1

Diagramma 1

Ambiente di Sviluppo

L'ambiente (development environment) in cui gli sviluppatori scrivono e provano il codice per la prima volta, di solito sul proprio computer o in uno spazio dedicato non visibile agli utenti finali. È il primo “gradino” prima che il codice arrivi al Test/QA, allo Staging e infine alla Produzione. → approfondito nella sezione 15 (Ambienti di sviluppo).

Ambiente di Test/QA

Un ambiente separato da quello di sviluppo, usato per verificare (Quality Assurance) che una funzionalità funzioni correttamente prima di mostrarla a chiunque altro. Qui si eseguono i test manuali e automatici in condizioni più simili a quelle reali. → approfondito nella sezione 15 (Ambienti di sviluppo). Si collega a: **Ambiente di Sviluppo** (che lo precede) e **Staging** (che ne consegue, prima della Produzione).

API

Sigla di Application Programming Interface: è un insieme di regole che permette a due programmi di “parlarsi” e scambiarsi informazioni, senza che uno debba conoscere i dettagli interni dell'altro. Pensala come un menu di un ristorante: tu scegli una voce (una richiesta) e la cucina (il sistema) ti restituisce il piatto (la risposta). → approfondito nella sezione 2 (Fondamenti di informatica).

Artifact (build)

Il “prodotto finito” generato da una pipeline dopo aver compilato e testato il codice: ad esempio un file eseguibile, un pacchetto installabile o un'immagine Docker pronta per essere distribuita. È diverso dagli Artifacts di Azure DevOps, che sono il servizio usato per conservarlo. → approfondito nella sezione 11 (CI/CD). Si collega a: **Pipeline** (che lo produce) e **Deploy/Rilascio** (che lo installa, identico, in ogni ambiente).

Artifacts (Azure DevOps)

Il servizio di Azure DevOps dedicato a creare, conservare e condividere i pacchetti software (le librerie o i componenti riutilizzabili) usati dai progetti del team. → approfondito nella sezione 10 (Azure DevOps).

Autenticazione

Il processo con cui un sistema verifica che tu sia davvero chi dici di essere, tipicamente chiedendo username e password (authentication). È il primo passo della sicurezza: prima si verifica l'identità, poi si decide cosa quella identità può fare. → approfondito nella sezione 14 (Sicurezza).

Autorizzazione

Il processo che stabilisce, dopo che sei stato riconosciuto (autenticazione), a quali risorse puoi accedere e quali azioni puoi compiere (authorization). Esempio: essere autenticati come dipendenti non significa automaticamente poter modificare i dati contabili dell'azienda. → approfondito nella sezione 14 (Sicurezza).

Backend

La parte “dietro le quinte” di un’applicazione: server, database e logica di business che l’utente non vede direttamente, ma che elabora le richieste e restituisce i risultati al Frontend. → approfondito nella sezione 12 (Architetture software).

Board Kanban

Una lavagna (fisica o digitale) divisa in colonne — tipicamente “Da fare”, “In corso”, “Fatto” — che rende visibile lo stato di avanzamento di ogni attività del team con dei cartellini (le card). → approfondito nella sezione 7 (Kanban).

Boards (Azure DevOps)

Il servizio di Azure DevOps che offre board Kanban, backlog e strumenti per pianificare e monitorare il lavoro del team, gestendo gli Work Item. → approfondito nella sezione 10 (Azure DevOps).

Branch

Una “linea di sviluppo” separata all’interno di un repository Git, che permette di lavorare su una modifica senza toccare il codice principale finché non è pronta. → approfondito nella sezione 4 (Git e GitLab). Si collega a: **Repository** (che lo contiene) e **Merge Request** (che ne propone l’unione al codice principale).

Branching

La pratica di creare e gestire più Branch per isolare il lavoro su funzionalità diverse, permettendo a più persone di lavorare in parallelo senza intralciarsi. → approfondito nella sezione 3 (Come nasce un software).

Bug

Un errore nel software che produce un comportamento diverso da quello previsto: dal termine inglese per “insetto”, nato da un aneddoto storico legato a un vero insetto trovato in un computer degli anni ’40. → approfondito nella sezione 3 (Come nasce un software).

Burndown Chart

Un grafico che mostra quanto lavoro **resta** da fare in uno Sprint (o in un progetto) man mano che passano i giorni: la linea idealmente “scende” verso lo zero. → approfondito nella sezione 8 (Project Management).

Burnup Chart

Un grafico simile al Burndown Chart, ma che mostra il lavoro **già completato** che “sale” verso il totale previsto, rendendo più facile notare se l’ambito del progetto (lo scope) cresce nel tempo. → approfondito nella sezione 8 (Project Management).

CALMS

Un acronimo (Culture, Automation, Lean, Measurement, Sharing) che riassume i cinque pilastri della cultura DevOps: cultura collaborativa, automazione, approccio lean, misurazione dei risultati e condivisione della conoscenza. → approfondito nella sezione 9 (DevOps).

CI (Continuous Integration)

La pratica di integrare frequentemente (più volte al giorno) il proprio codice con quello del resto del team, verificando automaticamente con dei test che tutto continui a funzionare insieme. → approfondito nella sezione 9 (DevOps) e nella sezione 11 (CI/CD). Si collega a: **Pipeline** (il meccanismo che la rende concreta) e **Quality Gate** (il controllo che ne verifica l'esito).

Ciclo di vita del software

L'insieme delle fasi che un software attraversa dalla sua nascita alla sua "pensione": analisi dei requisiti, progettazione, sviluppo, test, rilascio e manutenzione (software development lifecycle). → approfondito nella sezione 3 (Come nasce un software).

Client-Server

Un modello architetturale in cui un programma "client" (ad esempio il tuo browser) richiede informazioni o servizi a un programma "server", che risponde elaborando la richiesta. → approfondito nella sezione 12 (Architetture software).

Code Review

La pratica di far leggere e commentare il proprio codice a un collega prima di unirlo al progetto principale, per individuare errori, migliorare la qualità e condividere conoscenza nel team. → approfondito nella sezione 3 (Come nasce un software). Si collega a: **Merge Request** (che la richiede) e **Merge** (l'operazione che la conclude, una volta approvata).

Commit

Uno "scatto fotografico" delle modifiche fatte al codice in un preciso momento, salvato nella storia del repository con un messaggio che ne descrive il contenuto. → approfondito nella sezione 4 (Git e GitLab). Si collega a: **Branch** (su cui viene registrato) e **Merge Request** (che raccoglie una serie di commit da integrare).

Computer

Una macchina capace di eseguire istruzioni (programmi) per elaborare dati: è composta da componenti hardware (come CPU e RAM) che collaborano seguendo le indicazioni del software. → approfondito nella sezione 2 (Fondamenti di informatica).

Container

Un “pacchetto” leggero e isolato che contiene un’applicazione insieme a tutto ciò che le serve per funzionare (librerie, configurazioni), così da poterla eseguire in modo identico su qualsiasi computer. Docker è la tecnologia più usata per crearli. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Docker** (lo strumento più diffuso per crearli) e **Kubernetes** (che li orchestra su larga scala).

Continuous Delivery

La pratica che assicura che il software, dopo aver superato tutti i test automatici, sia sempre pronto per essere rilasciato in produzione in qualsiasi momento, anche se il rilascio finale resta una decisione manuale. → approfondito nella sezione 9 (DevOps). Si collega a: **CI (Continuous Integration)** (che la precede nel percorso verso il rilascio) e **Deploy/Rilascio** (l’operazione finale, qui ancora decisa manualmente).

Continuous Deployment

Un’evoluzione della Continuous Delivery in cui ogni modifica che supera i test viene rilasciata automaticamente in produzione, senza intervento manuale. → approfondito nella sezione 9 (DevOps). Si collega a: **Continuous Delivery** (di cui è l’evoluzione automatica) e **Deploy/Rilascio** (che qui avviene senza intervento umano).

CPU

Sigla di Central Processing Unit, il “cervello” del computer: il componente che esegue materialmente le istruzioni dei programmi, facendo calcoli a velocità elevatissima. → approfondito nella sezione 2 (Fondamenti di informatica).

Cultura DevOps

Un modo di lavorare che abbatte le barriere tra chi sviluppa il software (Dev) e chi lo gestisce in produzione (Ops), promuovendo collaborazione, responsabilità condivisa e automazione. → approfondito nella sezione 9 (DevOps).

Cycle Time

Il tempo che intercorre dal momento in cui il team **inizia effettivamente a lavorare** su un’attività al momento in cui la completa. È una misura più “interna” del Lead Time. → approfondito nella sezione 7 (Kanban).

Daily Scrum

Un breve incontro quotidiano (di solito 15 minuti) in cui il team Scrum sincronizza il lavoro, condivide progressi e segnala eventuali ostacoli. → approfondito nella sezione 6 (Scrum).

Database NoSQL

Un database che non organizza i dati in tabelle rigide come quelli relazionali, ma in formati più flessibili (documenti, chiavi-valore, grafi), utile quando i dati cambiano forma spesso o servono grandi volumi. → approfondito nella sezione 2 (Fondamenti di informatica).

Database relazionale

Un database che organizza i dati in tabelle collegate tra loro tramite relazioni, un po' come fogli Excel che si richiamano a vicenda; si interroga con il linguaggio SQL. → approfondito nella sezione 2 (Fondamenti di informatica).

Definition of Done (DoD)

L'elenco condiviso di criteri che un'attività deve soddisfare per essere considerata davvero "completata" (ad esempio: codice scritto, testato, revisionato e documentato), evitando ambiguità nel team. → approfondito nella sezione 6 (Scrum).

Definition of Ready (DoR)

L'elenco di criteri che una User Story deve soddisfare prima di poter essere accettata in uno Sprint (ad esempio: requisiti chiari, stima fatta, dipendenze note). → approfondito nella sezione 6 (Scrum).

Deploy/Rilascio

L'operazione di rendere disponibile una nuova versione del software agli utenti, spostandola dall'ambiente in cui è stata sviluppata e testata a quello di produzione. → approfondito nella sezione 3 (Come nasce un software). Si collega a: **Artifact (build)** (ciò che viene effettivamente distribuito) e **Rollback** (l'operazione da compiere se il rilascio va male).

DevOps

La fusione tra "Development" (sviluppo) e "Operations" (gestione dei sistemi): un approccio che unisce persone, processi e strumenti per rilasciare software in modo più rapido, frequente e affidabile. → approfondito nella sezione 9 (DevOps). Si collega a: **CALMS** (che ne riassume i pilastri) e **CI (Continuous Integration)** (una delle pratiche concrete che lo realizzano).

DevSecOps

Un'estensione del DevOps che integra la sicurezza (Security) in ogni fase del ciclo di vita del software, invece di considerarla solo un controllo finale prima del rilascio. → approfondito nella sezione 14 (Sicurezza).

Disco

Il componente del computer dove i dati vengono salvati in modo permanente (anche da spento), a differenza della RAM che perde i dati quando il computer si spegne. → approfondito nella sezione 2 (Fondamenti di informatica).

DNS

Sigla di Domain Name System: il “elenco telefonico” di internet, che traduce nomi di siti facili da ricordare (come esempio.it) negli indirizzi numerici (IP) che i computer usano davvero per comunicare. → approfondito nella sezione 2 (Fondamenti di informatica).

Docker

Lo strumento più diffuso per creare, eseguire e distribuire Container, cioè applicazioni “pacchettizzate” insieme a tutto ciò che serve loro per funzionare ovunque nello stesso modo. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Container** (ciò che crea e distribuisce) e **Artifact (build)** (un'immagine Docker è uno dei formati più comuni di artifact).

Epic

Un'attività molto grande, che raggruppa più User Story o Work Item legati a un unico obiettivo di ampio respiro, troppo estesa per essere completata in un singolo Sprint. → approfondito nella sezione 10 (Azure DevOps).

Feature

Una funzionalità del software: una caratteristica o capacità concreta che l'utente può usare (ad esempio “possibilità di esportare un report in PDF”). → approfondito nella sezione 3 (Come nasce un software).

File System

Il modo in cui un sistema operativo organizza e conserva i file su un disco, tramite cartelle e nomi, permettendo di ritrovarli e gestirli facilmente. → approfondito nella sezione 2 (Fondamenti di informatica).

Frontend

La parte di un'applicazione con cui l'utente interagisce direttamente: l'interfaccia visibile (pagine web, schermate, pulsanti), che comunica con il Backend per mostrare e inviare informazioni. → approfondito nella sezione 12 (Architetture software).

GDPR

Il Regolamento Generale sulla Protezione dei Dati (General Data Protection Regulation), la legge europea che stabilisce come le aziende devono raccogliere, trattare e proteggere i dati personali delle persone. → approfondito nella sezione 14 (Sicurezza).

Git Flow

Un modello ben definito di organizzazione dei Branch in Git, con rami dedicati per lo sviluppo, le funzionalità, i rilasci e le correzioni urgenti, utile in progetti con rilasci pianificati e meno frequenti. → approfondito nella sezione 4 (Git e GitLab).

HTTP

Sigla di HyperText Transfer Protocol: il linguaggio con cui il browser e i siti web si scambiano informazioni su internet, ad esempio quando richiedi una pagina web. → approfondito nella sezione 2 (Fondamenti di informatica).

HTTPS

La versione sicura e cifrata di HTTP (la “S” sta per Secure), che protegge le informazioni scambiate tra browser e sito web da occhi indiscreti, oggi usata praticamente ovunque. → approfondito nella sezione 2 (Fondamenti di informatica).

IaaS

Sigla di Infrastructure as a Service: un modello cloud in cui il fornitore ti dà “solo” server, storage e rete virtuali, e sei tu a occuparti di sistema operativo, applicazioni e configurazioni. → approfondito nella sezione 13 (Cloud).

IaC (Infrastructure as Code)

La pratica di descrivere l'infrastruttura informatica (server, reti, configurazioni) tramite file di testo/codice invece che configurandola manualmente, così da poterla creare, versionare e ripetere in modo automatico e affidabile. → approfondito nella sezione 9 (DevOps).

Increment

Il risultato concreto e utilizzabile prodotto durante uno Sprint: la somma di tutte le User Story completate, che si aggiunge a quanto già costruito nei Sprint precedenti. → approfondito nella sezione 6 (Scrum).

Issue

Una “segnalazione” aperta in un repository (su GitLab o Azure DevOps) per tracciare un bug, una richiesta di funzionalità o qualsiasi attività da discutere e risolvere. → approfondito nella sezione 4 (Git e GitLab).

JSON

Sigla di JavaScript Object Notation: un formato di testo semplice e leggibile usato moltissimo per scambiare dati tra programmi, ad esempio nelle risposte delle API. → approfondito nella sezione 2 (Fondamenti di informatica).

KPI

Sigla di Key Performance Indicator: un indicatore numerico che misura quanto bene un progetto o un'attività sta raggiungendo i propri obiettivi (ad esempio il tempo medio di risposta a un cliente). → approfondito nella sezione 8 (Project Management).

Kubernetes

Uno strumento che gestisce automaticamente grandi quantità di Container in produzione: li avvia, li riavvia se si bloccano, li distribuisce su più macchine e ne regola il numero in base al carico. → approfondito nella sezione 2 (Fondamenti di informatica).

Lead Time

Il tempo totale che intercorre dal momento in cui un'attività viene **richiesta** al momento in cui viene **consegnata**, includendo anche l'attesa prima che il lavoro inizi davvero. → approfondito nella sezione 7 (Kanban).

Logging

La pratica di registrare in modo continuo gli eventi che accadono in un sistema (errori, richieste, azioni) in file o strumenti dedicati, utile per capire cosa è successo quando qualcosa va storto. → approfondito nella sezione 9 (DevOps).

Manifesto Agile

Il documento pubblicato nel 2001 da un gruppo di sviluppatori che definisce i valori e i principi alla base dell'approccio Agile, come privilegiare le persone e la collaborazione rispetto a processi rigidi e documentazione eccessiva. → approfondito nella sezione 5 (Agile).

Merge

L'operazione con cui le modifiche fatte su un Branch vengono riunite (unite) al codice principale o a un altro branch. → approfondito nella sezione 3 (Come nasce un software).

Merge Request

Una richiesta formale di unire le proprie modifiche di codice (spesso su un Branch) al codice principale, che di solito viene prima discussa e revisionata dal team (Code Review) prima di essere accettata. Su Azure DevOps lo stesso concetto si chiama "Pull Request" (abbreviata PR). → approfondito nella sezione 3 (Come nasce un software). Si collega a: **Branch** (che la genera) e **Code Review** (che ne consegue, prima dell'approvazione).

Message Queue

Un sistema che permette a diverse parti di un'applicazione di scambiarsi messaggi senza dover comunicare direttamente e nello stesso istante: un componente "deposita" un messaggio in una coda e un altro lo legge quando è pronto. → approfondito nella sezione 12 (Architetture software).

MFA (autenticazione a più fattori)

Un metodo di sicurezza che richiede più di una prova d'identità per accedere a un sistema (ad esempio password più codice ricevuto sul telefono), rendendo molto più difficile un accesso non autorizzato anche se la password viene scoperta. → approfondito nella sezione 14 (Sicurezza).

Microservizi

Un'architettura software in cui un'applicazione è divisa in tanti piccoli servizi indipendenti, ciascuno responsabile di una funzione specifica, che comunicano tra loro (spesso via API), invece di essere un unico grande blocco di codice. → approfondito nella sezione 12 (Architetture software).

Milestone

Un punto di controllo significativo in un progetto, che segna il completamento di una fase importante o il raggiungimento di un obiettivo intermedio. → approfondito nella sezione 8 (Project Management).

Mindset Agile

L'atteggiamento mentale alla base di Agile: adattarsi al cambiamento, imparare dagli errori, collaborare con il cliente e migliorare continuamente, più che l'applicazione meccanica di regole e strumenti. → approfondito nella sezione 5 (Agile).

Monitoring

L'attività di osservare costantemente lo stato di un sistema (uso della CPU, tempi di risposta, errori) per accorgersi rapidamente se qualcosa non funziona come dovrebbe. → approfondito nella sezione 9 (DevOps).

Monolite

Un'architettura software in cui tutta l'applicazione è costruita come un unico grande blocco di codice, con tutte le funzionalità interdipendenti tra loro, contrapposta ai Microservizi. → approfondito nella sezione 12 (Architetture software).

Observability

La capacità di capire **cosa sta succedendo dentro** un sistema complesso osservandolo dall'esterno, combinando log, metriche e tracce; va oltre il semplice Monitoring perché aiuta a capire anche il "perché" di un problema, non solo il "cosa". → approfondito nella sezione 9 (DevOps).

OWASP

Sigla di Open Web Application Security Project: un'organizzazione che pubblica risorse gratuite (come la celebre lista "OWASP Top 10") sulle vulnerabilità di sicurezza più comuni nelle applicazioni web. → approfondito nella sezione 14 (Sicurezza).

PaaS

Sigla di Platform as a Service: un modello cloud in cui il fornitore gestisce anche il sistema operativo e gli strumenti di base, lasciandoti concentrare solo sullo sviluppo e sulla gestione della tua applicazione. → approfondito nella sezione 13 (Cloud).

Pay-as-you-go

Un modello di costo cloud in cui paghi solo per le risorse effettivamente usate (come una bolletta elettrica), invece di acquistare in anticipo hardware costoso che magari resterà sottoutilizzato. → approfondito nella sezione 13 (Cloud).

Pipeline

Una sequenza automatizzata di passaggi (compilare, testare, rilasciare) che il codice attraversa dal momento in cui viene scritto a quando arriva agli utenti, senza interventi manuali ripetitivi. → approfondito nella sezione 11 (CI/CD). Si collega a: **Trigger** (l'evento che ne fa scattare l'avvio) e **Artifact (build)** (il prodotto finale che genera).

Pipeline as Code

La pratica di definire i passaggi di una pipeline in un file di testo versionato insieme al codice, invece che configurarla a mano tramite un'interfaccia grafica, così da poterla tracciare e riutilizzare facilmente. → approfondito nella sezione 11 (CI/CD).

Pipelines (Azure DevOps)

Il servizio di Azure DevOps che permette di creare pipeline di Continuous Integration e Continuous Delivery per compilare, testare e rilasciare automaticamente le applicazioni. → approfondito nella sezione 10 (Azure DevOps).

Processo

Un programma in esecuzione sul computer, con la propria area di memoria dedicata: quando apri un'applicazione, il sistema operativo crea un processo per farla funzionare. → approfondito nella sezione 2 (Fondamenti di informatica).

Product Backlog

L'elenco ordinato di tutto ciò che potrebbe essere fatto su un prodotto: funzionalità, correzioni, miglioramenti. È gestito dal Product Owner e cambia continuamente in base alle priorità. → approfondito nella sezione 6 (Scrum). Si collega a: **Product Owner** (che lo gestisce) e **Sprint Backlog** (il sottoinsieme selezionato per lo sprint corrente).

Product Owner

La persona responsabile di definire cosa deve essere costruito e in quale ordine di priorità, rappresentando la voce del cliente e degli utenti all'interno del team Scrum. → approfondito nella sezione 6 (Scrum).

Produzione

L'ambiente “vero”, quello effettivamente usato dagli utenti finali del software; è l'ultimo gradino dopo Sviluppo, Test/QA e Staging, e qui gli errori hanno un impatto reale. → approfondito nella sezione 15 (Ambienti di sviluppo).

Quality Gate

Un controllo automatico inserito in una pipeline che blocca l'avanzamento del rilascio se certi criteri di qualità non sono soddisfatti (ad esempio troppi errori nei test o problemi di sicurezza rilevati). → approfondito nella sezione 11 (CI/CD). Si collega a: **CI (Continuous Integration)** (di cui è parte integrante) e **Artifact (build)** (che viene generato solo se il gate viene superato).

RACI

Una matrice usata in Project Management per chiarire i ruoli in un'attività: chi è Responsible (la esegue), chi è Accountable (ne risponde), chi va Consultato e chi va solo Informato. → approfondito nella sezione 8 (Project Management).

RAID Log

Un registro che il project manager tiene per tracciare Risks (rischi), Assumptions (assunzioni), Issues (problemi) e Dependencies (dipendenze) di un progetto, per avere sempre sotto controllo i punti critici. → approfondito nella sezione 8 (Project Management).

RAM

Sigla di Random Access Memory: la memoria “di lavoro” del computer, veloce ma temporanea, che perde i dati quando il computer viene spento — a differenza del Disco. → approfondito nella sezione 2 (Fondamenti di informatica).

Release

Una versione del software resa disponibile agli utenti, spesso accompagnata da un elenco delle novità e delle correzioni incluse rispetto alla versione precedente. → approfondito nella sezione 4 (Git e GitLab).

Repos (Azure DevOps)

Il servizio di Azure DevOps che ospita i repository Git del progetto, permettendo al team di gestire il codice, i Branch e le Pull Request (l'equivalente della Merge Request di GitLab). → approfondito nella sezione 10 (Azure DevOps).

Repository

Lo “spazio” (cartella speciale) dove Git conserva tutto il codice di un progetto insieme alla sua intera storia di modifiche (Commit). → approfondito nella sezione 4 (Git e GitLab). Si collega a: **Commit** (che ne costituisce la storia) e **Branch** (che ne organizza lo sviluppo in parallelo).

Requisiti (funzionali e non funzionali)

Le caratteristiche che un software deve avere: i requisiti funzionali descrivono **cosa** il sistema deve fare (es. "l'utente può resettare la password"), quelli non funzionali descrivono **come** deve farlo (es. velocità, sicurezza, disponibilità). → approfondito nella sezione 3 (Come nasce un software).

REST

Uno stile molto diffuso per progettare API web, basato su regole semplici e standard (come l'uso di HTTP) che rendono i servizi facili da capire, usare e collegare tra loro. → approfondito nella sezione 2 (Fondamenti di informatica).

Rete

Un insieme di computer e dispositivi collegati tra loro per scambiarsi dati e comunicare, dalla piccola rete di un ufficio fino a internet, la rete di reti più grande al mondo. → approfondito nella sezione 2 (Fondamenti di informatica).

Rischio

Un evento incerto che, se si verifica, può avere un impatto positivo o (più spesso) negativo su un progetto; identificarlo e pianificarne la gestione è uno dei compiti principali del project manager. → approfondito nella sezione 8 (Project Management).

Rollback

L'operazione di tornare indietro a una versione precedente e funzionante del software dopo che un rilascio ha causato problemi in produzione. → approfondito nella sezione 11 (CI/CD). Si collega a: **Deploy/Rilascio** (l'operazione precedente, che a volte va corretta) e **Monitoring** (che ne segnala spesso la necessità).

SaaS

Sigla di Software as a Service: un modello cloud in cui usi direttamente un'applicazione già pronta tramite browser (come una webmail), senza doverti preoccupare di server, installazioni o aggiornamenti. → approfondito nella sezione 13 (Cloud).

Scalabilità verticale e orizzontale

La capacità di un sistema di gestire più carico di lavoro: la scalabilità verticale significa potenziare la singola macchina (più CPU, più RAM), quella orizzontale significa aggiungere più macchine che si dividono il lavoro. → approfondito nella sezione 13 (Cloud).

Scrum Master

La persona che facilita l'applicazione di Scrum nel team, rimuove gli ostacoli che intralciano il lavoro e protegge il team da interferenze esterne, senza dare ordini diretti su cosa fare. → approfondito nella sezione 6 (Scrum).

Scrumban

Un approccio ibrido che combina elementi di Scrum (come i ruoli e le cerimonie) con elementi di Kanban (come il flusso continuo e i limiti di WIP), utile per team che vogliono più struttura di Kanban ma più flessibilità di Scrum. → approfondito nella sezione 7 (Kanban).

Semantic Versioning

Una convenzione per numerare le versioni del software nel formato MAJOR.MINOR.PATCH (es. 2.4.1), dove ogni numero comunica il tipo di cambiamento avvenuto rispetto alla versione precedente. → approfondito nella sezione 3 (Come nasce un software).

Serverless

Un modello cloud in cui scrivi solo il codice della funzione che ti serve e il fornitore si occupa di tutto il resto (server, scalabilità, manutenzione), facendoti pagare solo per l'effettivo utilizzo. → approfondito nella sezione 12 (Architetture software).

Sistema Operativo

Il software di base che gestisce le risorse del computer (CPU, RAM, Disco) e permette a te e alle altre applicazioni di usarle, facendo da intermediario tra hardware e programmi. Esempi: Windows, macOS, Linux. → approfondito nella sezione 2 (Fondamenti di informatica).

Sprint

Un periodo di tempo fisso e breve (di solito 1-4 settimane) durante il quale il team Scrum lavora per completare un insieme di attività scelte dal Product Backlog, prodotto poi come Increment. → approfondito nella sezione 6 (Scrum). Si collega a: **Product Backlog** (da cui trae le attività) e **Increment** (il risultato concreto che produce).

Sprint Backlog

L'elenco delle attività selezionate dal Product Backlog che il team si impegna a completare durante lo Sprint corrente. → approfondito nella sezione 6 (Scrum).

Sprint Planning

L'incontro all'inizio di ogni Sprint in cui il team decide cosa realizzare, selezionando le voci dal Product Backlog e definendo come portarle a termine. → approfondito nella sezione 6 (Scrum).

Sprint Retrospective

L'incontro alla fine di ogni Sprint in cui il team riflette su come ha lavorato (non su cosa ha costruito) per individuare cosa migliorare nel prossimo Sprint. → approfondito nella sezione 6 (Scrum).

Sprint Review

L'incontro alla fine di ogni Sprint in cui il team mostra agli stakeholder ciò che ha completato, raccogliendo feedback utile per i prossimi passi. → approfondito nella sezione 6 (Scrum).

SQL

Sigla di Structured Query Language: il linguaggio usato per “interrogare” un Database relazionale, ad esempio per chiedere, aggiungere o modificare dati nelle tabelle. → approfondito nella sezione 2 (Fondamenti di informatica).

Staging

Un ambiente che replica il più possibile le condizioni reali della Produzione, usato come ultima verifica prima del rilascio definitivo agli utenti. → approfondito nella sezione 15 (Ambienti di sviluppo).

Stakeholder

Qualsiasi persona o gruppo che ha un interesse nel progetto o ne è influenzato: può essere un cliente, un utente finale, un manager o un membro del team. → approfondito nella sezione 8 (Project Management).

Story Point

Un'unità di misura relativa (non temporale) usata per stimare quanto sia complessa una User Story rispetto alle altre, tenendo conto di difficoltà, incertezza e quantità di lavoro. → approfondito nella sezione 6 (Scrum).

Tag

Un'etichetta che Git può assegnare a un Commit specifico, tipicamente per marcare un punto importante come una Release (es. “v2.4.1”). → approfondito nella sezione 4 (Git e GitLab).

TCP/IP

La famiglia di regole (protocolli) che permette ai computer di tutto il mondo di scambiarsi dati su internet in modo ordinato e affidabile, anche passando per reti diverse. → approfondito nella sezione 2 (Fondamenti di informatica).

Test Plans (Azure DevOps)

Il servizio di Azure DevOps dedicato a pianificare, organizzare ed eseguire i test manuali ed esplorativi su un'applicazione. → approfondito nella sezione 10 (Azure DevOps).

Thread

Una “sotto-unità” di un Processo che può eseguire istruzioni in modo indipendente; più thread nello stesso processo permettono a un programma di fare più cose contemporaneamente in modo più efficiente. → approfondito nella sezione 2 (Fondamenti di informatica).

Trigger

L'evento che fa scattare automaticamente l'avvio di una Pipeline, ad esempio un nuovo Commit su un branch o l'apertura di una Merge Request. → approfondito nella sezione 11 (CI/CD).

Trunk Based Development

Una pratica in cui tutti gli sviluppatori integrano il proprio codice molto frequentemente su un unico ramo principale (il "trunk"), evitando Branch di lunga durata e favorendo una forte automazione dei test. → approfondito nella sezione 4 (Git e GitLab).

User Story

Una breve descrizione di una funzionalità scritta dal punto di vista dell'utente, spesso nel formato "Come [ruolo], voglio [obiettivo], per [beneficio]", usata per catturare i requisiti in modo semplice e centrato sulle persone. → approfondito nella sezione 6 (Scrum). Si collega a: **Requisiti (funzionali e non funzionali)** (da cui spesso nasce) e **Product Backlog** (dove viene inserita in attesa di essere pianificata).

Velocity

La quantità media di Story Point che un team Scrum riesce a completare in uno Sprint, misurata negli Sprint passati e usata per stimare quanto lavoro pianificare in futuro. → approfondito nella sezione 6 (Scrum).

Versioning

La pratica di assegnare un numero o un'etichetta univoca a ogni versione del software, per poter distinguere, confrontare e tornare a versioni precedenti quando necessario. → approfondito nella sezione 3 (Come nasce un software).

Virtual Machine (VM)

Un "computer dentro il computer": un ambiente simulato dal software che si comporta come una macchina indipendente, con proprio sistema operativo, pur condividendo l'hardware fisico con altre VM. → approfondito nella sezione 2 (Fondamenti di informatica).

Waterfall

Un approccio tradizionale alla gestione dei progetti in cui le fasi (analisi, progettazione, sviluppo, test, rilascio) si susseguono in modo lineare e sequenziale, una dopo l'altra, a differenza dell'iteratività di Agile. → approfondito nella sezione 5 (Agile).

WIP (Work In Progress)

Il numero di attività che sono attualmente "in corso" in una Board Kanban; limitare il WIP (fissando un tetto massimo per colonna) aiuta il team a concentrarsi e a completare il lavoro più rapidamente invece di iniziarne troppo in parallelo. → approfondito nella sezione 7 (Kanban).

Work Item

L'unità base di lavoro tracciata in Azure DevOps Boards: può essere una User Story, un Bug, un'attività (Task) o un Epic, a seconda del tipo. → approfondito nella sezione 10 (Azure DevOps).

XML

Sigla di eXtensible Markup Language: un formato di testo, più “verboso” di JSON, usato per organizzare e scambiare dati tramite tag simili a quelli dell'HTML. → approfondito nella sezione 2 (Fondamenti di informatica).



Collegamenti

- [17. Piano di studio](#) — un percorso suggerito per ripassare tutti questi concetti in modo organico

Risorse

- [Glossario Agile Alliance](#) — glossario dei termini Agile e Scrum (in inglese)
- [Microsoft Learn — Glossario Azure](#) — terminologia cloud e DevOps
- [Wikipedia — Glossario di informatica](#) — per approfondire i termini di base

17. Piano di studio

Le sezioni da 1 a 15 di questo corso contengono tutto il materiale teorico e pratico che ti serve. Questa sezione risponde a una domanda diversa e molto concreta: **in che ordine e con quale ritmo studiarlo, in 2-3 mesi, mentre affianchi la tua collega Scrum Master/PM sul progetto?**

Il piano che segue copre **8 settimane**. Non è un vincolo rigido: è una traccia pensata per darti un ritmo sostenibile, evitare di sentirti sopraffatto dalla quantità di argomenti nuovi (specialmente nelle prime due settimane, le più dense dal punto di vista tecnico) e assicurarti che, settimana dopo settimana, tu costruisca le competenze nell'ordine in cui ti serviranno davvero sul campo.

Come usare questo piano

- **È un percorso, non una gara.** Le ore indicate per ogni settimana (in media 6-10 ore) sono una stima per un neolaureato senza background informatico che studia part-time, in parallelo all'affiancamento quotidiano sul progetto. Se una settimana ti serve più tempo — è normalissimo, soprattutto per le sezioni 2 e 9, che sono le più dense — **rallenta**. È molto meglio arrivare alla settimana 8 con basi solide che aver “letto tutto” senza averlo capito.
- **Le settimane si possono spostare, non solo comprimere.** Se il progetto ti mette in un contesto reale prima del previsto (es. partecipi a un Sprint Planning già alla settimana 2), va benissimo anticipare la lettura della sezione corrispondente: il piano è una guida, il contesto reale è il miglior maestro che hai.
- **Il check-in con la tua collega è la parte più importante del piano**, non un'aggiunta facoltativa. Ogni settimana prevede almeno un momento di confronto (15-30 minuti bastano) in cui le racconti cosa hai capito, le fai le domande che ti sono rimaste in sospeso e — soprattutto — collega la teoria che hai letto a un esempio concreto del progetto (“quello che ho letto sulle user story, corrisponde a come scriviamo le card sul backlog?”). Senza questo passaggio, il rischio è restare con una conoscenza da manuale che non si aggancia mai alla pratica quotidiana.
- **Gli esercizi pratici non sono opzionali.** Leggere una sezione ti dà il vocabolario; l'esercizio è quello che lo trasforma in competenza. Se salti gli esercizi per fare prima, arriverai alla fine del percorso con parole difficili ma poca sicurezza nell'usarle.
- **Il Glossario (sezione 16) è il tuo compagno silenzioso per tutte le 8 settimane.** Ogni volta che incontri un termine che non ricordi — anche uno già visto — torna lì. Non è un segno di scarsa preparazione: è esattamente lo scopo per cui esiste.
- **Non aspettarti di “sapere tutto” alla settimana 8.** L'obiettivo del piano è darti l'autonomia per affiancare e poi sostituire la tua collega con sicurezza sulle basi, non farti diventare un esperto DevOps in due mesi. Il resto lo imparerai sul campo, con la sezione 19 (Risorse online) e la sezione 18 (Libri consigliati) a disposizione per gli approfondimenti che vorrai fare più avanti.

Vista d'insieme del percorso

Diagramma 1

Diagramma 1

Il percorso segue una logica a blocchi progressivi: prima il **linguaggio di base** del software (settimane 1-2), poi **il modo in cui il team si organizza** per lavorare (settimane 3-4), poi **gli strumenti e i processi specifici** della piattaforma DevOps che gestirai (settimane 5-6), poi il **contesto tecnico e trasversale** che ti serve per capire le decisioni del team (settimana 7), e infine un momento di **consolidamento** (settimana 8) prima di sentirti pronto/a a camminare sempre più da solo/a.

Settimana 1 — Le fondamenta: come funziona un computer, come nasce un software

Argomenti	Sezione 1 — Introduzione · Sezione 2 — Fondamenti di informatica
Tempo stimato	8-10 ore (la sezione 2 è la più densa dell'intero corso: CPU, RAM, rete, database, container, Docker, Kubernetes — non sentirti in colpa se ti serve più tempo)

Esercizi pratici

1. Fatti installare (o installa da solo/a, se hai i permessi) Docker Desktop sul tuo computer e prova a lanciare un container di esempio (es. `docker run hello-world`), anche senza capire ogni dettaglio: l'obiettivo è vedere con i tuoi occhi cos'è un container prima di studiarlo in teoria.
2. Chiedi a un collega developer di mostrarti, per 15 minuti, come si collegano tra loro un'applicazione, un database e un server nel progetto (anche solo a parole, con un disegno su una lavagna o su un foglio): non serve capire il codice, serve vedere la “geografia” del sistema.
3. Disegna a mano (anche su carta) uno schizzo con client, server, database e rete, usando le tue parole per etichettare ogni pezzo.

Obiettivi di apprendimento

Al termine della settimana devi saper spiegare a parole tue, senza usare il manuale:

- cosa fanno CPU e RAM e perché sono diverse;
- cos'è un indirizzo IP e a cosa serve una rete;
- cos'è un database e perché serve rispetto a “un file con dentro i dati”;
- cos'è un container e in cosa differisce da una macchina virtuale;
- a cosa serve, in generale, Kubernetes (senza ancora sapere usarlo).

Verifica finale della settimana

Sai spiegare a un amico non tecnico, in meno di due minuti, la differenza tra un container e una macchina virtuale, usando un'analogia (ad esempio quella degli appartamenti in un condominio vista nella sezione 2)?

Settimana 2 — Come nasce un software e il controllo di versione con Git

Argomenti	Sezione 3 — Come nasce un software · Sezione 4 — Git e GitLab
Tempo stimato	7-9 ore (la sezione 4 richiede pratica al computer, non solo lettura)

Esercizi pratici

1. Crea un account GitLab (se non lo hai già) e crea un repository di prova personale, anche vuoto.
2. Clona un repository di esempio (puoi usare uno dei tuoi o uno pubblico semplice) sul tuo computer con `git clone`.
3. Crea un nuovo branch, modifica un file (anche solo il README), fai un commit e apri una **merge request fittizia** verso il branch principale del tuo repository di prova.
4. Chiedi a un developer del team di farti vedere, sullo schermo, una vera merge request aperta sul progetto: osserva come è strutturata la descrizione, chi la revisiona, quali commenti riceve.
5. Prova a spiegare a voce, senza guardare gli appunti, cosa succede “dietro le quinte” quando fai un commit.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- descrivere le fasi principali del ciclo di vita di un software (analisi, progettazione, sviluppo, test, rilascio, manutenzione);
- distinguere un bug da una richiesta di nuova funzionalità;
- spiegare cosa sono repository, commit, branch e merge request;
- distinguere concettualmente Git Flow e Trunk Based Development (anche solo a grandi linee, senza padroneggiarli).

Verifica finale della settimana

Sai disegnare su un foglio il percorso di una modifica al codice, dal branch alla merge request fino al merge sul branch principale, spiegando ogni passaggio?

Settimana 3 — Agile e Scrum: il mindset e il framework che il team usa ogni giorno

Argomenti	Sezione 5 — Agile · Sezione 6 — Scrum
Tempo stimato	8-10 ore (la sezione 6 è la più importante del corso per il tuo ruolo: prendila con calma)

Esercizi pratici

1. Partecipa come **osservatore** a uno Sprint Planning e a una Daily Scrum del team, senza intervenire: prendi appunti su chi parla, cosa viene deciso, quanto dura ogni evento rispetto a quanto previsto dalla teoria.
2. Scrivi 3 user story per una feature immaginaria (ad esempio “un’app per prenotare una sala riunioni”), seguendo il formato “Come [utente], voglio [obiettivo], così che [beneficio]”, con relativi criteri di accettazione.
3. Prova a stimare le 3 user story che hai scritto con la scala di Story Point di Fibonacci (1, 2, 3, 5, 8...), motivando a voce alta perché hai scelto quel valore.
4. Confronta con la tua collega la Definition of Ready e la Definition of Done reali del progetto: sono scritte da qualche parte? Coincidono con quello che hai letto in teoria?

Obiettivi di apprendimento

Al termine della settimana devi saper:

- riassumere i 4 valori e i 12 principi del Manifesto Agile con parole tue;
- elencare e descrivere i tre ruoli di Scrum (Product Owner, Scrum Master, Team di sviluppo);
- descrivere i cinque eventi di Scrum (Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective) con durata e obiettivo di ciascuno;
- distinguere i tre artefatti (Product Backlog, Sprint Backlog, Increment);
- scrivere una user story ben formata e spiegare a cosa servono Story Point e Definition of Ready/Done.

Verifica finale della settimana

Sai spiegare, senza guardare gli appunti, a cosa serve ciascuno dei cinque eventi di Scrum e cosa succederebbe al team se uno di essi venisse eliminato?

Settimana 4 — Kanban e Project Management: un altro modo di visualizzare il lavoro, e come si tiene sotto controllo un progetto

Argomenti	Sezione 7 — Kanban · Sezione 8 — Project Management
Tempo stimato	6-8 ore

Esercizi pratici

1. Osserva la board del team (Kanban o Scrum board che sia) e identifica le colonne, i limiti di WIP (Work In Progress) se presenti, e chiedi a un collega di spiegarti come si sposta una card da una colonna all'altra.
2. Crea, anche solo su carta o con un tool gratuito online, una board Kanban di prova con 5 card immaginarie e sposta manualmente una card lungo le colonne, calcolando a mano un ipotetico lead time e cycle time.

3. Costruisci una tabella RACI di esempio (anche minimale, 4-5 righe) per un piccolo progetto immaginario, assegnando ruoli Responsible, Accountable, Consulted, Informed.
4. Chiedi alla tua collega di farti vedere un report o una dashboard reale del progetto (burndown, RAID log, o simili) e provate insieme a leggerla.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- descrivere la struttura di una board Kanban e a cosa serve il limite di WIP;
- distinguere lead time e cycle time;
- spiegare cos'è un RAID log e a cosa serve una matrice RACI;
- descrivere almeno due KPI tipici di un progetto software e leggere un report/dashboard di base.

Verifica finale della settimana

Sai spiegare a parole tue la differenza tra Sprint (Scrum) e flusso continuo (Kanban)? In quale scenario useresti l'uno o l'altro?

Settimana 5 — DevOps: la cultura e i concetti che uniscono sviluppo e operations

Argomenti	Sezione 9 — DevOps
Tempo stimato	8-10 ore (sezione densa e centrale: cultura, CI/CD concettuale, Infrastructure as Code, monitoring/observability — vale la pena dedicarle un'intera settimana da sola)

Esercizi pratici

1. Chiedi a un membro del team (developer o operations) di raccontarti, con parole semplici, un episodio concreto in cui l'automazione (una pipeline, un test automatico) ha evitato un problema in produzione: raccogli l'aneddoto, aiuta più di dieci definizioni.
2. Osserva, anche solo guardando lo schermo di un collega, una dashboard di monitoring reale del progetto: prova a distinguere cosa mostra il "monitoring" (è tutto ok?) da cosa servirebbe per fare "observability" (perché non va?).
3. Scrivi in 5 righe, con parole tue, cosa significa "you build it, you run it" e perché cambia il modo in cui un team lavora rispetto al modello tradizionale sviluppo/operations separati.
4. Prova a spiegare il modello CALMS (Culture, Automation, Lean, Measurement, Sharing) usando un esempio del progetto per ciascuna delle 5 lettere.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- spiegare cos'è la cultura DevOps e perché nasce come risposta al conflitto storico tra Dev e Ops;

- descrivere il modello CALMS;
- distinguere concettualmente Continuous Integration, Continuous Delivery e Continuous Deployment;
- spiegare cos'è l'Infrastructure as Code e perché è utile;
- distinguere monitoring, observability e logging.

Verifica finale della settimana

Sai spiegare a parole tue la differenza tra Continuous Delivery e Continuous Deployment, indicando esattamente dove sta il confine tra i due?

Settimana 6 — Azure DevOps e CI/CD in dettaglio: dalla teoria alla pipeline reale

Argomenti	Sezione 10 — Azure DevOps · Sezione 11 — CI/CD
Tempo stimato	8-10 ore

Esercizi pratici

1. Fatti fare un tour guidato (30-45 minuti) di Azure DevOps sul progetto reale da un collega: Boards, Repos, Pipelines, Test Plans, Artifacts — anche solo “a vista”, senza toccare nulla la prima volta.
2. Osserva una pipeline CI/CD reale del progetto in esecuzione (o la sua ultima esecuzione registrata) e **disegna uno schema** di come funziona: trigger, fasi (build, test, deploy), ambienti coinvolti, eventuali quality gate.
3. Trova nel progetto un file di pipeline YAML (o fattelo mostrare) e prova a identificare, senza capire ogni riga, dove sono definiti trigger, stage e job.
4. Chiedi cosa succede quando una pipeline “si rompe” (fallisce un quality gate, un test): chi viene notificato, cosa succede al rilascio.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- descrivere i cinque servizi principali di Azure DevOps e a cosa serve ciascuno;
- descrivere le fasi tipiche di una pipeline CI/CD (checkout, build, test, pubblicazione artifact, deploy);
- spiegare cosa fa scattare (trigger) una pipeline;
- spiegare cos'è un quality gate e perché può bloccare un rilascio;
- leggere, a grandi linee, la struttura di un file YAML di pipeline.

Verifica finale della settimana

Sai descrivere le fasi di una pipeline CI/CD del progetto, dal commit del developer fino al rilascio, indicando dove potrebbe fermarsi se qualcosa va storto?

Settimana 7 — Il contesto tecnico e trasversale: architetture, cloud, sicurezza, ambienti

Argomenti	Sezione 12 — Architetture software · Sezione 13 — Cloud · Sezione 14 — Sicurezza · Sezione 15 — Ambienti di sviluppo
Tempo stimato	8-10 ore (quattro sezioni più brevi, ma trasversali: prova a leggerle collegandole sempre al progetto reale)

Esercizi pratici

1. Fatti spiegare da un developer se l'architettura del progetto è più vicina a un monolite o a dei microservizi, e chiedi un esempio concreto di come frontend e backend comunicano.
2. Individua, con l'aiuto di un collega, quale provider cloud usa il progetto (Azure, AWS o altro) e quali servizi principali (IaaS, PaaS, SaaS) sono in uso — anche solo i nomi, senza approfondire ogni dettaglio tecnico.
3. Chiedi come funzionano autenticazione e autorizzazione nell'applicazione del progetto (login, ruoli, permessi) e prova a mappare i concetti visti nella sezione 14 su questo esempio reale.
4. Ricostruisci, con l'aiuto della tua collega, la sequenza reale degli ambienti del progetto (Dev, Test, Staging, Prod o equivalenti) e chi/cosa decide quando il codice passa da uno all'altro.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- distinguere monolite e microservizi, e frontend e backend;
- distinguere IaaS, PaaS e SaaS con un esempio ciascuno;
- spiegare la differenza tra autenticazione e autorizzazione, e cos'è il DevSecOps;
- descrivere a cosa serve ciascun ambiente (Dev, Test, Staging, Prod) nella catena di rilascio del progetto.

Verifica finale della settimana

Sai spiegare, usando il progetto come esempio, perché il codice passa attraverso più ambienti prima di arrivare in produzione, e cosa si rischierebbe saltando uno di questi passaggi?

Settimana 8 — Consolidamento: ripasso generale, autovalutazione e prossimi passi

Argomenti	Ripasso trasversale delle sezioni 1-15, con il supporto di Sezione 16 — Glossario · Sezione 18 — Libri consigliati · Sezione 19 — Risorse online
Tempo stimato	6-8 ore, distribuite tra ripasso e un colloquio conclusivo con la tua collega

Esercizi pratici

1. Scorri il Glossario (sezione 16) da cima a fondo, senza fretta: segna i termini che non ricordi al 100% e torna alla sezione corrispondente per un rapido ripasso mirato.
2. Prepara una “mappa mentale” (anche a mano, su un foglio) che collega i temi principali del corso: Agile → Scrum/Kanban → Project Management → DevOps → Azure DevOps/CI/CD → Cloud/Sicurezza. Non deve essere perfetta, deve aiutarti a vedere le connessioni.
3. Fissa un colloquio conclusivo di 45-60 minuti con la tua collega Scrum Master/PM in cui **tu** guidi la conversazione: racconta con parole tue come funziona il progetto, dal punto di vista di processo (Scrum/Kanban) e di piattaforma (Azure DevOps, pipeline, ambienti). Fatti correggere dove serve.
4. Scegli, guardando la sezione 18, un libro che approfondirai nei mesi successivi (non serve leggerlo entro questa settimana: è un impegno per dopo).
5. Salva nei preferiti 3-4 risorse online dalla sezione 19 che userai come riferimento rapido nel lavoro quotidiano.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- ricostruire una visione d'insieme di tutto il corso, collegando i temi delle diverse sezioni tra loro;
- usare il Glossario come strumento di consultazione rapida senza più bisogno di “studiarlo”;
- muoverti con sicurezza nella conversazione quotidiana del team, sia sul piano di processo che su quello tecnico di base;
- riconoscere quali aree richiederanno ancora approfondimento nei mesi successivi (e sapere dove trovare le risorse per farlo).

Verifica finale della settimana

Se dovessi spiegare in 5 minuti a un/una nuovo/a collega, arrivato/a oggi, “come funziona questo progetto” — dal processo di lavoro del team alla piattaforma tecnica che lo supporta — ci riusciresti senza guardare gli appunti? Se la risposta è “quasi”, individua i due punti più deboli e torna sulle sezioni corrispondenti.

Tabella riepilogativa delle 8 settimane

Settimana	Argomenti	Tempo stimato	Focus dell'esercizio pratico
1	Introduzione · Fondamenti di informatica	8-10 ore	Docker in locale, schema client/server/DB
2	Come nasce un software · Git e GitLab	7-9 ore	Repository, branch e merge request fittizia su GitLab
3	Agile · Scrum	8-10 ore	Osservazione Sprint Planning/Daily, scrittura di 3 user story
4	Kanban · Project Management	6-8 ore	Board Kanban di prova, tabella RACI di esempio
5	DevOps	8-10 ore	Osservazione dashboard di monitoring, modello CALMS applicato al progetto
6	Azure DevOps · CI/CD	8-10 ore	Osservazione pipeline reale e disegno dello schema
7	Architetture software · Cloud · Sicurezza · Ambienti di sviluppo	8-10 ore	Mappatura architettura, cloud provider e ambienti reali del progetto
8	Ripasso generale · Glossario · Libri · Risorse online	6-8 ore	Mappa mentale finale e colloquio conclusivo con la collega

Collegamenti

- [18. Libri consigliati](#) — approfondimenti da leggere con calma dopo aver completato le 8 settimane
- [19. Risorse online](#) — materiali di consultazione rapida da usare durante e dopo il piano di studio

Risorse

- [Learning How to Learn \(Coursera, Barbara Oakley\)](#) — corso gratuito molto pratico su tecniche di apprendimento efficace, utile per organizzare meglio lo studio in 8 settimane

- [Cal Newport — Come studiare in modo efficace](#) — riflessioni pratiche su concentrazione, ripasso attivo e gestione del tempo di studio
- [Atlassian — Guida per i nuovi Scrum Master](#) — utile come lettura di rinforzo trasversale durante tutto il percorso, in particolare nelle settimane 3-4

18. Libri consigliati

Le sezioni da 1 a 17 di questo corso ti hanno già dato tutto ciò che ti serve per muovere i primi passi nel ruolo: i fondamenti tecnici, i framework Agile/Scrum, il Project Management e il mondo DevOps. Questa sezione è diversa dalle altre: non introduce concetti nuovi e obbligatori, ma raccoglie **12 libri di riferimento**, indicati direttamente dal responsabile del team, pensati per **approfondire con più calma e più profondità** ciò che nel corso hai visto in forma sintetica.

La logica dei 5 livelli

I libri sono organizzati in **5 livelli di lettura progressiva**, costruiti seguendo la stessa logica “a mattoncini” con cui è organizzato il resto del corso (lo ricordi dalla sezione 1: non puoi costruire il tetto se non hai ancora le fondamenta):

- **Livello 0 – Fondamenti di informatica:** rafforza la sezione 2. Utile soprattutto se, arrivando da un percorso gestionale, senti ancora un po' di terreno instabile sotto i piedi quando si parla di computer, reti o architetture di base.
- **Livello 1 – Sviluppo software:** rafforza la sezione 3 (Come nasce un software) e la sezione 4 (Git e GitLab). Ti aiuta a capire **come lavorano davvero gli sviluppatori** che gestirai e con cui parlerai ogni giorno — non per scrivere codice tu stesso, ma per capire le loro priorità, i loro vincoli e il loro linguaggio.
- **Livello 2 – Agile:** rafforza le sezioni 5 (Agile), 6 (Scrum) e 7 (Kanban). È il livello più direttamente collegato al ruolo che andrai a occupare: Scrum Master.
- **Livello 3 – Project Management:** rafforza la sezione 8 (Project Management) e fa da ponte verso la sezione 9 (DevOps), mostrando cosa succede quando i metodi “leggeri” di Agile incontrano progetti complessi, reali, con tutte le loro frizioni.
- **Livello 4 – DevOps:** rafforza le sezioni 9-14 (il Blocco C del corso, quello più tecnico), con un'attenzione particolare a **perché** le pratiche DevOps funzionano, supportata da dati e ricerca, non solo da intuizione.

Non c'è un obbligo di leggerli tutti né di seguire l'ordine alla lettera: sono pensati per essere agganciati **al momento giusto del tuo percorso**, quando l'argomento del livello corrispondente è già “caldo” perché lo hai appena studiato o lo stai osservando dal vivo nel team. Per questo, ogni libro qui sotto include un suggerimento su **quando** leggerlo rispetto al [Piano di studio](#) a 8 settimane.

Come usare questa sezione: non è pensata per essere letta tutta subito. Torna qui ogni volta che completi un blocco di sezioni del corso (ad esempio dopo aver finito la sezione 6 su Scrum) e scegli il libro del livello corrispondente che ti interessa di più. Va benissimo leggerne solo alcuni, o leggerli fuori ordine, se un argomento ti appassiona particolarmente.

Livello 0 – Fondamenti di informatica

“Code” – Charles Petzold

Questo libro racconta, con un linguaggio quasi narrativo e senza dare per scontato nulla, come si arriva dai concetti più elementari (interruttori, codice binario) fino a un computer moderno funzionante. È il libro ideale se, dopo la sezione 2, ti restano ancora dei dubbi su **cosa succede davvero dentro la macchina** quando parliamo di CPU, memoria o istruzioni: qui lo vedrai costruito pezzo per pezzo, in modo che l'intuizione resti solida anche quando in team si parla di argomenti più avanzati come container o cloud.

- **Quando leggerlo:** consigliato durante la settimana 1-2, in parallelo o subito dopo la sezione 2 (Fondamenti di informatica).
- **Difficoltà:** Media (richiede attenzione ma nessuna competenza pregressa).
- **Tempo stimato:** 10-12 ore di lettura complessiva.

“Computer Science Distilled” – Wladston Ferreira Filho

Un libro molto compatto e visuale, pensato per chi vuole avere una mappa d'insieme dei concetti chiave dell'informatica (algoritmi, strutture dati, basi di reti e sistemi) senza addentrarsi nei dettagli tecnici che, nel tuo ruolo, non ti serviranno mai scrivere in prima persona. È un ottimo complemento “riassuntivo” alla sezione 2 del corso: dove il corso spiega con analogie, questo libro offre schemi e diagrammi che aiutano a fissare i concetti in memoria a lungo termine.

- **Quando leggerlo:** consigliato durante la settimana 2, come ripasso rapido dopo la sezione 2.
- **Difficoltà:** Facile (molto sintetico e visuale).
- **Tempo stimato:** 4-6 ore di lettura complessiva.

“Computer Networking: A Top-Down Approach” – Kurose & Ross (capitoli introduttivi)

Questo è il manuale universitario di riferimento per le reti informatiche: non serve leggerlo per intero (è pensato per un intero corso semestrale), ma i **capitoli introduttivi** sulla struttura di internet, sui protocolli TCP/IP e su HTTP/HTTPS offrono un livello di dettaglio superiore a quello della sezione 2, utile se nel team senti spesso discussioni su reti, ambienti e connessioni tra servizi e vuoi capirle a fondo, non solo a livello di analogia.

- **Quando leggerlo:** lettura di consolidamento dopo la settimana 2, oppure più avanti se emergono dubbi specifici su reti durante le sezioni su Cloud (13) o CI/CD (11).
- **Difficoltà:** Impegnativo (è un testo universitario, anche se i capitoli iniziali sono i più accessibili).
- **Tempo stimato:** 6-8 ore per i soli capitoli introduttivi.

Livello 1 – Sviluppo software

“Clean Code” – Robert C. Martin

Non diventerai uno sviluppatore, ma capire **cosa rende il codice “buono” o “scadente”** agli occhi di chi lo scrive ti aiuta moltissimo a comprendere discussioni di code review, stime più lunghe del previsto per “ripulire” una parte di codice (il cosiddetto debito tecnico) o perché il team insiste su determinate pratiche. Questo libro, uno dei più citati nel mondo dello sviluppo software, ti dà il vocabolario e la sensibilità per capire quelle conversazioni dall’interno, collegandosi direttamente a quanto visto nella sezione 3 sul ciclo di vita del software e sulla code review.

- **Quando leggerlo:** consigliato durante la settimana 3, dopo aver completato la sezione 3 (Come nasce un software) e la sezione 4 (Git e GitLab).
- **Difficoltà:** Media (alcuni esempi di codice, ma i concetti generali sono comprensibili anche senza saperlo scrivere).
- **Tempo stimato:** 10-12 ore di lettura complessiva.

“The Pragmatic Programmer” – Andrew Hunt & David Thomas

Un classico che va oltre il “come scrivere codice” per parlare di **come pensano e lavorano gli sviluppatori**: gestione degli errori, automazione, comunicazione con il team, cura dei dettagli. È particolarmente utile per un futuro Scrum Master perché offre uno sguardo diretto sulla mentalità delle persone che faciliterai e coordinerai, aiutandoti a capire le loro priorità quando negozi scadenze o priorità del backlog.

- **Quando leggerlo:** consigliato durante la settimana 3-4, come lettura di accompagnamento a “Clean Code” o subito dopo.
- **Difficoltà:** Media.
- **Tempo stimato:** 8-10 ore di lettura complessiva.

Livello 2 – Agile

“Scrum: The Art of Doing Twice the Work in Half the Time” – Jeff Sutherland

Scritto da uno dei co-creatori di Scrum, questo libro racconta la nascita del framework attraverso storie concrete (compresi progetti falliti prima dell'adozione di Scrum), rendendo tangibile il “perché” dietro le pratiche che hai studiato nella sezione 6 in modo più formale. È una lettura quasi narrativa, perfetta per consolidare la motivazione dietro Sprint, Daily Scrum e Retrospective proprio mentre inizi a osservarle dal vivo nel team, secondo la Fase 1 (Osservazione) del tuo percorso di crescita.

- **Quando leggerlo:** consigliato durante la settimana 4-5, subito dopo la sezione 6 (Scrum), mentre osservi le prime cerimonie del team.
- **Difficoltà:** Facile (scorrevole, orientato agli aneddoti più che alla teoria pura).
- **Tempo stimato:** 6-8 ore di lettura complessiva.

“Essential Scrum” – Kenneth Rubin

Se il libro di Sutherland è la storia e la motivazione, questo è il **manuale di riferimento**: copre in modo sistematico ruoli, eventi e artefatti di Scrum, con diagrammi chiari e casi limite che il corso, per motivi di sintesi, non può trattare in dettaglio (ad esempio come gestire Sprint con più team, o come stimare backlog molto grandi). È il libro da tenere sul comodino durante la Fase 2 (Affiancamento attivo), quando comincerai a condurre tu stesso alcune cerimonie.

- **Quando leggerlo**: lettura di consolidamento dopo la settimana 5, quando inizi a facilitare direttamente le prime attività.
 - **Difficoltà**: Media (più denso e sistematico del libro di Sutherland).
 - **Tempo stimato**: 12-15 ore di lettura complessiva.
-

Livello 3 – Project Management

“Making Things Happen” – Scott Berkun

Un libro sul project management che non segue la logica rigida del Waterfall universitario, ma parla di gestione dei progetti nel mondo reale: comunicazione, gestione delle pressioni, decisioni sotto incertezza. Si collega direttamente alla sezione 8, dove hai visto che il project management in un contesto Agile è più simile a “un navigatore che aggiorna la rotta” che a un piano rigido: questo libro approfondisce esattamente quella metafora con esempi pratici su stakeholder, rischi e comunicazione.

- **Quando leggerlo**: consigliato durante la settimana 5-6, subito dopo la sezione 8 (Project Management).
- **Difficoltà**: Media.
- **Tempo stimato**: 10-12 ore di lettura complessiva.

“The Phoenix Project” – Gene Kim, Kevin Behr, George Spafford

Un romanzo aziendale (sì, è scritto come una storia, non come un manuale) che segue un IT manager alle prese con un progetto in crisi, mostrando in modo molto realistico le tensioni tra sviluppo, operations e business che la cultura DevOps cerca di risolvere. È probabilmente il libro più “riconoscibile” di questa lista per chi lavora nel team: molte delle dinamiche raccontate (rilasci in ritardo, colpe scaricate tra reparti, pressione degli stakeholder) sono facilmente riconducibili a episodi reali che osserverai. Fa da ponte naturale verso la sezione 9 (DevOps).

- **Quando leggerlo**: consigliato durante la settimana 6, come transizione tra il blocco Project Management e il blocco DevOps del corso.
 - **Difficoltà**: Facile (si legge come un romanzo, nonostante i temi tecnici).
 - **Tempo stimato**: 8-10 ore di lettura complessiva.
-

Livello 4 – DevOps

“The DevOps Handbook” – Gene Kim, Jez Humble, Patrick Debois, John Willis

Se “The Phoenix Project” racconta la storia in forma di romanzo, questo libro ne è il **manuale operativo**: spiega in modo sistematico i principi e le pratiche DevOps (i tre flussi, l'automazione, il modello CALMS che hai già incontrato nella sezione 9) con esempi reali di aziende che li hanno adottati. È il testo di riferimento per capire in profondità tutto il Blocco C del corso (sezioni 9-14), collegando cultura, pratiche tecniche e risultati di business in un unico quadro coerente.

- **Quando leggerlo**: consigliato durante la settimana 7, dopo aver completato la sezione 9 (DevOps) e la sezione 11 (CI/CD).
- **Difficoltà**: Impegnativo (denso, ma organizzato in modo molto chiaro).
- **Tempo stimato**: 15-18 ore di lettura complessiva.

“Accelerate” – Nicole Forsgren, Jez Humble, Gene Kim

Un libro diverso dagli altri: qui la cultura DevOps non viene raccontata per aneddoti ma **misurata con dati e ricerca scientifica**, mostrando quali pratiche (deployment frequenti, automazione, cultura blameless) sono statisticamente collegate a team e organizzazioni ad alte prestazioni. È la lettura ideale per chi, come te, viene da un percorso gestionale abituato a ragionare per KPI e metriche: ti dà argomenti solidi per spiegare al cliente o ai responsabili **perché** certe pratiche DevOps valgono l'investimento, andando oltre l'intuizione.

- **Quando leggerlo**: lettura di consolidamento dopo la settimana 7-8, come chiusura del percorso DevOps del corso e apertura verso l'autonomia della Fase 3.
 - **Difficoltà**: Impegnativo (alcuni capitoli hanno un taglio statistico/metodologico).
 - **Tempo stimato**: 10-12 ore di lettura complessiva.
-

Tabella riepilogativa

#	Livello	Titolo	Autore/i	Difficoltà	Tempo stimato
1	0 – Fondamenti di informatica	Code	Charles Petzold	Media	10-12 ore
2	0 – Fondamenti di informatica	Computer Science Distilled	Wladston Ferreira Filho	Facile	4-6 ore
3	0 – Fondamenti di informatica	Computer Networking: A Top-Down Approach (cap. introduttivi)	Kurose & Ross	Impegnativo	6-8 ore
4	1 – Sviluppo software	Clean Code	Robert C. Martin	Media	10-12 ore
5	1 – Sviluppo software	The Pragmatic Programmer	Andrew Hunt & David Thomas	Media	8-10 ore
6	2 – Agile	Scrum: The Art of Doing Twice the Work in Half the Time	Jeff Sutherland	Facile	6-8 ore
7	2 – Agile	Essential Scrum	Kenneth Rubin	Media	12-15 ore
8	3 – Project Management	Making Things Happen	Scott Berkun	Media	10-12 ore
9	3 – Project Management	The Phoenix Project	Gene Kim, Kevin Behr, George Spafford	Facile	8-10 ore
10	4 – DevOps	The DevOps Handbook	Gene Kim, Jez Humble, Patrick Debois, John Willis	Impegnativo	15-18 ore
11	4 – DevOps	Accelerate	Nicole Forsgren, Jez Humble, Gene Kim	Impegnativo	10-12 ore

Nota importante: questi 12 libri sono un **arricchimento facoltativo**, non un requisito per completare il corso o per essere operativo nel ruolo — ma sono **fortemente raccomandati** dal responsabile del team, perché offrono una profondità che nessun corso di onboarding può dare in poche settimane. Non c'è alcuna fretta: molti di questi libri richiedono decine di ore di lettura e hanno tutto il senso di essere letti **con calma, anche ben oltre le 8 settimane iniziali** del [Piano di studio](#), magari uno al mese nei primi 6-12 mesi di lavoro. L'importante è tornare a questa lista ogni volta che un argomento del corso ti ha incuriosito e vuoi andare più a fondo.

Collegamenti

- [19. Risorse online](#) — altri materiali gratuiti (articoli, video, documentazione ufficiale) da consultare insieme o in alternativa ai libri
- [17. Piano di studio](#) — il programma delle 8 settimane a cui agganciare le letture suggerite in questa sezione

Risorse

- [Goodreads](#) — per leggere recensioni, valutazioni e trovare edizioni (anche in italiano) di ciascun libro prima di acquistarlo
- [O'Reilly Online Learning](#) — molti di questi titoli (in particolare quelli sullo sviluppo software e su DevOps) sono disponibili in versione e-book con abbonamento, spesso già incluso in piani aziendali
- Tutti i libri elencati sono inoltre disponibili sui principali store online (versione cartacea e e-book) e in molte biblioteche universitarie, essendo testi di riferimento ampiamente diffusi nel settore

19. Risorse online

Come usare questa sezione

Questa non è una sezione da leggere tutta d'un fiato, e non è nemmeno un test: è una **cassetta degli attrezzi**. Raccoglie link a documentazione ufficiale, corsi gratuiti, video e articoli, organizzati per area tematica seguendo lo stesso ordine delle sezioni del corso.

Il modo giusto di usarla è così:

- Mentre studi una sezione (ad esempio la 6, su Scrum), torna qui e apri l'area corrispondente ("Agile, Scrum, Kanban") per trovare materiale di approfondimento se un argomento ti ha incuriosito o non ti è chiaro al 100%.
- Se in una riunione qualcuno cita uno strumento o un concetto che non conosci, puoi tornare qui in un secondo momento (non durante la riunione!) e cercare la risorsa giusta per approfondire con calma.
- Non serve leggere ogni link elencato: sono alternative e integrazioni, non un elenco di compiti a casa. Scegli quelle che si adattano al tuo modo di imparare (alcune persone preferiscono leggere documentazione, altre guardare video, altre seguire corsi strutturati passo passo).
- Molte risorse sono in inglese: è normale, ed è anche una buona palestra, perché la quasi totalità della documentazione tecnica "primaria" (Microsoft, GitLab, AWS, OWASP...) nasce in inglese e le traduzioni arrivano dopo, quando arrivano.

Le risorse sono raggruppate in 6 aree, che corrispondono ai grandi blocchi del corso.

1. Fondamenti di informatica

(per la sezione 2 — Fondamenti di informatica)

- **MDN Web Docs (Mozilla)** — Documentazione ufficiale — Il riferimento più completo e autorevole per capire come funzionano web, browser, HTML/CSS/JS e i concetti di base di Internet, scritto in modo accessibile anche a chi non è sviluppatore. — <https://developer.mozilla.org/>
- **roadmap.sh — Computer Science roadmap** — Corso gratuito / percorso guidato — Una mappa visuale gratuita che elenca, in ordine logico, tutti i concetti di base dell'informatica (hardware, reti, sistemi operativi) con link di approfondimento per ciascuno. — <https://roadmap.sh/>
- **freeCodeCamp** — Corso gratuito — Piattaforma gratuita con corsi introduttivi su informatica, programmazione e reti, pensata per chi parte da zero e vuole progredire con esercizi pratici. — <https://www.freecodecamp.org/>
- **Coursera** — Corso gratuito (con opzione certificazione a pagamento) — Ospita corsi introduttivi di informatica di università reali (ad esempio corsi di "Computer Science 101" o "IT fundamentals"), spesso frequentabili gratuitamente in modalità "audit". — <https://www.coursera.org/>

- **Video — canale ufficiale di un'università o di un provider cloud** — Video — Cerca su YouTube playlist introduttive tipo “how computers work” o “networking basics” pubblicate da canali istituzionali (università, Microsoft, AWS, Google): sono generalmente più affidabili di canali amatoriali. — <https://www.youtube.com/>
 - **Microsoft Learn — Fondamenti di Azure (AZ-900)** — Documentazione ufficiale / corso gratuito — Percorso gratuito e guidato di Microsoft che parte dai concetti di base del cloud, utile anche solo per capire il vocabolario di base dell'informatica moderna. — <https://learn.microsoft.com/training/courses/az-900t01>
-

2. Sviluppo software e Git/GitLab

(per le sezioni 3-4 — Come nasce un software, Git e GitLab)

- **Git — documentazione ufficiale** — Documentazione ufficiale — Il sito ufficiale del progetto Git, con la guida di riferimento e il libro gratuito “Pro Git” scaricabile in italiano e altre lingue. — <https://git-scm.com/>
 - **GitLab Docs** — Documentazione ufficiale — La documentazione ufficiale di GitLab: guide passo passo su repository, merge request, issue e tutte le funzionalità della piattaforma. — <https://docs.gitlab.com/>
 - **Learn Git Branching** — Corso gratuito interattivo — Un tutorial visuale e interattivo (gratuito) che fa vedere graficamente cosa succede a branch e commit man mano che esegui comandi Git; ottimo per capire il “modello mentale” di Git senza usare il terminale. — <https://learngitbranching.js.org/>
 - **GitLab Docs — Tutorials** — Corso gratuito — Tutorial interattivi ufficiali di GitLab, per imparare facendo (merge request, collaborazione, GitLab CI/CD). — <https://docs.gitlab.com/tutorials/>
 - **Atlassian Git Tutorials** — Articolo / documentazione — Una serie di guide molto chiare (con immagini) sui concetti fondamentali di Git: branching, merge, workflow di team. — <https://www.atlassian.com/git>
 - **roadmap.sh — Backend / Full Stack roadmap** — Corso gratuito / percorso guidato — Mappe gratuite che mostrano, in ordine, i concetti che uno sviluppatore impara per costruire software, utili per capire il “vocabolario” dei tuoi colleghi sviluppatori. — <https://roadmap.sh/>
-

3. Agile, Scrum, Kanban

(per le sezioni 5-7 — Agile, Scrum, Kanban)

- **Manifesto for Agile Software Development** — Documentazione ufficiale — Il testo originale del Manifesto Agile (disponibile anche in italiano), con i 4 valori e i 12 principi che hanno dato origine a tutte le metodologie Agile. — <https://agilemanifesto.org/>
- **Scrum Guide (Scrum Guides)** — Documentazione ufficiale — La Guida Scrum ufficiale, scritta dai creatori di Scrum (Ken Schwaber e Jeff Sutherland), disponibile gratuitamente in molte lingue incluso l'italiano. — <https://scrumguides.org/>

- **Scrum.org** — Documentazione ufficiale / community — Il sito dell'organizzazione fondata da Ken Schwaber, con articoli, blog e risorse gratuite su Scrum e sul ruolo di Scrum Master. — <https://www.scrum.org/>
 - **Atlassian — Agile Coach** — Articolo / corso gratuito — Una guida online gratuita e molto completa su Agile, Scrum e Kanban, scritta con un linguaggio pratico e molti esempi concreti. — <https://www.atlassian.com/agile>
 - **Atlassian — Kanban Guide** — Articolo — Guida dedicata specificamente a Kanban: colonne, WIP limit, flusso continuo, con esempi visuali. — <https://www.atlassian.com/agile/kanban>
 - **Kanbanize (Businessmap) — Kanban Guide** — Articolo — Una risorsa gratuita e molto dettagliata dedicata interamente al metodo Kanban, alle sue origini (Toyota) e alla sua applicazione nello sviluppo software. — <https://kanbanize.com/kanban-resources>
 - **Scrum Alliance** — Community / documentazione — Una delle principali organizzazioni internazionali dedicate a Scrum, con articoli, risorse gratuite e informazioni sulle certificazioni (utile anche solo per capire il vocabolario delle certificazioni che potresti sentir citare). — <https://www.scrumalliance.org/>
-

4. Project Management

(per la sezione 8 — Project Management)

- **PMI — Project Management Institute** — Documentazione ufficiale / community — L'organizzazione internazionale di riferimento per il project management, con articoli gratuiti, standard di settore e informazioni sulle certificazioni (come il PMP). — <https://www.pmi.org/>
 - **PMI — What is Project Management** — Articolo — Pagina introduttiva ufficiale del PMI che spiega cos'è il project management, i suoi principi e le competenze richieste, utile come base comune di vocabolario. — <https://www.pmi.org/about/learn-about-pmi/what-is-project-management>
 - **Atlassian — Project Management Guide** — Articolo — Guida pratica e gratuita, orientata al software, su tecniche di project management applicate a team di sviluppo (roadmap, stime, gestione del rischio). — <https://www.atlassian.com/agile/project-management>
 - **Coursera — corsi di Project Management** — Corso gratuito (con opzione certificazione) — Diverse università e aziende (incluso Google) pubblicano su Coursera corsi introduttivi di project management, spesso frequentabili gratuitamente in modalità "audit". — <https://www.coursera.org/>
 - **freeCodeCamp — articoli su Project Management** — Articolo — Il blog di freeCodeCamp pubblica periodicamente guide gratuite e accessibili su tecniche di project management applicate al software. — <https://www.freecodecamp.org/news/>
-

5. DevOps, Azure DevOps, CI/CD

(per le sezioni 9-11 — DevOps, Azure DevOps, CI/CD)

- **Microsoft Learn — Cos'è DevOps** — Documentazione ufficiale — La spiegazione ufficiale Microsoft dei principi DevOps, con il modello CALMS e le pratiche principali (CI/CD, monitoraggio, cultura collaborativa). — <https://learn.microsoft.com/devops/what-is-devops>
- **Microsoft Learn — Documentazione Azure DevOps** — Documentazione ufficiale — La documentazione completa e ufficiale di Azure DevOps: Boards, Repos, Pipelines, Artifacts e Test Plans, con guide passo passo. — <https://learn.microsoft.com/azure/devops/>
- **Microsoft Learn — Percorsi di formazione DevOps** — Corso gratuito — Percorsi formativi gratuiti e ufficiali Microsoft su DevOps e Azure DevOps, organizzati per moduli con esercizi pratici. — <https://learn.microsoft.com/training/browse/?terms=devops>
- **GitLab Docs — CI/CD** — Documentazione ufficiale — La documentazione ufficiale di GitLab CI/CD, lo strumento di pipeline integrato in GitLab, utile per confrontare un approccio alternativo a quello di Azure Pipelines. — <https://docs.gitlab.com/ci/>
- **Martin Fowler — Continuous Integration** — Articolo — Un articolo di riferimento, scritto da uno degli autori più citati nel mondo dello sviluppo software, che spiega in profondità cos'è la Continuous Integration e perché conta. — <https://martinfowler.com/articles/continuousIntegration.html>
- **Atlassian — CI/CD Pipeline** — Articolo — Guida pratica con schemi visuali su cosa sono le pipeline di CI/CD e come si differenziano CI, delivery continua e deployment continuo. — <https://www.atlassian.com/continuous-delivery/continuous-integration>
- **Docker — documentazione ufficiale** — Documentazione ufficiale — Se nelle pipeline del team senti parlare di container e immagini Docker, questa è la documentazione ufficiale con cui iniziare a orientarsi. — <https://docs.docker.com/>
- **roadmap.sh — DevOps roadmap** — Corso gratuito / percorso guidato — Una mappa gratuita che elenca, in ordine logico, tutti i concetti e strumenti dell'ecosistema DevOps, con link di approfondimento per ciascuno. — <https://roadmap.sh/devops>

6. Architetture, Cloud, Sicurezza

(per le sezioni 12-14 — Architetture software, Cloud, Sicurezza)

- **Microsoft Learn — Azure Architecture Center** — Documentazione ufficiale — Raccolta ufficiale di pattern architetturali, best practice e glossario dedicati alle architetture cloud moderne (utile anche solo per orientarsi tra i diagrammi che il team potrebbe mostrarti). — <https://learn.microsoft.com/azure/architecture/>
- **Microsoft Learn — Fondamenti di Azure** — Corso gratuito — Il percorso formativo gratuito e ufficiale Microsoft per capire i concetti base del cloud: risorse, regioni, modelli di servizio (IaaS/PaaS/SaaS). — <https://learn.microsoft.com/training/azure/>
- **AWS — Cloud concepts / What is Cloud Computing** — Documentazione ufficiale — La spiegazione ufficiale di Amazon Web Services su cosa significa “cloud computing”, utile per un confronto con la terminologia usata da Microsoft Azure. — <https://aws.amazon.com/what-is-cloud-computing/>

- **Martin Fowler — bliki (Microservices e architetture)** — Articolo — Articoli di riferimento sul tema delle architetture software moderne, inclusi i microservizi, scritti con un linguaggio più discorsivo rispetto alla documentazione tecnica pura. — <https://martinfowler.com/microservices/>
- **OWASP Top 10** — Documentazione ufficiale — La classifica ufficiale e gratuita delle vulnerabilità di sicurezza più critiche per le applicazioni web, un riferimento citato in moltissimi contratti e audit di sicurezza. — <https://owasp.org/www-project-top-ten/>
- **OWASP Foundation** — Documentazione ufficiale / community — Il sito della fondazione OWASP, con decine di progetti e guide gratuite sulla sicurezza applicativa, oltre a eventi e community locali. — <https://owasp.org/>
- **Microsoft Learn — Zero Trust e concetti di sicurezza** — Documentazione ufficiale — Introduzione ufficiale Microsoft ai principi moderni di sicurezza applicati al cloud, utile per capire il vocabolario che i colleghi tecnici usano nelle discussioni su sicurezza e accessi. — <https://learn.microsoft.com/security/zero-trust/>
- **Microsoft Learn — DevSecOps** — Documentazione ufficiale — Approfondimento su come la sicurezza si integra nel ciclo DevOps, con pratiche come lo “shift left” già incontrato nella sezione 14. — <https://learn.microsoft.com/devops/operate/security-in-devops>

Come restare aggiornato nel tempo

Il mondo DevOps si evolve rapidamente: nuove funzionalità di Azure DevOps, nuove versioni degli strumenti, nuove pratiche di sicurezza. Non serve rincorrere ogni novità, ma un aggiornamento periodico e leggero ti aiuterà a non restare indietro. Qualche suggerimento pratico:

- **Segui i blog ufficiali degli strumenti che usa il team:** sia Microsoft (Azure DevOps Blog, Azure Blog) sia GitLab (GitLab Blog) pubblicano regolarmente novità, cambi di funzionalità e guide pratiche — bastano 10 minuti ogni tanto per restare al passo.
- **Iscriviti a una newsletter di settore:** esistono diverse newsletter gratuite (via email) dedicate ad Agile, DevOps o Project Management che riassumono settimanalmente le notizie più rilevanti, così non devi andare a cercarle una per una.
- **Partecipa a community e, se possibile, a un evento o webinar all'anno:** gruppi locali o online di Scrum Master/Agile Coach, conferenze di settore (anche solo guardando le registrazioni gratuite pubblicate dopo l'evento) e community come quella di Scrum.org o PMI ti mettono in contatto con altre persone che affrontano le tue stesse sfide quotidiane.

Collegamenti

- [16. Glossario](#) — per ripassare rapidamente la terminologia incontrata in queste risorse
- [17. Piano di studio](#) — il percorso settimanale in cui inserire, se vuoi, l'approfondimento di alcune di queste risorse